



Webseiten Projekt

Grundlagen HTML / CSS

Webseiten Projekt

Version 1.0.0 · Stand 2026-10-04 · Commit 82c4d7a

© 2026 Jens Reiner · Lizenz: CC BY-SA 4.0



Inhalt

Teil 1 • HTML & CSS	1
1.1 Das Skelett	2
1.2 Text & Semantik	11
1.3 Navigation & mehrere Seiten	21
1.4 Bilder	34
1.5 Tabellen & Formulare	45
1.6 CSS-Grundlagen	58
1.7 CSS-Fundament	69
1.8 Layout	85
1.9 Responsive	100
1.10 Design-System	116
1.11 Nicht jeder Browser kann alles	132
1.12 Interaktion ohne JavaScript	143
1.13 Barrierefreiheit prüfen	159
Anhang	176
A Glossar	177



Teil 1 • HTML & CSS



1.1 Das Skelett

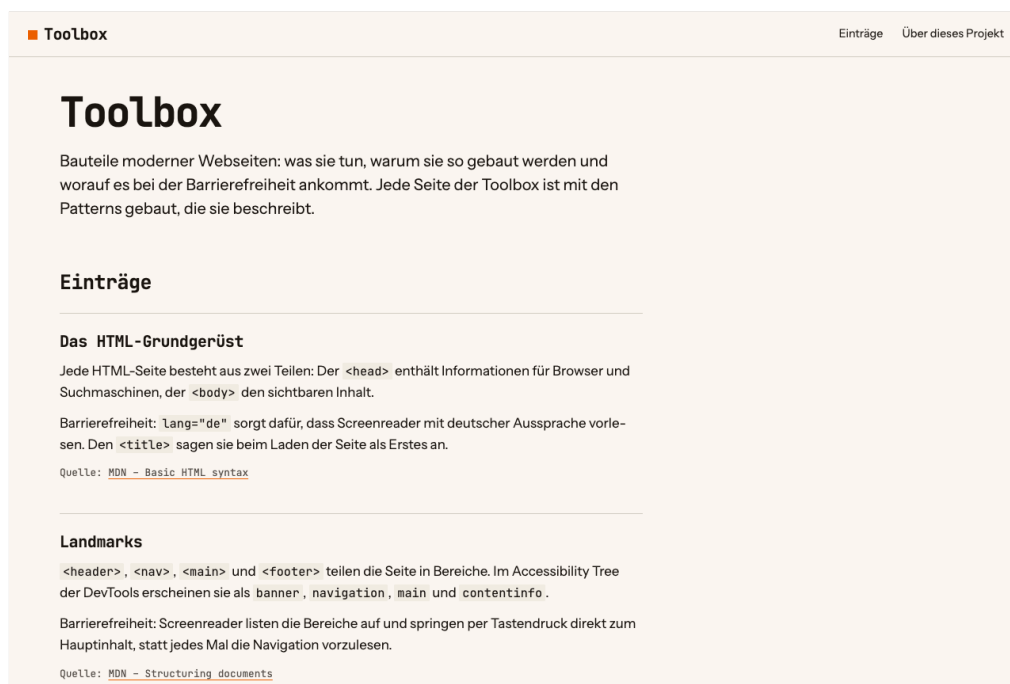
Voraussetzungen: Dateisystem & Pfade, VS Code, Browser & DevTools, Git & GitHub, Markdown & README

Lernziele

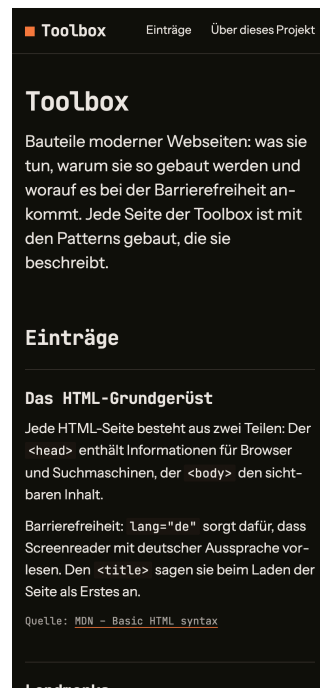
- Das Grundgerüst eines HTML-Dokuments aufbauen und die Wirkung von viewport nachweisen.
- Eine Seite mit den Landmarks `<header>`, `<nav>`, `<main>` und `<footer>` strukturieren und ihre Rollen im Accessibility Tree nachweisen.
- Quelltext und DOM unterscheiden und eine Seite mit Validator und DevTools prüfen.

1. Vorschau

Zielstand nach diesem Kapitel: die erste Seite der Toolbox mit zwei Einträgen.



Dieselbe Seite bei Smartphone-Breite im Dark Mode. Das Start-Stylesheet enthält beide Darstellungen.



Die Texte der Einträge im Bild sind Beispiele.

2. Warum

Eine Webseite ist zuerst Struktur und erst danach Gestaltung. Browser, Suchmaschinen und Screenreader werten die Struktur einer Seite aus. Dazu gehören die Lage der Navigation, der Beginn des Hauptinhalts und die Sprache des Texts.

Weiterführend: [MDN – Basic HTML syntax](#) · [MDN – Structuring documents](#)

3. Bauen

3.1 Projekt vorbereiten

Ausgangsstand ist die Toolbox aus dem Kapitel Markdown & README mit README.md, learning-notizen.md und .gitignore.

1. Öffne die Toolbox in VS Code.
2. Lade [assets.zip](#) herunter und entpacke die Datei in den Projektordner (siehe Dateisystem & Pfade). Der Ordner assets enthält das Start-Stylesheet und die Schriften.

Das Projekt hat danach diesen Aufbau:

```

toolbox/
├── README.md
├── learning-notizen.md
├── .gitignore
├── assets/
│   ├── css/
│   │   └── base.css
│   └── fonts/
│       └── ... (Schriften und Lizenzen)

```

Die Datei `base.css` und die Schriften sind vorgegeben. Dieses Kapitel setzt kein Verständnis des Stylesheets voraus.

3.2 Das Grundgerüst

Lege im Ordner `toolbox` die Datei `index.html` mit diesem Inhalt an:

```

<!doctype html>
<html lang="de">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Toolbox</title>
    <meta name="description" content="Bauteile moderner Webseiten zum
Nachschlagen – gebaut mit genau diesen Bauteilen.">
    <link rel="stylesheet" href="assets/css/base.css">
  </head>
  <body>

  </body>
</html>

```

HTML beschreibt eine Seite mit **Elementen**. Ein Element beginnt mit einem öffnenden **Tag** wie `<title>` und endet mit einem schließenden Tag wie `</title>`. `<meta>` und `<link>` haben keinen schließenden Tag. Ein **Attribut** im öffnenden Tag ergänzt eine Angabe, zum Beispiel `lang="de"`. `<head>` enthält Angaben für Browser und Suchmaschinen, `<body>` den sichtbaren Inhalt.

Angabe	Aufgabe
<code><!doctype html></code>	Der Browser stellt die Seite nach dem aktuellen HTML-Standard dar.
<code>lang="de"</code>	Sprache der Seite. Screenreader lesen den Text mit der Aussprache dieser Sprache vor. Browser verwenden sie für Übersetzung und Silbentrennung.
<code>charset="utf-8"</code>	<u>Zeichenkodierung</u> der Datei: UTF-8
<code>viewport</code>	Breite, mit der der Browser die Seite auf einem Smartphone darstellt
<code><title></code>	Titel im Tab, in Lesezeichen und in Suchergebnissen
<code>description</code>	Kurzbeschreibung für Suchmaschinen
<code><link rel="stylesheet"></code>	Bindet das Stylesheet über einen relativen Pfad ein (siehe Dateisystem & Pfade).

Öffne `index.html` in Chrome. Der Tab zeigt den Titel „Toolbox“, die Seite ist leer. Zeigt Chrome den Quelltext als Text, heißt die Datei `index.html.txt` (siehe Dateisystem & Pfade).

3.3 Die Landmarks

Der sichtbare Teil der Seite besteht aus drei Bereichen: Header mit Navigation, Hauptinhalt und Footer. Die Elemente `<header>`, `<nav>`, `<main>` und `<footer>` heißen Landmarks: Seitenbereiche mit einer festen Rolle im Accessibility Tree.

Ersetze in `index.html` die Zeilen von `<body>` bis `</body>` durch diesen Inhalt:

```
<body>
  <header>
    <p><a href="index.html">Toolbox</a></p>
    <nav>
      <ul>
        <li><a href="#eintraege">Einträge</a></li>
        <li><a href="#ueber">Über dieses Projekt</a></li>
      </ul>
    </nav>
  </header>

  <main>
    <h1>Toolbox</h1>
    <p>Ein Satz zum Zweck der Toolbox in eigenen Worten.</p>

    <section id="eintraege">
      <h2>Einträge</h2>
      <!-- Toolbox-Einträge aus Abschnitt 5 -->
    </section>

    <section id="ueber">
      <h2>Über dieses Projekt</h2>
      <p>Zweck der Toolbox.</p>
    </section>
  </main>

  <footer>
    <p>Toolbox · Bauteile für das Web</p>
  </footer>
</body>
```

- Eine Seite enthält genau ein Element `<h1>`: die Hauptüberschrift.
- `<p>` ist ein Absatz, `<section>` ein Abschnitt mit eigener Überschrift `<h2>`. Überschriften, Listen und Links behandelt das Kapitel Text & Semantik (Kapitel 1.2).
- Die Navigation ist eine Liste von Links, weil sie Ziele aufzählt.
- `href="#eintraege"` verweist auf das Element mit `id="eintraege"` auf derselben Seite.
- Den Kommentar `<!-- ... -->` zeigt der Browser nicht an. Die Einrückung dient nur der Lesbarkeit.

Ersetze die Sätze unter `<h1>` und unter „Über dieses Projekt“ durch eigene Sätze zum Zweck der Toolbox.

3.4 Seite im Browser prüfen

Lade die Seite in Chrome neu. Der Header enthält die **Wordmark** „Toolbox“, den Link mit dem farbigen Quadrat, und die Navigation. Darunter folgen die Überschrift „Toolbox“ und die Ab-

schnitte „Einträge“ und „Über dieses Projekt“. Die zwei Einträge der Vorschau entstehen im Abschnitt Toolbox-Eintrag.

Die Gestaltung stammt aus `base.css`. Weicht sie von der Vorschau ab, vergleiche den Pfad in `<link>` mit dem Verzeichnisbaum in Projekt vorbereiten. Die Datei `base.css` liegt in `toolbox/assets/css/`.

3.5 Commit

Die Toolbox hat jetzt eine Seite. Ergänze in `README.md` nach dem Absatz zum Zweck den Abschnitt zum Start des Projekts (siehe Markdown & README):

Ansehen

``index.html`` im Browser öffnen (Doppelklick oder in den Browser ziehen). Die Seite braucht keinen Server und keine Installation.

Ergänze im Strukturbaum die Datei `index.html` und den Ordner `assets` mit `css/base.css` und `fonts/`. Der Abschnitt `## Struktur` hat danach diesen Inhalt:

Struktur

```
...
toolbox/
├── README.md           ← diese Datei
├── learning-notizen.md ← Lernprotokoll: Vorhersagen und Beobachtungen
├── .gitignore          ← Systemdateien, die nicht ins Repository gehören
├── index.html          ← Startseite mit allen Einträgen
├── assets/
│   ├── css/
│   │   └── base.css    ← Start-Stylesheet (vorgegeben)
│   └── fonts/          ← Schriften + Lizenzen (vorgegeben)
...
```

1. Übertrage alle Änderungen mit `git add .` in die Staging Area (siehe Git & GitHub).
2. Prüfe mit `git status`, dass alle geänderten Dateien unter **Changes to be committed** stehen.
3. Erstelle einen Commit mit der Nachricht `feat: Grundgerüst mit Landmarks anlegen` und übertrage ihn mit `git push`.

4. Verstehen

Lege im Lernprotokoll die Überschrift `## Das Skelett (Datum)` an. Der Ablauf je Experiment ist: Vorhersage notieren, ausführen, Ergebnis und Erklärung notieren.

Nimm jede Änderung nach dem Experiment zurück: `Strg/Cmd + Z` oder `git restore index.html`.

Break-it 1: Landmarks durch `<div>` ersetzen

Öffne in den DevTools das Panel **Elements** und den Tab **Accessibility**. Wähle `<header>`, `<main>` und `<footer>` nacheinander aus und lies unter **Computed properties** jeweils **Role**.

Ersetze danach die Elemente `<header>`, `<main>` und `<footer>` durch `<div>`, jeweils im öffnenden und im schließenden Tag.

- **Vorhersage:** Wie ändert sich die Darstellung? Was ändert sich für einen Screenreader?
- **Prüfen:**
 1. Lade die Seite neu.
 2. Wähle die drei Elemente `<div>` nacheinander aus und vergleiche den Tab **Accessibility** mit dem Stand vor der Änderung.

→ [Auflösung 1](#) am Ende des Kapitels

Break-it 2: Viewport entfernen

Lösche die Zeile `<meta name="viewport" ...>`.

- **Vorhersage:** Was ändert sich auf dem Desktop? Was ändert sich auf einem Smartphone?
- **Prüfen:**
 1. Schalte in den DevTools die **Device Toolbar** ein.
 2. Wähle ein Smartphone.
 3. Lade die Seite neu.

→ [Auflösung 2](#) am Ende des Kapitels

Break-it 3: Falscher Pfad zum Stylesheet

Ändere `href="assets/css/base.css"` in `href="css/base.css"`.

- **Vorhersage:** Welche Fehlermeldung zeigt die Seite?
- **Prüfen:**
 1. Öffne in den DevTools das Panel **Network**.
 2. Lade die Seite neu.
 3. Öffne das Panel **Console**.

→ [Auflösung 3](#) am Ende des Kapitels

Predict 1: Quelltext und DOM

Füge in `index.html` direkt vor der Zeile `</main>` diese Zeile ein:

```
<p>Hallo<div>Welt</div></p>
```

- **Vorhersage:** Welche Struktur erzeugt der Browser daraus? Wie viele Elemente `<p>` enthält sie?
- **Prüfen:**
 1. Lade die Seite neu.
 2. Öffne den Quelltext der Seite: `Strg + U` (macOS: `Cmd + Option + U`).
 3. Vergleiche ihn mit dem Panel **Elements** der DevTools.
 4. Wähle im [W3C-Validator](#) unter **Check by** die Option **file upload** und danach die Datei `index.html`.
 5. Klicke auf **Check**.

→ [Auflösung 4](#) am Ende des Kapitels

5. Toolbox-Eintrag

Ersetze in `index.html` den Kommentar `<!-- Toolbox-Einträge aus Abschnitt 5 -->` durch zwei Einträge: „Das HTML-Grundgerüst“ und „Landmarks“. Jeder Eintrag ist ein Element `<article>`:

```
<article>
  <h3>Landmarks</h3>
  <p>Beschreibung und Zweck in eigenen Worten.</p>
  <p>Barrierefreiheit: Auswirkung auf Screenreader und Tastaturbedienung.</p>
  <p>Quelle: <a href="https://developer.mozilla.org/...">MDN - ...</a></p>
</article>
```

`<article>` umschließt einen Eintrag, der auch für sich allein verständlich ist. `<h3>` ist seine Überschrift, eine Ebene unter `<h2>` „Einträge“.

Jeder Eintrag enthält:

1. **Beschreibung und Zweck** in zwei bis fünf Sätzen, in eigenen Worten.
2. **Barrierefreiheit**: die Auswirkung auf Screenreader und Tastaturbedienung.
3. **Quelle**: einen Link auf die zugehörige MDN-Seite.

Für das Grundgerüst beschreibt der Punkt Barrierefreiheit die Wirkung von `lang` aus der Tabelle der Angaben, für Landmarks die Auflösung von Break-it 1. Ohne Screenreader ändern beide nichts an der Bedienung mit der Tastatur. Die zwei Einträge nennen deshalb nur den Screenreader. Quelle für das Grundgerüst ist MDN – Basic HTML syntax, für Landmarks MDN – Structuring documents.

Schreibe spitze Klammern im Text als `<` und `>`, zum Beispiel `<main>`. Sonst liest der Browser sie als Tag.

Maßstab für die Beschreibung, am Beispiel des Viewports:

Beispiel	
Unzureichend	„Der Viewport passt die Seite an das Gerät an.“
Ausreichend	„Ohne <code><meta name="viewport"></code> rechnet ein Smartphone mit einer Breite von rund 980 Pixel und verkleinert die Seite. Mit der Angabe verwendet die Seite die Breite des Geräts.“

Die Referenz-Toolbox zeigt die vollständige Toolbox, wie sie am Ende aussehen kann.

Lade die Seite neu. Unter „Einträge“ stehen die Überschriften „Das HTML-Grundgerüst“ und „Landmarks“ mit je drei Absätzen.

1. Setze in `README.md` in der Tabelle `## Fortschritt` „Das Skelett“ auf „erledigt“ und ergänze die Zeile „Text & Semantik“ mit „offen“.
2. Übertrage alle Änderungen mit `git add .` in die Staging Area.
3. Prüfe mit `git status`, dass alle geänderten Dateien unter **Changes to be committed** stehen, auch `learning-notizen.md`.
4. Erstelle einen Commit mit der Nachricht `feat: Einträge Grundgerüst und Landmarks ergänzen` und übertrage ihn mit `git push`.

6. Prüfen

- ☐ Der [W3C-Validator](#) zeigt für `index.html` die Meldung „Document checking completed. No errors or warnings to show.“
- ☐ Der Accessibility Tree zeigt `banner`, `navigation`, `main` und `contentinfo`.
- ☐ Die Seite enthält genau ein `<h1>`.
- ☐ Die `README.md` beschreibt Zweck, Aufbau und Öffnen des Projekts.
- ☐ `learning-notizen.md` enthält zu jedem Experiment Vorhersage, Ergebnis und Erklärung.
- ☐ Die Einträge „Das HTML-Grundgerüst“ und „Landmarks“ stehen unter „Einträge“.
- ☐ Die Commits `feat: Grundgerüst mit Landmarks anlegen` und `feat: Einträge Grundgerüst und Landmarks ergänzen` sind auf GitHub.

Quellen

- [MDN – Basic HTML syntax](#)
- [MDN – Structuring documents](#)
- [MDN – What’s in the head? Web page metadata](#)
- [MDN – Accessibility tree](#)
- [MDN – `<meta name="viewport">`](#)
- [MDN – `lang`](#)
- [W3C Nu HTML Checker](#)

Auflösungen

Auflösung 1

Zu Break-it 1: Landmarks durch <div> ersetzen

Die Gestaltung geht teilweise verloren: Die Wordmark verliert ihr Quadrat, Header und Footer verlieren ihre Linien, und die Navigation steht unter der Wordmark. Das Stylesheet `base.css` gestaltet die Elemente `<header>`, `<main>` und `<footer>` selbst. Ein `<div>` an ihrer Stelle erhält diese Gestaltung nicht.

Im Accessibility Tree fehlen die Rollen `banner`, `main` und `contentinfo`. Ein Screenreader kann die Bereiche nicht mehr direkt ansteuern, zum Beispiel den Hauptinhalt.

Auflösung 2

Zu Break-it 2: Viewport entfernen

Auf dem Desktop ändert sich nichts. Auf dem Smartphone rechnet der Browser mit einer Breite von rund 980 Pixel und verkleinert die gesamte Seite. Der Text ist ohne Zoomen nicht lesbar. Mit der Angabe verwendet die Seite die tatsächliche Breite des Geräts.

Auflösung 3

Zu Break-it 3: Falscher Pfad zum Stylesheet

Die Seite zeigt keine Fehlermeldung. Der Browser stellt sie ohne Stylesheet dar. Der Fehler steht nur in der Console (`ERR_FILE_NOT_FOUND`) und im Panel **Network** als fehlgeschlagene Anfrage.

Auflösung 4

Zu Predict 1: Quelltext und DOM

Der Quelltext ist der Inhalt der Datei. Der DOM ist die Struktur, die der Browser daraus erzeugt:

```
<p>Hallo</p><div>Welt</div><p></p>
```

Ein Element `<p>` darf kein `<div>` enthalten. Der Browser schließt das `<p>` deshalb vor dem `<div>` und erzeugt aus dem verbleibenden `</p>` ein leeres `<p>`. Der Validator meldet einen Fehler.

Der Browser korrigiert fehlerhaftes HTML, ohne es zu melden. Der Validator macht diese Fehler sichtbar.



1.2 Text & Semantik

Voraussetzungen: [Das Skelett](#) (Kapitel 1.1)

Lernziele

- Inhalte mit einer lückenlosen Überschriften-Hierarchie gliedern und die Gliederung im Accessibility Tree nachweisen.
- Listen und Textauszeichnung nach Bedeutung statt nach Aussehen wählen.
- Links mit aussagekräftigem Linktext schreiben und für relative Pfade, absolute URLs und Anchors vorhersagen, welches Ziel der Browser öffnet.

1. Vorschau

Zielstand nach diesem Kapitel: Die Einträge der Toolbox sind mit Listen und Auszeichnungen gegliedert. Drei Einträge sind neu.

■ Toolbox

EinträgeÜber dieses Projekt

Toolbox

Bauteile moderner Webseiten: was sie tun, warum sie so gebaut werden und worauf es bei der Barrierefreiheit ankommt. Jede Seite der Toolbox ist mit den Patterns gebaut, die sie beschreibt.

Einträge

Das HTML-Grundgerüst

Jede HTML-Seite besteht aus zwei Teilen: Der `<head>` enthält Informationen für Browser und Suchmaschinen, der `<body>` den sichtbaren Inhalt.

Barrierefreiheit: `lang="de"` sorgt dafür, dass Screenreader mit deutscher Aussprache vorlesen. Den `<title>` sagen sie beim Laden der Seite als Erstes an.

Quelle: [MDN - Basic HTML syntax](#)

Landmarks

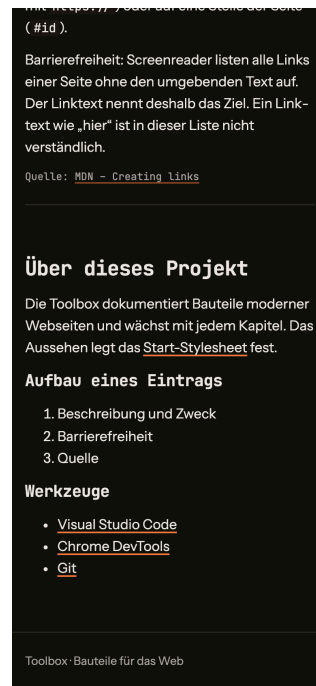
Landmarks teilen die Seite in Bereiche. Jedes Element erhält im Accessibility Tree eine Rolle:

`<header>`
Rolle banner

`<nav>`
Rolle navigation

`<main>`

Der Abschnitt „Über dieses Projekt“ bei Smartphone-Breite im Dark Mode:



Die Texte der Einträge im Bild sind Beispiele.

2. Warum

HTML-Elemente beschreiben die Bedeutung eines Inhalts, nicht sein Aussehen. Eine Überschrift ist eine Überschrift, weil sie einen Abschnitt benennt, und nicht, weil sie groß und fett erscheint. Screenreader und Suchmaschinen werten diese Bedeutung aus. Das Aussehen legt ausschließlich CSS fest.

Weiterführend: [MDN – Headings and paragraphs](#) · [MDN – Lists](#) · [MDN – Creating links](#)

3. Bauen

Ausgangsstand ist die Toolbox aus dem Kapitel [Das Skelett](#) (Kapitel 1.1): die Datei `index.html` mit zwei Einträgen.

3.1 Überschriften-Hierarchie

Die Elemente `<h1>` bis `<h6>` bilden die Gliederung einer Seite, die [Überschriften-Hierarchie](#). Die Toolbox verwendet bisher drei Ebenen:

```
h1  Toolbox
  h2  Einträge
    h3  Das HTML-Grundgerüst
    h3  Landmarks
  h2  Über dieses Projekt
```

Für die Hierarchie gelten drei Regeln:

Regel	Begründung
Eine Seite enthält genau ein <code><h1></code> .	Das <code><h1></code> benennt das Thema der Seite.
Die Hierarchie überspringt keine Ebene.	Auf <code><h2></code> folgt <code><h2></code> oder <code><h3></code> , nicht <code><h4></code> . Auf <code><h3></code> darf wieder <code><h2></code> folgen. Bei einer Lücke vermuten Lesende einen fehlenden Abschnitt.
Die Ebene folgt aus der Struktur, nicht aus der Schriftgröße.	Die Größe einer Überschrift legt CSS fest.

Ergänze den Abschnitt „Über dieses Projekt“ um zwei Unterabschnitte. Füge nach dem Absatz unter `<h2>Über dieses Projekt</h2>` diese Zeilen ein:

```
<h3>Aufbau eines Eintrags</h3>

<h3>Werkzeuge</h3>
```

Lade die Seite neu. Unter „Über dieses Projekt“ stehen die Überschriften „Aufbau eines Eintrags“ und „Werkzeuge“, noch ohne Text. Ihre Inhalte entstehen in [Listen](#) und [Links](#).

3.2 Absätze und Auszeichnung

Das Element `<p>` umschließt einen Absatz. Innerhalb eines Absatzes kennzeichnen weitere Elemente die Bedeutung einzelner Wörter:

Element	Bedeutung	Nicht verwenden für
<code></code>	Der Inhalt ist wichtig oder dringend.	fette Schrift ohne inhaltlichen Grund
<code></code>	Betonung, die den Sinn des Satzes ändert	kursive Schrift ohne inhaltlichen Grund
<code><code></code>	Code: Elementnamen, Attribute, Dateinamen	

Ein Beispiel für `<em>`: Die Ebene folgt aus der Struktur des Inhalts, `<em>nicht</em>` aus der gewünschten Schriftgröße. Die Betonung liegt auf dem Gegensatz.

Die Elemente `<b>` und `<i>` erzeugen dieselbe Darstellung wie `<strong>` und `<em>`, tragen aber keine Bedeutung. Das Element `<br>` erzeugt einen Zeilenumbruch innerhalb eines Absatzes, zum Beispiel in einer Adresse. Es ist kein Mittel für Abstände und kein Ersatz für Absätze oder Listen.

Überarbeite deine beiden Einträge:

1. Kennzeichne jeden Elementnamen und jedes Attribut mit `<code>`, zum Beispiel `<main>` und `lang="de"`. Schreibe die spitzen Klammern im Text als `<` und `>`, wie im [Toolbox-Eintrag](#) (Kapitel 1.1) von *Das Skelett*.
2. Kennzeichne höchstens eine Aussage je Eintrag mit `<strong>`, wenn sie wichtig ist.

Beispiel aus dem Eintrag „Landmarks“:

```
<p>Barrierefreiheit: Screenreader listen die Bereiche auf und springen
direkt zum Hauptinhalt. Eine Seite enthält <strong>genau ein</strong>
<code>&lt;main&gt;</code>.</p>
```

Lade die Seite neu. Die Wörter mit `<code>` erscheinen in Festbreitenschrift mit eigenem Hintergrund, die Aussage mit `<strong>` fett.

3.3 Listen

HTML kennt drei Arten von Listen. Die Art folgt aus dem Inhalt:

Element	Inhalt	Beispiel
mit	Einträge ohne feste Reihenfolge	Werkzeuge, Zutaten
mit	Einträge mit fester Reihenfolge	Arbeitsschritte, Rangfolge
<dl> mit <dt> und <dd>	Paare aus Begriff und Beschreibung	Glossar, Element und Rolle

Probe für : Ändert eine andere Reihenfolge die Aussage, ist die Reihenfolge fest.

Schritt 1: Die Teile eines Eintrags haben eine feste Reihenfolge. Ergänze unter <h3>Aufbau eines Eintrags</h3>:

```
<ol>
  <li>Beschreibung und Zweck</li>
  <li>Barrierefreiheit</li>
  <li>Quelle</li>
</ol>
```

Schritt 2: Die vier Elemente der Landmarks und ihre Rollen sind Paare aus Begriff und Beschreibung. Ersetze im Eintrag „Landmarks“ den ersten Absatz unter <h3>Landmarks</h3> durch diesen Code. Der Code ersetzt deine Beschreibung. Der Absatz „Barrierefreiheit“ folgt danach unverändert:

```
<p>Landmarks teilen die Seite in Bereiche. Jedes Element erhält im
Accessibility Tree eine Rolle:</p>
<dl>
  <dt><code>&lt;header&gt;</code></dt>
  <dd>Rolle <code>banner</code></dd>
  <dt><code>&lt;nav&gt;</code></dt>
  <dd>Rolle <code>navigation</code></dd>
  <dt><code>&lt;main&gt;</code></dt>
  <dd>Rolle <code>main</code></dd>
  <dt><code>&lt;footer&gt;</code></dt>
  <dd>Rolle <code>contentinfo</code></dd>
</dl>
```

Lade die Seite neu. Unter „Aufbau eines Eintrags“ stehen drei nummerierte Punkte. Die Beschreibungen <dd> stehen ohne Einrückung unter ihrem Begriff, weil das Start-Stylesheet den Abstand am Rand entfernt. Die Bedeutung ist davon unabhängig: Der Accessibility Tree enthält die Rollen `term` und `definition`.

3.4 Links

Das Element <a> verweist mit dem Attribut href auf ein Ziel. Die Form des Werts bestimmt die Art des Ziels:

Ziel	Wert von href	Beispiel
Datei im eigenen Projekt	relativer Pfad	assets/css/base.css
andere Website	absolute URL mit Schema (https://)	https://git-scm.com/
Stelle auf derselben Seite (Anchor)	# und die id des Elements	#eintraege

Der Linktext nennt das Ziel. Er ist ohne den umgebenden Satz verständlich.

Beispiel

Unzureichend	Das Stylesheet steht <code>hier</code> .
Ausreichend	Das Aussehen legt das <code>Start-Stylesheet</code> fest.

Schritt 1: Ergänze am Ende des Absatzes unter `<h2>Über dieses Projekt</h2>`, direkt vor `</p>`, einen Satz mit einem Link mit relativem Pfad. ... steht für deinen bisherigen Text:

```
<p>... Das Aussehen legt das <a href="assets/css/base.css">Start-Stylesheet</a> fest.</p>
```

Schritt 2: Ergänze unter `<h3>Werkzeuge</h3>` eine Liste mit drei Links auf andere Websites. Die Werkzeuge haben keine feste Reihenfolge:

```
<ul>
  <li><a href="https://code.visualstudio.com/">Visual Studio Code</a></li>
  <li><a href="https://developer.chrome.com/docs/devtools">Chrome DevTools</a></li>
  <li><a href="https://git-scm.com/">Git</a></li>
</ul>
```

Schritt 3: Lade die Seite neu und klicke nacheinander auf die vier neuen Links. Mit dem Zurück-Pfeil des Browsers kommst du jeweils zur Toolbox zurück.

- „Start-Stylesheet“ zeigt den Text der Datei `base.css`.
- Die Links unter „Werkzeuge“ öffnen die Websites von Visual Studio Code, Chrome DevTools und Git.

Die Navigation im Header verwendet bereits Anchors: Nach einem Klick auf „Über dieses Projekt“ endet die Adresse auf `#ueber`.

3.5 Commit

1. Übertrage alle Änderungen mit `git add .` in die Staging Area.
2. Prüfe mit `git status`, dass alle geänderten Dateien unter **Changes to be committed** stehen.
3. Erstelle einen Commit mit der Nachricht `feat: Inhalte mit Überschriften, Listen und Links gliedern` und übertrage ihn mit `git push`.

4. Verstehen

Lege im Lernprotokoll die Überschrift **## Text & Semantik** (Datum) an. Der Ablauf je Experiment ist: Vorhersage notieren, ausführen, Ergebnis und Erklärung notieren.

Nimm jede Änderung nach dem Experiment zurück: `Strg/Cmd + Z` oder `git restore index.html`.

Die Experimente lesen Rollen im Accessibility Tree ab. Wähle dazu das Element mit dem **Inspect mode** der DevTools aus. Öffne danach im Panel **Elements** den Tab **Accessibility** und lies unter **Computed properties** die **Role** (siehe Browser & DevTools, Accessibility).

Break-it 1: Überschrift durch fetten Absatz ersetzen

Wähle `<h3>Werkzeuge</h3>` aus und lies die Role.

Ersetze danach `<h3>Werkzeuge</h3>` durch `<p><strong>Werkzeuge</strong></p>`.

- **Vorhersage:** Welche Rolle hat das Element nach der Änderung? Was ändert sich für einen Screenreader?
- **Prüfen:**
 1. Lade die Seite neu.
 2. Wähle den Text „Werkzeuge“ aus. Der Inspect mode wählt das innere Element `<strong>`.
 3. Klicke im Panel **Elements** auf die Zeile `<p>` direkt darüber und lies die Role.

→ Auflösung 1 am Ende des Kapitels

Predict 1: Eine Ebene überspringen

Ersetze die beiden Elemente `<h3>` im Abschnitt „Über dieses Projekt“ durch `<h5>`, jeweils im öffnenden und im schließenden Tag.

- **Vorhersage:** Wer meldet die übersprungenen Ebenen: der Browser, der Validator oder keiner von beiden?
- **Prüfen:**
 1. Lade die Seite neu und öffne das Panel **Console**.
 2. Wähle die Überschrift „Werkzeuge“ aus und lies unter **Computed properties** die Werte **Role** und **Level**.
 3. Wähle im W3C-Validator unter **Check by** die Option **file upload** und danach die Datei `index.html`.
 4. Klicke auf **Check**.

→ Auflösung 2 am Ende des Kapitels

Break-it 2: Liste durch Zeilenumbrüche ersetzen

Wähle den Link „Visual Studio Code“ aus. Klicke im Panel **Elements** auf die Zeile `<li>` darüber und danach auf die Zeile `<ul>`. Lies jeweils die Role.

Ersetze danach die Liste unter „Werkzeuge“ durch einen Absatz mit Zeilenumbrüchen:

```
<p>
  <a href="https://code.visualstudio.com/">Visual Studio Code</a><br>
  <a href="https://developer.chrome.com/docs/devtools">Chrome DevTools</a><br>
  <a href="https://git-scm.com/">Git</a>
</p>
```

- **Vorhersage:** Die drei Links stehen weiterhin untereinander. Welche Rolle hat das Element, das die Links enthält? Meldet der Validator einen Fehler?
- **Prüfen:**
 1. Lade die Seite neu.
 2. Wähle den Link „Visual Studio Code“ aus. Klicke im Panel **Elements** auf die Zeile `<p>` darüber und lies die Role.
 3. Wähle im W3C-Validator unter **Check by** die Option **file upload** und danach die Datei `index.html`.

4. Klicke auf **Check**.

→ Auflösung 3 am Ende des Kapitels

Predict 2: Drei Links mit dem Text „hier“

Ändere die drei Listeneinträge unter „Werkzeuge“ nach diesem Muster:

```
<li>Visual Studio Code: <a href="https://code.visualstudio.com/">hier</a></li>
```

- **Vorhersage:** Welchen Namen haben die drei Links im Accessibility Tree?
- **Prüfen:**
 1. Lade die Seite neu.
 2. Wähle nacheinander die drei Links aus und lies unter **Computed properties** den **Name**.

→ Auflösung 4 am Ende des Kapitels

Break-it 3: URL ohne Schema

Ändere im Link auf Git den Wert von href von `https://git-scm.com/` in `git-scm.com`.

- **Vorhersage:** Welche Adresse öffnet der Browser nach einem Klick auf den Link?
- **Prüfen:**
 1. Lade die Seite neu.
 2. Klicke auf den Link „Git“.
 3. Lies die Adresszeile.

→ Auflösung 5 am Ende des Kapitels

5. Toolbox-Eintrag

Schreibe in `<section id="eintraege">` nach dem Eintrag „Landmarks“ drei weitere Einträge: „Überschriften-Hierarchie“, „Listen“ und „Links“. Der Aufbau entspricht den bisherigen Einträgen:

1. **Beschreibung und Zweck** in zwei bis fünf Sätzen, in eigenen Worten.
2. **Barrierefreiheit:** die Auswirkung auf Screenreader und Tastaturbedienung.
3. **Quelle:** einen Link auf die zugehörige MDN-Seite.

Die drei Einträge nennen unter Barrierefreiheit nur den Screenreader: „Überschriften-Hierarchie“ die Auflösung von Break-it 1, „Listen“ die von Break-it 2, „Links“ die von Predict 2. Quelle für die Überschriften-Hierarchie ist MDN – Heading elements, für Listen MDN – Lists, für Links MDN – Creating links.

Verwende in den Einträgen die Elemente dieses Kapitels: Elementnamen stehen in `<code>`, der Eintrag „Listen“ enthält mindestens eine Liste:

```

<article>
  <h3>Listen</h3>
  <p>HTML kennt drei Arten von Listen:</p>
  <ul>
    <li><code>&lt;ul>&gt;</code> für Einträge ohne feste Reihenfolge</li>
    <li><code>&lt;ol>&gt;</code> für Einträge mit fester Reihenfolge</li>
    <li><code>&lt;dl>&gt;</code> für Paare aus Begriff und Beschreibung</li>
  </ul>
  <p>Barrierefreiheit: ...</p>
  <p>Quelle: <a href="https://developer.mozilla.org/...">MDN - ...</a></p>
</article>

```

Die Referenz-Toolbox zeigt die vollständige Toolbox, wie sie am Ende aussehen kann.

Lade die Seite neu. Unter „Einträge“ stehen fünf Einträge, zuletzt „Überschriften-Hierarchie“, „Listen“ und „Links“.

1. Setze in README.md in der Tabelle ## Fortschritt „Text & Semantik“ auf „erledigt“ und ergänze die Zeile „Navigation & mehrere Seiten“ mit „offen“.
2. Übertrage alle Änderungen mit `git add .` in die Staging Area.
3. Prüfe mit `git status`, dass alle geänderten Dateien unter **Changes to be committed** stehen, auch learning-notizen.md.
4. Erstelle einen Commit mit der Nachricht feat: Einträge Überschriften, Listen und Links ergänzen und übertrage ihn mit `git push`.

6. Prüfen

- ☐ Der W3C-Validator zeigt für index.html die Meldung „Document checking completed. No errors or warnings to show.“
- ☐ Die Seite enthält genau ein <h1> und überspringt keine Ebene.
- ☐ Unter „Aufbau eines Eintrags“ steht eine nummerierte Liste, unter „Werkzeuge“ eine Liste mit Aufzählungszeichen. Der Eintrag „Landmarks“ enthält die Paare aus Element und Rolle.
- ☐ Jeder Linktext nennt das Ziel des Links.
- ☐ learning-notizen.md enthält zu jedem Experiment Vorhersage, Ergebnis und Erklärung.
- ☐ Die Einträge „Überschriften-Hierarchie“, „Listen“ und „Links“ stehen unter „Einträge“.
- ☐ Die Commits feat: Inhalte mit Überschriften, Listen und Links gliedern und feat: Einträge Überschriften, Listen und Links ergänzen sind auf GitHub.

Quellen

- MDN – Headings and paragraphs
- MDN – Emphasis and importance
- MDN – Lists
- MDN – Creating links
- MDN – Heading elements
- W3C WAI – Headings
- W3C WAI – Understanding Link Purpose (In Context)
- W3C Nu HTML Checker

Auflösungen

Auflösung 1

Zu Break-it 1: Überschrift durch fetten Absatz ersetzen

Vor der Änderung hat das Element die Rolle `heading` mit der Ebene 3. Nach der Änderung hat es die Rolle `paragraph` und enthält ein Element mit der Rolle `strong`. Die Seite hat eine Überschrift weniger.

Ein Screenreader führt den Abschnitt „Werkzeuge“ nicht mehr in der Liste der Überschriften. Der Abschnitt ist nicht mehr direkt ansteuerbar. Fette Schrift erzeugt für Sehende den Eindruck einer Überschrift, für den Accessibility Tree aber keine.

Auflösung 2

Zu Predict 1: Eine Ebene überspringen

Der Browser stellt die Seite ohne Meldung dar. Die Überschriften erscheinen kleiner, weil das Start-Stylesheet `<h5>` nicht gestaltet. Der Accessibility Tree führt beide Überschriften mit der Ebene 5.

Der Validator meldet einen Fehler: `The heading "h5" (with computed level 5) follows the heading "h2" (with computed level 2), skipping 2 heading levels.`

Der Browser korrigiert oder meldet Fehler in der Gliederung nicht. Die Prüfung mit dem Validator gehört deshalb zu jedem Kapitel.

Auflösung 3

Zu Break-it 2: Liste durch Zeilenumbrüche ersetzen

Die Rollen `list` und `listitem` fehlen. Die drei Links liegen in einem Element mit der Rolle `paragraph`.

Ein Screenreader sagt eine Liste mit der Anzahl ihrer Einträge an, zum Beispiel „Liste mit 3 Einträgen“, und erlaubt den Sprung zum Ende der Liste. Bei einem Absatz mit Zeilenumbrüchen entfällt beides. Der Validator meldet keinen Fehler: Das HTML ist gültig, aber die Bedeutung fehlt.

Auflösung 4

Zu Predict 2: Drei Links mit dem Text „hier“

Alle drei Links haben den Namen „hier“. Der Name eines Links ist sein Linktext. Der Text vor dem Link gehört nicht dazu.

Screenreader bieten eine Liste aller Links einer Seite an. Diese Liste enthält dreimal „hier“. Das Ziel der Links ist darin nicht erkennbar.

Auflösung 5

Zu Break-it 3: URL ohne Schema

Der Browser zeigt eine Fehlerseite (`ERR_FILE_NOT_FOUND`). Die Adresse lautet `file:///.../toolbox/git-scm.com`.

Ohne Schema ist der Wert ein relativer Pfad. Der Browser sucht eine Datei mit dem Namen `git-scm.com` im Ordner der Seite. Erst das Schema `https://` kennzeichnet den Wert als Adresse einer anderen Website.



1.3 Navigation & mehrere Seiten

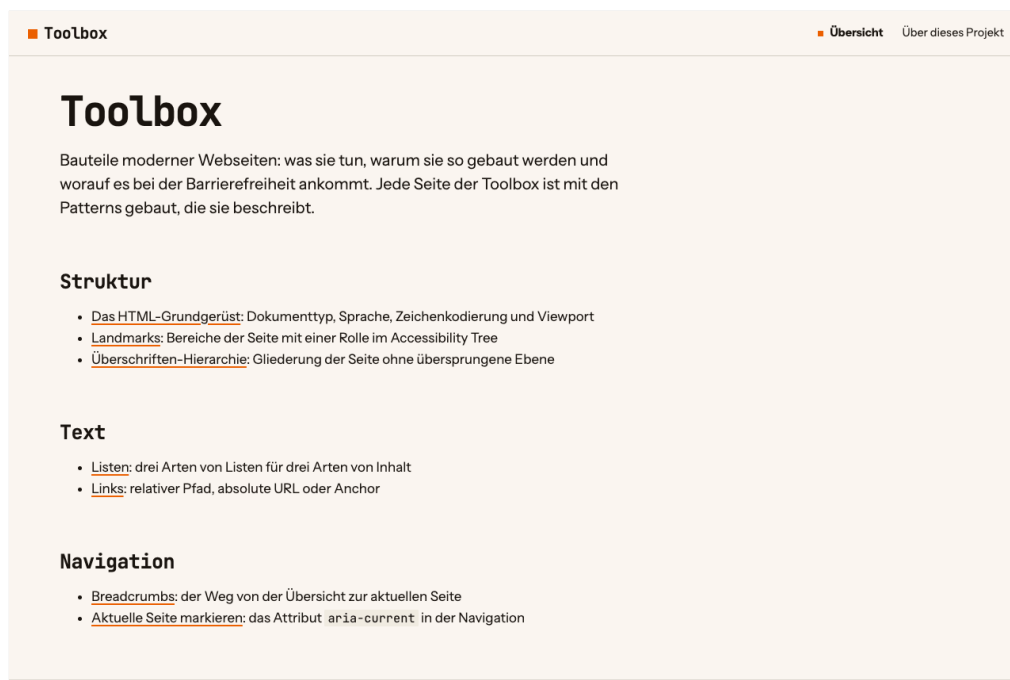
Voraussetzungen: Text & Semantik (Kapitel 1.2)

Lernziele

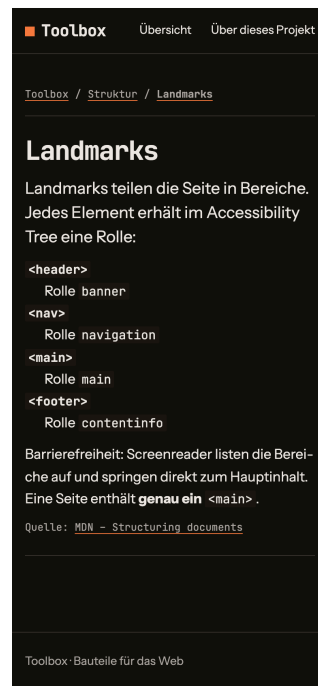
- Eine Website in mehrere Seiten und Ordner aufteilen und Stylesheet und Navigation von jeder Seite aus über relative Pfade erreichen.
- Die aktuelle Seite in einer Navigation mit `aria-current="page"` markieren und die Markierung in den DevTools nachweisen.
- Mehrere Navigationen einer Seite mit `<nav>` und `aria-label` benennen und im Accessibility Tree unterscheiden.

1. Vorschau

Zielstand nach diesem Kapitel: Die Startseite der Toolbox ist eine Übersicht. Jeder Eintrag hat eine eigene Seite.



Die Seite des Eintrags „Landmarks“ bei Smartphone-Breite im Dark Mode:



Die Texte der Einträge im Bild sind Beispiele.

2. Warum

Eine Seite im Unterordner, deren Pfade nicht angepasst sind, erscheint ohne Gestaltung. Eine Navigation nennt die Ziele der Website und die Lage der aktuellen Seite. Beide Angaben stehen im HTML, damit Screenreader sie auswerten. Das Stylesheet leitet die sichtbare Markierung aus diesen Angaben ab.

Weiterführend: [MDN – The Navigation Section element](#) · [MDN – ARIA: aria-current attribute](#) · [W3C WAI – Breadcrumb Pattern](#)

3. Bauen

Ausgangsstand ist die Toolbox aus dem Kapitel [Text & Semantik](#) (Kapitel 1.2): die Datei `index.html` mit fünf Einträgen und dem Abschnitt „Über dieses Projekt“.

3.1 Seitenstruktur

Das Projekt hat am Ende von [Bauen](#) diesen Aufbau:

```

toolbox/
├── .gitignore
├── README.md
├── learning-notizen.md
├── index.html           ← Übersicht mit Links auf alle Einträge
├── ueber.html          ← Über dieses Projekt
├── patterns/
│   ├── html-grundgeruest.html ← je Eintrag eine Seite
│   └── landmarks.html

```



Dateinamen von Seiten sind Teil der URL. Sie enthalten nur Kleinbuchstaben, Ziffern und Bindestriche.

1. Lade die Datei `base.css` aus dem Material herunter und ersetze damit `assets/css/base.css` (siehe Dateisystem & Pfade).
2. Lege im Projektordner den Ordner `patterns` an.

Die Version 2.1.0 des Start-Stylesheets gestaltet zusätzlich die Markierung der aktuellen Seite, die Breadcrumbs und das Element `<dl>`. Zeile 3 der Datei nennt Version 2.1.0.

3.2 Die Seite „Über dieses Projekt“

Die erste neue Seite liegt im selben Ordner wie `index.html`.

1. Kopiere `index.html` und nenne die Kopie `ueber.html`.
2. Lösche in `index.html` das Element `<section id="ueber">`.
3. Ersetze in `ueber.html` den Inhalt von `<main>` durch den Inhalt von `<section id="ueber">` aus derselben Datei, ohne die Tags `<section>` und `</section>`.
4. Ändere in `ueber.html` die Ebenen der Überschriften.

Der Abschnitt ist jetzt der Inhalt einer eigenen Seite. Seine Überschrift ist das `<h1>` der Seite. Die beiden Unterüberschriften sind `<h2>`:

```

<main>
  <h1>Über dieses Projekt</h1>
  <p>Die Toolbox dokumentiert Bauteile moderner Webseiten und wächst mit
  jedem Kapitel. Das Aussehen legt das <a href="assets/css/base.css">Start-
  Stylesheet</a> fest.</p>

  <h2>Aufbau eines Eintrags</h2>
  <ol>
    <li>Beschreibung und Zweck</li>
    <li>Barrierefreiheit</li>
    <li>Quelle</li>
  </ol>

  <h2>Werkzeuge</h2>
  <ul>
    <li><a href="https://code.visualstudio.com/">Visual Studio Code</a></li>
    <li><a href="https://developer.chrome.com/docs/devtools">Chrome
  DevTools</a></li>
    <li><a href="https://git-scm.com/">Git</a></li>
  </ul>
</main>

```

Jede Seite hat einen eigenen `<title>` und eine eigene `description`. Der `<title>` nennt zuerst die Seite und danach die Website. Tabs, Lesezeichen und Suchergebnisse zeigen ihn an. Ändere beide Angaben in `ueber.html`:

```
<title>Über dieses Projekt - Toolbox</title>
<meta name="description" content="Zweck, Aufbau und Werkzeuge der Toolbox.">
```

Die Navigation im Header, die Main Navigation, verweist bisher auf zwei Anchors derselben Seite. Beide Ziele sind jetzt Dateien. Ersetze in `index.html` und in `ueber.html` die beiden Listeneinträge der Navigation:

```
<li><a href="index.html">Übersicht</a></li>
<li><a href="ueber.html">Über dieses Projekt</a></li>
```

Öffne `index.html` im Browser und wechsle über die Navigation zwischen beiden Seiten. Der Tab zeigt abwechselnd „Toolbox“ und „Über dieses Projekt – Toolbox“.

3.3 Aktuelle Seite markieren

Das Attribut `aria-current="page"` kennzeichnet den Link, der auf die geöffnete Seite verweist. In jeder Datei trägt deshalb ein anderer Link das Attribut.

Ergänze das Attribut in `ueber.html`:

```
<ul>
  <li><a href="index.html">Übersicht</a></li>
  <li><a href="ueber.html" aria-current="page">Über dieses Projekt</a></li>
</ul>
```

Ergänze das Attribut in `index.html`:

```
<li><a href="index.html" aria-current="page">Übersicht</a></li>
```

Das Start-Stylesheet stellt den markierten Link fett und mit einem Quadrat davor dar. Den Aufbau der Regeln erklärt das Kapitel CSS-Grundlagen (Kapitel 1.6). Beide Regeln stehen bereits in `base.css`. Diese erzeugt die fette Schrift:

```
nav a[aria-current="page"] {
  font-weight: 700;
}
```

Weise die Markierung in den DevTools nach:

1. Wähle in `ueber.html` den Link „Über dieses Projekt“ mit dem **Inspect mode** aus.
2. Öffne im Panel **Elements** den Tab **Accessibility**.
3. Lies den Abschnitt **ARIA attributes**. Er enthält den Eintrag `aria-current: page`.

3.4 Eine Seite im Unterordner

Die Seite eines Eintrags liegt im Ordner `patterns`. Alle Pfade zu Dateien außerhalb dieses Ordners beginnen deshalb mit `../` (siehe Dateisystem & Pfade).

1. Kopiere ueber.html in den Ordner patterns und nenne die Kopie landmarks.html.
2. Öffne patterns/landmarks.html im Browser. Die Seite erscheint ohne Gestaltung.
3. Ergänze ../ in den vier Pfaden der Tabelle. Die Pfade in <main> bleiben, weil der Eintrag „Landmarks“ den Inhalt von <main> ersetzt.
4. Entferne aria-current="page" aus der Main Navigation.

Stelle	Pfad in ueber.html	Pfad in patterns/landmarks.html
Stylesheet	assets/css/base.css	../assets/css/base.css
Wordmark	index.html	../index.html
Navigation, erster Link	index.html	../index.html
Navigation, zweiter Link	ueber.html	../ueber.html

```
<link rel="stylesheet" href="../assets/css/base.css">
```

```
<p><a href="../index.html">Toolbox</a></p>
```

```
<li><a href="../index.html">Übersicht</a></li>
<li><a href="../ueber.html">Über dieses Projekt</a></li>
```

Lade patterns/landmarks.html neu. Die Seite erscheint mit Gestaltung.

Die Seite „Landmarks“ ist kein Ziel der Main Navigation. Kein Link der Main Navigation trägt deshalb auf dieser Seite aria-current.

Verschiebe jetzt den Eintrag auf die Seite:

1. Schneide in index.html das Element <article> des Eintrags „Landmarks“ aus.
2. Ersetze in patterns/landmarks.html den Inhalt von <main> durch dieses <article>.
3. Ändere die Überschrift des Eintrags von <h3> in <h1>.
4. Ändere <title> und description.

```
<title>Landmarks - Toolbox</title>
<meta name="description" content="Landmarks teilen eine Seite in Bereiche mit
einer Rolle im Accessibility Tree.">
```

```
<article>
  <h1>Landmarks</h1>
  <p>Landmarks teilen die Seite in Bereiche. Jedes Element erhält im
  Accessibility Tree eine Rolle:</p>
  ""
</article>
```

Der Eintrag ist der einzige Inhalt der Seite. Seine Überschrift ist deshalb das <h1>.

Lade die Seite neu. Der Tab zeigt „Landmarks – Toolbox“, unter dem Header steht die Überschrift „Landmarks“.

3.5 Breadcrumbs

Breadcrumbs nennen die Lage einer Seite innerhalb der Website: jede übergeordnete Seite von der Übersicht bis zur Seite selbst. Die Reihenfolge der Links ist fest. Die Liste ist deshalb ein `<ol>`.

Füge in `patterns/landmarks.html` am Anfang von `<main>` eine zweite Navigation ein:

```
<nav aria-label="Breadcrumbs">
  <ol>
    <li><a href=".. /index.html">Toolbox</a></li>
    <li><a href=".. /index.html#struktur">Struktur</a></li>
    <li><a href="landmarks.html" aria-current="page">Landmarks</a></li>
  </ol>
</nav>
```

- Der zweite Link verbindet einen Pfad mit einem Anchor. Das Ziel `id="struktur"` entsteht in Übrige Seiten und Übersicht.
- Der dritte Link verweist auf die geöffnete Seite und trägt `aria-current="page"`.
- Die Schrägstriche zwischen den Links erzeugt das Stylesheet. Sie stehen nicht im HTML.

Die Seite enthält jetzt zwei Elemente `<nav>`. Das Attribut `aria-label` gibt jeder Navigation einen Namen im Accessibility Tree. Ergänze den Namen der Main Navigation in `index.html`, `ueber.html` und `patterns/landmarks.html`:

```
<nav aria-label="Main">
```

Der Name enthält das Wort „Navigation“ nicht. Screenreader sagen die Rolle `navigation` zusätzlich zum Namen an.

Lade die Seite neu. Über der Überschrift stehen die Breadcrumbs „Toolbox / Struktur / Landmarks“, „Landmarks“ ist fett.

3.6 Übrige Seiten und Übersicht

Die vier weiteren Einträge erhalten je eine Seite nach dem Muster von `landmarks.html`:

Datei in patterns	Eintrag	Kategorie	Anchor
<code>html-grundgeruest.html</code>	Das HTML-Grundgerüst	Struktur	<code>#struktur</code>
<code>ueberschriften.html</code>	Überschriften-Hierarchie	Struktur	<code>#struktur</code>
<code>listen.html</code>	Listen	Text	<code>#text</code>
<code>links.html</code>	Links	Text	<code>#text</code>

Führe für jede Zeile der Tabelle diese Schritte aus:

1. Kopiere `landmarks.html` und nenne die Kopie wie in der Tabelle.
2. Ändere `<title>` und `description`. Der `<title>` nennt den Eintrag aus der Tabelle, zum Beispiel Listen – Toolbox. Die `description` beschreibt den Eintrag in einem Satz in eigenen Worten.
3. Ändere `href` und Linktext des zweiten und des dritten Links der Breadcrumbs nach der Tabelle.
4. Schneide in `index.html` das `<article>` des Eintrags aus und ersetze damit das `<article>` der Kopie.

5. Ändere die Überschrift des Eintrags von `<h3>` in `<h1>`.

Die Breadcrumbs von `listen.html`:

```
<nav aria-label="Breadcrumbs">
  <ol>
    <li><a href="../index.html">Toolbox</a></li>
    <li><a href="../index.html#text">Text</a></li>
    <li><a href="listen.html" aria-current="page">Listen</a></li>
  </ol>
</nav>
```

Die Datei `index.html` enthält danach keinen Eintrag mehr. Ersetze `<section id="eintraege">` durch eine Übersicht mit einem Abschnitt je Kategorie:

```
<section id="struktur">
  <h2>Struktur</h2>
  <ul>
    <li><a href="patterns/html-grundgeruest.html">Das HTML-Grundgerüst</a>:
    Dokumenttyp, Sprache, Zeichenkodierung und Viewport</li>
    <li><a href="patterns/landmarks.html">Landmarks</a>: Bereiche der Seite
    mit einer Rolle im Accessibility Tree</li>
    <li><a href="patterns/ueberschriften.html">Überschriften-Hierarchie</
a>: Gliederung der Seite ohne übersprungene Ebene</li>
  </ul>
</section>

<section id="text">
  <h2>Text</h2>
  <ul>
    <li><a href="patterns/listen.html">Listen</a>: drei Arten von Listen
    für drei Arten von Inhalt</li>
    <li><a href="patterns/links.html">Links</a>: relativer Pfad, absolute
    URL oder Anchor</li>
  </ul>
</section>
```

Lade `index.html` neu. Unter „Struktur“ stehen drei Links, unter „Text“ zwei Links. Die Überschrift „Einträge“ fehlt.

Die `id` jedes Abschnitts ist das Ziel des zweiten Links der Breadcrumbs. Prüfe in `index.html`, dass jeder Anchor aus der Tabelle als `id` eines Abschnitts vorkommt. Ein Link auf einen fehlenden Anchor öffnet die Übersicht ohne Sprung und ohne Meldung. Prüfe danach auf jeder der sieben Seiten jeden Link der Main Navigation und der Breadcrumbs.

3.7 Commit

1. Übertrage alle Änderungen mit `git add .` in die Staging Area.
2. Prüfe mit `git status`, dass alle geänderten Dateien unter **Changes to be committed** stehen.
3. Erstelle einen Commit mit der Nachricht `feat: Toolbox in mehrere Seiten aufteilen` und übertrage ihn mit `git push`.

4. Verstehen

Lege im Lernprotokoll die Überschrift ## Navigation & mehrere Seiten (Datum) an. Der Ablauf je Experiment ist: Vorhersage notieren, ausführen, Ergebnis und Erklärung notieren.

Nimm jede Änderung nach dem Experiment zurück: Strg/Cmd + Z oder git restore mit dem Pfad der Datei.

Break-it 1: Pfad mit führendem Schrägstrich

Ändere in index.html und in patterns/landmarks.html den Pfad zum Stylesheet in href="/assets/css/base.css".

- **Vorhersage:** Der Pfad ist in beiden Dateien gleich und enthält kein .. /. Welche der beiden Seiten findet das Stylesheet?
- **Prüfen:**
 1. Öffne index.html und in den DevTools das Panel **Network**.
 2. Lade die Seite neu und lies die Adresse der fehlgeschlagenen Anfrage.
 3. Wiederhole Schritt 1 und 2 mit patterns/landmarks.html.

→ [Auflösung 1](#) am Ende des Kapitels

Break-it 2: aria-current durch eine Klasse ersetzen

Ersetze in der Main Navigation von ueber.html das Attribut aria-current="page" durch class="active". Das Attribut class gibt einem Element einen selbst gewählten Namen, die Klasse.

- **Vorhersage:** Was ändert sich in der Darstellung? Was ändert sich im Tab **Accessibility**?
- **Prüfen:**
 1. Lade die Seite neu.
 2. Wähle den Link „Über dieses Projekt“ aus.
 3. Lies im Tab **Accessibility** den Abschnitt **ARIA attributes**.

→ [Auflösung 2](#) am Ende des Kapitels

Predict 1: Zwei markierte Links

Ergänze in der Main Navigation von ueber.html das Attribut aria-current="page" zusätzlich im Link „Übersicht“.

- **Vorhersage:** Zwei Links tragen die Markierung der aktuellen Seite. Meldet der Validator einen Fehler? Meldet er einen Fehler, wenn ein Wert yes lautet?
- **Prüfen:**
 1. Lade die Seite neu.
 2. Wähle im [W3C-Validator](#) unter **Check by** die Option **file upload** und danach die Datei ueber.html. Klicke auf **Check**.
 3. Ändere einen der beiden Werte in aria-current="yes" und prüfe die Datei erneut.

→ [Auflösung 3](#) am Ende des Kapitels

Break-it 3: Navigationen ohne Namen

Wähle in `patterns/landmarks.html` einen Link der Main Navigation aus. Klicke im Panel **Elements** auf die Zeile `<nav aria-label="Main">` oberhalb des Links und lies im Tab **Accessibility** unter **Computed properties Role** und **Name**. Wiederhole das mit einem Link der Breadcrumbs.

Lösche danach in `patterns/landmarks.html` das Attribut `aria-label` in beiden Elementen `<nav>`.

- **Vorhersage:** Woran unterscheidet der Accessibility Tree die beiden Navigationen nach der Änderung? Meldet der Validator einen Fehler?
- **Prüfen:**
 1. Lade die Seite neu.
 2. Wähle beide Elemente `<nav>` auf demselben Weg aus und lies **Role** und **Name**.
 3. Wähle im W3C-Validator unter **Check by** die Option **file upload** und danach die Datei `landmarks.html`. Klicke auf **Check**.

→ [Auflösung 4](#) am Ende des Kapitels

Predict 2: Einen Punkt in der Main Navigation ergänzen

Die Main Navigation soll einen dritten Link „Kontakt“ erhalten. Das Ziel ist eine neue Seite `kontakt.html` im Projektordner. Führe die Änderung nicht aus.

- **Vorhersage:** Wie viele bestehende Dateien sind zu ändern? Welchen Wert hat `href` in diesen Dateien?
- **Prüfen:**
 1. Öffne in VS Code die View **Search**.
 2. Suche nach `aria-label="Main"`.
 3. Zähle die HTML-Dateien mit einem Treffer.

→ [Auflösung 5](#) am Ende des Kapitels

5. Toolbox-Eintrag

Schreibe zwei weitere Einträge: „Breadcrumbs“ und „Aktuelle Seite markieren“. Jeder Eintrag erhält eine eigene Seite:

Datei in patterns	Eintrag	Kategorie	Anchor
<code>breadcrumbs.html</code>	Breadcrumbs	Navigation	<code>#navigation</code>
<code>aktuelle-seite.html</code>	Aktuelle Seite markieren	Navigation	<code>#navigation</code>

Lege beide Seiten mit den Schritten 1 bis 3 aus [Übrige Seiten und Übersicht](#) an. Ersetze danach in beiden Seiten das `<article>` durch einen neuen Eintrag mit der Überschrift `<h1>`. Der Aufbau entspricht den bisherigen Einträgen:

1. **Beschreibung und Zweck** in zwei bis fünf Sätzen, in eigenen Worten.
2. **Barrierefreiheit:** die Auswirkung auf Screenreader und Tastaturbedienung.
3. **Quelle:** einen Link auf die zugehörige MDN-Seite.

Die Attribute `aria-current` und `aria-label` ändern die Bedienung mit der Tastatur nicht. Die beiden Einträge nennen deshalb unter Barrierefreiheit nur den Screenreader: „Breadcrumbs“ die Auflösung von Break-it 3, „Aktuelle Seite markieren“ die von Break-it 2. Quelle für Breadcrumbs

ist MDN – The Navigation Section element, für Aktuelle Seite markieren MDN – ARIA: aria-current attribute.

```
<article>
  <h1>Aktuelle Seite markieren</h1>
  <p>Das Attribut <code>aria-current="page"</code> kennzeichnet in einer
  Navigation den Link, der auf die geöffnete Seite verweist. ...</p>
  <p>Barrierefreiheit: ...</p>
  <p>Quelle: <a href="https://developer.mozilla.org/...">MDN - ...</a></p>
</article>
```

Ergänze in `index.html` nach dem `</section>` des Abschnitts „Text“, direkt vor `</main>`, den Abschnitt der neuen Kategorie:

```
<section id="navigation">
  <h2>Navigation</h2>
  <ul>
    <li><a href="patterns/breadcrumbs.html">Breadcrumbs</a>: der Weg von
    der Übersicht zur aktuellen Seite</li>
    <li><a href="patterns/aktuelle-seite.html">Aktuelle Seite markieren</a>:
    das Attribut <code>aria-current</code> in der Navigation</li>
  </ul>
</section>
```

Lade `index.html` neu. Unter „Navigation“ stehen die Links „Breadcrumbs“ und „Aktuelle Seite markieren“.

Die Referenz-Toolbox zeigt die vollständige Toolbox, wie sie am Ende aussehen kann.

Ersetze in `README.md` den Abschnitt `## Struktur` durch diesen Inhalt:

```
## Struktur

...
toolbox/
├── README.md           ← diese Datei
├── learning-notizen.md ← Lernprotokoll: Vorhersagen und Beobachtungen
├── .gitignore          ← Systemdateien, die nicht ins Repository gehören
├── index.html          ← Übersicht mit Links auf alle Einträge
├── ueber.html          ← Über dieses Projekt
├── patterns/           ← je Eintrag eine Seite
│   ├── html-grundgeruest.html
│   ├── landmarks.html
│   ├── ueberschriften.html
│   ├── listen.html
│   ├── links.html
│   ├── breadcrumbs.html
│   └── aktuelle-seite.html
├── assets/
│   ├── css/
│   │   └── base.css    ← Start-Stylesheet 2.1.0 (vorgegeben)
│   └── fonts/          ← Schriften + Lizenzen (vorgegeben)
...

```

Header, Navigation und Footer stehen in jeder HTML-Datei. Eine Änderung an der Navigation betrifft alle neun Seiten.

1. Setze in README.md in der Tabelle ## Fortschritt „Navigation & mehrere Seiten“ auf „erledigt“ und ergänze die Zeile „Bilder“ mit „offen“.
2. Übertrage alle Änderungen mit `git add .` in die Staging Area.
3. Prüfe mit `git status`, dass alle geänderten Dateien unter **Changes to be committed** stehen, auch `learning-notizen.md`.
4. Erstelle einen Commit mit der Nachricht `feat: Einträge Breadcrumbs und Aktuelle Seite ergänzen` und übertrage ihn mit `git push`.

6. Prüfen

- ☐ Der W3C-Validator zeigt für jede der neun Seiten die Meldung „Document checking completed. No errors or warnings to show.“
- ☐ Jede Seite hat einen eigenen `<title>` und genau ein `<h1>`.
- ☐ Jeder Link der Main Navigation und der Breadcrumbs führt von jeder Seite aus zum erwarteten Ziel. Jeder Anchor der Breadcrumbs steht in `index.html` als `id` eines Abschnitts.
- ☐ Je Navigation trägt höchstens ein Link `aria-current="page"`. Dieser Link verweist auf die geöffnete Seite.
- ☐ Der Accessibility Tree zeigt auf jeder Seite eines Eintrags die Navigationen „Main“ und „Breadcrumbs“.
- ☐ Kein Pfad beginnt mit `/`.
- ☐ `learning-notizen.md` enthält zu jedem Experiment Vorhersage, Ergebnis und Erklärung.
- ☐ Die Einträge „Breadcrumbs“ und „Aktuelle Seite markieren“ stehen unter „Navigation“.
- ☐ Die Commits `feat: Toolbox in mehrere Seiten aufteilen` und `feat: Einträge Breadcrumbs und Aktuelle Seite ergänzen` sind auf GitHub.

Quellen

- MDN – The Navigation Section element
- MDN – ARIA: navigation role
- MDN – ARIA: aria-current attribute
- MDN – ARIA: aria-label attribute
- W3C WAI – ARIA Authoring Practices: Breadcrumb Pattern
- W3C WAI – Menu Structure
- W3C Nu HTML Checker

Auflösungen

Auflösung 1

Zu Break-it 1: Pfad mit führendem Schrägstrich

Keine der beiden Seiten findet das Stylesheet. Der Browser fordert diese Adresse an:

Windows	macOS
file:///C:/assets/css/base.css	file:///assets/css/base.css

Ein Pfad mit führendem / beginnt nicht im Projektordner, sondern an der Wurzel. Lokal ist das die Wurzel des Dateisystems, auf einem Webserver die Wurzel der Domain.

Der Pfad gilt deshalb nur, wenn das Projekt an der Wurzel einer Domain liegt. Liegt die Toolbox unter `https://example.org/toolbox/`, fordert der Browser `https://example.org/assets/css/base.css` an. Relative Pfade gelten unabhängig vom Ort des Projektordners.

Auflösung 2

Zu Break-it 2: aria-current durch eine Klasse ersetzen

Der Link ist nicht mehr fett, und das Quadrat fehlt. Das Stylesheet wählt das Attribut `aria-current` aus und enthält keine Regel für die Klasse `active`. Der Abschnitt **ARIA attributes** enthält keinen Eintrag `aria-current` mehr.

Eine Klasse mit einer passenden CSS-Regel stellt nur die sichtbare Markierung wieder her. Ein Screenreader erhält weiterhin keine Angabe zur aktuellen Seite.

Auflösung 3

Zu Predict 1: Zwei markierte Links

Der Browser stellt beide Links markiert dar. Der Validator meldet keinen Fehler.

Den Wert `yes` meldet der Validator: `Bad value "yes" for attribute "aria-current" on element "a"`.

Der Validator prüft, ob ein Wert erlaubt ist. Er prüft nicht, ob die Aussage stimmt. Eine Markierung am falschen Link entsteht beim Kopieren einer Seite, wenn das Attribut in der Kopie stehen bleibt. Prüfen enthält deshalb eine Prüfung der Markierung auf jeder Seite.

Auflösung 4

Zu Break-it 3: Navigationen ohne Namen

Beide Elemente haben die Rolle `navigation` und keinen Namen. Vor der Änderung heißen sie „Main“ und „Breadcrumbs“. Der Validator meldet keinen Fehler.

Ein Screenreader führt beide Navigationen in der Liste der Landmarks. Ohne Namen enthält die Liste zwei gleiche Einträge. Der Inhalt einer Navigation ist erst nach dem Sprung dorthin erkennbar.

Auflösung 5

Zu Predict 2: Einen Punkt in der Main Navigation ergänzen

Sieben Dateien sind zu ändern: `index.html`, `ueber.html` und die fünf Seiten im Ordner `patterns`. Der Wert von `href` ist in den zwei Dateien des Projektordners `kontakt.html` und in den fünf Dateien des Ordners `patterns` `../kontakt.html`.

HTML enthält kein Element, das einen gemeinsamen Header in mehrere Seiten einfügt. Jede Seite enthält Header, Navigation und Footer vollständig. Eine Änderung an der Navigation betrifft deshalb jede Datei der Website.



1.4 Bilder

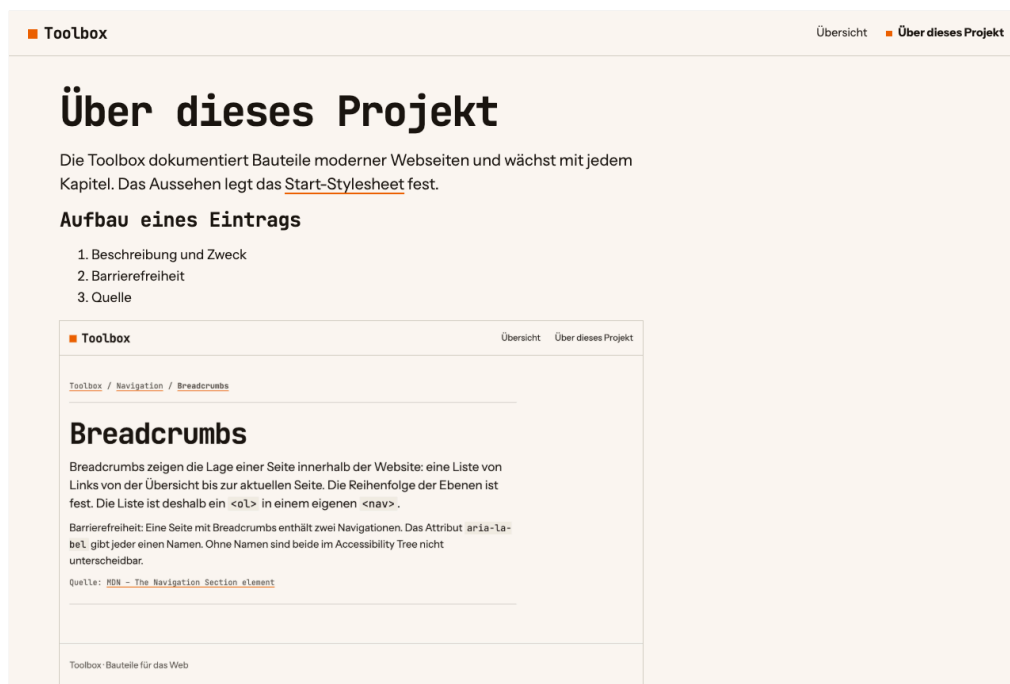
Voraussetzungen: Navigation & mehrere Seiten (Kapitel 1.3)

Lernziele

- Für ein Bild einen Alt-Text formulieren oder es mit `alt=""` als dekorativ kennzeichnen und die Wirkung im Accessibility Tree nachweisen.
- Mit `width` und `height` den Platz eines Bildes vor dem Laden reservieren und das abgeleitete Seitenverhältnis in den DevTools nachweisen.
- Mit `<picture>` und `<source media>` je nach Viewport eine andere Bilddatei ausliefern und die geladene Datei in den DevTools nachweisen.

1. Vorschau

Zielstand nach diesem Kapitel: Die Seite „Über dieses Projekt“ zeigt einen Eintrag der Toolbox als Bild mit Caption.



Dieselbe Seite bei Smartphone-Breite im Dark Mode. Der Browser lädt dort eine andere Bilddatei:



2. Warum

Ein Bild ohne Maße verschiebt beim Laden den Text, den jemand gerade liest. Ein Screenshot für den Desktop ist auf einem Smartphone nicht lesbar. Ein Screenreader liest statt des Bildes einen Text vor, den das HTML dazu enthält. Das HTML enthält deshalb drei Angaben zu einem Bild: den Alt-Text, die Maße und bei Bedarf weitere Dateien für andere Viewports.

Weiterführend: [MDN – HTML images](#) · [W3C WAI – Images Tutorial](#) · [MDN – Using responsive images in HTML](#)

3. Bauen

Ausgangsstand ist die Toolbox aus dem Kapitel [Navigation & mehrere Seiten](#) (Kapitel 1.3): neun Seiten ohne Bild.

3.1 Material

Das Projekt hat am Ende von [Bauen](#) diesen Aufbau:

```

toolbox/
├── index.html
├── ueber.html
├── patterns/
│   └── landmarks.html
│   ...
├── assets/
│   ├── css/
│   │   └── base.css
│   ├── fonts/
│   └── img/

```

← erhält ein Bild in zwei Varianten

← erhält ein Schema

← Version 2.2.0

← neu

```

├── landmarks.svg
├── eintrag-desktop.png
└── eintrag-mobil.png

```

1. Lade die Datei [base.css](#) aus dem Material herunter und ersetze damit `assets/css/base.css` (siehe Dateisystem & Pfade).
2. Lade [img.zip](#) herunter und entpacke die Datei in den Ordner `assets`.

Der Ordner `assets/img` enthält danach drei Dateien:

Datei	Inhalt	Größe in Pixeln
<code>landmarks.svg</code>	Schema der vier Landmarks einer Seite	640 × 400
<code>eintrag-desktop.png</code>	Screenshot des Eintrags „Breadcrumbs“ in einem breiten Viewport	1280 × 800
<code>eintrag-mobil.png</code>	Screenshot desselben Eintrags in einem schmalen Viewport	640 × 800

Die Dateien sind für die Toolbox erstellt und frei verwendbar.

Die Version 2.2.0 des Start-Stylesheets gestaltet zusätzlich die Elemente `<figure>` und `<figcaption>`. Ist ein Bild breiter als der verfügbare Platz, verkleinert sie es, ohne es zu verzerren. Zeile 3 der Datei nennt Version 2.2.0.

3.2 Ein Bild einbinden

Das Element `<img>` bindet eine Bilddatei ein. Es hat keinen Inhalt und keinen schließenden Tag. Das Element `<figure>` verbindet das Bild mit einer [Caption](#) im Element `<figcaption>`.

Füge in `patterns/landmarks.html` zwischen `</dl>` und dem Absatz „Barrierefreiheit: ...“ ein Bild mit Caption ein:

```

<figure>
  
  <figcaption>Die vier Landmarks einer Seite der Toolbox</figcaption>
</figure>

```

- `src` enthält den Pfad zur Bilddatei. Die Seite liegt im Ordner `patterns`. Der Pfad beginnt deshalb mit `../`.
- `alt` enthält den Alt-Text ([Alt-Text](#)).
- `width` und `height` nennen die Größe der Datei in Pixeln ([Platz reservieren](#)).

Öffne die Seite im Browser. Das Schema steht unter der Liste der Rollen, die Caption unter dem Schema.

3.3 Alt-Text

Der Alt-Text ersetzt das Bild: Ein Screenreader liest ihn vor, und der Browser zeigt ihn an, wenn die Datei nicht lädt. Der Alt-Text nennt deshalb die Aussage des Bildes, nicht sein Aussehen.

Art des Bildes	Alt-Text	Beispiel
informativ: Das Bild enthält eine Aussage, die nicht im Text steht.	die Aussage in einem Satz	Schema, Diagramm, Foto eines Produkts
dekorativ: Das Bild enthält keine Aussage.	leer: <code>alt=""</code>	Schmuckgrafik, Muster

Weitere Fälle enthält der [alt Decision Tree](#) des W3C.

Das Schema ist informativ: Es zeigt die Anordnung der vier Bereiche. Die Liste über dem Schema nennt diese Anordnung nicht. Der Alt-Text nennt sie.

- Der Alt-Text beginnt nicht mit „Bild von“. Ein Screenreader sagt die Rolle `image` zusätzlich zum Alt-Text an.
- Caption und Alt-Text haben verschiedene Aufgaben. Die Caption ist für alle sichtbar und ergänzt das Bild. Der Alt-Text ersetzt das Bild.

Weise den Alt-Text im [Accessibility Tree](#) nach:

1. Wähle das Schema mit dem **Inspect mode** aus.
2. Öffne im Panel **Elements** den Tab **Accessibility**.
3. Lies unter **Computed properties** die Werte **Name** und **Role**. Die Rolle ist `image`, der Name ist der Alt-Text.

3.4 Platz reservieren

Der Browser kennt die Größe eines Bildes erst, wenn die Datei geladen ist. Ohne weitere Angabe hat das Bild bis dahin die Höhe 0. Beim Laden verschiebt sich der folgende Inhalt um die Höhe des Bildes nach unten. Diese Verschiebung heißt [Layout Shift](#).

Die Attribute `width` und `height` nennen die Größe der Datei in Pixeln, ohne Einheit. Der Browser berechnet daraus das Seitenverhältnis und reserviert die Fläche vor dem Laden.

Mit dem Start-Stylesheet legen die Attribute nicht die dargestellte Größe fest. Es begrenzt die Breite eines Bildes auf den verfügbaren Platz. Die Eigenschaft `block-size` ist die Höhe. Mit dem Wert `auto` folgt sie aus dem Seitenverhältnis. Die Regel steht bereits in `base.css`:

```
img {
  block-size: auto;
}
```

Weise das Seitenverhältnis und die dargestellte Größe nach:

1. Wähle das Schema mit dem **Inspect mode** aus.
2. Öffne im Panel **Elements** den Tab **Computed** und gib `aspect-ratio` in das Feld **Filter** ein. Der Wert ist `auto 640 / 400`. Was `auto` bedeutet, zeigt [Predict 2](#).
3. Schalte die **Device Toolbar** ein und setze die Breite auf 1280.
4. Bewege den Mauszeiger im Panel **Elements** über das Element `<img>`. Der Browser zeigt auf der Seite einen Tooltip mit der dargestellten Größe.
5. Setze die Breite auf 390 und lies die Größe erneut.

Breite des Viewports	Dargestellte Größe des Schemas (px)
1280 px	640 × 400
390 px	358 × 223.75

Das Seitenverhältnis ist in beiden Zeilen 640 : 400.

3.5 Zwei Varianten eines Bildes

Die Datei `eintrag-desktop.png` ist 1280 px breit. In einem Viewport von 390 px Breite stellt der Browser sie 358 px breit dar. Der Text im Screenshot ist dann nicht lesbar. Das Element `<picture>` stellt dem Browser deshalb eine zweite Datei zur Wahl.

Füge in `ueber.html` nach der Liste unter „Aufbau eines Eintrags“ ein Bild mit zwei Varianten ein:

```
<figure>
  <picture>
    <source media="(width ≥ 40rem)" srcset="assets/img/eintrag-
desktop.png" width="1280" height="800">
    
  </picture>
  <figcaption>Der Eintrag „Breadcrumbs“ als Beispiel</figcaption>
</figure>
```

Der Screenshot ist informativ. Die Liste darüber nennt die Teile eines Eintrags, aber nicht ihre Anordnung auf der Seite. Ersetze ... im Attribut `alt` durch einen Satz in eigenen Worten, der diese Anordnung nennt. Öffne dazu `assets/img/eintrag-desktop.png` im Browser. Der Alt-Text des Schemas in [Ein Bild einbinden](#) nennt eine Anordnung auf dieselbe Weise: von oben nach unten.

- Ein `<source>` nennt in `media` eine Bedingung und in `srcset` eine Datei. Die Bedingung (`width ≥ 40rem`) trifft zu, wenn der Viewport mindestens 40rem breit ist. Ein rem entspricht der Schriftgröße, die der Browser ohne eigene Einstellung verwendet: 16 px. 40rem sind deshalb 640 px.
- Der Browser prüft die Elemente `<source>` in der Reihenfolge des Quelltexts. Er lädt die Datei des ersten Elements, dessen Bedingung zutrifft.
- Trifft keine Bedingung zu, lädt der Browser die Datei aus `src` des `<img>`.
- Der Alt-Text steht am `<img>` und gilt für beide Dateien.
- `width` und `height` stehen an beiden Elementen. Die beiden Dateien haben verschiedene Seitenverhältnisse.
- Die Pfade enthalten kein `../`, weil `ueber.html` im Projektordner liegt.

Weise die geladene Datei nach:

1. Öffne `ueber.html` im Browser und öffne das Panel **Network**.
2. Schalte die **Device Toolbar** ein, setze die Breite auf 390 und lade die Seite neu. Die Spalte **Name** enthält `eintrag-mobil.png`.
3. Setze die Breite auf 1280 und lade die Seite neu. Die Spalte **Name** enthält `eintrag-desktop.png`.

3.6 Commit

1. Übertrage alle Änderungen mit `git add .` in die Staging Area.
2. Prüfe mit `git status`, dass alle geänderten Dateien unter **Changes to be committed** stehen.

3. Erstelle einen Commit mit der Nachricht `feat: Bilder in die Toolbox aufnehmen` und übertrage ihn mit `git push`.

4. Verstehen

Lege im Lernprotokoll die Überschrift `## Bilder (Datum)` an. Der Ablauf je Experiment ist: Vorhersage notieren, ausführen, Ergebnis und Erklärung notieren.

Nimm jede Änderung nach dem Experiment zurück: `Strg/Cmd + Z` oder `git restore` mit dem Pfad der Datei.

Predict 1: Drei Varianten von alt

Ändere in `patterns/landmarks.html` das Attribut `alt` des Schemas nacheinander in drei Varianten:

1. Das Attribut fehlt.
 2. Das Attribut ist leer: `alt=""`.
 3. Das Attribut enthält ein Wort: `alt="Bild"`.
- **Vorhersage:** Welche Rolle und welchen Namen zeigt der Tab **Accessibility** je Variante? Meldet der Validator für eine der Varianten einen Fehler?
 - **Prüfen:** Führe je Variante diese Schritte aus:
 1. Lade die Seite neu, wähle das Schema aus und lies unter **Computed properties** die Werte **Name** und **Role**.
 2. Wähle im W3C-Validator unter **Check by** die Option **file upload** und danach die Datei `landmarks.html`. Klicke auf **Check**.

→ Auflösung 1 am Ende des Kapitels

Break-it 1: Falscher Pfad

Ändere in `patterns/landmarks.html` den Dateinamen im Attribut `src` in `landmark.svg`.

- **Vorhersage:** Was zeigt der Browser an der Stelle des Schemas? Wie groß ist die Fläche des Elements `<img>`?
- **Prüfen:**
 1. Öffne das Panel **Network**, setze in der **Device Toolbar** die Breite auf 1280 und lade die Seite neu.
 2. Lies im Panel **Elements** die dargestellte Größe des `<img>` wie in Platz reservieren.
 3. Lösche zusätzlich die Attribute `width` und `height`.
 4. Lade die Seite neu und lies die Größe erneut.

→ Auflösung 2 am Ende des Kapitels

Predict 2: Falsches Seitenverhältnis

Ändere in `patterns/landmarks.html` das Attribut `height="400"` in `height="200"`.

- **Vorhersage:** Stellt der Browser das Schema gestaucht dar? Wie groß ist die Fläche, wenn die Datei zusätzlich fehlt?
- **Prüfen:**
 1. Setze in der **Device Toolbar** die Breite auf 1280 und lade die Seite neu.

2. Lies die dargestellte Größe und im Tab **Computed** den Wert von aspect-ratio.
3. Ändere zusätzlich den Dateinamen in `landmark.svg` wie in Break-it 1.
4. Lade die Seite neu und lies die Größe erneut.

→ [Auflösung 3](#) am Ende des Kapitels

Break-it 2: <picture> ohne

Lösche in `ueber.html` die Zeile mit dem Element `<img>`. Das Element `<picture>` enthält danach nur das `<source>`.

- **Vorhersage:** Welche Datei zeigt die Seite bei 1280 px Breite des Viewports? Meldet der Validator einen Fehler?
- **Prüfen:**
 1. Öffne das Panel **Network**, setze in der **Device Toolbar** die Breite auf 1280 und lade die Seite neu.
 2. Bewege den Mauszeiger im Panel **Elements** über die Zeile `<picture>` und lies die dargestellte Größe.
 3. Wähle im W3C-Validator unter **Check by** die Option **file upload** und danach die Datei `ueber.html`. Klicke auf **Check**.

→ [Auflösung 4](#) am Ende des Kapitels

5. Toolbox-Eintrag

Schreibe zwei weitere Einträge: „Bilder“ und „Responsive Images“. Jeder Eintrag erhält eine eigene Seite:

Datei in patterns	Eintrag	Kategorie	Anchor
<code>bilder.html</code>	Bilder	Medien	<code>#medien</code>
<code>responsive-images.html</code>	Responsive Images	Medien	<code>#medien</code>

Lege beide Seiten mit den Schritten 1 bis 3 aus [Navigation & mehrere Seiten](#), [Übrige Seiten und Übersicht](#) (Kapitel 1.3) an. Die Schritte kopieren `landmarks.html`, ändern `<title>` und `description` und passen die Breadcrumbs an. Ersetze danach in beiden Seiten das ganze `<article>`, mit dem Schema aus [Ein Bild einbinden](#), durch einen neuen Eintrag mit der Überschrift `<h1>`. Der Aufbau entspricht den bisherigen Einträgen:

1. **Beschreibung und Zweck** in zwei bis fünf Sätzen, in eigenen Worten.
2. **Barrierefreiheit:** die Auswirkung auf Screenreader.
3. **Quelle:** einen Link auf die zugehörige MDN-Seite.

Unter Barrierefreiheit nennt „Bilder“ die Auflösung von Predict 1, „Responsive Images“ die Aussage zum Alt-Text aus [Zwei Varianten eines Bildes](#). Quelle für Bilder ist [MDN – The Image Embed element](#), für Responsive Images [MDN – The Picture element](#).

```

<article>
  <h1>Bilder</h1>
  <p>Das Element <img> bindet eine Bilddatei ein. ...</p>
  <p>Barrierefreiheit: ...</p>
  <p>Quelle: <a href="https://developer.mozilla.org/...">MDN - ...</a></p>
</article>

```

Ergänze in `index.html` nach dem `</section>` des Abschnitts „Navigation“, direkt vor `</main>`, den Abschnitt der neuen Kategorie:

```

<section id="medien">
  <h2>Medien</h2>
  <ul>
    <li><a href="patterns/bilder.html">Bilder</a>: Alt-Text, Maße und
Caption</li>
    <li><a href="patterns/responsive-images.html">Responsive Images</a>: je
Breite des Viewports eine andere Bilddatei</li>
  </ul>
</section>

```

Lade `index.html` neu. Unter „Medien“ stehen die Links „Bilder“ und „Responsive Images“.

Die Referenz-Toolbox zeigt die vollständige Toolbox, wie sie am Ende aussehen kann.

Ersetze in `README.md` den Abschnitt `## Struktur` durch diesen Inhalt:

```

## Struktur

...

toolbox/
├── README.md           ← diese Datei
├── learning-notizen.md ← Lernprotokoll: Vorhersagen und Beobachtungen
├── .gitignore          ← Systemdateien, die nicht ins Repository gehören
├── index.html          ← Übersicht mit Links auf alle Einträge
├── ueber.html          ← Über dieses Projekt
├── patterns/           ← je Eintrag eine Seite
│   ├── html-grundgeruest.html
│   ├── landmarks.html
│   ├── ueberschriften.html
│   ├── listen.html
│   ├── links.html
│   ├── breadcrumbs.html
│   ├── aktuelle-seite.html
│   ├── bilder.html
│   └── responsive-images.html
├── assets/
│   ├── css/
│   │   └── base.css    ← Start-Stylesheet 2.2.0 (vorgegeben)
│   ├── fonts/         ← Schriften + Lizenzen (vorgegeben)
│   └── img/            ← Bilder (vorgegeben, CC0 1.0)
└── ...

```

Header, Navigation und Footer stehen in jeder HTML-Datei. Eine Änderung an der Navigation betrifft alle elf Seiten.

1. Setze in README.md in der Tabelle ## Fortschritt „Bilder“ auf „erledigt“ und ergänze die Zeile „Tabellen & Formulare“ mit „offen“.
2. Übertrage alle Änderungen mit `git add` in die Staging Area.
3. Prüfe mit `git status`, dass alle geänderten Dateien unter **Changes to be committed** stehen, auch learning-notizen.md.
4. Erstelle einen Commit mit der Nachricht feat: Einträge Bilder und Responsive Images ergänzen und übertrage ihn mit `git push`.

6. Prüfen

- ☐ Der W3C-Validator zeigt für jede der elf Seiten die Meldung „Document checking completed. No errors or warnings to show.“
- ☐ Jedes `<img>` hat das Attribut `alt`. Der Accessibility Tree zeigt für beide Bilder die Rolle `image` und den Alt-Text als Namen.
- ☐ Der Alt-Text des Screenshots nennt in einem Satz, was über und was unter der Überschrift des Eintrags steht.
- ☐ Jedes `<img>` und jedes `<source>` hat die Attribute `width` und `height` mit der Größe der Datei.
- ☐ Das Panel **Network** zeigt auf keiner Seite eine fehlgeschlagene Anfrage.
- ☐ ueber.html lädt bei 390 px Breite eintrag-mobil.png und bei 1280 px Breite eintrag-desktop.png.
- ☐ Bei 390 px Breite ist kein Bild verzerrt.
- ☐ learning-notizen.md enthält zu jedem Experiment Vorhersage, Ergebnis und Erklärung.
- ☐ Die Einträge „Bilder“ und „Responsive Images“ stehen unter „Medien“.
- ☐ Die Commits feat: Bilder in die Toolbox aufnehmen und feat: Einträge Bilder und Responsive Images ergänzen sind auf GitHub.

Quellen

- MDN – The Image Embed element
- MDN – The Figure with Optional Caption element
- MDN – The Picture element
- MDN – Using responsive images in HTML
- MDN – aspect-ratio
- MDN – Image file type and format guide
- W3C WAI – Images Tutorial
- W3C WAI – An alt Decision Tree
- web.dev – Optimize Cumulative Layout Shift
- W3C Nu HTML Checker

Auflösungen

Auflösung 1

Zu Predict 1: Drei Varianten von alt

Variante	Tab Accessibility	Validator
alt fehlt	Rolle image, kein Name	keine Meldung
alt=""	„Accessibility node not exposed“ mit dem Grund „Element has empty alt text“	keine Meldung
alt="Bild"	Rolle image, Name „Bild“	keine Meldung

Der Validator meldet für keine Variante einen Fehler. Ein fehlendes alt meldet er nur außerhalb eines <figure> mit <figcaption>: An “img” element must have an “alt” attribute, except under certain conditions. Der HTML-Standard lässt die Caption als Ersatz zu. Im Accessibility Tree hat das Bild trotzdem keinen Namen.

Ein leeres alt ist eine Angabe: Das Bild ist dekorativ. Der Browser entfernt es aus dem Accessibility Tree. Für das Schema ist diese Angabe falsch. Der Wert Bild wiederholt die Rolle und nennt keine Aussage.

Der Validator prüft, ob das Attribut vorhanden ist. Er prüft nicht, ob der Text das Bild ersetzt. Prüfen enthält deshalb eine Prüfung jedes Bildes im Accessibility Tree.

Auflösung 2

Zu Break-it 1: Falscher Pfad

Der Browser zeigt den Alt-Text an der Stelle des Schemas. Das Panel **Network** enthält eine fehlgeschlagene Anfrage für landmark.svg.

Attribute	Dargestellte Größe bei 1280 px Breite des Viewports (px)
mit width und height	640 × 400
ohne width und height	736 × 54

Mit den Attributen reserviert der Browser die Fläche des Bildes, auch wenn die Datei fehlt. Ohne die Attribute ist die Fläche so hoch wie die zwei Zeilen des Alt-Texts.

Auch während des Ladens reserviert der Browser mit den Attributen 640 × 400 px. Ein Bild ohne die Attribute hat dann die Höhe 0. Nach dem Laden ist es 400 px hoch, und der folgende Inhalt steht 400 px tiefer. Eine lokale Datei lädt in wenigen Millisekunden. Die Verschiebung ist deshalb im Experiment kaum zu sehen.

Auflösung 3

Zu Predict 2: Falsches Seitenverhältnis

Der Browser stellt das Schema nicht gestaucht dar. Die dargestellte Größe bleibt 640 × 400. Der Wert von aspect-ratio ist auto 640 / 200.

Das Schlüsselwort auto bedeutet: Nach dem Laden gilt das Seitenverhältnis der Datei. Das Verhältnis aus den Attributen gilt nur vor dem Laden und bei fehlender Datei. Mit dem falschen Dateinamen ist die Fläche 640 × 200 groß.

Falsche Werte verzerren das Bild deshalb nicht. Sie reservieren die falsche Fläche: Beim Laden wächst das Bild von 200 px auf 400 px Höhe. Der Validator meldet falsche Werte nicht.

Das gilt nur mit der Regel `block-size: auto` aus Platz reservieren. Ohne diese Regel gilt die Höhe aus dem Attribut, und der Browser stellt das Schema 640 × 200 px groß dar.

Auflösung 4

Zu Break-it 2: <picture> ohne

Die Seite zeigt kein Bild. Das Element `<picture>` hat die Höhe 0 (`picture 736 × 0`), und das Panel **Network** enthält keine Anfrage für eine Bilddatei. Der Validator meldet: Element “picture” is missing a required instance of child element “img”.

Ein `<source>` stellt kein Bild dar. Der Browser stellt immer das `<img>` dar, ein `<source>` nennt ihm nur eine andere Datei. Alt-Text und Fläche gehören zum `<img>`. Ohne `<img>` fehlen beide. Das Glossar fasst das Pattern unter Responsive Images zusammen.



1.5 Tabellen & Formulare

Voraussetzungen: [Bilder](#) (Kapitel 1.4)

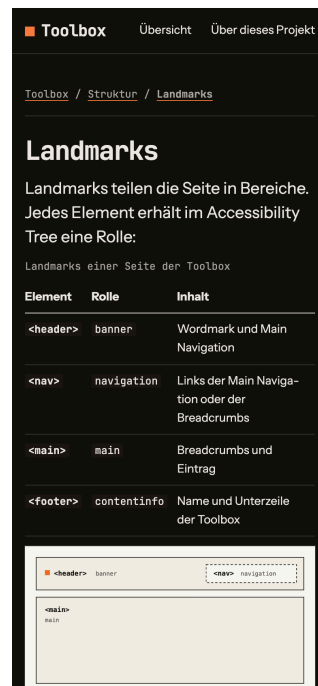
Lernziele

- Tabellarische Daten mit `<table>`, `<caption>` und `<th scope>` auszeichnen und den Namen der Tabelle und die Rollen der Zellen im Accessibility Tree nachweisen.
- Ein Formular mit `<label>`, `name` und `<fieldset>` aufbauen und den Namen jedes Felds im Accessibility Tree und die gesendeten Daten in der URL nachweisen.
- Ein Pflichtfeld mit `required` vom Browser validieren lassen und den Zeitpunkt nachweisen, zu dem `:invalid` und `:user-invalid` zutreffen.

1. Vorschau

Zielstand nach diesem Kapitel: Die Seite „Über dieses Projekt“ enthält ein Formular, das die Quellen der Einträge durchsucht.

Die Seite des Eintrags „Landmarks“ bei Smartphone-Breite im Dark Mode. Eine Tabelle nennt je Element die Rolle und den Inhalt:



2. Warum

Preislisten, Stundenpläne, Bestellungen und Anmeldungen bestehen aus Tabellen und Formularen, deren Struktur Browser und Screenreader auswerten. In einer Tabelle gehört jeder Wert zu einer Spalte und zu einer Zeile. In einem Formular hat jedes Feld einen Namen für die Person und einen Schlüssel für die gesendeten Daten. Regeln für die Eingabe stehen im HTML, und der Browser prüft sie vor dem Absenden.

Weiterführend: [MDN – HTML table basics](#) · [MDN – Your first form](#) · [W3C WAI – Forms Tutorial](#)

3. Bauen

Ausgangsstand ist die Toolbox aus dem Kapitel [Bilder](#) (Kapitel 1.4): elf Seiten ohne Tabelle und ohne Formular.

3.1 Material

Das Projekt hat am Ende von [Bauen](#) diesen Aufbau:

```

toolbox/
├── ueber.html           ← erhält ein Formular
├── patterns/
│   ├── landmarks.html ← erhält eine Tabelle
│   └── ...
├── assets/
│   └── css/
│       └── base.css    ← Version 2.3.0

```

Lade die Datei [base.css](#) aus dem Material herunter und ersetze damit `assets/css/base.css` (siehe Dateisystem & Pfade).

Die Version 2.3.0 des Start-Stylesheets gestaltet zusätzlich Tabellen, die Felder und Buttons eines Formulars und die Markierung einer ungültigen Eingabe. Zeile 3 der Datei nennt Version 2.3.0.

3.2 Eine Tabelle

Die Seite „Landmarks“ nennt je Element eine Rolle. Das Element `<dl>` ordnet einem Begriff Beschreibungen zu, benennt diese Beschreibungen aber nicht. Eine Tabelle ordnet Daten in Zeilen und Spalten: Jede Zeile beschreibt ein Element, jede Spalte enthält eine Angabe mit einem Namen.

Ersetze in `patterns/landmarks.html` das Element `<dl>` durch diese Tabelle:

```
<table>
  <caption>Landmarks einer Seite der Toolbox</caption>
  <thead>
    <tr>
      <th scope="col">Element</th>
      <th scope="col">Rolle</th>
      <th scope="col">Inhalt</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <th scope="row"><code>&lt;header&gt;</code></th>
      <td><code>banner</code></td>
      <td>Wordmark und Main Navigation</td>
    </tr>
    <tr>
      <th scope="row"><code>&lt;nav&gt;</code></th>
      <td><code>navigation</code></td>
      <td>Links der Main Navigation oder der Breadcrumbs</td>
    </tr>
    <tr>
      <th scope="row"><code>&lt;main&gt;</code></th>
      <td><code>main</code></td>
      <td>Breadcrumbs und Eintrag</td>
    </tr>
    <tr>
      <th scope="row"><code>&lt;footer&gt;</code></th>
      <td><code>contentinfo</code></td>
      <td>Name und Unterzeile der Toolbox</td>
    </tr>
  </tbody>
</table>
```

- `<tr>` ist eine Zeile der Tabelle, `<td>` eine Datenzeile.
- `<thead>` enthält die Zeile mit den Header-Zellen der Spalten, `<tbody>` die Zeilen mit den Daten.
- `<th>` ist eine Header-Zelle, `<caption>` die Caption der Tabelle (Header-Zellen und Caption).

Öffne die Seite im Browser. Die Tabelle steht zwischen dem ersten Absatz und dem Schema.

3.3 Header-Zellen und Caption

Eine Header-Zelle benennt eine Spalte oder eine Zeile. Das Attribut `scope` nennt die Richtung: `col` für die Spalte, `row` für die Zeile. Die Tabelle hat Header-Zellen in beiden Richtungen. Die erste Zeile benennt die Spalten, die erste Zeile jeder weiteren Zeile benennt diese Zeile.

Ein Screenreader sagt zu einer Datenzeile die zugehörigen Header-Zellen an. Der Wert `navigation` ist dann ohne die übrige Tabelle verständlich: Er gehört zur Zeile `<nav>` und zur Spalte „Rolle“.

Das Element `<caption>` steht als erstes Element in `<table>`. Es ist für alle sichtbar und gibt der Tabelle ihren Namen im Accessibility Tree.

Weise Namen und Rollen in den DevTools nach:

1. Wähle die Caption mit dem **Inspect mode** aus. Klicke im Panel **Elements** auf die Zeile `<table>` direkt darüber.
2. Öffne den Tab **Accessibility** und lies unter **Computed properties** die Werte **Name** und **Role**. Die Rolle ist `table`, der Name ist die Caption.
3. Wähle nacheinander die Zellen der folgenden Tabelle mit dem **Inspect mode** aus und lies die Rolle. Enthält eine Zelle `<code>`, wählt der Inspect mode das innere Element `<code>`. Klicke dann im Panel **Elements** auf die Zeile `<th>` oder `<td>` direkt darüber.

Element	Rolle
<code><th scope="col"></code>	columnheader
<code><th scope="row"></code>	rowheader
<code><td></code>	cell

3.4 Ein Formular

Ein Formular sendet Eingaben an eine Adresse. Die Toolbox hat keinen eigenen Server, der Eingaben verarbeitet. Das Formular sendet sie deshalb an die Suchmaschine DuckDuckGo und sucht einen Begriff in einer Quelle der Einträge.

Füge in `ueber.html` direkt vor `</main>`, nach der Liste unter „Werkzeuge“, diesen Abschnitt ein:

```
<h2>Quellen durchsuchen</h2>
<p>Das Formular sucht einen Begriff in einer Quelle der Einträge. Die Suche
führt DuckDuckGo aus.</p>
<form action="https://duckduckgo.com/" method="get">
  <p>
    <label for="q">Suchbegriff</label>
    <input type="search" id="q" name="q" required>
  </p>
  <fieldset>
    <legend>Quelle</legend>
    <p>
      <input type="radio" id="sites-mdn" name="sites"
value="developer.mozilla.org" checked>
      <label for="sites-mdn">MDN</label>
    </p>
    <p>
      <input type="radio" id="sites-wai" name="sites" value="www.w3.org/
```

```
WAI">
    <label for="sites-wai">W3C WAI</label>
  </p>
  <p>
    <input type="radio" id="sites-webdev" name="sites" value="web.dev">
    <label for="sites-webdev">web.dev</label>
  </p>
</fieldset>
<p><button type="submit">Mit DuckDuckGo suchen</button></p>
</form>
```

- action enthält die Adresse, an die der Browser die Eingaben sendet.
- method="get" hängt die Eingaben an diese Adresse an (Label und Name).
- <input> ist ein Feld. Das Attribut type legt die Art des Felds fest: search für einen Suchbegriff, radio für einen Radio Button, einen runden Knopf zur Auswahl einer Antwort. Alle Werte nennt MDN – The HTML Input element.
- <button type="submit"> ist der Submit Button. Ein Klick sendet das Formular ab, ebenso die Taste Enter in einem Feld.

Ersetze in ueber.html die Zeile mit description durch diese Zeile. Sie nennt zusätzlich die Quellen:

```
<meta name="description" content="Zweck, Aufbau, Werkzeuge und Quellen der
Toolbox.">
```

Öffne die Seite im Browser, gib label in das Feld ein und sende das Formular ab. Der Browser öffnet DuckDuckGo. Das Suchfeld dort enthält label site:developer.mozilla.org. DuckDuckGo macht aus der gewählten Quelle den Zusatz site:. Er beschränkt die Suche auf diese Website.

3.5 Label und Name

Jedes Feld trägt zwei Angaben mit verschiedenen Aufgaben:

Angabe	Aufgabe	Nachweis
<label for="q">	Name des Felds für die Person: sichtbar und im Accessibility Tree	Tab Accessibility
name="q"	Schlüssel, unter dem der Browser den Wert sendet	URL nach dem Absenden

Das Label verbindet das Attribut for mit dem Feld: for enthält die id des Felds. Eine id kommt auf einer Seite nur einmal vor. Ein Klick auf das Label setzt den Fokus in das Feld. Bei einem Radio Button wählt der Klick ihn aus.

Weise beide Angaben nach:

1. Kehre mit dem Zurück-Pfeil des Browsers zu ueber.html zurück und lade die Seite neu.
2. Wähle das Feld unter „Suchbegriff“ mit dem **Inspect mode** aus.
3. Lies im Tab **Accessibility** die Werte **Name** und **Role**. Die Rolle ist searchbox, der Name ist „Suchbegriff“.
4. Klicke auf der Seite auf das Wort „Suchbegriff“. Der Fokus steht im Feld: Ein Rahmen in der Akzentfarbe umgibt das Feld.
5. Gib label ein, wähle „W3C WAI“ und sende das Formular ab.

6. Lies die Adresse in der Adresszeile des Browsers. Sie beginnt so:

```
https://duckduckgo.com/?q=label&sites=www.w3.org%2FWAI
```

Nach dem ? folgt je Feld ein Paar aus Schlüssel und Wert. Ein & trennt die Paare. Der Browser kodiert Zeichen, die in einer URL eine eigene Bedeutung haben: %2F steht für /. Stehen am Ende der Adresse weitere Paare, stammen sie von DuckDuckGo.

3.6 Gruppe mit <fieldset>

Die drei Radio Buttons bilden eine Auswahl. Dazu tragen sie denselben Wert im Attribut name. Der Browser lässt in einer Auswahl höchstens einen Radio Button ausgewählt. Das Attribut checked legt die Vorauswahl fest, value den gesendeten Wert.

Die Labels „MDN“, „W3C WAI“ und „web.dev“ nennen die Antworten. Die Frage dazu nennt das Element <legend>. Das Element <fieldset> fasst die Felder zu einer Gruppe zusammen, und <legend> gibt der Gruppe ihren Namen.

Weise die Gruppe nach:

1. Kehre mit dem Zurück-Pfeil des Browsers zu ueber.html zurück und lade die Seite neu.
2. Wähle den Text „Quelle“ mit dem **Inspect mode** aus. Klicke im Panel **Elements** auf die Zeile <fieldset> direkt darüber.
3. Lies im Tab **Accessibility** die Werte **Name** und **Role**. Die Rolle ist group, der Name ist „Quelle“.
4. Klicke in das Feld „Suchbegriff“ und drücke Tab. Der Fokus steht auf dem ausgewählten Radio Button: Der Rahmen umgibt jetzt den Radio Button.
5. Drücke die Pfeiltaste nach unten. Der nächste Radio Button ist ausgewählt.
6. Drücke Tab. Der Fokus steht auf dem Submit Button.

3.7 Pflichtfeld

Das Attribut required kennzeichnet ein Pflichtfeld. Der Browser prüft die Felder vor dem Absenden. Diese Prüfung heißt Validierung. Bei einem leeren Pflichtfeld sendet der Browser das Formular nicht ab, setzt den Fokus in das Feld und zeigt dort eine Fehlermeldung.

1. Lade ueber.html neu.
2. Klicke auf den Submit Button, ohne einen Suchbegriff einzugeben.

Der Browser bleibt auf der Seite und zeigt am Feld eine Fehlermeldung. Der Text der Fehlermeldung stammt vom Browser. Das Feld trägt danach am linken Rand einen Balken in der Akzentfarbe, der Farbe des Quadrats vor der Wordmark. Den Balken erzeugt das Start-Stylesheet. Den Aufbau der Regeln erklärt das Kapitel CSS-Grundlagen (Kapitel 1.6). Hier zählt nur die erste Zeile mit dem Selektor input:user-invalid:

```
input:user-invalid {
  border-color: var(--color-accent);
  border-inline-start-width: var(--space-xs);
  padding-inline-start: calc(var(--space-sm) - var(--space-xs) + var(--rule-width));
}
```

Der Selektor `:user-invalid` trifft auf ein Feld mit ungültigem Wert zu. Predict 3 vergleicht ihn mit dem Selektor `:invalid`.

Weitere Regeln für Eingaben sind `type="email"`, `minlength` und `pattern`. Eine Übersicht enthält [MDN – Client-side form validation](#).

3.8 Commit

1. Übertrage alle Änderungen mit `git add .` in die Staging Area.
2. Prüfe mit `git status`, dass alle geänderten Dateien unter **Changes to be committed** stehen.
3. Erstelle einen Commit mit der Nachricht `feat: Tabelle und Formular in die Toolbox aufnehmen` und übertrage ihn mit `git push`.

4. Verstehen

Lege im Lernprotokoll die Überschrift **## Tabellen & Formulare (Datum)** an. Der Ablauf je Experiment ist: Vorhersage notieren, ausführen, Ergebnis und Erklärung notieren.

Nimm jede Änderung nach dem Experiment zurück: `Strg/Cmd + Z` oder `git restore` mit dem Pfad der Datei.

Predict 1: Tabelle ohne scope, ohne `<th>`, ohne `<caption>`

Ändere in `patterns/landmarks.html` die Tabelle nacheinander in drei Varianten:

1. Lösche jedes Attribut `scope`.
2. Ersetze zusätzlich jedes `<th>` durch `<td>`, im öffnenden und im schließenden Tag.
3. Nimm beide Änderungen zurück und lösche die Zeile mit `<caption>`.

Variante 2 ändert vierzehn Tags. Öffne in VS Code das Ersetzen in der Datei mit `Strg + H` (macOS: `Cmd + Option + F`, siehe VS Code, Suchen und Ersetzen). Gib unter **Find** `<th>` und unter **Replace** `<td>` ein und klicke auf **Replace All**. Wiederhole das für `</th>` und `</td>`.

- **Vorhersage:** Welche Rolle haben die Zellen der ersten Zeile und der ersten Spalte je Variante? Welchen Namen hat die Tabelle? Meldet der Validator für eine der Varianten einen Fehler?
- **Prüfen:** Führe je Variante diese Schritte aus:
 1. Lade die Seite neu und lies Rollen und Namen wie in [Header-Zellen und Caption](#).
 2. Wähle im [W3C-Validator](#) unter **Check by** die Option **file upload** und danach die Datei `landmarks.html`. Klicke auf **Check**.

→ [Auflösung 1](#) am Ende des Kapitels

Predict 2: Feld ohne passendes Label

Ändere in `ueber.html` das Feld „Suchbegriff“ nacheinander in drei Varianten:

1. Das Attribut `for` nennt eine andere `id`: `for="suche"`.
2. Die Zeile mit `<label>` fehlt. Das `<input>` enthält dafür das Attribut `placeholder="Suchbegriff"`, einen Beispieltext im leeren Feld.
3. Die Zeile mit `<label>` und das Attribut `placeholder` fehlen.

- **Vorhersage:** Welchen Namen zeigt der Tab **Accessibility** für das Feld je Variante? Für welche Variante meldet der Validator einen Fehler?
- **Prüfen:** Führe je Variante diese Schritte aus:
 1. Lade ueber.html neu und lies den Namen des Felds wie in Label und Name.
 2. Nur in Variante 1: Klicke auf das Wort „Suchbegriff“.
 3. Nur in Variante 2: Gib ein Zeichen in das Feld ein.
 4. Wähle im W3C-Validator unter **Check by** die Option **file upload** und danach die Datei ueber.html. Klicke auf **Check**.

→ Auflösung 2 am Ende des Kapitels

Break-it 1: Feld ohne name

Lösche in ueber.html im Feld „Suchbegriff“ das Attribut name="q".

- **Vorhersage:** Welche Paare aus Schlüssel und Wert enthält die Adresse nach dem Absenden?
- **Prüfen:** Lade ueber.html neu, gib label ein und sende das Formular ab. Lies die Adresse in der Adresszeile.

→ Auflösung 3 am Ende des Kapitels

Break-it 2: Radio Buttons mit verschiedenen Werten von name

Ändere in ueber.html im Radio Button „W3C WAI“ das Attribut name="sites" in name="quelle".

- **Vorhersage:** Wie viele Radio Buttons sind nach einem Klick auf „W3C WAI“ ausgewählt? Welche Paare enthält die Adresse nach dem Absenden?
- **Prüfen:** Lade ueber.html neu, gib label ein und klicke auf „W3C WAI“. Zähle die ausgewählten Radio Buttons. Sende das Formular ab und lies die Adresse.

→ Auflösung 4 am Ende des Kapitels

Predict 3: :invalid statt :user-invalid

Der Selektor :invalid trifft auf jedes Feld zu, dessen Wert eine Regel für die Eingabe verletzt, zum Beispiel required. Ändere in assets/css/base.css in der Zeile input:user-invalid { den Selektor in input:invalid. Die Suche mit Strg/Cmd + F findet die Zeile.

- **Vorhersage:** Ab welchem Zeitpunkt trägt das Feld „Suchbegriff“ den Balken: mit :invalid und mit :user-invalid?
- **Prüfen:** Führe die Schritte zuerst mit input:invalid aus. Nimm danach die Änderung in base.css zurück und führe sie mit input:user-invalid erneut aus.
 1. Lade ueber.html neu und betrachte das Feld, ohne es anzuklicken.
 2. Klicke in das Feld und verlasse es mit Tab, ohne ein Zeichen einzugeben.
 3. Klicke in das Feld, gib zwei Zeichen ein und lösche sie. Betrachte das Feld, bevor du es verlässt.
 4. Verlasse das Feld mit Tab.
 5. Lade die Seite neu und klicke auf den Submit Button.

→ Auflösung 5 am Ende des Kapitels

Predict 4: Leerzeichen im Pflichtfeld

Gib in ueber.html in das Feld „Suchbegriff“ drei Leerzeichen ein.

- **Vorhersage:** Sendet der Browser das Formular nach einem Klick auf den Submit Button ab?
- **Prüfen:** Klicke auf den Submit Button. Lies die Adresse in der Adresszeile.

→ [Auflösung 6](#) am Ende des Kapitels

5. Toolbox-Eintrag

Schreibe zwei weitere Einträge: „Tabellen“ und „Label und Fehlermeldung“. Jeder Eintrag erhält eine eigene Seite:

Datei in patterns	Eintrag	Kategorie	Anchor
tabellen.html	Tabellen	Text	#text
label-fehlermeldung.html	Label und Fehlermeldung	Formulare	#formulare

Lege beide Seiten mit den Schritten 1 bis 3 aus [Navigation & mehrere Seiten](#), [Übrige Seiten und Übersicht](#) (Kapitel 1.3) an. Die Schritte kopieren landmarks.html, ändern <title> und description und passen die Breadcrumbs an. Ersetze danach in beiden Seiten das ganze <article>, mit der Tabelle aus [Eine Tabelle](#) und dem Schema, durch einen neuen Eintrag mit der Überschrift <h1>. Der Aufbau entspricht den bisherigen Einträgen:

1. **Beschreibung und Zweck** in zwei bis fünf Sätzen, in eigenen Worten.
2. **Barrierefreiheit:** die Auswirkung auf Screenreader und Tastaturbedienung.
3. **Quelle:** einen Link auf die zugehörige MDN-Seite.

Unter Barrierefreiheit nennt „Tabellen“ nur den Screenreader: die Aussage aus [Header-Zellen und Caption](#). „Label und Fehlermeldung“ nennt die Auflösung von Predict 2 und den Klick auf das Label aus [Label und Name](#). Die Beschreibung von „Label und Fehlermeldung“ nennt das Label mit for und das Pflichtfeld mit required. Beide Einträge bestehen aus drei Absätzen wie im Code-Block, ohne Tabelle und ohne Formular. Quelle für Tabellen ist [MDN – HTML table basics](#), für Label und Fehlermeldung [MDN – The Label element](#).

```
<article>
  <h1>Label und Fehlermeldung</h1>
  <p>Das Element <code>&lt;label&gt;</code> gibt einem Feld einen Namen. ...
</p>
  <p>Barrierefreiheit: ...</p>
  <p>Quelle: <a href="https://developer.mozilla.org/...">MDN - ...</a></p>
</article>
```

Ergänze in index.html am Ende der Liste unter „Text“, nach dem Eintrag „Links“, diese Zeile:

```
<li><a href="patterns/tabellen.html">Tabellen</a>: Daten in Zeilen und
Spalten mit Header-Zellen</li>
```

Ergänze in index.html nach dem </section> des Abschnitts „Medien“, direkt vor </main>, den Abschnitt der neuen Kategorie:


```

<section id="formulare">
  <h2>Formulare</h2>
  <ul>
    <li><a href="patterns/label-fehlermeldung.html">Label und
Fehlermeldung</a>: der Name eines Felds und die Validierung durch den Browser</
li>
  </ul>
</section>

```

Lade index.html neu. Unter „Text“ steht als letzter Link „Tabellen“, unter „Formulare“ der Link „Label und Fehlermeldung“.

Die Referenz-Toolbox zeigt die vollständige Toolbox, wie sie am Ende aussehen kann.

Ersetze in README.md den Abschnitt ## Struktur durch diesen Inhalt:

```

## Struktur

...

toolbox/
├── README.md           ← diese Datei
├── learning-notizen.md ← Lernprotokoll: Vorhersagen und Beobachtungen
├── .gitignore          ← Systemdateien, die nicht ins Repository gehören
├── index.html          ← Übersicht mit Links auf alle Einträge
├── ueber.html          ← Über dieses Projekt
├── patterns/           ← je Eintrag eine Seite
│   ├── html-grundgeruest.html
│   ├── landmarks.html
│   ├── ueberschriften.html
│   ├── listen.html
│   ├── links.html
│   ├── tabellen.html
│   ├── breadcrumbs.html
│   ├── aktuelle-seite.html
│   ├── bilder.html
│   ├── responsive-images.html
│   └── label-fehlermeldung.html
├── assets/
│   ├── css/
│   │   └── base.css    ← Start-Stylesheet 2.3.0 (vorgegeben)
│   ├── fonts/          ← Schriften + Lizenzen (vorgegeben)
│   └── img/             ← Bilder (vorgegeben, CC0 1.0)
└── ...

```

Header, Navigation und Footer stehen in jeder HTML-Datei. Eine Änderung an der Navigation betrifft alle dreizehn Seiten.

1. Setze in README.md in der Tabelle ## Fortschritt „Tabellen & Formulare“ auf „erledigt“ und ergänze die Zeile „CSS-Fundament“ mit „offen“.
2. Übertrage alle Änderungen mit `git add .` in die Staging Area.
3. Prüfe mit `git status`, dass alle geänderten Dateien unter **Changes to be committed** stehen, auch learning-notizen.md.
4. Erstelle einen Commit mit der Nachricht feat: Einträge zu Tabellen und Formularen ergänzen und übertrage ihn mit `git push`.

6. Prüfen

- ☐ Der W3C-Validator zeigt für jede der dreizehn Seiten die Meldung „Document checking completed. No errors or warnings to show.“
- ☐ Der Accessibility Tree zeigt für die Tabelle die Rolle `table` mit der Caption als Namen und für die Header-Zellen die Rollen `columnheader` und `rowheader`.
- ☐ Der Accessibility Tree zeigt für jedes Feld einen Namen und für das `<fieldset>` die Rolle `group` mit dem Namen „Quelle“.
- ☐ Ein Klick auf jedes Label setzt den Fokus in das zugehörige Feld oder wählt den Radio Button aus.
- ☐ Die Adresse nach dem Absenden enthält die Schlüssel `q` und `sites`.
- ☐ Mit leerem Feld „Suchbegriff“ sendet der Browser das Formular nicht ab und zeigt eine Fehlermeldung.
- ☐ Tastatur: Tab erreicht das Feld, den ausgewählten Radio Button und den Submit Button.
- ☐ Viewport: Bei 320 px Breite in der **Device Toolbar** endet die Tabelle in `landmarks.html` vor dem rechten Rand. Jeder Code in einer Zelle steht in einer Zeile.
- ☐ `learning-notizen.md` enthält zu jedem Experiment Vorhersage, Ergebnis und Erklärung.
- ☐ Die Einträge „Tabellen“ unter „Text“ und „Label und Fehlermeldung“ unter „Formulare“ stehen in der Übersicht.
- ☐ Die Commits `feat: Tabelle und Formular in die Toolbox aufnehmen` und `feat: Einträge zu Tabellen und Formularen ergänzen` sind auf GitHub.

Quellen

- [MDN – HTML table basics](#)
- [MDN – HTML table accessibility](#)
- [MDN – The Table Header element](#)
- [MDN – Your first form](#)
- [MDN – Sending form data](#)
- [MDN – The Label element](#)
- [MDN – The Field Set element](#)
- [MDN – The HTML Input element](#)
- [MDN – Client-side form validation](#)
- [MDN – `:user-invalid`](#)
- [MDN – `:invalid`](#)
- [W3C WAI – Tables Tutorial](#)
- [W3C WAI – Forms Tutorial](#)
- [W3C Nu HTML Checker](#)

Auflösungen

Auflösung 1

Zu Predict 1: Tabelle ohne scope, ohne <th>, ohne <caption>

Variante	Accessibility Tree	Validator
ohne scope	columnheader und rowheader wie zuvor	keine Meldung
<td> statt <th>	jede Zelle hat die Rolle cell	keine Meldung
ohne <caption>	Rolle table, kein Name	keine Meldung

Chrome leitet die Richtung einer Header-Zelle aus ihrer Position in der Tabelle ab. Die Rollen bleiben deshalb ohne scope gleich. Das Attribut nennt die Richtung ausdrücklich. Das W3C empfiehlt es für Tabellen mit Header-Zellen in beiden Richtungen.

Mit <td> verlieren die Zellen die fette Schrift und ihre Rolle. Die Tabelle enthält dann keine Angabe, welche Zelle eine Spalte oder eine Zeile benennt.

Der Validator prüft den Aufbau der Tabelle. Er erkennt nicht, welche Zelle eine Header-Zelle ist und ob die Tabelle einen Namen braucht. Prüfen enthält deshalb eine Prüfung im Accessibility Tree.

Auflösung 2

Zu Predict 2: Feld ohne passendes Label

Variante	Name im Accessibility Tree	Validator
for="suche"	kein Name	The value of the "for" attribute of the "label" element must be the ID of a non-hidden form control.
nur placeholder	„Suchbegriff“	keine Meldung
weder <label> noch placeholder	kein Name	keine Meldung

In Variante 1 steht der Text weiterhin über dem Feld. Er ist trotzdem nicht der Name des Felds, und ein Klick auf den Text setzt keinen Fokus.

In Variante 2 bildet Chrome den Namen aus dem Placeholder. Der Placeholder ersetzt das Label trotzdem nicht. Er verschwindet mit dem ersten eingegebenen Zeichen, und die Seite enthält keinen Text, der den Fokus in das Feld setzt.

Der Validator meldet ein fehlendes Label nicht. Prüfen enthält deshalb eine Prüfung des Namens jedes Felds.

Auflösung 3

Zu Break-it 1: Feld ohne name

Die Adresse beginnt so:

```
https://duckduckgo.com/?sites=developer.mozilla.org
```

Der Suchbegriff fehlt. Der Browser sendet nur Felder mit dem Attribut name.

Das Feld ist sichtbar, hat ein Label und nimmt eine Eingabe an. Der Validator meldet keinen Fehler.

Auflösung 4

Zu Break-it 2: Radio Buttons mit verschiedenen Werten von name

Zwei Radio Buttons sind ausgewählt: „MDN“ und „W3C WAI“. Die Adresse beginnt so:

```
https://duckduckgo.com/?q=label&sites=developer.mozilla.org&quelle=www.w3.org%2FWAI
```

Der Browser bildet eine Auswahl aus allen Radio Buttons mit demselben Wert in name. „W3C WAI“ gehört nach der Änderung zu einer eigenen Auswahl mit einem einzigen Radio Button. Ein zweiter Klick wählt ihn nicht ab.

Das <fieldset> fasst die Felder für die Person zusammen. Es bildet keine Auswahl. Der Validator meldet keinen Fehler.

Auflösung 5

Zu Predict 3: :invalid statt :user-invalid

Zeitpunkt	input:invalid	input:user-invalid
nach dem Laden der Seite	Balken	kein Balken
Feld ohne Eingabe verlassen	Balken	kein Balken
Eingabe gelöscht, Fokus im Feld	Balken	kein Balken
Eingabe gelöscht, Feld verlassen	Balken	Balken
Submit Button geklickt, Feld leer	Balken	Balken

Ein leeres Pflichtfeld verletzt die Regel aus required vom Laden der Seite an. :invalid markiert es deshalb, bevor die Person ein Zeichen eingegeben hat.

:user-invalid trifft erst zu, nachdem die Person den Wert geändert und das Feld verlassen hat. Der Selektor trifft auch nach einem Klick auf den Submit Button zu. Ein Feld nur zu betreten und zu verlassen, reicht nicht.

Beide Markierungen enden, sobald das Feld einen gültigen Wert enthält.

Auflösung 6

Zu Predict 4: Leerzeichen im Pflichtfeld

Der Browser sendet das Formular ab. Die Adresse beginnt so:

```
https://duckduckgo.com/?q=+++&sites=developer.mozilla.org
```

Ein + steht in den gesendeten Daten für ein Leerzeichen.

required prüft, ob das Feld einen Wert enthält. Drei Leerzeichen sind ein Wert. Das Attribut prüft nicht, ob der Wert ein Suchbegriff ist.

Die Validierung im Browser prüft nur die Regeln aus dem HTML. Das Programm, das die Daten erhält, prüft sie deshalb erneut.



1.6 CSS-Grundlagen

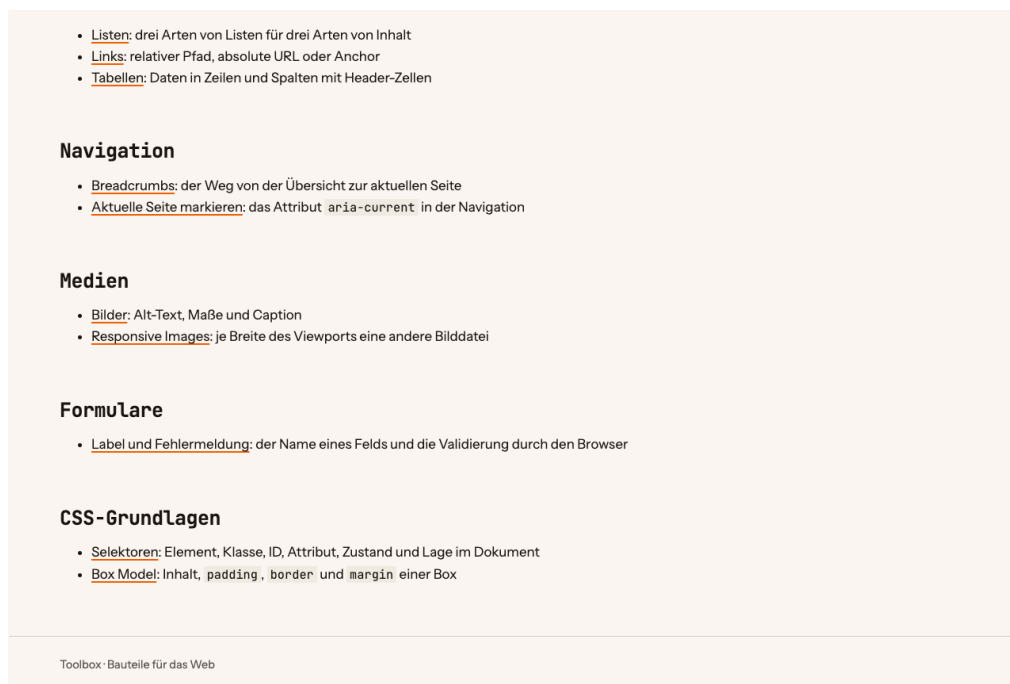
Voraussetzungen: Tabellen & Formulare (Kapitel 1.5)

Lernziele

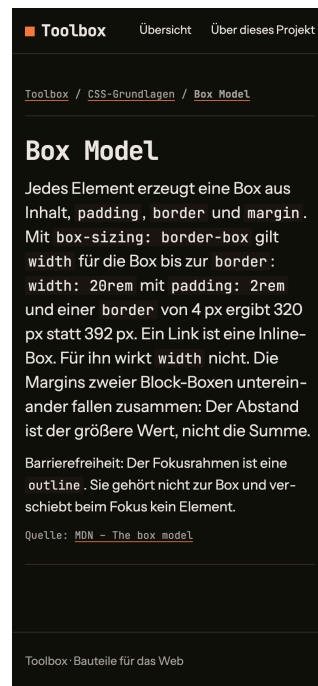
- CSS über `<link>`, `<style>` und das Attribut `style` einbinden und im Tab **Styles** die Quelle jeder Regel nachweisen.
- Für Selektoren aus Typ, Klasse, ID, Attribut, Pseudoklasse und Kombinatoren vorhersagen, welche Elemente sie auswählen, und die Auswahl in der Console nachweisen.
- Breite und Abstände eines Elements aus Inhalt, `padding`, `border` und `margin` abhängig von `box-sizing` und `display` vorhersagen und im Tab **Computed** nachweisen.

1. Vorschau

Zielstand nach diesem Kapitel: Die Übersicht enthält die Kategorie „CSS-Grundlagen“ mit zwei Einträgen.



Die Seite des Eintrags „Box Model“ bei Smartphone-Breite im Dark Mode:



2. Warum

HTML legt fest, was ein Element ist. CSS legt fest, wie der Browser es darstellt. Ein Stylesheet in einer eigenen Datei gilt für jede Seite, die es einbindet. Jede Regel wählt ihre Elemente mit einem Selektor aus und gestaltet deren Box.

Weiterführend: [MDN – What is CSS?](#) · [MDN – Basic CSS selectors](#) · [MDN – The box model](#)

3. Bauen

Ausgangsstand ist die Toolbox aus dem Kapitel [Tabellen & Formulare](#) (Kapitel 1.5): 13 Seiten und das Start-Stylesheet `base.css` in der Version 2.3.0.

Die Abschnitte [Was CSS ist](#) bis [Box Model](#) lesen das Start-Stylesheet und weisen seine Wirkung in den DevTools nach. Sie ändern keine Datei. Neue Dateien entstehen im [Toolbox-Eintrag](#).

3.1 Was CSS ist

[CSS](#) (Cascading Style Sheets) ist eine regelbasierte Sprache. Ein Stylesheet besteht aus [Regeln](#). Jede Regel nennt, welche Elemente sie auswählt und welche Werte für diese Elemente gelten. Das Stylesheet enthält keinen Ablauf von Anweisungen.

Der Browser stellt eine Seite in vier Schritten dar:

1. Er erzeugt aus dem HTML den [DOM](#).
2. Er prüft für jedes Element, welche Regeln zutreffen.
3. Er berechnet für jedes Element den Wert jeder Eigenschaft.
4. Er ordnet die Elemente an und zeichnet sie.

Eine Regel gilt deshalb für jedes passende Element, auch für ein Element, das nach dem Schreiben der Regel in das HTML kommt. Eine Deklaration, die der Browser nicht versteht, ignoriert er ohne Meldung (siehe [Nicht jeder Browser kann alles, Was der Browser verwirft](#) (Kapitel 1.11)).

Das Ergebnis von Schritt 3 zeigt der Tab **Computed** (siehe Browser & DevTools, Elements). Wähle auf `index.html` die Überschrift „Toolbox“ mit dem **Inspect mode** aus. Der Tab **Computed** nennt für `color` einen Wert, obwohl keine Regel in `base.css` die Farbe von `h1` festlegt. Die Regel `html` legt die Farbe fest, und `<h1>` erbt den Wert über `<body>` und `<main>` ([Vererbung](#)).

3.2 CSS einbinden

Eine Seite erhält CSS auf drei Wegen:

Weg	Beispiel	gilt für
extern: Element <code><link></code> im <code><head></code>	<code><link rel="stylesheet" href="assets/css/base.css"></code>	jede Seite, die die Datei einbindet
intern: Element <code><style></code> im <code><head></code>	<code><style> h1 { color: red; } </style></code>	diese Seite
Attribut <code>style</code> an einem Element	<code><h1 style="color: green"></code>	dieses Element

Die Toolbox verwendet nur `<link>`. Eine Änderung in `base.css` wirkt auf allen 13 Seiten. Der Browser lädt die Datei einmal und verwendet sie für jede weitere Seite. `<style>` und das Attribut `style` wiederholen dieselben Regeln in jeder Seite.

Der Tab **Styles** nennt die Quelle jeder Regel (siehe Browser & DevTools, Elements):

Quelle	Angabe im Tab Styles
<code><link></code>	Datei und Zeile, zum Beispiel <code>base.css:272</code>
<code><style></code>	Seite und Zeile, zum Beispiel <code>index.html:10</code>
Attribut <code>style</code>	<code>element.style</code>
Vorgaben des Browsers	user agent stylesheet

Wähle auf `index.html` die Überschrift „Toolbox“ aus. Der Tab **Styles** zeigt die Regel `h1` mit `base.css:272` und die Regel `h1, h2, h3` mit `base.css:260`. Darüber steht die leere Regel `element.style`.

3.3 Eine Regel lesen

Diese Regel aus `base.css` gestaltet die Caption eines Bildes:

```
figcaption {
  margin-block-start: var(--space-xs);
  font-family: var(--font-mono);
  font-size: var(--text-xs);
  color: var(--color-muted);
}
```

Begriff	Im Beispiel	Aufgabe
Selektor	<code>figcaption</code>	wählt die Elemente aus, für die die Regel gilt

Begriff	Im Beispiel	Aufgabe
Deklaration	<code>font-size: var(--text-xs);</code>	legt eine Eigenschaft auf einen Wert fest
Eigenschaft	<code>font-size</code>	Merkmal der Darstellung
Wert	<code>var(--text-xs)</code>	Festlegung für dieses Merkmal

Die geschweiften Klammern umschließen die Deklarationen. Jede Deklaration endet mit `;`. `var(--text-xs)` setzt einen Wert ein, der an einer anderen Stelle des Stylesheets steht. Das Kapitel [CSS-Fundament](#) (Kapitel 1.7) erklärt diese Werte.

Ein Kommentar in CSS steht zwischen `/*` und `*/`. Der Browser ignoriert ihn.

In der Datei ist die Regel um zwei Leerzeichen eingerückt. Sie steht dort in einem Block `@layer base { ... }`. Das Kapitel [CSS-Fundament](#) (Kapitel 1.7) erklärt diese Blöcke.

3.4 Selektoren

Ein [Selektor](#) wählt Elemente nach einem Merkmal aus:

Art	Selektor	wählt aus
Typ	<code>figcaption</code>	jedes Element <code><figcaption></code>
Klasse	<code>.code</code>	jedes Element mit <code>class="code"</code>
ID	<code>#medien</code>	das Element mit <code>id="medien"</code>
Attribut	<code>input[type="radio"]</code>	jedes <code><input></code> mit dem Attribut <code>type</code> und dem Wert <code>radio</code>
Pseudoklasse	<code>a:hover</code>	jedes <code><a></code> unter dem Mauszeiger
alle	<code>*</code>	jedes Element

Eine Pseudoklasse wählt ein Element in einem Zustand oder an einer Stelle aus. `:focus-visible` wählt das Element mit sichtbarem Fokus aus, `article:last-child` jedes `<article>`, das das letzte Element in seinem umgebenden Element ist.

Ein Kombinator verbindet zwei Selektoren nach ihrer Lage im DOM:

Kombinator	Selektor	wählt aus
Leerzeichen (Nachfahre)	<code>nav a</code>	jedes <code><a></code> innerhalb von <code><nav></code> , auch tiefer verschachtelt
<code>></code> (Kind)	<code>body > footer</code>	jedes <code><footer></code> direkt in <code><body></code>
<code>+</code> (nächstes Geschwister)	<code>h1 + p</code>	jedes <code><p></code> , das direkt auf ein <code><h1></code> im selben umgebenden Element folgt

Ein Komma verbindet Selektoren zu einer Liste. Die Regel `main > * + *`, `article > * + *` gilt für jedes Element, auf das einer der Selektoren zutrifft.

`main > * + *` wählt jedes Element direkt in `<main>` aus, außer dem ersten. Ein Selektor kann Typ, Attribut und Pseudoklasse verbinden: `nav a[aria-current="page"]` wählt in einer Navigation den Link auf die aktuelle Seite aus.

Die Toolbox enthält noch keine Klasse und in `base.css` keinen Selektor mit ID. Das Kapitel [CSS-Fundament](#) (Kapitel 1.7) ergänzt die erste Klasse.

Die Console prüft einen Selektor gegen den DOM der Seite:

1. Öffne `index.html` und das Panel **Console**.

2. Gib `document.querySelectorAll("section > h2").length` ein und drücke Enter. Die Console zeigt 5: die Überschriften der fünf Kategorien.

Die Methode `document.querySelectorAll()` nimmt jeden Selektor an, der auch in einer Regel steht.

3.5 Box Model

Der Browser stellt jedes Element als Box dar. Das Box Model beschreibt ihre vier Bereiche von innen nach außen:

Bereich	Eigenschaft	Aufgabe
Inhalt	<code>width, height</code>	Text und Kinder des Elements
Padding (Innenabstand)	<code>padding</code>	Abstand zwischen Inhalt und Border, mit dem Hintergrund des Elements
Border (Rahmen)	<code>border</code>	Linie um das Padding
Margin (Außenabstand)	<code>margin</code>	Abstand zu den Nachbarn, immer durchsichtig

Der Fokusrahmen aus `:focus-visible` ist eine `outline`. Sie liegt außerhalb der Box und zählt nicht zu ihrer Breite.

Weise die Bereiche in den DevTools nach:

1. Stelle in der **Device Toolbar** die Breite 1280 px ein und lade `index.html` neu.
2. Wähle im Panel **Elements** die Zeile `<main>` aus.
3. Öffne den Tab **Computed**. Das Diagramm oben zeigt die vier Bereiche als Rechtecke. Das Padding beträgt oben 40, rechts 16, unten 64 und links 64.

Die Werte stammen aus der Regel `main` und aus dem Block `@media` am Ende von `base.css`.

box-sizing: Mit dem Anfangswert `content-box` gilt `width` nur für den Inhalt, und `padding` und `border` kommen hinzu. `base.css` setzt für jedes Element `border-box` (in der Datei eingerückt ab Zeile 118):

```
*,
*::before,
*::after {
  box-sizing: border-box;
}
```

Mit `border-box` gilt `width` für die Box bis einschließlich Border. Der Tab **Computed** nennt für `<main>` deshalb `width 1280px`, und der Inhalt ist schmaler.

display: Der Wert von `display` legt fest, wie die Box im Textfluss steht. `<p>`, `<section>` und `<main>` sind Block-Boxen mit `display: block`. Eine Block-Box beginnt in einer neuen Zeile und nimmt die ganze verfügbare Breite ein. `<a>`, `<code>` und `<strong>` sind Inline-Boxen mit `display: inline`. Eine Inline-Box steht in der Zeile des umgebenden Texts.

Logische Eigenschaften: `base.css` legt Abstände mit Eigenschaften wie `margin-block-start` und `padding-inline` fest. Sie folgen der Schreibrichtung des Texts. In einem Text von oben nach unten und von links nach rechts gilt:

Logische Eigenschaft	Seite	Der Tab Computed zeigt
margin-block-start	oben	margin-top
margin-block-end	unten	margin-bottom
padding-inline-start	links	padding-left
padding-inline	links und rechts	padding-left, padding-right

4. Verstehen

Lege im Lernprotokoll die Überschrift ## CSS-Grundlagen (Datum) an. Der Ablauf je Experiment ist: Vorhersage notieren, ausführen, Ergebnis und Erklärung notieren.

Die Experimente schreiben Regeln in ein Element `<style>` direkt vor `</head>` in `index.html`. `base.css` bleibt unverändert. Nimm jede Änderung nach dem Experiment zurück, auch das Element `<style>`: Strg/Cmd + Z oder `git restore index.html`.

Predict 1: Drei Orte, eine Eigenschaft

Ändere `index.html` nacheinander in zwei Varianten:

1. Füge vor `</head>` dieses Element ein:

```
<style>
  h1 {
    color: red;
  }
</style>
```

2. Ergänze zusätzlich am Element `<h1>` das Attribut `style="color: green"`.
 - **Vorhersage:** Welche Farbe hat die Überschrift „Toolbox“ je Variante?
 - **Prüfen:**
 1. Lade `index.html` nach jeder Änderung neu.
 2. Wähle die Überschrift aus und lies im Tab **Styles** die Quelle jeder Regel `h1`.

→ [Auflösung 1](#) am Ende des Kapitels

Predict 2: Selektoren zählen

Öffne `patterns/landmarks.html`.

- **Vorhersage:** Wie viele Elemente wählt jeder der fünf Selektoren auf dieser Seite aus: `nav a`, `nav > a`, `li:first-child`, `[aria-current="page"]`, `td code`?
- **Prüfen:** Gib in der Console für jeden Selektor `document.querySelectorAll("...").length` ein, zum Beispiel `document.querySelectorAll("nav a").length`.

→ [Auflösung 2](#) am Ende des Kapitels

Predict 3: width plus padding

Füge vor `</head>` ein Element `<style>` mit dieser Regel für den ersten Absatz nach der Überschrift ein:

```
h1 + p {
  width: 20rem;
  padding: 2rem;
  border: 4px solid;
}
```

- **Vorhersage:** Wie breit ist der Absatz mit Rahmen in Pixel? Wie breit ist er mit der zusätzlichen Deklaration `box-sizing: content-box;` in dieser Regel?
- **Prüfen:**
 1. Lade `index.html` neu und wähle den Absatz aus. Fahre mit dem Mauszeiger über die Zeile im Panel **Elements**. Der Tooltip nennt die Breite.
 2. Ergänze die Deklaration `box-sizing: content-box;` und lade die Seite neu.

→ [Auflösung 3](#) am Ende des Kapitels

Break-it 1: width an einem Link

Füge vor `</head>` ein Element `<style>` mit dieser Regel für die Links der Übersicht ein:

```
main li a {
  width: 20rem;
  padding-block: 1rem;
}
```

- **Vorhersage:** Wie breit sind die Links danach? Verschieben sich die Einträge der Listen?
- **Prüfen:**
 1. Lade `index.html` neu. Fahre im Panel **Elements** mit dem Mauszeiger über die Zeile des Links „Das HTML-Grundgerüst“. Der Tooltip nennt Breite und Höhe.
 2. Ergänze in der Regel die Deklaration `display: inline-block;` und lade die Seite neu.

→ [Auflösung 4](#) am Ende des Kapitels

Predict 4: Zwei Abstände

Füge vor `</head>` ein Element `<style>` mit dieser Regel ein:

```
h2 {
  margin-block-end: 2rem;
}
```

Die Liste unter jeder Überschrift `<h2>` hat aus `base.css` den Abstand `margin-block-start: var(--space-md)`, also 16 px.

- **Vorhersage:** Wie groß ist der Abstand zwischen der Überschrift „Struktur“ und der Liste darunter in Pixel?
- **Prüfen:**
 1. Lade `index.html` neu.
 2. Gib in der Console `document.querySelector("#struktur ul").getBoundingClientRect().top - document.querySelector("#struktur h2").getBoundingClientRect().bottom` ein. Die Console zeigt den Abstand in Pixel.

→ [Auflösung 5](#) am Ende des Kapitels

5. Toolbox-Eintrag

Schreibe zwei weitere Einträge: „Selektoren“ und „Box Model“. Jeder Eintrag erhält eine eigene Seite:

Datei in patterns	Eintrag	Kategorie	Anchor
selektoren.html	Selektoren	CSS-Grundlagen	#css-grundlagen
box-model.html	Box Model	CSS-Grundlagen	#css-grundlagen

Lege beide Seiten mit den Schritten 1 bis 3 aus [Navigation & mehrere Seiten](#), [Übrige Seiten und Übersicht](#) (Kapitel 1.3) an. Die Schritte kopieren `landmarks.html`, ändern `<title>` und `description` und passen die Breadcrumbs an. Ersetze danach in beiden Seiten das ganze `<article>` durch einen neuen Eintrag mit der Überschrift `<h1>`. Jeder Eintrag enthält:

1. **Beschreibung und Zweck** in zwei bis fünf Sätzen, in eigenen Worten.
2. **Barrierefreiheit**: die Auswirkung auf Screenreader und Tastaturbedienung.
3. **Quelle**: einen Link auf die zugehörige MDN-Seite.

Eintrag	Beschreibung nennt	Barrierefreiheit	Quelle
Selektoren	die Arten von Selektoren und die Auflösung von Predict 2	nur Screenreader: [aria-current="page"] gestaltet die aktuelle Seite über das Attribut, das ein Screenreader vorliest	MDN – Basic CSS selectors
Box Model	die vier Bereiche und die Auflösungen von Predict 3 , Break-it 1 und Predict 4	nur Tastatur: Der Fokusrahmen ist eine outline, gehört nicht zur Box und verschiebt kein Element	MDN – The box model

Schreibe spitze Klammern im Text als `<` und `>`, auch im Selektor `nav > a`.

Ergänze in `index.html` nach dem `</section>` des Abschnitts „Formulare“, direkt vor `</main>`, den Abschnitt der neuen Kategorie:

```
<section id="css-grundlagen">
  <h2>CSS-Grundlagen</h2>
  <ul>
    <li><a href="patterns/selektoren.html">Selektoren</a>: Typ, Klasse, ID,
    Attribut, Pseudoklasse und Kombinator</li>
    <li><a href="patterns/box-model.html">Box Model</a>: Inhalt,
    <code>padding</code>, <code>border</code> und <code>margin</code> einer Box</li>
  </ul>
</section>
```

Lade `index.html` neu. Unter „CSS-Grundlagen“ stehen die Links „Selektoren“ und „Box Model“. Prüfe auf beiden neuen Seiten jeden Link der Breadcrumbs.

Die Referenz-Toolbox zeigt die vollständige Toolbox, wie sie am Ende aussehen kann.

Ersetze in `README.md` im Abschnitt `## Struktur` die Zeile `| label-fehlermeldung.html` durch diese drei Zeilen:

```

| ┌ label-fehlermeldung.html
| ┌ selektoren.html
| └ box-model.html

```

Ändere im Satz darunter „alle dreizehn Seiten“ in „alle fünfzehn Seiten“.

1. Setze in `README.md` in der Tabelle `## Fortschritt` „CSS-Grundlagen“ auf „erledigt“ und ergänze die Zeile „CSS-Fundament“ mit „offen“.
2. Übertrage alle Änderungen mit `git add .` in die Staging Area.
3. Prüfe mit `git status`, dass alle geänderten und neuen Dateien unter **Changes to be committed** stehen, auch `learning-notizen.md`.
4. Erstelle einen Commit mit der Nachricht `feat: Einträge zu Selektoren und Box Model ergänzen` und übertrage ihn mit `git push`.

6. Prüfen

- ☐ Der [W3C-Validator](#) zeigt für `index.html`, `patterns/selektoren.html` und `patterns/box-model.html` die Meldung „Document checking completed. No errors or warnings to show.“
- ☐ Keine Seite enthält ein Element `<style>` oder ein Attribut `style`. Die View **Search** findet `<style` und `style=` in keiner Datei.
- ☐ `learning-notizen.md` enthält zu jedem Experiment Vorhersage, Ergebnis und Erklärung.
- ☐ Die Einträge „Selektoren“ und „Box Model“ stehen unter „CSS-Grundlagen“ in der Übersicht.
- ☐ Der Commit `feat: Einträge zu Selektoren und Box Model ergänzen` ist auf GitHub.

Quellen

- [MDN – What is CSS?](#)
- [MDN – <link>](#)
- [MDN – <style>](#)
- [MDN – style](#)
- [MDN – Basic CSS selectors](#)
- [MDN – Attribute selectors](#)
- [MDN – Pseudo-classes and pseudo-elements](#)
- [MDN – Combinators](#)
- [MDN – querySelectorAll\(\)](#)
- [MDN – The box model](#)
- [MDN – box-sizing](#)
- [MDN – display](#)
- [MDN – Mastering margin collapsing](#)
- [MDN – Basic concepts of logical properties and values](#)
- [Chrome DevTools – View and change CSS](#)
- [W3C Nu HTML Checker](#)

Auflösungen

Auflösung 1

Zu Predict 1: Drei Orte, eine Eigenschaft

Variante	Überschrift auf index.html	Erste Regel im Tab Styles
<style>	rot	h1 mit index.html:...
<style> und Attribut style	grün	element.style mit color: green

Das Attribut `style` gilt nur für sein Element und gewinnt gegen jede Regel eines Stylesheets. Der Tab **Styles** stellt die Deklaration aus `<style>` darunter durchgestrichen dar.

Die Regel in `<style>` gewinnt gegen die geerbte Farbe aus der Regel `html`. Ein geerbter Wert gilt nur, wenn keine Regel die Eigenschaft für das Element selbst festlegt.

Auflösung 2

Zu Predict 2: Selektoren zählen

Selektor	Anzahl	Elemente
nav a	5	zwei Links der Main Navigation, drei Links der Breadcrumbs
nav > a	0	keines
li:first-child	2	erster Eintrag der Main Navigation und der Breadcrumbs
[aria-current="page"]	1	Link „Landmarks“ in den Breadcrumbs
td code	4	die Rollen in der zweiten Spalte der Tabelle

`nav > a` wählt nur ein `<a>` aus, das direkt in `<nav>` steht. Jeder Link einer Navigation steht in einem `<li>`, das `<li>` in einer Liste. Das Leerzeichen in `nav a` wählt dagegen Nachfahren in jeder Tiefe aus.

Ein Selektor ohne Treffer erzeugt keine Fehlermeldung. Eine Regel mit `nav > a` bleibt ohne Wirkung, und die Console zeigt nichts an.

Auflösung 3

Zu Predict 3: width plus padding

box-sizing	Breite des Absatzes mit Rahmen
border-box aus base.css	320 px
content-box	392 px

20rem sind bei einer Schriftgröße von 16 px genau 320 px. Mit `border-box` umfasst `width` Inhalt, Padding und Rahmen. Der Inhalt ist $320 - 2 \times 32 - 2 \times 4 = 248$ px breit.

Mit `content-box` gilt `width` nur für den Inhalt. Padding und Rahmen kommen hinzu: $320 + 2 \times 32 + 2 \times 4 = 392$ px. `content-box` ist der Anfangswert. Ohne die Regel aus `base.css` rechnet jede Box so.

Auflösung 4

Zu [Break-it 1: width an einem Link](#)

display	Breite des Links	Einträge der Listen
inline	unverändert, so breit wie der Linktext	unverändert; die Hervorhebung des ausgewählten Links ragt in die Zeilen darüber und darunter
inline-block	320 px	stehen weiter auseinander

Ein `<a>` ist eine Inline-Box. Ihre Breite ergibt sich aus dem Text, und `width` hat keine Wirkung. Die Höhe einer Zeile ergibt sich aus Schriftgröße und `line-height`. Das Padding oben und unten einer Inline-Box zählt dabei nicht mit.

`display: inline-block` macht den Link zu einer Box in der Zeile, für die `width` und `height` gelten. Ihr Padding vergrößert die Zeile.

Auflösung 5

Zu [Predict 4: Zwei Abstände](#)

Der Abstand beträgt 32 px, nicht $32 + 16 = 48$ px.

Die Margins zweier Block-Boxen untereinander fallen zusammen ([Margin Collapsing](#)). Es gilt der größere der beiden Werte. Der Tab **Computed** zeigt für `<h2>` weiterhin `margin-bottom 32px` und für die Liste `margin-top 16px`.

Padding und Rahmen fallen nie zusammen.



1.7 CSS-Fundament

Voraussetzungen: CSS-Grundlagen (Kapitel 1.6)

Lernziele

- Für konkurrierende Regeln vorhersagen, welche Deklaration gilt (Ebene, Spezifität, Reihenfolge), und das Ergebnis im Tab **Styles** nachweisen.
- Ein Stylesheet mit `@import` und `@layer` in Dateien und Ebenen aufteilen und eine Komponente mit Klasse in einer eigenen Ebene ergänzen.
- Werte als Custom Properties definieren und mit `var()` verwenden und vorhersagen, wo eine Änderung wirkt und was ein ungültiger Wert bewirkt.

1. Vorschau

Zielstand nach diesem Kapitel: Das Stylesheet besteht aus sechs Dateien. Die Seite „Über dieses Projekt“ zeigt die Datei `main.css` in einem Code-Block.

Der Eintrag „Breadcrumbs“ als Beispiel

Stylesheet

Das Stylesheet besteht aus sechs Dateien. Die Datei `main.css` legt die Reihenfolge der Ebenen fest und importiert die übrigen Dateien.

```
main.css

@layer tokens, base, layout, components, overrides;

@import url("tokens.css") layer(tokens);
@import url("base.css") layer(base);
@import url("layout.css") layer(layout);
@import url("components.css") layer(components);
@import url("overrides.css") layer(overrides);
```

Werkzeuge

- [Visual Studio Code](#)
- [Chrome DevTools](#)
- [Git](#)

Quellen durchsuchen

Das Formular sucht einen Begriff in einer Quelle der Einträge. Die Suche führt DuckDuckGo aus.

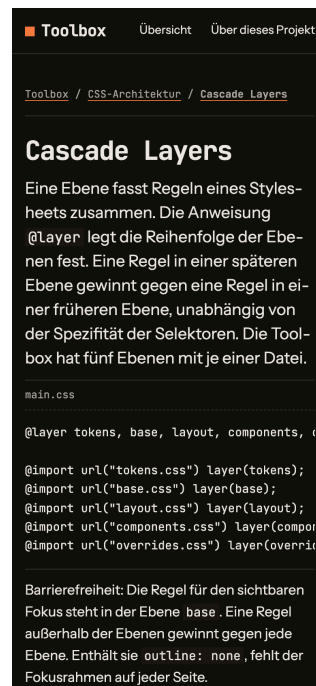
Suchbegriff

Quelle

☒ MDN

☐ W3C WAI

Die Seite des Eintrags „Cascade Layers“ bei Smartphone-Breite im Dark Mode:



```

toolbox/
├── assets/
│   └── css/
│       ├── main.css      ← einzige Datei, die die Seiten einbinden
│       ├── tokens.css    ← Custom Properties
│       ├── base.css      ← Schriften, Reset, HTML-Elemente ohne Klassen
│       ├── layout.css    ← Seitenaufbau
│       ├── components.css ← Komponenten mit einer Klasse
│       └── overrides.css ← eigene Gestaltung (freiwillig)

```

2. Warum

Eine neue Regel bleibt ohne Wirkung, wenn eine andere Regel dieselbe Eigenschaft für das Element festlegt und gewinnt. Die Kaskade entscheidet, welche Deklaration gilt. Ebenen legen diese Entscheidung an einer Stelle des Stylesheets fest, unabhängig von der Spezifität der einzelnen Selektoren. Mit Custom Properties steht jeder Wert der Gestaltung an einer Stelle.

Weiterführend: [MDN – Handling conflicts](#) · [MDN – Cascade layers](#) · [MDN – Using CSS custom properties](#)

3. Bauen

Ausgangsstand ist die Toolbox aus dem Kapitel CSS-Grundlagen (Kapitel 1.6): 15 Seiten und das Start-Stylesheet `base.css` in einer Datei.

Dieses Kapitel enthält kein Stylesheet zum Ersetzen. Ab hier ist das Stylesheet eigener Code der Toolbox.

3.1 Stylesheet aufteilen

Die Datei `base.css` enthält drei Blöcke: `@layer tokens { ... }`, `@layer base { ... }` und `@layer layout { ... }`. Jeder Block ist eine Ebene (Die Kaskade in drei Schritten). Dieser Abschnitt verschiebt jede Ebene in eine eigene Datei. Die Darstellung der Seiten ändert sich dabei nicht.

Lege im Ordner `assets/css` die Dateien `main.css`, `tokens.css`, `layout.css`, `components.css` und `overrides.css` an. Schreibe diesen Inhalt in `main.css`:

```
@layer tokens, base, layout, components, overrides;

@import url("tokens.css") layer(tokens);
@import url("base.css") layer(base);
@import url("layout.css") layer(layout);
@import url("components.css") layer(components);
@import url("overrides.css") layer(overrides);
```

Zeile in <code>main.css</code>	Aufgabe
<code>@layer tokens, base, layout, components, overrides;</code>	legt die Reihenfolge der Ebenen fest
<code>@import url("tokens.css") layer(tokens);</code>	lädt die Datei und ordnet jede ihrer Regeln der Ebene <code>tokens</code> zu

Der Pfad in `url()` ist relativ zur Datei `main.css`. Die Zuordnung zu einer Ebene steht damit in `main.css`. Die Blöcke `@layer ... { }` in `base.css` entfallen deshalb.

Verschiebe danach die Ebenen in ihre Dateien:

1. Schneide in `base.css` die Zeilen zwischen `@layer tokens {` und der schließenden `}` dieses Blocks aus. Füge sie in `tokens.css` ein. Die schließende `}` des Blocks ist die erste Zeile danach, die nur `}` ohne Einrückung enthält.
2. Schneide die Zeilen zwischen `@layer layout {` und der schließenden `}` dieses Blocks aus. Füge sie in `layout.css` ein.
3. Lösche in `base.css` die Zeile `@layer tokens, base, layout;` mit dem Kommentar darüber, die beiden leeren Blöcke `@layer tokens { }` und `@layer layout { }` und die Kommentare über diesen Blöcken.
4. Lösche in `base.css` den Kommentar über `@layer base {`, die Zeile `@layer base {` und die schließende `}` dieses Blocks. Die Regeln des Blocks bleiben in der Datei.
5. Markiere in `tokens.css` und `layout.css` alle Zeilen (`Strg + A`, macOS: `Cmd + A`) und drücke `Shift + Tab`. Markiere in `base.css` die Zeilen ab dem Selektor `*`, bis zum Ende der Datei und drücke `Shift + Tab`. Die Einrückung ändert die Darstellung nicht.
6. Beginne jede der sechs Dateien mit einem Kommentar, der ihre Aufgabe nennt, in den fünf importierten Dateien auch ihre Ebene, zum Beispiel `/* tokens.css - Custom Properties, Ebene tokens */`. Ein Kommentar darf auch vor `@layer` und `@import` stehen. Die Aufgabe jeder Datei nennt der Verzeichnisbaum in der [Vorschau](#). In `base.css` ersetzt der Kommentar den bisherigen Kommentar am Anfang der Datei.
7. Ersetze in allen HTML-Dateien `css/base.css` durch `css/main.css`. Die View **Search** von VS Code ersetzt alle 16 Stellen in einem Schritt (siehe VS Code, Suchen und Ersetzen).

Die Regeln `@font-face` für die Schriften bleiben in `base.css`, danach folgt der Reset: Regeln, die Vorgaben des Browsers für alle Elemente zurücksetzen. Die Datei `components.css` enthält bis

Eine Komponente mit Klasse nur den Kommentar, die Datei overrides .css bis Custom Properties.

Prüfe das Ergebnis:

1. Lade `index.html` neu. Die Darstellung entspricht dem Stand vor dem Aufteilen.
2. Öffne das Panel **Network** und lade die Seite neu. Der Browser lädt sechs CSS-Dateien: `main.css` und die fünf importierten Dateien.

Fehlt die ganze Gestaltung, zeigt **Network** eine fehlgeschlagene Anfrage: Ein Pfad in `main.css` oder im `<link>` ist falsch. Fehlt ein Teil, vergleiche `base.css` mit dem letzten Commit: Ein Klick auf die Datei unter **Source Control** zeigt alte und neue Zeilen nebeneinander.

1. Übertrage alle Änderungen mit `git add .` in die Staging Area.
2. Prüfe mit `git status`, dass alle geänderten und neuen Dateien unter **Changes to be committed** stehen.
3. Erstelle einen Commit mit der Nachricht `refactor: Stylesheet in Dateien und Ebenen aufteilen`.

3.2 Die Kaskade in drei Schritten

Legen mehrere Regeln dieselbe Eigenschaft für ein Element fest, gilt genau eine Deklaration. Der Browser vergleicht die Regeln in drei Schritten. Ein Schritt entscheidet nur, wenn der vorherige Schritt keinen Unterschied ergibt.

Schritt	Es gewinnt	Beispiel aus der Toolbox
1. Ebene	die Regel in der späteren Ebene	<code>main > * + *</code> in layout gegen <code>*</code> in base (Abstand oben)
2. Spezifität	die Regel mit der höheren Spezifität	<code>nav ol a</code> gegen <code>nav a</code> , beide in layout (<code>text-decoration</code>)
3. Reihenfolge	die Regel, die im Quelltext später steht	<code>article > * + *</code> steht in <code>layout.css</code> zweimal (Abstand oben)

Eine Regel außerhalb jeder Ebene gewinnt in Schritt 1 gegen alle Ebenen. In der Toolbox steht deshalb jede Regel in einer Ebene.

Die Spezifität eines Selektors besteht aus drei Zahlen. Der Browser vergleicht sie von links nach rechts.

Zahl	Zählt
erste	IDs (<code>#q</code>)
zweite	Klassen (<code>.code</code>), Attribute (<code>[aria-current="page"]</code>) und Pseudoklassen (<code>:hover</code>)
dritte	Elemente (<code>a</code>) und Pseudoelemente (<code>::before</code>)

Der Selektor `#q` wählt auf `ueber.html` das Feld „Suchbegriff“ aus. Ein Pseudoelement wie `::before` wählt einen Inhalt aus, den das Stylesheet vor dem Inhalt des Elements erzeugt.

Bestimme mit dieser Tabelle die Spezifität von `nav ol a`, `.code` und `nav a[aria-current="page"]`. Vergleiche danach mit der Lösung.

→ Lösung 1 am Ende des Kapitels

Eine Klasse gewinnt gegen jede Anzahl von Elementen: `0,1,0` ist höher als `0,0,3`. Die Zeichen `>` und `+` und der Selektor `*` zählen nicht.

Weise Schritt 3 in den DevTools nach:

1. Öffne `patterns/landmarks.html` und wähle mit dem **Inspect mode** den Absatz aus, der mit „Barrierefreiheit.“ beginnt.
2. Suche im Tab **Styles** die beiden Regeln aus `layout.css`, die `margin-block-start` festlegen. Der Tab nennt zu jeder Regel die Ebene.
3. Lies im Tab **Computed** den Wert von `margin-top`. Er beträgt 12px. Der Tab **Computed** zeigt `margin-block-start` als `margin-top` (siehe [CSS-Grundlagen, Box Model](#) (Kapitel 1.6)).

Beide Regeln stehen in der Ebene `layout`, und beide Selektoren haben die Spezifität 0,0,1. Es gilt die spätere Regel mit dem Wert `var(--space-sm)`. Der Tab **Styles** stellt die Deklaration der früheren Regel durchgestrichen dar.

Die Kaskade hat weitere Schritte: die Vorgaben des Browsers, das Attribut `style` und `!` `important`. Sie stehen in [MDN – Introduction to the CSS cascade](#).

3.3 Eine Komponente mit Klasse

Die bisherigen Regeln wählen Elemente nach Typ, Attribut und Lage aus. Ein Code-Block ist ein `<figure>` wie ein Bild mit Caption, hat aber eine andere Aufgabe und eine andere Darstellung. Eine Klasse unterscheidet beide (siehe [CSS-Grundlagen, Selektoren](#) (Kapitel 1.6)). Der Name einer Klasse nennt die Aufgabe der Komponente, nicht ihr Aussehen.

Ergänze in `components.css` diese Regeln:

```
.code {
  border-block: var(--rule-width) solid var(--color-rule);
}

.code > figcaption {
  margin-block-start: 0;
  padding-block: var(--space-xs);
  border-block-end: var(--rule-width) dashed var(--color-rule);
}

/* Eine lange Zeile scrollt im Code-Block, nicht die Seite. */
.code pre {
  padding-block: var(--space-md);
  overflow-x: auto;
  font-size: var(--text-sm);
  line-height: 1.55;
}

.code code {
  padding-inline: 0;
  background-color: transparent;
  font-size: inherit;
}
```

Füge in `ueber.html` vor der Zeile `<h2>Werkzeuge</h2>` diesen Abschnitt ein:

```

<h2>Stylesheet</h2>
<p>Das Stylesheet besteht aus sechs Dateien. Die Datei <code>main.css</code> legt die Reihenfolge der Ebenen fest und importiert die übrigen Dateien.</p>
<figure class="code">
  <figcaption>main.css</figcaption>
  <pre tabindex="0"><code>@layer tokens, base, layout, components,
overrides;

@import url("tokens.css") layer(tokens);
@import url("base.css") layer(base);
@import url("layout.css") layer(layout);
@import url("components.css") layer(components);
@import url("overrides.css") layer(overrides);</code></pre>
</figure>

```

Ändere im ersten Absatz von ueber.html den Linktext „Start-Stylesheet“ in „Stylesheet“ und ergänze die description der Seite:

```

<meta name="description" content="Zweck, Aufbau, Stylesheet, Werkzeuge und Quellen der Toolbox.">

```

- `<pre>` behält Leerzeichen und Zeilenumbrüche des Quelltexts. Die Zeilen des Codes beginnen deshalb ohne Einrückung am Zeilenanfang.
- `<code>` kennzeichnet den Inhalt als Code.
- `tabindex="0"` nimmt den Code-Block in die Reihenfolge der Taste Tab auf. Mit dem Fokus im Code-Block scrollen die Pfeiltasten eine lange Zeile.
- Enthält der Code das Zeichen `<`, steht im HTML `<`; (siehe [Text & Semantik](#) (Kapitel 1.2)). Das Zeichen `>` steht im Code-Block unverändert.

Für zwei Eigenschaften legen eine Regel aus base und eine Regel aus components verschiedene Werte fest. Bestimme mit den drei Schritten der Kaskade, welcher Wert jeweils gilt. Vergleiche danach mit der Lösung.

Element und Eigenschaft	Regel in base	Regel in components
Caption, margin-block-start	figcaption: var(--space-xs)	.code > figcaption: 0
<code>, background-color	code: var(--color-paper-2)	.code code: transparent

→ [Lösung 2](#) am Ende des Kapitels

Schrift, Schriftgröße und Farbe der Caption legt die Komponente nicht fest. Für sie gelten die Deklarationen aus base.

Weise das Ergebnis nach: Wähle auf ueber.html die Caption „main.css“ aus und lies im Tab **Computed** den Wert von margin-top. Er beträgt 0px, bei der Caption des Bildes 8px.

3.4 Custom Properties

Die Datei tokens.css enthält eine Regel mit dem Selektor `:root`. Er wählt das Element `<html>` aus. Eine Deklaration, deren Name mit `--` beginnt, definiert eine Custom Property. Die Funktion `var()` setzt den Wert in eine andere Deklaration ein. Jedes Element der Seite erbt die Custom Properties von `<html>`.

Die Breite des Fokusrahmens steht bisher als fester Wert in `base.css`. Definiere sie als Custom Property. Ergänze dazu in `tokens.css` nach der Zeile `--rule-width: 1px`; diese Zeile:

```
--focus-width: 3px;
```

Ändere in `base.css` die Regel für den Fokus:

```
:focus-visible {
  outline: var(--focus-width) solid var(--color-accent);
  outline-offset: 2px;
}
```

Lade `index.html` neu und drücke Tab. Der Fokusrahmen ist 3 px breit wie zuvor.

Eine Custom Property kann eine andere verwenden. Die Akzentfarbe enthält den Farbton aus `--accent-hue`:

```
--color-accent: light-dark(
  oklch(66% 0.19 var(--accent-hue)),
  oklch(72% 0.17 var(--accent-hue))
);
```

`--accent-hue` ist ein Winkel im Farbkreis von 0 bis 360. Die Funktionen `light-dark()` und `oklch()` sind nicht Teil dieses Kapitels.

Die Datei `overrides.css` nimmt eigene Gestaltung auf. Schreibe diese Regel in die Datei:

```
:root {
  --accent-hue: 45;
}
```

Die Regel definiert `--accent-hue` ein zweites Mal, mit demselben Selektor wie in `tokens.css`. Die Ebene `overrides` steht in `main.css` zuletzt. Schritt 1 der Kaskade entscheidet: Es gilt der Wert aus `overrides.css`.

Eine eigene Akzentfarbe ist freiwillig:

1. Ändere in `overrides.css` den Wert von `--accent-hue`, zum Beispiel in 150 (Grün) oder 250 (Blau).
2. Lade `ueber.html` neu, klicke in das leere Feld „Suchbegriff“ und drücke Enter. Prüfe die sechs Stellen der folgenden Liste.
 - Quadrat vor der Wordmark
 - Quadrat vor dem Link der aktuellen Seite
 - Linie unter jedem Link im Hauptinhalt
 - Fokusrahmen
 - ausgewählter Radio Button
 - Balken am Feld mit ungültiger Eingabe

Keine dieser Stellen enthält einen eigenen Farbwert. Jede verwendet `var(--color-accent)`.

Für jede weitere eigene Gestaltung gilt dasselbe Vorgehen: Die Regel steht in `overrides.css`, die übrigen Dateien bleiben unverändert. Die folgenden Kapitel ändern `overrides.css` nicht. Ihre Messwerte und die dieses Kapitels gelten für die Toolbox ohne eigene Gestaltung und für die

Schriftgröße 16 px in den Einstellungen des Browsers. Weicht ein Ergebnis ab, setze die Zeile mit `overrides.css` in `main.css` in einen Kommentar (`/* davor, */ dahinter`) und lade die Seite neu.

3.5 Commit

1. Übertrage alle Änderungen mit `git add .` in die Staging Area.
2. Prüfe mit `git status`, dass alle geänderten Dateien unter **Changes to be committed** stehen.
3. Erstelle einen Commit mit der Nachricht `feat: Komponente Code-Block ergänzen` und übertrage beide Commits mit `git push`.

4. Verstehen

Lege im Lernprotokoll die Überschrift `## CSS-Fundament (Datum)` an. Der Ablauf je Experiment ist: Vorhersage notieren, ausführen, Ergebnis und Erklärung notieren.

Nimm jede Änderung nach dem Experiment zurück: `Strg/Cmd + Z` oder `git restore` mit dem Pfad der Datei.

Predict 1: Reihenfolge und Spezifität

Die Datei `layout.css` legt `text-decoration` für Links in einer Navigation in drei Regeln fest. Zuerst steht `nav a` mit dem Wert `none`, weiter unten `nav ol a` mit dem Wert `underline`. Die Regel `nav a:hover` dazwischen gilt nur für den Link unter dem Mauszeiger und bleibt unverändert. Ändere die Datei nacheinander in drei Varianten:

1. Verschiebe die Regel `nav ol a` vor die Regel `nav a`.
 2. Ändere zusätzlich den Selektor `nav ol a` in `nav a`.
 3. Verschiebe die geänderte Regel zurück an ihre ursprüngliche Stelle.
- **Vorhersage:** Welche Links sind je Variante unterstrichen: die Links der Main Navigation, die Links der Breadcrumbs?
 - **Prüfen:** Lade `patterns/landmarks.html` nach jeder Änderung neu. Der Mauszeiger steht dabei nicht über einem Link.

→ [Auflösung 3](#) am Ende des Kapitels

Predict 2: Ebenen tauschen

Ändere in `main.css` die erste Zeile in `@layer tokens, layout, base, components, overrides;`. Die Zeilen mit `@import` bleiben unverändert.

- **Vorhersage:** Was ändert sich an der Darstellung von `index.html`?
- **Prüfen:** Lade `index.html` neu. Wähle den Absatz unter der Überschrift „Toolbox“ aus und lies im Tab **Computed** den Wert von `margin-top`.

→ [Auflösung 4](#) am Ende des Kapitels

Break-it 1: Regel ohne Ebene

Füge am Ende von `main.css` diese Regel ein:

```
* {
  outline: none;
}
```

Den Fokusrahmen legt die Regel `:focus-visible` in `base.css` fest. Ihre Spezifität ist 0,1,0.

- **Vorhersage:** Ist der Fokusrahmen nach der Änderung sichtbar?
- **Prüfen:**
 1. Lade `index.html` neu und drücke Tab.
 2. Verschiebe die Regel an das Ende von `base.css`.
 3. Lade die Seite neu und drücke Tab.

→ [Auflösung 5](#) am Ende des Kapitels

Break-it 2: Regel vor @import

Füge in `main.css` vor dem ersten `@import` diese Regel ein:

```
h1 {
  text-transform: uppercase;
}
```

- **Vorhersage:** Welche Meldung zeigt das Panel **Console**? Wie viele CSS-Dateien lädt der Browser?
- **Prüfen:**
 1. Öffne die Panels **Console** und **Network** und lade `index.html` neu.
 2. Verschiebe die Regel zwischen den zweiten und den dritten `@import`.
 3. Lade die Seite neu.

→ [Auflösung 6](#) am Ende des Kapitels

Break-it 3: Ungültiger Wert in einer Custom Property

Ändere in `overrides.css` den Wert von `--accent-hue` in orange.

- **Vorhersage:** Welche Farbe haben die sechs Stellen aus [Custom Properties](#)?
- **Prüfen:** Öffne das Panel **Console**. Lade `ueber.html` neu, klicke in das leere Feld „Suchbegriff“ und drücke Enter.

→ [Auflösung 7](#) am Ende des Kapitels

Predict 3: Custom Property an einem Element überschreiben

Füge am Ende von `layout.css` diese Regel ein:

```
main {
  --accent-hue: 250;
}
```

Verwende in beiden Deklarationen den Farbton 150 statt 250, wenn deine Akzentfarbe bereits den Farbton 250 hat.

- **Vorhersage:** Welche Farbe haben die Linien unter den Links im Hauptinhalt? Welche Farbe hat das Quadrat vor der Wordmark?
- **Prüfen:**
 1. Lade `index.html` neu.
 2. Ersetze die Deklaration durch `--color-accent: oklch(66% 0.19 250);`.
 3. Lade die Seite neu.

→ [Auflösung 8](#) am Ende des Kapitels

Predict 4: Zwei Tippfehler

Die Regel `body > footer` in `layout.css` enthält die Deklaration `color: var(--color-muted);`. Ändere die Regel nacheinander in drei Varianten:

1. Füge nach dieser Deklaration die Zeile `color: grau;` ein.
 2. Ersetze die eingefügte Zeile durch `color: var(--color-mutd);`.
 3. Ersetze sie durch `color: var(--color-mutd, red);`.
- **Vorhersage:** Welche Farbe hat der Text im Footer je Variante?
 - **Prüfen:** Lade `index.html` nach jeder Änderung neu und vergleiche den Text im Footer mit dem Text im Hauptinhalt.

→ [Auflösung 9](#) am Ende des Kapitels

5. Toolbox-Eintrag

Schreibe zwei weitere Einträge: „Cascade Layers“ und „Custom Properties“. Jeder Eintrag erhält eine eigene Seite:

Datei in patterns	Eintrag	Kategorie	Anchor
<code>cascade-layers.html</code>	Cascade Layers	CSS-Architektur	<code>#css-architektur</code>
<code>custom-properties.html</code>	Custom Properties	CSS-Architektur	<code>#css-architektur</code>

Lege beide Seiten mit den Schritten 1 bis 3 aus [Navigation & mehrere Seiten](#), [Übrige Seiten und Übersicht](#) (Kapitel 1.3) an. Die Schritte kopieren `landmarks.html`, ändern `<title>` und `description` und passen die Breadcrumbs an. Ersetze danach in beiden Seiten das ganze `<article>` durch einen neuen Eintrag mit der Überschrift `<h1>`. Jeder Eintrag enthält:

1. **Beschreibung und Zweck** in zwei bis fünf Sätzen, in eigenen Worten.
2. **Code:** einen Code-Block mit einem Ausschnitt aus dem eigenen Stylesheet.
3. **Barrierefreiheit:** die Auswirkung auf Screenreader und Tastaturbedienung.
4. **Quelle:** einen Link auf die zugehörige MDN-Seite.

Eintrag	Code-Block mit Caption	Barrierefreiheit	Quelle
Cascade Layers	die ganze Datei <code>main.css</code>	nur Tastatur: Auflösung von Break-it 1	MDN – Cascade layers
Custom Properties	zwei Code-Blöcke: Regel <code>:root</code> nur mit der Zeile <code>--focus-width</code> <code>tokens.css</code> (Auszug);	nur Tastatur: Auflösung von Break-it 3	MDN – Using CSS custom properties

Eintrag	Code-Block mit Caption	Barrierefreiheit	Quelle
	Regel :focus-visible base.css (Auszug)		

```

<article>
  <h1>Cascade Layers</h1>
  <p>Eine Ebene fasst Regeln eines Stylesheets zusammen. ...</p>
  <figure class="code">
    <figcaption>main.css</figcaption>
    <pre tabindex="0"><code>...</code></pre>
  </figure>
  <p>Barrierefreiheit: ...</p>
  <p>Quelle: <a href="https://developer.mozilla.org/...">MDN - ...</a></p>
</article>

```

Ein Eintrag zu CSS hat damit vier Teile. Ergänze in ueber.html in der Liste unter „Aufbau eines Eintrags“ nach Beschreibung und Zweck diese Zeile:

```
<li>Code (Einträge zu CSS)</li>
```

Ergänze in index.html nach dem </section> des Abschnitts „CSS-Grundlagen“, direkt vor </main>, den Abschnitt der neuen Kategorie:

```

<section id="css-architektur">
  <h2>CSS-Architektur</h2>
  <ul>
    <li><a href="patterns/cascade-layers.html">Cascade Layers</a>: die
    Reihenfolge der Ebenen eines Stylesheets</li>
    <li><a href="patterns/custom-properties.html">Custom Properties</a>:
    ein Wert unter einem Namen, eingesetzt mit <code>var()</code></li>
  </ul>
</section>

```

Lade index.html neu. Unter „CSS-Architektur“ stehen die Links „Cascade Layers“ und „Custom Properties“. Prüfe auf beiden neuen Seiten jeden Link der Breadcrumbs.

Die Referenz-Toolbox zeigt die vollständige Toolbox, wie sie am Ende aussehen kann.

Ersetze in README.md den Abschnitt ## Struktur durch diesen Inhalt:

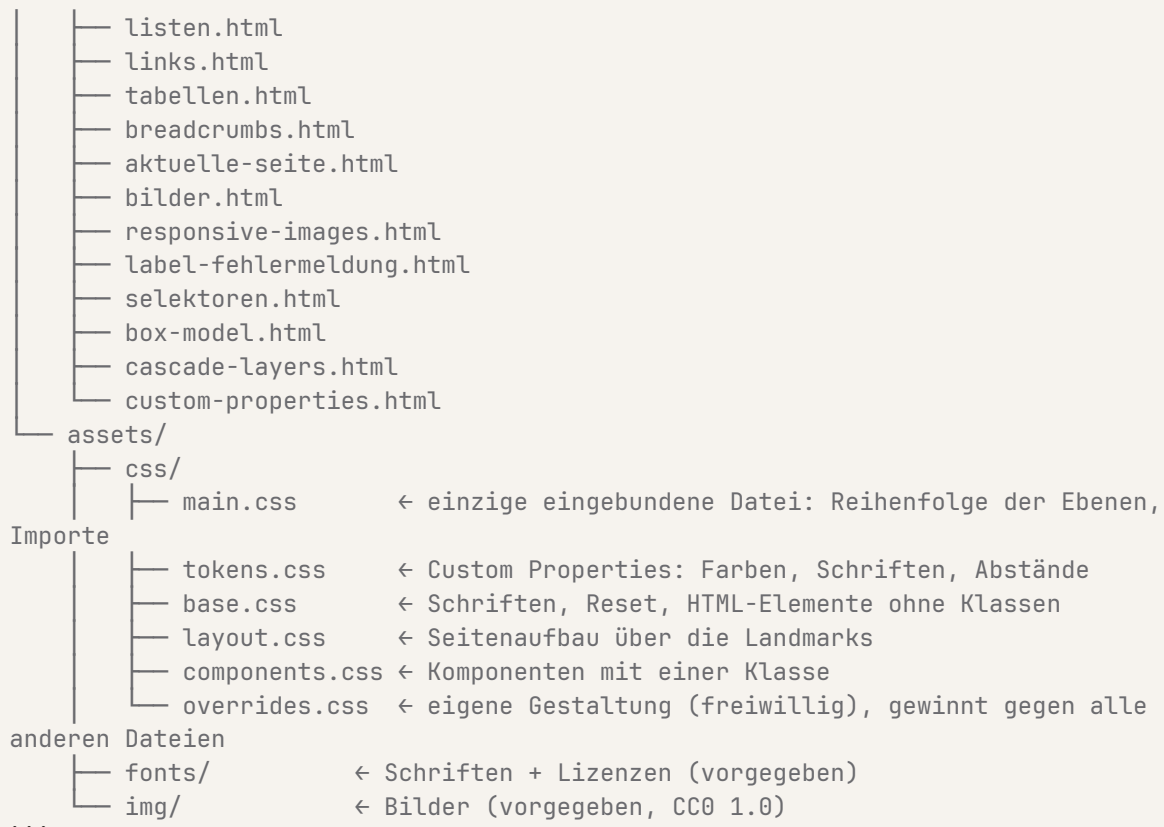
```

## Struktur

...

toolbox/
├── README.md           ← diese Datei
├── learning-notizen.md ← Lernprotokoll: Vorhersagen und Beobachtungen
├── .gitignore          ← Systemdateien, die nicht ins Repository gehören
├── index.html          ← Übersicht mit Links auf alle Einträge
├── ueber.html          ← Über dieses Projekt
├── patterns/           ← je Eintrag eine Seite
│   ├── html-grundgeruest.html
│   ├── landmarks.html
│   └── ueberschriften.html

```



1. Setze in README.md in der Tabelle ## Fortschritt „CSS-Fundament“ auf „erledigt“ und ergänze die Zeile „Layout“ mit „offen“.
2. Übertrage alle Änderungen mit `git add .` in die Staging Area.
3. Prüfe mit `git status`, dass alle geänderten Dateien unter **Changes to be committed** stehen, auch learning-notizen.md.
4. Erstelle einen Commit mit der Nachricht feat: Einträge zu Cascade Layers und Custom Properties ergänzen und übertrage ihn mit `git push`.

6. Prüfen

- ☐ Der [W3C-Validator](#) zeigt für jede der siebzehn Seiten die Meldung „Document checking completed. No errors or warnings to show.“
- ☐ Jede Seite bindet nur `main.css` ein. Das Panel **Network** zeigt sechs CSS-Dateien und keine fehlgeschlagene Anfrage.
- ☐ `main.css` enthält keine Regel. Jede Regel der fünf anderen Dateien gehört über `layer()` zu einer Ebene.
- ☐ Die Funktion `oklch()` und die Namen der Schriften kommen nur in `tokens.css`, in `overrides.css` und in den Regeln `@font-face` vor.

- ☐ Tastatur: Tab erreicht den Code-Block in ueber.html, und der Fokusrahmen ist sichtbar.
- ☐ Viewport: Bei 320 px Breite in der **Device Toolbar** verschieben die Pfeiltasten mit dem Fokus im Code-Block von ueber.html die lange Zeile.
- ☐ Learning-notizen.md enthält zu jedem Experiment Vorhersage, Ergebnis und Erklärung.
- ☐ Die Einträge „Cascade Layers“ und „Custom Properties“ stehen unter „CSS-Architektur“ in der Übersicht.
- ☐ Die Commits refactor: Stylesheet in Dateien und Ebenen aufteilen, feat: Komponente Code-Block ergänzen und feat: Einträge zu Cascade Layers und Custom Properties ergänzen sind auf GitHub.

Quellen

- [MDN – Handling conflicts](#)
- [MDN – Introduction to the CSS cascade](#)
- [MDN – Specificity](#)
- [MDN – Cascade layers](#)
- [MDN – @layer](#)
- [MDN – @import](#)
- [MDN – Basic CSS selectors](#)
- [MDN – Using CSS custom properties](#)
- [MDN – var\(\)](#)
- [MDN – The Preformatted Text element](#)
- [Chrome DevTools – View and change CSS](#)
- [W3C Nu HTML Checker](#)

Auflösungen

Lösung 1

Zu [3.2 Die Kaskade in drei Schritten](#)

Selektor	Spezifität
*	0,0,0
a	0,0,1
main > * + *	0,0,1
nav ol a	0,0,3
.code	0,1,0
nav a[aria-current="page"]	0,1,2
#q	1,0,0

Lösung 2

Zu [3.3 Eine Komponente mit Klasse](#)

Es gilt 0 für die Caption und transparent für `<code>`. Die Ebene components steht in `main.css` nach der Ebene base. Schritt 1 der Kaskade entscheidet.

Auflösung 3

Zu [Predict 1: Reihenfolge und Spezifität](#)

Variante	Main Navigation	Breadcrumbs
1: nav ol a vor nav a	nicht unterstrichen	unterstrichen
2: zweimal nav a, underline zuerst	nicht unterstrichen	nicht unterstrichen
3: zweimal nav a, underline zuletzt	unterstrichen	unterstrichen

Variante 1 ändert nichts. `nav ol a` hat die Spezifität 0,0,3 und gewinnt gegen `nav a` mit 0,0,2, unabhängig von der Reihenfolge.

In den Varianten 2 und 3 haben beide Regeln denselben Selektor und damit dieselbe Spezifität. Die Reihenfolge entscheidet: Es gilt die spätere Regel, und sie gilt für beide Navigationen.

Auflösung 4

Zu [Predict 2: Ebenen tauschen](#)

Die Abstände zwischen Überschriften, Absätzen, Listen und Abschnitten fehlen. `margin-top` beträgt 0px statt 16px. Schriften, Farben und der Aufbau des Headers bleiben.

`base.css` enthält die Regel `* { margin: 0; }`. `layout.css` legt die Abstände mit `main > * + *` fest. Der Selektor `*` hat die Spezifität 0,0,0 und `main > * + *` die Spezifität 0,0,1. Nach dem Tausch steht die Ebene base nach der Ebene layout. Schritt 1 entscheidet vor Schritt 2: Die Regel mit der niedrigeren Spezifität gewinnt.

Die Reihenfolge der Ebenen legt die Anweisung `@layer` fest. Die Reihenfolge der Zeilen mit `@import` ändert sie nicht.

Auflösung 5

Zu Break-it 1: Regel ohne Ebene

Ort der Regel	Fokusrahmen
Ende von <code>main.css</code>	fehlt
Ende von <code>base.css</code>	sichtbar

In `main.css` steht die Regel in keiner Ebene. Eine Regel außerhalb jeder Ebene gewinnt gegen alle Ebenen. Die Spezifität entscheidet dabei nicht.

In `base.css` steht die Regel in derselben Ebene wie `:focus-visible`. Dort entscheidet die Spezifität, und 0,1,0 gewinnt gegen 0,0,0.

Ohne Fokusrahmen erkennt eine Person mit Tastatur nicht, welches Element den Fokus hat. CSS aus einer fremden Quelle, das ohne Ebene im Stylesheet steht, gewinnt gegen jede Regel der Toolbox.

Auflösung 6

Zu Break-it 2: Regel vor @import

Ort der Regel	Geladene CSS-Dateien	Darstellung
vor dem ersten <code>@import</code>	<code>main.css</code>	ohne Gestaltung, Überschrift in Großbuchstaben
zwischen dem zweiten und dem dritten <code>@import</code>	<code>main.css</code> , <code>tokens.css</code> , <code>base.css</code>	Schriften und Farben vorhanden, Aufbau der Seite und Abstände fehlen

Das Panel **Console** zeigt in beiden Fällen keine Meldung. Das Panel **Network** enthält keine fehlgeschlagene Anfrage.

`@import` ist nur vor allen Regeln einer Datei gültig. Davor darf nur die Anweisung `@layer` mit der Liste der Ebenen stehen. Der Browser ignoriert jedes `@import` nach einer Regel und fordert die Datei nicht an.

Ein falscher Pfad im `@import` erzeugt dagegen eine Meldung im Panel **Console**, wie ein falscher Pfad im `<link>`.

Auflösung 7

Zu Break-it 3: Ungültiger Wert in einer Custom Property

Stelle	Darstellung
Quadrat vor der Wordmark und vor der aktuellen Seite	fehlt
Linie unter Links	Textfarbe
Fokusrahmen	fehlt
ausgewählter Radio Button	Standardfarbe des Browsers
Balken am Feld mit ungültiger Eingabe	Textfarbe

Das Panel **Console** zeigt keine Meldung.

Eine Custom Property nimmt jeden Wert an. Der Browser prüft den Wert erst in der Deklaration, die ihn mit `var()` einsetzt. `oklch(66% 0.19 orange)` ist keine Farbe. Jede Deklaration mit `var(--color-accent)` ist deshalb ungültig.

Die Eigenschaft erhält dann den geerbten Wert, wenn sie vererbt wird, sonst ihren Anfangswert: `background-color` den Wert `transparent`, `text-decoration-color` die Textfarbe, `outline` den Wert `none`.

Nach einem ungültigen Wert für `--accent-hue` in `overrides.css` fehlt der Fokusrahmen auf jeder Seite.

Auflösung 8

Zu Predict 3: Custom Property an einem Element überschreiben

Deklaration in <code>main</code>	Linien im Hauptinhalt	Quadrat vor der Wordmark
<code>--accent-hue: 250;</code>	unverändert	unverändert
<code>--color-accent: oklch(66% 0.19 250);</code>	blau (mit 150: grün)	unverändert

Die erste Variante ändert nichts. Der Browser berechnet `--color-accent` am Element `<html>`, an dem die Custom Property definiert ist. Dort hat `--accent-hue` den Wert aus `overrides.css`. `<main>` erbt den berechneten Wert von `--color-accent`, nicht die Formel.

Die zweite Variante definiert `--color-accent` am Element `<main>` neu. Der Wert gilt für `<main>` und für jedes Element darin. Der Header liegt außerhalb von `<main>` und behält den Wert von `<html>`.

Auflösung 9

Zu Predict 4: Zwei Tippfehler

Variante	Farbe des Texts im Footer
<code>color: grau;</code>	gedämpft wie zuvor
<code>color: var(--color-mutd);</code>	Textfarbe des Hauptinhalts
<code>color: var(--color-mutd, red);</code>	rot

`grau` ist kein Wert der Eigenschaft `color`. Der Browser verwirft die Deklaration beim Lesen der Datei, und die vorherige Deklaration gilt.

Eine Deklaration mit `var()` verwirft der Browser beim Lesen nicht. Sie ersetzt die vorherige Deklaration. Erst beim Einsetzen fehlt der Wert: Die Custom Property `--color-mutd` ist nicht definiert. `color` erhält dann den geerbten Wert, die Textfarbe der Seite.

Der zweite Wert in `var()` ist ein Ersatzwert. Er gilt, wenn die Custom Property nicht definiert ist.



1.8 Layout

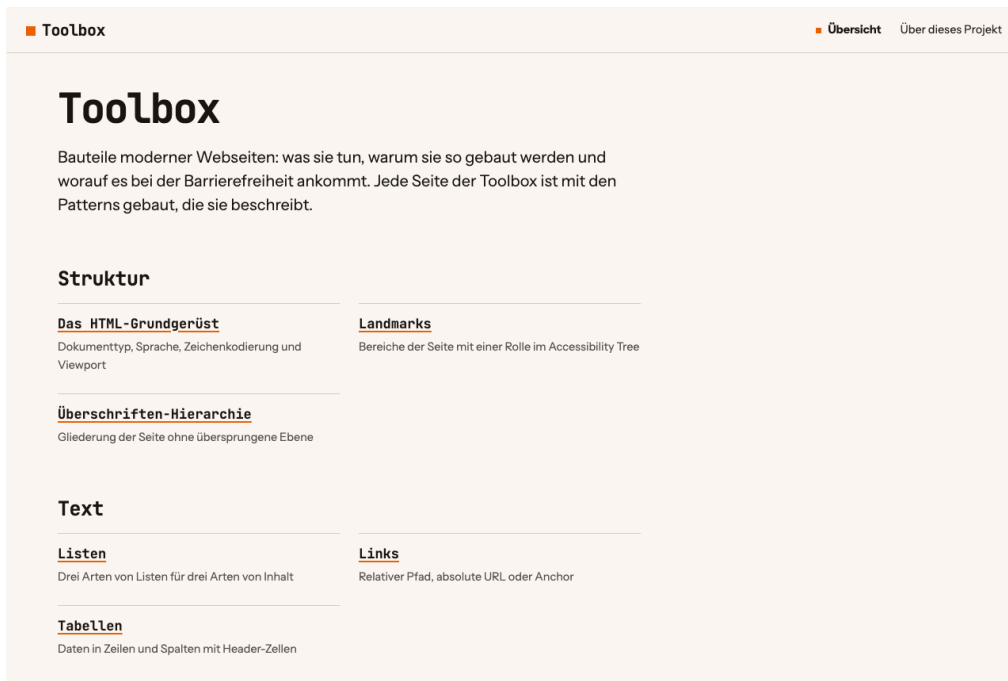
Voraussetzungen: CSS-Fundament (Kapitel 1.7)

Lernziele

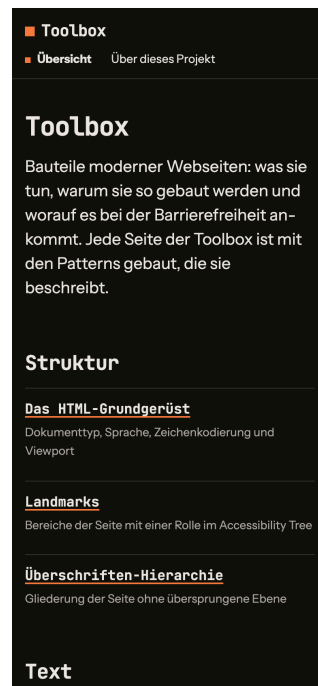
- Für einen Flex-Container vorhersagen, wie Umbruch, Verteilung des freien Platzes und Schrumpfen die Items anordnen, und die Anordnung mit dem Overlay der DevTools nachweisen.
- Ein Grid mit Spalten, Zeilen und Grid Areas definieren und eine Liste bauen, deren Spaltenzahl sich ohne Media Query an den Platz anpasst.
- Zwischen Flexbox und Grid entscheiden und begründen, warum die Reihenfolge im Quelltext die Reihenfolge für Tastatur und Screenreader bleibt.

1. Vorschau

Zielstand nach diesem Kapitel: Die Übersicht zeigt die Einträge jeder Kategorie in zwei Spalten.



Dieselbe Seite bei Smartphone-Breite im Dark Mode. Die Einträge stehen in einer Spalte:



2. Warum

Ohne Layout stehen die Elemente einer Seite untereinander, in der Reihenfolge des Quelltexts. Flexbox ordnet Elemente in einer Richtung an, Grid in Spalten und Zeilen. Beide Verfahren passen die Anordnung an den vorhandenen Platz an.

Weiterführend: [MDN – Introduction to CSS layout](#) · [MDN – Flexbox](#) · [MDN – CSS grid layout](#)

3. Bauen

Ausgangsstand ist die Toolbox aus dem Kapitel [CSS-Fundament](#) (Kapitel 1.7): 17 Seiten und ein Stylesheet in sechs Dateien.

Die Datei `layout.css` enthält drei Flex-Container und ein Grid, die bisher kein Kapitel erklärt. [Flexbox lesen](#) und [Grid lesen](#) erklären diese Regeln. [Seitenlayout mit Grid Areas](#) und [Pattern-Liste](#) ergänzen neue Regeln.

Die Messwerte dieses Kapitels gelten für die Toolbox ohne eigene Gestaltung in `overrides.css` und für die Schriftgröße 16 px in den Einstellungen des Browsers. Die Breite des Viewports stellt die Device Toolbar ein (siehe Browser & DevTools, Device Toolbar).

3.1 Flexbox lesen

Die Regel `body > header` in `layout.css` enthält diese Deklarationen:

```
display: flex;
flex-wrap: wrap;
align-items: center;
justify-content: space-between;
gap: var(--space-xs) var(--space-lg);
```

`display: flex` legt den Header als Flex-Container fest ([Flexbox](#)). Seine Kinder sind die Items: der Absatz mit der Wordmark und die Main Navigation. Die Items stehen nebeneinander auf der Hauptachse, in der Reihenfolge des Quelltexts. Im Header verläuft die Hauptachse waagrecht von links nach rechts, die Querachse senkrecht dazu.

Deklaration	Wirkung
<code>flex-wrap: wrap</code>	Ein Item wechselt in eine neue Zeile, wenn der Platz in der Zeile nicht reicht.
<code>justify-content: space-between</code>	Der freie Platz der Zeile liegt zwischen den Items. Das erste Item steht am Anfang, das letzte am Ende.
<code>align-items: center</code>	Die Items stehen auf der Querachse in der Mitte.
<code>gap: var(--space-xs) var(--space-lg)</code>	Das Gap ist der Abstand zwischen den Items. Der erste Wert gilt zwischen den Zeilen, der zweite zwischen den Items einer Zeile.

Zwei weitere Regeln legen Flex-Container fest: `nav ul` für die Liste der Main Navigation und `nav ol` für die Breadcrumbs. Die Items sind dort die Elemente `<li>`.

Weise die Anordnung in den DevTools nach:

1. Öffne `index.html` und das Panel **Elements**. Neben dem Element `<header>` steht das Badge `flex`.
2. Klicke auf das Badge. Das Overlay zeigt die Items und den freien Platz zwischen ihnen.
3. Stelle die Breite des Viewports auf 1280 px. Die Wordmark steht am linken Rand, die Navigation am rechten Rand.
4. Stelle die Breite auf 320 px. Die Navigation steht in einer zweiten Zeile.

Die weiteren Funktionen des Overlays nennt [Chrome DevTools – Inspect and debug CSS flexbox layouts](#).

3.2 Grid lesen

Die Regel `body` in `layout.css` beginnt mit diesen Zeilen:

```
display: grid;
/* minmax(0, 1fr): Die Spalte wird nie breiter als der Viewport. */
grid-template-columns: minmax(0, 1fr);
grid-template-rows: auto 1fr auto;
```

`display: grid` legt `<body>` als Grid-Container fest. Seine Kinder sind die Items: Header, Hauptinhalt und Footer. Ein Grid besteht aus Spalten und Zeilen. Ohne weitere Angabe belegt jedes Item die nächste freie Zelle, in der Reihenfolge des Quelltexts.

Die Einheit `fr` ist ein Anteil am freien Platz des Containers. Die Funktion `minmax()` nennt die kleinste und die größte Größe einer Spalte oder Zeile.

Deklaration	Wirkung
<code>grid-template-columns: minmax(0, 1fr)</code>	Das Grid hat eine Spalte. Ihre Breite liegt zwischen 0 und dem gesamten freien Platz.
<code>grid-template-rows: auto 1fr auto</code>	Das Grid hat drei Zeilen. Die erste und die dritte Zeile sind so hoch wie ihr Inhalt. Die mittlere Zeile erhält den freien Platz.

Deklaration	Wirkung
<code>min-block-size: 100dvh</code> (letzte Zeile der Regel)	Das Grid ist mindestens so hoch wie der Viewport. 100dvh ist die ganze Höhe des Viewports.

Weise die Zeilen in den DevTools nach:

1. Öffne `patterns/listen.html` bei 1280 px Breite und das Panel **Elements**. Neben dem Element `<body>` steht das Badge `grid`.
2. Klicke auf das Badge und aktiviere im Tab **Layout** die Option **Show track sizes** (siehe Browser & DevTools, Elements).
3. Lies die Höhe der drei Zeilen. Die erste Zeile ist 57px hoch, die dritte 72px. Die mittlere Zeile belegt den Rest der Höhe des Viewports.

Ist der Viewport höher als der Inhalt der Seite, steht der Footer am unteren Rand des Viewports. Die weiteren Funktionen des Overlays nennt [Chrome DevTools – Inspect CSS grid layouts](#).

3.3 Seitenlayout mit Grid Areas

Ein Grid Area ist ein Bereich des Grids mit einem Namen. Die Eigenschaft `grid-template-areas` legt die Namen im Container fest. Die Eigenschaft `grid-area` ordnet ein Item einem Grid Area zu, unabhängig von der Position des Items im Quelltext.

Ergänze in `layout.css` in der Regel `body` die Deklaration `grid-template-areas`. Die Regel lautet danach:

```
body {
  display: grid;
  /* minmax(0, 1fr): Die Spalte wird nie breiter als der Viewport. */
  grid-template-columns: minmax(0, 1fr);
  grid-template-rows: auto 1fr auto;
  grid-template-areas:
    "header"
    "main"
    "footer";
  min-block-size: 100dvh;
}
```

Jede Zeichenkette in Anführungszeichen, zum Beispiel `"header"`, beschreibt eine Zeile des Grids. Jeder Name in einer Zeichenkette steht für eine Spalte dieser Zeile.

Ergänze in `layout.css` in drei Regeln je eine Deklaration als erste Zeile. Jede Regel steht direkt unter dem Kommentar, der mit `Header`, `Hauptinhalt` oder `Footer` beginnt. Die Regeln im Block `@media` am Ende der Datei bleiben unverändert.

Regel	Neue Deklaration
<code>body > header</code>	<code>grid-area: header;</code>
<code>main</code>	<code>grid-area: main;</code>
<code>body > footer</code>	<code>grid-area: footer;</code>

Ergänze im Kommentar am Anfang von `layout.css` direkt vor dem `*/` diesen Satz: „body ist ein Grid mit drei Grid Areas. Header, Main Navigation und Breadcrumbs sind Flex-Container.“ Der Satz steht damit innerhalb des Kommentars.

Prüfe das Ergebnis:

1. Lade `index.html` neu. Die Darstellung entspricht dem Stand vor der Änderung.
2. Klicke im Panel **Elements** auf das Badge grid neben `<body>` und aktiviere im Tab **Layout** die Option **Show area names**. Das Overlay zeigt die Namen header, main und footer.

Mit Grid Areas steht die Anordnung der Seite an einer Stelle des Stylesheets. Eine andere Anordnung, zum Beispiel mit einer Sidebar, ändert nur die Regel body.

3.4 Pattern-Liste

Die Übersicht enthält je Kategorie eine Liste. Ein Eintrag besteht bisher aus einer Zeile: Link, Doppelpunkt, Beschreibung. Dieser Abschnitt trennt Link und Beschreibung und ordnet die Einträge in Spalten an.

Ändere in `index.html` den Abschnitt „Struktur“:

```
<section id="struktur">
  <h2>Struktur</h2>
  <ul class="pattern-list">
    <li>
      <a href="patterns/html-grundgeruest.html">Das HTML-Grundgerüst</a>
      <p>Dokumenttyp, Sprache, Zeichenkodierung und Viewport</p>
    </li>
    <li>
      <a href="patterns/landmarks.html">Landmarks</a>
      <p>Bereiche der Seite mit einer Rolle im Accessibility Tree</p>
    </li>
    <li>
      <a href="patterns/ueberschriften.html">Überschriften-Hierarchie</a>
      <p>Gliederung der Seite ohne übersprungene Ebene</p>
    </li>
  </ul>
</section>
```

Ändere die Listen der sechs anderen Kategorien (Text, Navigation, Medien, Formulare, CSS-Grundlagen, CSS-Architektur) auf dieselbe Weise. Elemente `<code>` in einer Beschreibung bleiben erhalten:

1. Ergänze `class="pattern-list"` am Element `<ul>`.
2. Entferne den Doppelpunkt nach dem Link.
3. Setze die Beschreibung in ein Element `<p>` und beginne sie mit einem Großbuchstaben.

Ergänze am Ende von `components.css` diese Regeln:

```
/* -----
   Pattern-Liste: Einträge der Übersicht in Spalten
   Die Spaltenzahl folgt dem Platz im Container.
   ----- */

.pattern-list {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(16rem, 1fr));
  gap: var(--space-lg);
  padding: 0;
  list-style: none;
```

```

}

.pattern-list > li {
  padding-block-start: var(--space-sm);
  border-block-start: var(--rule-width) solid var(--color-rule);
}

.pattern-list a {
  font-family: var(--font-mono);
  font-weight: 700;
}

.pattern-list p {
  margin-block-start: var(--space-2xs);
  font-size: var(--text-sm);
  color: var(--color-muted);
}

```

Deklaration	Wirkung
grid-template-columns: repeat(auto-fill, minmax(16rem, 1fr))	minmax(16rem, 1fr) beschreibt eine Spalte: mindestens 16rem breit, darüber ein gleicher Anteil am freien Platz. Ein rem sind 16 px (siehe Bilder (Kapitel 1.4)), 16rem also 256 px. repeat(auto-fill, ...) wiederholt die Spalte so oft, wie sie in den Container passt.
gap: var(--space-lg)	Das Gap gilt zwischen den Spalten und zwischen den Zeilen.
padding: 0 und list-style: none	Die Liste hat keine Einrückung und keine Aufzählungszeichen.

Die Regel nennt keine Spaltenzahl und keine Breite des Viewports. Eine [Media Query](#) (`@media`) wendet Regeln erst ab einer bestimmten Breite des Viewports an. Die Pattern-Liste enthält keine Media Query.

Prüfe das Ergebnis:

1. Lade `index.html` bei 1280 px Breite neu. Jede Liste hat zwei Spalten.
2. Klicke im Panel **Elements** auf das Badge `grid` neben einem Element `<ul>` und aktiviere im Tab **Layout** die Option **Show track sizes**. Das Overlay zeigt zwei Spalten mit je 356px Breite und das Gap.
3. Stelle die Breite auf 320 px. Jede Liste hat eine Spalte mit 288px Breite. Jede Beschreibung endet vor dem rechten Rand.

Die Regel `main > *` in `layout.css` begrenzt jedes Element im Hauptinhalt mit `max-inline-size: var(--content-width)`. `tokens.css` legt `--content-width: 46rem` fest, das sind 736 px. Zwei Spalten mit 16rem und ein Gap mit 1,5rem benötigen 33,5rem. Drei Spalten mit zwei Gaps benötigen 51rem.

3.5 Flexbox oder Grid

Aufgabe	Verfahren	Ort in der Toolbox
Items in einer Richtung, jedes Item so breit wie sein Inhalt, mit Umbruch	Flexbox	Header, Main Navigation, Breadcrumbs
Items in gemeinsamen Spalten, in jeder Zeile gleich breit	Grid	Pattern-Liste
Bereiche einer Seite mit Namen	Grid	Seitenlayout

Bei Flexbox bestimmt der Inhalt der Items die Aufteilung einer Zeile. Bei Grid bestimmt der Container die Spalten und Zeilen für alle Items. Den Vergleich vertieft [MDN – Relationship of grid layout to other layout methods](#).

Beide Verfahren ändern nur die Darstellung. Tab und Screenreader folgen der Reihenfolge im Quelltext. Drücke auf `index.html` mehrmals Tab: Der Fokus folgt den Einträgen zeilenweise von links nach rechts.

3.6 Commit

1. Übertrage alle Änderungen mit `git add .` in die Staging Area.
2. Prüfe mit `git status`, dass alle geänderten Dateien unter **Changes to be committed** stehen.
3. Erstelle einen Commit mit der Nachricht `feat: Seitenlayout mit Grid Areas und Pattern-Liste` ergänzen und übertrage ihn mit `git push`.

4. Verstehen

Lege im Lernprotokoll die Überschrift `## Layout (Datum)` an. Der Ablauf je Experiment ist: Vorhersage notieren, ausführen, Ergebnis und Erklärung notieren.

Nimm jede Änderung nach dem Experiment zurück: `Strg/Cmd + Z` oder `git restore` mit dem Pfad der Datei.

Predict 1: justify-content nach dem Umbruch

Bei 1280 px Breite stehen Wordmark und Navigation in einer Zeile, die Navigation am rechten Rand. Bei 320 px Breite steht die Navigation in einer zweiten Zeile am linken Rand.

- **Vorhersage:** Wo stehen Wordmark und Navigation bei 320 px Breite, wenn `justify-content` den Wert `flex-end` hat?
- **Prüfen:**
 1. Lade `index.html` bei 320 px Breite.
 2. Ändere in `layout.css` in der Regel `body > header` den Wert von `justify-content` in `flex-end`. Der Wert setzt die Items an das Ende der Zeile.
 3. Lade die Seite neu.

→ [Auflösung 1](#) am Ende des Kapitels

Break-it 1: flex-wrap im Header entfernen

Entferne in `layout.css` in der Regel `body > header` die Deklaration `flex-wrap: wrap;`.

- **Vorhersage:** Passt der Header von `index.html` bei 320 px Breite in den Viewport? Wie stehen die Wordmark und die zwei Links der Navigation?
- **Prüfen:**
 1. Lade `index.html` bei 320 px Breite.
 2. Entferne zusätzlich `flex-wrap: wrap;` in der Regel `nav ul`.
 3. Lade die Seite neu.

→ [Auflösung 2](#) am Ende des Kapitels

Predict 2: order und die Reihenfolge von Tab

Füge am Ende von `layout.css` diese Regel ein:

```
body > header > nav {
  order: -1;
}
```

- **Vorhersage:** Welches Item steht im Header links? Welches Element erhält beim ersten Druck auf Tab den Fokus?
- **Prüfen:** Lade `index.html` bei 1280 px Breite neu und drücke dreimal Tab.

→ [Auflösung 3](#) am Ende des Kapitels

Break-it 2: 1fr statt minmax(0, 1fr)

Ändere in `layout.css` in der Regel `body` den Wert von `grid-template-columns` in `1fr`.

- **Vorhersage:** Was ändert sich auf `ueber.html` bei 1280 px Breite? Was ändert sich bei 320 px Breite?
- **Prüfen:**
 1. Lade `ueber.html` bei beiden Breiten neu und prüfe den Code-Block unter der Überschrift „Stylesheet“.
 2. Klicke bei 320 px auf das Badge `grid` neben `<body>`. **Show track sizes** zeigt die Breite der Spalte.
 3. Fahre im Panel **Elements** mit der Maus über das Element `<pre>` des Code-Blocks. Der Tooltip nennt seine Breite.

→ [Auflösung 4](#) am Ende des Kapitels

Predict 3: Zeilen ohne grid-template-rows

Entferne in `layout.css` in der Regel `body` die Deklaration `grid-template-rows: auto 1fr auto;`. Stelle in der Device Toolbar einen Viewport mit `1280 × 860 px` ein.

- **Vorhersage:** Wo steht der Footer von `patterns/listen.html`? Wie hoch sind Header und Footer?
- **Prüfen:**
 1. Lade `patterns/listen.html` neu. Klicke auf das Badge `grid` neben `<body>` und lies die Höhen im Overlay mit **Show track sizes**.
 2. Setze die Deklaration wieder ein und entferne `min-block-size: 100dvh;`
 3. Lade die Seite neu.

→ [Auflösung 5](#) am Ende des Kapitels

Break-it 3: Tippfehler im Namen eines Grid Area

Ändere in `layout.css` in der Regel `main` den Wert von `grid-area` in `mian`.

- **Vorhersage:** Wo steht der Hauptinhalt? Zeigt das Panel **Console** eine Meldung?
- **Prüfen:**
 1. Lade `index.html` bei 1280 px Breite neu und öffne das Panel **Console**.
 2. Entferne die Deklaration `grid-area: mian;`.
 3. Lade die Seite neu.

→ [Auflösung 6](#) am Ende des Kapitels

Predict 4: auto-fit und Flexbox für die Pattern-Liste

Die Kategorie „Formulare“ enthält einen Eintrag, die Kategorie „Struktur“ drei Einträge. Ändere die Regeln in `components.css` nacheinander in zwei Varianten:

1. Ändere `auto-fill` in `auto-fit`.
 2. Nimm die Änderung zurück. Ersetze in der Regel `.pattern-list` die Deklarationen `display` und `grid-template-columns` durch `display: flex;` und `flex-wrap: wrap;`. Ergänze in der Regel `.pattern-list > li` die Deklaration `flex: 1 1 16rem;`. Die drei Werte von `flex` legen den Anteil am freien Platz, das Schrumpfen und die Ausgangsbreite fest.
- **Vorhersage:** Wie breit ist je Variante der Eintrag unter „Formulare“? Wie breit ist der dritte Eintrag unter „Struktur“? Welche Variante wählst du, wenn die Einträge aller Listen in denselben Spalten stehen sollen? Begründe die Wahl.
 - **Prüfen:** Lade `index.html` bei 1280 px Breite nach jeder Änderung neu.

→ [Auflösung 7](#) am Ende des Kapitels

5. Toolbox-Eintrag

Schreibe drei weitere Einträge. Jeder Eintrag erhält eine eigene Seite:

Datei in patterns	Eintrag	Kategorie	Anchor
<code>flexbox.html</code>	Flexbox	Layout	<code>#layout</code>
<code>seitenlayout.html</code>	Seitenlayout mit Grid	Layout	<code>#layout</code>
<code>auto-fill-grid.html</code>	Spalten ohne Media Query	Layout	<code>#layout</code>

Lege die Seiten mit den Schritten 1 bis 3 aus [Navigation & mehrere Seiten](#), [Übrige Seiten und Übersicht](#) (Kapitel 1.3) an. Die Schritte kopieren `landmarks.html`, ändern `<title>` und `description` und passen die Breadcrumbs an. Ersetze danach in jeder Seite das ganze `<article>` durch einen neuen Eintrag mit der Überschrift `<h1>`. Jeder Eintrag enthält:

1. **Beschreibung und Zweck** in zwei bis fünf Sätzen, in eigenen Worten.
2. **Code:** einen Code-Block mit einem Ausschnitt aus dem eigenen Stylesheet.
3. **Barrierefreiheit:** die Auswirkung auf Screenreader und Tastaturbedienung.
4. **Quelle:** einen Link auf die zugehörige MDN-Seite.

Eintrag	Code-Block mit Caption	Barrierefreiheit	Quelle
Flexbox	Regel body > header · layout.css (Auszug)	Auflösung von Predict 2	MDN – Flexbox
Seitenlayout mit Grid	Regel body · layout.css (Auszug)	Auflösung von Predict 2	MDN – Grid template areas
Spalten ohne Media Query	Regel .pattern-list · components.css (Auszug)	Auflösung von Predict 2	MDN – repeat()

Das Zeichen > im Selektor steht im Code-Block unverändert, nur < braucht <.

```
<article>
  <h1>Flexbox</h1>
  <p>Flexbox ordnet die Kinder eines Containers in einer Richtung an. ...</p>
  <figure class="code">
    <figcaption>layout.css (Auszug)</figcaption>
    <pre tabindex="0"><code>...</code></pre>
  </figure>
  <p>Barrierefreiheit: ...</p>
  <p>Quelle: <a href="https://developer.mozilla.org/...">MDN - ...</a></p>
</article>
```

Ergänze in index.html nach dem </section> des Abschnitts „CSS-Architektur“, direkt vor </main>, den Abschnitt der neuen Kategorie:

```
<section id="layout">
  <h2>Layout</h2>
  <ul class="pattern-list">
    <li>
      <a href="patterns/flexbox.html">Flexbox</a>
      <p>Items in einer Richtung, mit Umbruch und verteiltem Platz</p>
    </li>
    <li>
      <a href="patterns/seitenlayout.html">Seitenlayout mit Grid</a>
      <p>Header, Hauptinhalt und Footer als benannte Grid Areas</p>
    </li>
    <li>
      <a href="patterns/auto-fill-grid.html">Spalten ohne Media Query</a>
      <p>Die Spaltenzahl folgt dem Platz im Container</p>
    </li>
  </ul>
</section>
```

Lade index.html neu. Unter „Layout“ stehen die Links „Flexbox“, „Seitenlayout mit Grid“ und „Spalten ohne Media Query“ in zwei Spalten. Prüfe auf den drei neuen Seiten jeden Link der Breadcrumbs.

Die Referenz-Toolbox zeigt die vollständige Toolbox, wie sie am Ende aussehen kann.

Ersetze in README.md den Abschnitt ## Struktur durch diesen Inhalt:

Struktur

```

...
toolbox/
├── README.md           ← diese Datei
├── learning-notizen.md ← Lernprotokoll: Vorhersagen und Beobachtungen
├── .gitignore          ← Systemdateien, die nicht ins Repository gehören
├── index.html          ← Übersicht mit Links auf alle Einträge
├── ueber.html          ← Über dieses Projekt
├── patterns/           ← je Eintrag eine Seite
│   ├── html-grundgeruest.html
│   ├── landmarks.html
│   ├── ueberschriften.html
│   ├── listen.html
│   ├── links.html
│   ├── tabellen.html
│   ├── breadcrumbs.html
│   ├── aktuelle-seite.html
│   ├── bilder.html
│   ├── responsive-images.html
│   ├── label-fehlermeldung.html
│   ├── selektoren.html
│   ├── box-model.html
│   ├── cascade-layers.html
│   ├── custom-properties.html
│   ├── flexbox.html
│   ├── seitenlayout.html
│   └── auto-fill-grid.html
└── assets/
    ├── css/
    │   ├── main.css      ← einzige eingebundene Datei: Reihenfolge der Ebenen,
    │   └── Importe
    │       ├── tokens.css ← Custom Properties: Farben, Schriften, Abstände
    │       ├── base.css   ← Schriften, Reset, HTML-Elemente ohne Klassen
    │       └── layout.css  ← Seitenlayout: Grid mit drei Grid Areas, Flexbox im
    │
    ├── Header
    │   └── components.css ← Komponenten mit einer Klasse: Code-Block, Pattern-
    │
    └── Liste
        ├── overrides.css ← eigene Gestaltung (freiwillig), gewinnt gegen alle
        │   anderen Dateien
        ├── fonts/        ← Schriften + Lizenzen (vorgegeben)
        └── img/          ← Bilder (vorgegeben, CC0 1.0)
...

```

Header, Navigation und Footer stehen in jeder HTML-Datei. Eine Änderung an der Navigation betrifft alle zwanzig Seiten.

Jede Regel des Stylesheets steht in einer der fünf Ebenen `tokens`, `base`, `layout`, `components` und `overrides`. Eigene Gestaltung steht nur in `overrides.css`; dort legt `--accent-hue` die Akzentfarbe fest.

1. Setze in README.md in der Tabelle ## Fortschritt „Layout“ auf „erledigt“ und ergänze die Zeile „Responsive“ mit „offen“.
2. Übertrage alle Änderungen mit `git add .` in die Staging Area.

3. Prüfe mit `git status`, dass alle geänderten Dateien unter **Changes to be committed** stehen, auch `learning-notizen.md`.
4. Erstelle einen Commit mit der Nachricht `feat: Einträge zu Flexbox, Seitenlayout und Spalten ohne Media Query ergänzen` und übertrage ihn mit `git push`.

6. Prüfen

- ☐ Der [W3C-Validator](#) zeigt für jede der zwanzig Seiten die Meldung „Document checking completed. No errors or warnings to show.“
- ☐ Das Stylesheet enthält keine Änderung aus einem Experiment: Auf GitHub ändert der Commit `feat: Einträge zu Flexbox, Seitenlayout und Spalten ohne Media Query ergänzen` keine Datei im Ordner `assets/css`.
- ☐ Das Overlay mit **Show area names** zeigt für `<body>` die Namen `header`, `main` und `footer`.
- ☐ Viewport mit 1280 px Breite: Jede Liste der Übersicht hat zwei Spalten. Die Spalten aller Listen stimmen überein.
- ☐ Viewport mit 320 px Breite: Jede Liste der Übersicht hat eine Spalte. Jede Beschreibung endet vor dem rechten Rand.
- ☐ Tastatur: Tab erreicht die Links der Übersicht in der sichtbaren Reihenfolge, zeilenweise von links nach rechts.
- ☐ `learning-notizen.md` enthält zu jedem Experiment Vorhersage, Ergebnis und Erklärung.
- ☐ Die Einträge „Flexbox“, „Seitenlayout mit Grid“ und „Spalten ohne Media Query“ stehen unter „Layout“ in der Übersicht.
- ☐ Die Commits `feat: Seitenlayout mit Grid Areas und Pattern-Liste ergänzen` und `feat: Einträge zu Flexbox, Seitenlayout und Spalten ohne Media Query ergänzen` sind auf GitHub.

Quellen

- [MDN – Introduction to CSS layout](#)
- [MDN – Flexbox](#)
- [MDN – Basic concepts of flexbox](#)
- [MDN – Controlling ratios of flex items along the main axis](#)
- [MDN – order](#)
- [MDN – CSS grid layout](#)
- [MDN – Grid template areas](#)
- [MDN – repeat\(\)](#)
- [MDN – minmax\(\)](#)
- [MDN – Grid layout and accessibility](#)
- [MDN – Relationship of grid layout to other layout methods](#)
- [Chrome DevTools – Inspect and debug CSS flexbox layouts](#)
- [Chrome DevTools – Inspect CSS grid layouts](#)
- [W3C Nu HTML Checker](#)

Auflösungen

Auflösung 1

Zu [Predict 1: justify-content nach dem Umbruch](#)

Wert von justify-content	Wordmark	Navigation
space-between	am linken Rand	am linken Rand
flex-end	am rechten Rand	am rechten Rand

justify-content verteilt den freien Platz je Zeile. Nach dem Umbruch enthält jede Zeile ein Item. space-between setzt das erste Item einer Zeile an den Anfang. Für das Ende der Zeile fehlt ein zweites Item.

Auflösung 2

Zu [Break-it 1: flex-wrap im Header entfernen](#)

Stand	Header im Viewport	Wordmark	Links der Navigation
ohne flex-wrap in body > header	ja	zwei Zeilen: „Tool-“ und „box“	untereinander
zusätzlich ohne flex-wrap in nav ul	nein, der zweite Link ist am rechten Rand abgeschnitten	drei Zeilen: Quadrat, „Tool-“ und „box“	nebeneinander

Ohne flex-wrap bleiben alle Items in einer Zeile. Reicht der Platz nicht, schrumpft jedes Item bis zur kleinsten Breite seines Inhalts. Die Eigenschaft flex-shrink hat dafür den [Anfangswert 1](#).

Im ersten Stand schrumpfen beide Items: Der Browser trennt das Wort der Wordmark. Die Liste der Navigation ist selbst ein Flex-Container mit flex-wrap und setzt den zweiten Link in eine neue Zeile.

Im zweiten Stand ist die kleinste Breite der Navigation die Breite beider Links mit dem Gap: 235 px. Die Deklaration white-space: nowrap in der Regel nav a verhindert einen Umbruch im Linktext. Der Inhalt ist breiter als der Container und ragt 12 px über den Viewport. Die Seite scrollt trotzdem nicht: Die Deklaration overflow-x: clip in der Regel html in base.css schneidet den Inhalt am Rand des Viewports ab.

Auflösung 3

Zu [Predict 2: order und die Reihenfolge von Tab](#)

Die Navigation steht links, die Wordmark rechts. Der erste Druck auf Tab setzt den Fokus auf die Wordmark am rechten Rand. Der zweite und der dritte Druck setzen ihn auf die Links „Übersicht“ und „Über dieses Projekt“ am linken Rand.

order ändert die Position eines Items in der Darstellung. Die Reihenfolge für Tab und für den Accessibility Tree bleibt die Reihenfolge im Quelltext.

Eine Person mit Tastatur sieht den Fokus von rechts nach links wechseln. Eine andere sichtbare Reihenfolge entsteht deshalb im HTML, nicht mit order.

Auflösung 4

Zu [Break-it 2: 1fr statt minmax\(0, 1fr\)](#)

Breite des Viewports	Seite	Code-Block
1280 px	unverändert	unverändert
320 px	Spalte 473px breit, Text und Überschriften am rechten Rand abgeschnitten	441px breit, scrollt nicht

1fr entspricht `minmax(auto, 1fr)`. Das Minimum `auto` ist die kleinste Breite des Inhalts. Die längste Zeile des Code-Blocks bricht nicht um und ist 441 px breit. Die Regel `main` legt mit `padding links` und `rechts` je 16 px Abstand zwischen Rand und Inhalt fest. Mit diesem Padding ist die Spalte 473 px breit.

Mit dem Minimum 0 ist die Spalte so breit wie der Viewport, auch wenn ihr Inhalt breiter ist. Erst dann wirkt die Deklaration `overflow-x: auto` am Code-Block: Die lange Zeile scrollt im Code-Block. Die Seite selbst scrollt nie horizontal: `overflow-x: clip` in `base.css` schneidet zu breiten Inhalt ab.

`index.html` bleibt bei 320 px Breite unverändert. Kein Inhalt dieser Seite ist breiter als der Viewport.

Auflösung 5

Zu [Predict 3: Zeilen ohne grid-template-rows](#)

Stand	Zeile des Headers	Zeile des Hauptinhalts	Zeile des Footers	Ort des Footers
Ausgangsstand	57px	731px	72px	unterer Rand des Viewports
ohne <code>grid-template-rows</code>	139px	567px	154px	unterer Rand des Viewports
ohne <code>min-block-size</code>	57px	485px	72px	direkt unter dem Hauptinhalt

Ohne `grid-template-rows` haben alle drei Zeilen die Höhe `auto`. Das Grid ist durch `min-block-size 246 px` höher als sein Inhalt. Der Browser verteilt diesen freien Platz zu gleichen Teilen auf die Zeilen mit der Höhe `auto`: je 82 px. Der Footer steht weiter am unteren Rand, aber Header und Footer sind höher als ihr Inhalt.

Mit `auto 1fr auto` erhält nur die mittlere Zeile den freien Platz.

Ohne `min-block-size` ist das Grid so hoch wie sein Inhalt. Es gibt keinen freien Platz, und unter dem Footer bleiben 246 px des Viewports leer.

Auflösung 6

Zu [Break-it 3: Tippfehler im Namen eines Grid Area](#)

Stand	Header und Footer	Hauptinhalt
<code>grid-area: mian;</code>	464 px breit, der Footer direkt unter dem Header	beginnt 464 px nach dem linken Rand, unterhalb des Footers

Stand	Header und Footer	Hauptinhalt
ohne grid-area in der Regel main	wie im Ausgangsstand	wie im Ausgangsstand

Das Panel **Console** zeigt in beiden Fällen keine Meldung.

main ist ein gültiger Name, aber kein Grid Area trägt ihn. Der Browser legt für das Item eine zusätzliche Spalte und eine zusätzliche Zeile an. Header und Footer bleiben in der ersten Spalte. Die neue Spalte ist so breit wie der Hauptinhalt mit seinem Padding: 816 px. Die erste Spalte mit $\text{minmax}(0, 1\text{fr})$ erhält den Rest: 464 px.

Ohne grid-area belegt das Item die nächste freie Zelle. Header und Footer belegen ihre Grid Areas, frei bleibt die Zelle main.

Das Overlay mit **Show area names** zeigt die Namen, die das Grid enthält.

Auflösung 7

Zu [Predict 4: auto-fit und Flexbox für die Pattern-Liste](#)

Variante	Eintrag unter „Formulare“	dritter Eintrag unter „Struktur“
Ausgangsstand mit auto-fill	356 px	356 px
1: auto-fit	736 px	356 px
2: Flexbox	736 px	736 px

auto-fill behält eine Spalte, die kein Item enthält. auto-fit setzt eine leere Spalte auf die Breite 0. Die verbleibende Spalte erhält den gesamten Platz. In der Liste unter „Struktur“ enthalten beide Spalten ein Item; dort gibt es keinen Unterschied.

Flexbox hat keine Spalten. Jede Zeile verteilt ihren Platz für sich. `flex: 1 1 16rem` gibt jedem Item die Ausgangsbreite 16rem und den gleichen Anteil am freien Platz der Zeile. Ein Item, das allein in einer Zeile steht, erhält die ganze Zeile.

Nur mit auto-fill stimmen die Spalten aller Listen der Seite überein: Der Container legt die Spalten fest, auch wenn eine Spalte leer bleibt. Bei 320 px Breite sind die drei Varianten gleich.



1.9 Responsive

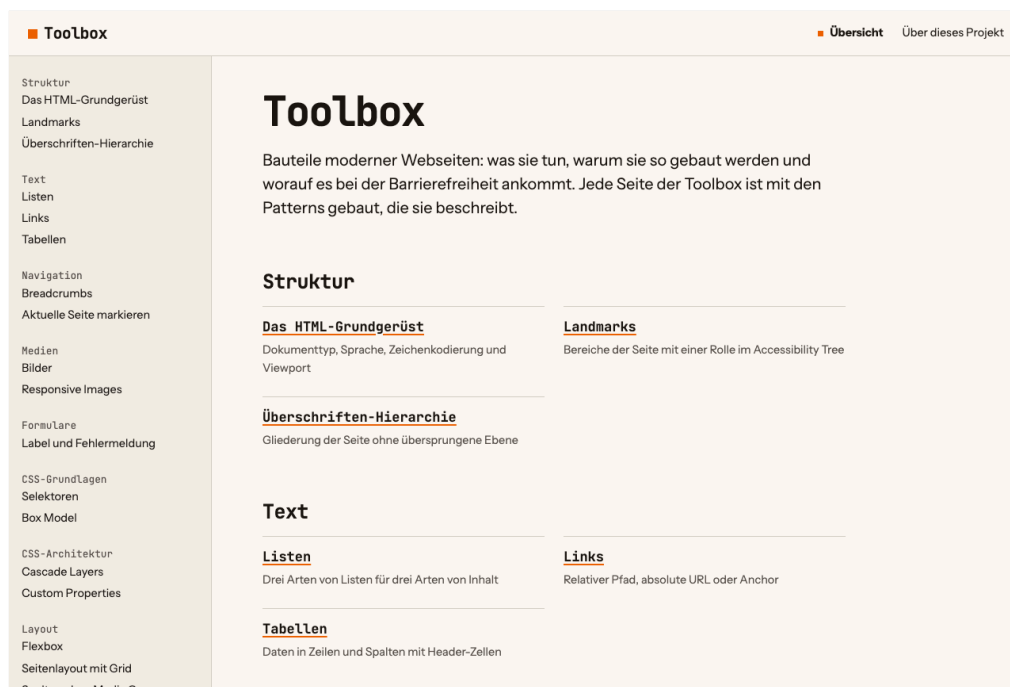
Voraussetzungen: [Layout](#) (Kapitel 1.8)

Lernziele

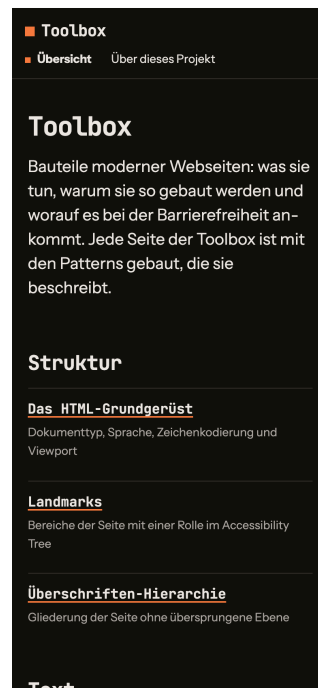
- Ein Seitenlayout nach Mobile-first mit einer Media Query erweitern und für eine gegebene Breite des Viewports vorhersagen, welche Regel gewinnt.
- Den Wert von `clamp()` für eine Breite des Viewports berechnen und eine Größe ohne Break-point zwischen Minimum und Maximum wachsen lassen.
- Eine Komponente mit einer Container Query an die Breite ihres Query Containers anpassen und begründen, wann eine Media Query dafür nicht ausreicht.

1. Vorschau

Zielstand nach diesem Kapitel: Ab 1024 px Breite des Viewports steht links eine Sidebar mit allen Patterns.



Dieselbe Seite bei Smartphone-Breite im Dark Mode. Die Seite hat eine Spalte und keine Sidebar:



2. Warum

Eine Webseite erscheint in Viewports von 320 px bis über 2000 px Breite. Responsive Design passt Layout und Größen an den vorhandenen Platz an, mit demselben HTML. Dieses Kapitel verwendet dafür drei Mittel: eine Media Query für den Wechsel des Layouts, `clamp()` für Größen und eine Container Query für eine Komponente. Sie ersetzen feste Breiten in Pixeln und eine eigene Media Query für jede Komponente.

Weiterführend: [MDN – Responsive web design](#) · [MDN – Media query fundamentals](#) · [MDN – CSS container queries](#)

3. Bauen

Ausgangsstand ist die Toolbox aus dem Kapitel [Layout](#) (Kapitel 1.8): 20 Seiten, ein Seitenlayout mit drei Grid Areas und die Komponente Pattern-Liste.

Die Messwerte dieses Kapitels gelten für die Toolbox ohne eigene Gestaltung in `overrides.css` und für die Schriftgröße 16 px in den Einstellungen des Browsers. Das ist der Wert, solange niemand die Einstellung ändert. Die Breite des Viewports stellt die Device Toolbar ein (siehe Browser & DevTools).

3.1 Media Query lesen

Die Datei `layout.css` endet mit einem Block aus dem Kapitel [Das Skelett](#) (Kapitel 1.1). Er beginnt mit dieser Zeile:

```
@media (width ≥ 64rem) {
```


Eine Media Query nennt eine Bedingung. Der Browser wendet die Regeln im Block nur an, solange die Bedingung zutrifft. Die Bedingung (`width ≥ 64rem`) trifft zu, wenn der Viewport mindestens 64rem breit ist. Das sind 1024 px. Diese Breite ist der Breakpoint.

Der Block enthält zwei Regeln:

Selektoren	Deklaration	Wirkung ab 1024 px
<code>body > header, body > footer</code>	<code>padding-inline: var(--space-lg)</code>	Header und Footer haben 24 px Abstand zum Rand statt 16 px.
<code>main, body > footer</code>	<code>padding-inline-start: var(--space-2xl)</code>	Hauptinhalt und Footer beginnen 64 px nach dem linken Rand statt 16 px.

Weise den Breakpoint in den DevTools nach:

1. Öffne `index.html` und stelle die Breite des Viewports auf 1023 px.
2. Wähle im Panel **Elements** das Element `<main>` aus und lies im Tab **Computed** den Wert von `padding-left`. Er ist 16px: Die Überschrift beginnt 16 px nach dem linken Rand. Bei Text von links nach rechts ist `padding-inline-start` aus der Tabelle das Padding links, `padding-left`.
3. Stelle die Breite auf 1024 px. Der Wert ist 64px.

Die Regeln ohne Bedingung beschreiben die Seite für einen schmalen Viewport. Der Block ergänzt Regeln für einen breiten Viewport. Diese Reihenfolge heißt Mobile-first: Die schmale Form ist die Grundlage, jede Media Query erweitert sie.

Ältere Stylesheets schreiben dieselbe Bedingung als (`min-width: 64rem`). Die Schreibweisen nennt MDN – Using media queries.

3.2 Größen mit `clamp()`

Die Media Query ändert den Abstand des Hauptinhalts in einer Stufe: 16 px bis 1023 px Breite, 64 px ab 1024 px. Die Funktion `clamp()` lässt eine Größe mit dem Viewport wachsen. Die Datei `tokens.css` verwendet sie für die Schriftgröße der Hauptüberschrift:

```
--text-display: clamp(2rem, 1.4rem + 2.6vw, 3.25rem);
```

Angabe	Bedeutung	Im Beispiel
erste	Minimum	2rem
zweite	Wert	1.4rem + 2.6vw
dritte	Maximum	3.25rem

Die Einheit `vw` ist ein Prozent der Breite des Viewports. Liegt der Wert unter dem Minimum, gilt das Minimum. Liegt er über dem Maximum, gilt das Maximum.

Ergänze in `tokens.css` nach der Zeile `--space-2xl` eine Custom Property für den Abstand des Inhalts zum Rand:

```
/* clamp(Minimum, Wert, Maximum): 1rem bis 320 px Breite des Viewports, 4rem ab 1280 px */
--space-page: clamp(1rem, 5vw, 4rem);
```

Ersetze in `layout.css` in den Regeln `main` und `body > footer` in der Deklaration `padding` den Wert `var(--space-md)` durch `var(--space-page)`. Die Deklaration lautet danach in der Regel `main`:

```
padding: var(--space-xl) var(--space-page) var(--space-2xl);
```

In der Regel `body > footer` lautet sie:

```
padding: var(--space-lg) var(--space-page);
```

Ändere danach den Block `@media`:

1. Lösche die Regel mit den Selektoren `main` und `body > footer`.
2. Lösche in der anderen Regel den Selektor `body > footer` und das Komma davor.

Der Block enthält danach eine Regel:

```
body > header {
  padding-inline: var(--space-lg);
}
```

Prüfe das Ergebnis mit `index.html`. Lies wie in [Media Query lesen](#) den Wert von `padding-left` am Element `<main>`:

Breite des Viewports	padding-left
320 px	16px
768 px	38.4px
1280 px	64px

Zwischen 1023 px und 1024 px ändert sich der Abstand um 0,05 px. Die Stufe am Breakpoint entfällt.

3.3 Sidebar im Stylesheet

Die Sidebar ist ein Element `<nav>` zwischen Header und Hauptinhalt. Dieser Abschnitt ergänzt die Regeln der Sidebar im Stylesheet, das Element `<nav>` folgt in Sidebar im HTML. Unter 64rem stellt der Browser die Sidebar nicht dar, ab 64rem belegt sie eine eigene Spalte des Grids.

Ergänze in `tokens.css` nach der Zeile `--content-width`:

```
--sidebar-width: 16rem;
```

Füge in `layout.css` vor dem Kommentar Hauptinhalt diese Regeln ein:

```
/* ---- Sidebar: Navigation mit allen Patterns ----
   Unter 64rem stellt der Browser sie nicht dar. */
body > nav {
  grid-area: nav;
  display: none;
  padding: var(--space-lg) var(--space-md);
  border-inline-end: var(--rule-width) solid var(--color-rule);
  background-color: var(--color-paper-2);
```

```

}

body > nav > p {
  font-family: var(--font-mono);
  font-size: var(--text-xs);
  color: var(--color-muted);
}

body > nav > ul + p {
  margin-block-start: var(--space-lg);
}

```

Der Selektor `body > nav` trifft nur ein Element `<nav>`, das ein Kind von `<body>` ist. Die Main Navigation im Header und die Breadcrumbs im Hauptinhalt trifft er nicht. Der Selektor `ul + p` trifft ein `<p>`, das direkt auf eine `<ul>` folgt: den Namen jeder Kategorie außer der ersten.

Ersetze den Block `@media` am Ende der Datei und den Kommentar `/* ---- Ab 64rem: mehr Luft an den Seiten ---- */` darüber durch diesen Block:

```

/* ---- Ab 64rem: Sidebar neben dem Hauptinhalt ---- */
@media (width ≥ 64rem) {
  body {
    grid-template-columns: var(--sidebar-width) minmax(0, 1fr);
    grid-template-areas:
      "header header"
      "nav main"
      "nav footer";
  }

  body > header {
    padding-inline: var(--space-lg);
  }

  body > nav {
    display: block;
  }
}

```

Regel im Block	Wirkung ab 64rem
<code>body</code>	Das Grid hat zwei Spalten: 16rem für die Sidebar, der Rest für den Hauptinhalt. Der Header belegt beide Spalten. Das Grid Area <code>nav</code> reicht über die zweite und die dritte Zeile.
<code>body > header</code>	unverändert aus Größen mit clamp()
<code>body > nav</code>	Der Browser stellt die Sidebar dar.

Die Regel `body` am Anfang der Datei bleibt unverändert. Der Block überschreibt zwei ihrer Deklarationen. Die übrigen Deklarationen gelten bei jeder Breite.

Prüfe das Ergebnis:

1. Lade `index.html` bei 1280 px Breite neu. Die erste Spalte ist 256 px breit und leer. Die Überschrift beginnt 320 px nach dem linken Rand.
2. Stelle die Breite auf 1023 px. Die Seite hat eine Spalte.

3.4 Sidebar im HTML

Füge in `index.html` zwischen `</header>` und `<main>` die Sidebar ein. Sie beginnt so:

```
<nav aria-label="Patterns">
  <p>Struktur</p>
  <ul class="pattern-list">
    <li><a href="patterns/html-grundgeruest.html">Das HTML-Grundgerüst</a></li>
    <li><a href="patterns/landmarks.html">Landmarks</a></li>
    <li><a href="patterns/ueberschriften.html">Überschriften-Hierarchie</a></li>
  </ul>
```

Ergänze die sieben weiteren Kategorien nach demselben Muster: ein Element `<p>` mit dem Namen der Kategorie und eine Liste mit einem Link je Eintrag. Titel und Pfade stehen in den Abschnitten der Übersicht. Schließe die Sidebar mit `</nav>`. Das Tag `</nav>` steht wie `<nav>` mit vier Leerzeichen am Anfang der Zeile.

- Das Attribut `aria-label="Patterns"` gibt der Navigation ihren Namen im Accessibility Tree.
- Die Listen verwenden die Komponente `.pattern-list` aus dem Kapitel [Layout](#) (Kapitel 1.8), ohne Beschreibung.

Lade `index.html` bei 1280 px Breite neu. Die Sidebar steht links und enthält 18 Links. Ihre Listen haben noch die Form aus der Übersicht: Die Spalte ist 256 px breit, der Platz in der Sidebar 223 px. Die Hairlines, die Linien über jedem Eintrag, ragen 16 px über den rechten Rand der Sidebar. Der Abschnitt [Container Query](#) behebt das.

Übertrage die Sidebar in die übrigen 19 Seiten. Die View **Search** ersetzt einen Text in allen Dateien (siehe VS Code, Suchen und Ersetzen). **Achtung:** Ein Fehler beim Ersetzen trifft alle 19 Seiten. `git restore ueber.html patterns` setzt sie dann auf den letzten Commit zurück.

1. Markiere in `index.html` den Text ab `</header>` bis zum `</nav>` der Sidebar, mit allen Zeilen dazwischen, und kopiere ihn.
2. Öffne die View **Search** und suche `</header>`. Blende mit **Toggle Replace** das Feld **Replace** ein.
3. Füge den kopierten Text in das Feld **Replace** ein. Das Feld übernimmt alle Zeilen.
4. Blende mit **Toggle Search Details** die Felder **files to include** und **files to exclude** ein. Trage unter **files to exclude** `index.html` ein. Die View zeigt **19 results in 19 files**.
5. Klicke auf **Replace All** neben dem Feld **Replace**.
6. Suche `href="patterns/"` und ersetze durch `href="`. Leere **files to exclude** und trage unter **files to include** `patterns` ein. Die View zeigt **324 results in 18 files**. Klicke auf **Replace All**. Die Pfade der Seiten im Ordner `patterns` beginnen danach ohne Ordner.
7. Ergänze auf jeder Seite im Ordner `patterns` das Attribut `aria-current="page"` am Link der Sidebar, der auf diese Seite verweist.

Prüfe das Ergebnis mit `patterns/listen.html`:

1. Lade die Seite bei 1280 px Breite. Der Link „Listen“ trägt als einziger Link der Sidebar das Akzent-Quadrat. Die Regeln dafür stammen aus dem Kapitel [Navigation & mehrere Seiten](#) (Kapitel 1.3) und stehen in `layout.css`.
2. Drücke mehrmals Tab. Der Fokus erreicht die drei Links im Header, danach die 18 Links der Sidebar, danach die Breadcrumbs.

3. Stelle die Breite auf 390 px und lade die Seite neu. Der Fokus erreicht die drei Links im Header und danach die Breadcrumbs.

`display: none` entfernt ein Element aus der Darstellung, aus dem Accessibility Tree und aus der Reihenfolge von Tab. Unter 64rem führt der Link „Übersicht“ im Header zu allen Patterns.

3.5 Container Query

Die Pattern-Liste steht jetzt an zwei Orten: in der Sidebar mit 223 px Platz und im Hauptinhalt mit bis zu 736 px. Bei 1280 px Breite des Viewports sind beide Orte sichtbar.

Eine Container Query fragt die Breite eines Vorfahren ab, eines Elements, das die Komponente enthält. Dieser Vorfahre ist der Query Container. Füge in `layout.css` nach den Regeln der Sidebar diese Regel ein:

```
/* Query Container: Eine Container Query fragt die Breite dieser Elemente ab. */
body > nav,
main {
  container-type: inline-size;
}
```

Ersetze in `components.css` im Kommentar über `.pattern-list` die zwei Zeilen nach der ersten Zeile aus Strichen durch diese Zeilen:

```
Pattern-Liste: Links auf die Einträge, in der Übersicht und in der Sidebar
Die Form folgt der Breite des Query Containers.
```

Ersetze danach die Regeln `.pattern-list`, `.pattern-list > li` und `.pattern-list a` durch diese Regeln:

```
.pattern-list {
  display: grid;
  gap: var(--space-2xs);
  padding: 0;
  list-style: none;
}

/* Ab 16rem Breite des Query Containers: Spalten, Hairline, Titel in Mono */
@container (width >= 16rem) {
  .pattern-list {
    grid-template-columns: repeat(auto-fill, minmax(16rem, 1fr));
    gap: var(--space-lg);
  }

  .pattern-list > li {
    padding-block-start: var(--space-sm);
    border-block-start: var(--rule-width) solid var(--color-rule);
  }

  .pattern-list a {
    font-family: var(--font-mono);
    font-weight: 700;
  }
}
```

```
}
}
```

Die Regel `.pattern-list p` bleibt unverändert.

Teil	Wirkung
<code>container-type: inline-size</code>	Das Element ist ein Query Container. Seine Nachfahren fragen seine Breite ab.
Regel <code>.pattern-list</code> ohne Bedingung	die schmale Form: eine Spalte, kleines Gap, keine Hairline
<code>@container (width ≥ 16rem)</code>	Die Regeln im Block gelten für Elemente, deren nächster Query Container mindestens 16rem breit ist.

Die Reihenfolge entspricht Mobile-first, übertragen auf den Container: Die schmale Form ist die Grundlage, der Block erweitert sie. Die Schwelle 16rem ist das Minimum einer Spalte in `minmax(16rem, 1fr)`.

Prüfe das Ergebnis mit `index.html`:

Breite des Viewports	Liste in der Sidebar	Liste im Hauptinhalt
1280 px	eine Spalte mit 223 px, ohne Hairline	zwei Spalten mit je 356 px, mit Hairline
320 px	nicht dargestellt	eine Spalte mit 288 px, mit Hairline

Bei 320 px Breite hat der Hauptinhalt 288 px Platz. Das sind 18rem, die Bedingung trifft zu.

Der Eintrag „Spalten ohne Media Query“ zeigt die Regel `.pattern-list` aus dem Kapitel [Layout](#) (Kapitel 1.8). Ersetze in `patterns/auto-fill-grid.html` das Element `<pre>` des Code-Blocks durch diese Zeilen. Das Zeichen `>` steht im Code-Block unverändert:

```
<pre tabindex="0"><code>@container (width ≥ 16rem) {
  .pattern-list {
    grid-template-columns: repeat(auto-fill, minmax(16rem, 1fr));
    gap: var(--space-lg);
  }
}</code></pre>
```

Ergänze in der Beschreibung des Eintrags, dass die Spalten erst ab 16rem Breite des Query Containers entstehen.

3.6 Media Query, `clamp()` oder Container Query

Aufgabe	Mittel	Ort in der Toolbox
Das Layout der Seite wechselt ab einer Breite des Viewports.	Media Query	Sidebar ab 64rem
Eine Größe wächst mit dem Viewport zwischen zwei Grenzen.	<code>clamp()</code>	Hauptüberschrift, Abstand des Inhalts zum Rand
Die Spaltenzahl folgt dem Platz im Container.	Grid mit <code>auto-fill</code>	Pattern-Liste im Hauptinhalt
Eine Komponente hat je nach Breite ihres Containers eine andere Form.	Container Query	Pattern-Liste in Sidebar und Hauptinhalt

Die Toolbox enthält nach diesem Kapitel eine Media Query und eine Container Query. Alle anderen Anpassungen an den Platz entstehen ohne Bedingung.

3.7 Commit

1. Übertrage alle Änderungen mit `git add .` in die Staging Area.
2. Prüfe mit `git status`, dass alle geänderten Dateien unter **Changes to be committed** stehen.
3. Erstelle einen Commit mit der Nachricht `feat: Sidebar, clamp() und Container Query` ergänzen und übertrage ihn mit `git push`.

4. Verstehen

Lege im Lernprotokoll die Überschrift **## Responsive** (Datum) an. Der Ablauf je Experiment ist: Vorhersage notieren, ausführen, Ergebnis und Erklärung notieren.

Nimm jede Änderung nach dem Experiment zurück: `Strg/Cmd + Z` oder `git restore` mit dem Pfad der Datei.

Predict 1: rem im Breakpoint

Füge in `base.css` in der Regel `html` die Deklaration `font-size: 125%;` ein. Das Element `<html>` hat danach die Schriftgröße 20 px.

- **Vorhersage:** Ab welcher Breite des Viewports erscheint die Sidebar: ab 1024 px oder ab 1280 px? Wie breit ist sie?
- **Prüfen:** Lade `index.html` bei 1023 px, bei 1024 px und bei 1280 px Breite neu. Klicke im Panel **Elements** auf das Badge `grid` neben `<body>`. **Show track sizes** im Tab **Layout** zeigt die Breite der ersten Spalte.

→ [Auflösung 1](#) am Ende des Kapitels

Break-it 1: Custom Property im Breakpoint

Ergänze in `tokens.css` nach der Zeile `--sidebar-width` die Zeile `--breakpoint: 64rem;`. Ändere in `layout.css` die Bedingung des Blocks:

```
@media (width ≥ var(--breakpoint)) {
```

- **Vorhersage:** Zeigt `index.html` bei 1280 px Breite die Sidebar? Zeigt das Panel **Console** eine Meldung?
- **Prüfen:** Lade `index.html` bei 1280 px und bei 1440 px Breite neu und öffne das Panel **Console**.

→ [Auflösung 2](#) am Ende des Kapitels

Predict 2: Block @media am Anfang der Datei

Verschiebe in `layout.css` den Block `@media` vom Ende der Datei vor die Regel `body`.

- **Vorhersage:** Welches Layout hat `index.html` bei 1280 px Breite?
- **Prüfen:** Lade `index.html` bei 1280 px Breite neu.

→ [Auflösung 3](#) am Ende des Kapitels

Predict 3: clamp() berechnen

Die Hauptüberschrift hat die Schriftgröße `clamp(2rem, 1.4rem + 2.6vw, 3.25rem)`. 1rem entspricht 16 px, 1vw einem Prozent der Breite des Viewports.

- **Vorhersage:** Welche Schriftgröße in Pixeln hat die `<h1>` bei 320 px, bei 768 px und bei 1280 px Breite?
- **Prüfen:** Wähle auf `index.html` die `<h1>` im Panel **Elements** aus und lies im Tab **Computed** den Wert von `font-size` bei jeder Breite.

→ [Auflösung 4](#) am Ende des Kapitels

Break-it 2: Wert ohne rem

Ändere in `tokens.css` den Wert von `--text-display` in `clamp(2rem, 2.6vw, 3.25rem)`.

- **Vorhersage:** Welche Schriftgröße hat die `<h1>` bei 320 px, bei 768 px und bei 1280 px Breite?
- **Prüfen:** Lade `index.html` bei jeder Breite neu und lies wie in Predict 3 im Tab **Computed** den Wert von `font-size`.

→ [Auflösung 5](#) am Ende des Kapitels

Break-it 3: container-type entfernen

Entferne in `layout.css` die Deklaration `container-type: inline-size;`.

- **Vorhersage:** Wie ändern sich bei 1280 px Breite die Listen im Hauptinhalt von `index.html`? Ändern sich die Listen in der Sidebar? Zeigt das Panel **Console** eine Meldung?
- **Prüfen:** Lade `index.html` bei 1280 px Breite neu und öffne das Panel **Console**.

→ [Auflösung 6](#) am Ende des Kapitels

Predict 4: @media statt @container

Ändere in `components.css` die Zeile `@container (width ≥ 16rem) {` in `@media (width ≥ 16rem) {`.

- **Vorhersage:** Bei 1280 px Breite soll die Liste in der Sidebar eine Spalte ohne Hairline haben, die Listen im Hauptinhalt zwei Spalten mit Hairline. Gibt es dafür einen Wert in der Bedingung?
- **Prüfen:** Lade `index.html` bei 1280 px Breite neu. Ändere den Wert in der Bedingung in `60rem` und danach in `90rem` und lade die Seite jeweils neu.

→ [Auflösung 7](#) am Ende des Kapitels

5. Toolbox-Eintrag

Schreibe drei weitere Einträge. Jeder Eintrag erhält eine eigene Seite:

Datei in patterns	Eintrag	Kategorie	Anchor
<code>media-queries.html</code>	Media Queries	Responsive	<code>#responsive</code>

Datei in patterns	Eintrag	Kategorie	Anchor
clamp.html	Größen mit clamp()	Responsive	#responsive
container-queries.html	Container Queries	Responsive	#responsive

Lege die Seiten mit den Schritten 1 bis 3 aus [Navigation & mehrere Seiten](#), [Übrige Seiten](#) und [Übersicht](#) (Kapitel 1.3) an. Die Schritte kopieren landmarks.html, ändern <title> und description und passen die Breadcrumbs an. Ersetze danach in jeder Seite das ganze <article> durch einen neuen Eintrag mit der Überschrift <h1>. Jeder Eintrag enthält:

1. **Beschreibung und Zweck** in zwei bis fünf Sätzen, in eigenen Worten.
2. **Code:** einen Code-Block mit einem Ausschnitt aus dem eigenen Stylesheet.
3. **Barrierefreiheit:** die Auswirkung auf Screenreader und Tastaturbedienung.
4. **Quelle:** einen Link auf die zugehörige MDN-Seite.

Eintrag	Code-Block mit Caption	Barrierefreiheit	Quelle
Media Queries	Block @media · layout.css (Auszug)	Prüfschritte zu Tab aus Sidebar im HTML , Auflösung von Predict 1	MDN – Using media queries
Größen mit clamp()	Regel :root nur mit den Zeilen --text-display und --space-page · tokens.css (Auszug)	Auflösung von Break-it 2	MDN – clamp()
Container Queries	zwei Code-Blöcke: Regel mit container-type · layout.css (Auszug); Block @container · components.css (Auszug)	Eine Container Query ändert nur die Darstellung, Tab folgt dem Quelltext (wie in Layout , Predict 2 (Kapitel 1.8))	MDN – CSS container queries

Das Zeichen > im Selektor steht im Code-Block unverändert, nur < braucht < ;.

```
<article>
  <h1>Container Queries</h1>
  <p>Eine Container Query wendet Regeln ab einer Breite des Query Containers an, nicht des Viewports. ...</p>
  <figure class="code">
    <figcaption>components.css (Auszug)</figcaption>
    <pre tabindex="0"><code>...</code></pre>
  </figure>
  <p>Barrierefreiheit: ...</p>
  <p>Quelle: <a href="https://developer.mozilla.org/...">MDN - ...</a></p>
</article>
```

Ergänze in index.html nach dem </section> des Abschnitts „Layout“, direkt vor </main>, den Abschnitt der neuen Kategorie:

```
<section id="responsive">
  <h2>Responsive</h2>
  <ul class="pattern-list">
    <li>
```

```

    <a href="patterns/media-queries.html">Media Queries</a>
    <p>Mobile-first: Regeln ab einem Breakpoint</p>
  </li>
  <li>
    <a href="patterns/clamp.html">Größen mit clamp()</a>
    <p>Ein Wert zwischen Minimum und Maximum, ohne Breakpoint</p>
  </li>
  <li>
    <a href="patterns/container-queries.html">Container Queries</a>
    <p>Regeln ab einer Breite des Query Containers</p>
  </li>
</ul>
</section>

```

Ergänze in der Sidebar aller 23 Seiten nach dem `</ul>` der Kategorie „Layout“ die neue Kategorie. In `index.html` und `ueber.html` endet die Sidebar danach so:

```

<p>Responsive</p>
<ul class="pattern-list">
  <li><a href="patterns/media-queries.html">Media Queries</a></li>
  <li><a href="patterns/clamp.html">Größen mit clamp()</a></li>
  <li><a href="patterns/container-queries.html">Container Queries</a></li>
</ul>
</nav>

```

Die View **Search** ändert alle Seiten wie in Sidebar im HTML:

1. Aktiviere im Suchfeld die Option **Use Regular Expression** (Symbol `.*`) und suche `^</nav>`. Die Suche trifft nur das `</nav>` der Sidebar mit vier Leerzeichen davor; jedes andere `</nav>` ist tiefer eingerückt.
2. Füge den Code-Block in das Feld **Replace** ein. Leere **files to include** und **files to exclude** und trage unter **files to include** `index.html`, `ueber.html` ein. Die View zeigt **2 results in 2 files**. Zeigt sie weniger, prüfe vor **Replace All** in der fehlenden Datei die Einrückung des `</nav>` der Sidebar. Klicke auf **Replace All**.
3. Lösche im Feld **Replace** die drei Vorkommen von `patterns/`. Ersetze unter **files to include** den Eintrag durch `patterns`. Die View zeigt **21 results in 21 files**. Klicke auf **Replace All**.
4. Deaktiviere **Use Regular Expression**.
5. Die drei neuen Seiten sind Kopien von `landmarks.html`. Entferne dort `aria-current="page"` am Link „Landmarks“ der Sidebar und setze es an den eigenen Link.

Lade `index.html` bei 1280 px Breite neu. Unter „Responsive“ stehen die Links „Media Queries“, „Größen mit clamp()“ und „Container Queries“ in zwei Spalten, in der Sidebar untereinander. Prüfe auf den drei neuen Seiten jeden Link der Breadcrumbs.

Die Referenz-Toolbox zeigt die vollständige Toolbox, wie sie am Ende aussehen kann.

Ersetze in `README.md` den Abschnitt `## Struktur` durch diesen Inhalt:

```

## Struktur
...
toolbox/
├─ README.md      ← diese Datei

```

```

├── learning-notizen.md ← Lernprotokoll: Vorhersagen und Beobachtungen
├── .gitignore          ← Systemdateien, die nicht ins Repository gehören
├── index.html          ← Übersicht mit Links auf alle Einträge
├── ueber.html          ← Über dieses Projekt
├── patterns/           ← je Eintrag eine Seite
│   ├── html-grundgeruest.html
│   ├── landmarks.html
│   ├── ueberschriften.html
│   ├── listen.html
│   ├── links.html
│   ├── tabellen.html
│   ├── breadcrumbs.html
│   ├── aktuelle-seite.html
│   ├── bilder.html
│   ├── responsive-images.html
│   ├── label-fehlermeldung.html
│   ├── selektoren.html
│   ├── box-model.html
│   ├── cascade-layers.html
│   ├── custom-properties.html
│   ├── flexbox.html
│   ├── seitenlayout.html
│   ├── auto-fill-grid.html
│   ├── media-queries.html
│   ├── clamp.html
│   └── container-queries.html
├── assets/
│   └── css/
│       ├── main.css      ← einzige eingebundene Datei: Reihenfolge der Ebenen,
Importe
│       ├── tokens.css    ← Custom Properties: Farben, Schriften, Abstände
│       ├── base.css      ← Schriften, Reset, HTML-Elemente ohne Klassen
│       └── layout.css     ← Seitenlayout: Grid Areas, Sidebar ab 64rem, Flexbox
im Header
│       └── components.css ← Komponenten mit einer Klasse: Code-Block, Pattern-
Liste
│           └── overrides.css ← eigene Gestaltung (freiwillig), gewinnt gegen alle
anderen Dateien
│               ├── fonts/      ← Schriften + Lizenzen (vorgegeben)
│               └── img/        ← Bilder (vorgegeben, CC0 1.0)
...

```

Header, Sidebar und Footer stehen in jeder HTML-Datei. Eine Änderung an der Sidebar betrifft alle dreiundzwanzig Seiten.

Jede Regel des Stylesheets steht in einer der fünf Ebenen `tokens`, `base`, `layout`, `components` und `overrides`. Eigene Gestaltung steht nur in `overrides.css`; dort legt `--accent-hue` die Akzentfarbe fest.

Das Stylesheet folgt Mobile-first: Die Regeln ohne Bedingung beschreiben die Seite mit einer Spalte. Ab dem Breakpoint 64rem stellt der Browser die Sidebar dar.

1. Setze in README.md in der Tabelle ## Fortschritt „Responsive“ auf „erledigt“ und ergänze die Zeile „Design-System“ mit „offen“.
2. Übertrage alle Änderungen mit `git add` in die Staging Area.
3. Prüfe mit `git status`, dass alle geänderten Dateien unter **Changes to be committed** stehen, auch `learning-notizen.md`.
4. Erstelle einen Commit mit der Nachricht `feat: Einträge zu Media Queries, clamp() und Container Queries ergänzen` und übertrage ihn mit `git push`.

6. Prüfen

- ☐ Der W3C-Validator zeigt für jede der 23 Seiten die Meldung „Document checking completed. No errors or warnings to show.“
- ☐ Das Stylesheet enthält keine Änderung aus einem Experiment: Auf GitHub ändert der Commit `feat: Einträge zu Media Queries, clamp() und Container Queries ergänzen` keine Datei im Ordner `assets/css`.
- ☐ Viewport mit 1280 px Breite: Jede Seite zeigt die Sidebar. Ihre Listen haben eine Spalte ohne Hairline, die Listen der Übersicht zwei Spalten.
- ☐ Viewport mit 1023 px Breite: Keine Seite zeigt die Sidebar.
- ☐ Viewport mit 320 px Breite: Auf jeder Seite hat `<main>` im Tab **Computed** den Wert `padding-left: 16px`. Jede Beschreibung der Übersicht endet vor dem rechten Rand.
- ☐ Viewport mit 768 px Breite: Auf jeder Seite hat `<main>` den Wert `padding-left: 38.4px`.
- ☐ Jede Seite im Ordner `patterns` markiert ihren Link in der Sidebar mit `aria-current="page"`.
- ☐ Tastatur: Bei 1280 px Breite erreicht Tab nach den Links im Header die Links der Sidebar. Bei 1023 px Breite erreicht Tab keinen Link der Sidebar.
- ☐ `learning-notizen.md` enthält zu jedem Experiment Vorhersage, Ergebnis und Erklärung.
- ☐ Die Einträge „Media Queries“, „Größen mit `clamp()`“ und „Container Queries“ stehen unter „Responsive“ in der Übersicht und in der Sidebar.
- ☐ Die Commits `feat: Sidebar, clamp() und Container Query ergänzen` und `feat: Einträge zu Media Queries, clamp() und Container Queries ergänzen` sind auf GitHub.

Quellen

- [MDN – Responsive web design](#)
- [MDN – Media query fundamentals](#)
- [MDN – Using media queries](#)
- [MDN – clamp\(\)](#)
- [MDN – CSS container queries](#)
- [MDN – container-type](#)
- [MDN – @container](#)
- [web.dev – CSS min\(\), max\(\), and clamp\(\)](#)
- [VS Code – Search across files](#)
- [W3C Nu HTML Checker](#)

Auflösungen

Auflösung 1

Zu [Predict 1: rem im Breakpoint](#)

Breite des Viewports	Sidebar
1023 px	nicht dargestellt
1024 px	320px breit
1280 px	320px breit

Der Breakpoint bleibt bei 1024 px. In einer Media Query rechnet rem mit der Schriftgröße aus den Einstellungen des Browsers, nicht mit font-size am Element `<html>`. In einer Deklaration rechnet rem mit font-size am Element `<html>`: Die Sidebar ist 16×20 px breit.

Stellt eine Person im Browser die Schriftgröße 20 px ein, liegt der Breakpoint bei 1280 px. Ein Breakpoint in rem folgt dieser Einstellung, ein Breakpoint in px nicht.

Auflösung 2

Zu [Break-it 1: Custom Property im Breakpoint](#)

Die Seite hat bei beiden Breiten eine Spalte und keine Sidebar. Der Header hat 16 px Abstand zum Rand. Das Panel **Console** zeigt keine Meldung.

`var()` wirkt nur im Wert einer Deklaration. In der Bedingung einer Media Query ist `var()` ungültig. Eine ungültige Bedingung trifft nie zu, und der Browser wendet keine Regel des Blocks an.

Der Wert des Breakpoints steht deshalb im Block selbst.

Auflösung 3

Zu [Predict 2: Block @media am Anfang der Datei](#)

Die Seite hat eine Spalte und keine Sidebar. Der Header hat 16 px Abstand zum Rand.

Die Bedingung trifft zu, und der Browser wendet die Regeln des Blocks an. Eine Media Query erhöht die Spezifität einer Regel nicht. Die Regeln `body, body > header` und `body > nav` stehen zweimal in der Datei, mit gleicher Spezifität und in derselben Ebene. Schritt 3 der Kaskade, die Reihenfolge im Quelltext, entscheidet: Die spätere Regel gewinnt (siehe [CSS-Fundament](#) (Kapitel 1.7)).

Bei Mobile-first stehen die Regeln ohne Bedingung zuerst. Jeder Block `@media` steht danach.

Auflösung 4

Zu [Predict 3: clamp\(\) berechnen](#)

Breite des Viewports	$1.4\text{rem} + 2.6\text{vw}$	Schriftgröße
320 px	$22,4 \text{ px} + 8,32 \text{ px} = 30,72 \text{ px}$	32px (Minimum)
768 px	$22,4 \text{ px} + 19,968 \text{ px} = 42,368 \text{ px}$	42.368px
1280 px	$22,4 \text{ px} + 33,28 \text{ px} = 55,68 \text{ px}$	52px (Maximum)

Bis 369 px Breite gilt das Minimum, ab 1139 px das Maximum. Zwischen diesen Breiten wächst die Schriftgröße mit jedem Pixel.

Auflösung 5

Zu [Break-it 2: Wert ohne rem](#)

Wert	320 px	768 px	1280 px
1.4rem + 2.6vw (Ausgangsstand)	32px	42.368px	52px
2.6vw	32px	32px	33.28px

2.6vw erreicht das Minimum von 32 px erst bei 1231 px Breite und das Maximum bei 2000 px. Der Anteil 1.4rem verschiebt den Bereich, in dem die Schriftgröße wächst, in die Breiten von 369 px bis 1139 px.

Der Anteil in rem hat eine zweite Aufgabe: Mit ihm folgt die Schriftgröße der Einstellung im Browser. Eine Schriftgröße nur in vw folgt dieser Einstellung nicht.

Auflösung 6

Zu [Break-it 3: container-type entfernen](#)

Ort	Ausgangsstand	ohne container-type
Hauptinhalt	zwei Spalten mit je 356 px, Hairline, Titel in Mono	eine Spalte mit 736 px, ohne Hairline, Titel in der Schrift des Fließtexts
Sidebar	eine Spalte mit 223 px	unverändert

Das Panel **Console** zeigt keine Meldung.

Ohne container-type hat die Pattern-Liste keinen Query Container. Die Bedingung (`width ≥ 16rem`) trifft dann nie zu, auch nicht im Hauptinhalt mit 736 px Platz. Alle Listen haben die schmale Form.

Eine Container Query hat zwei Teile: die Deklaration `container-type` an einem Vorfahren und den Block `@container`.

Auflösung 7

Zu [Predict 4: @media statt @container](#)

Bedingung	Sidebar	Hauptinhalt
<code>(width ≥ 16rem), (width ≥ 60rem)</code>	eine Spalte mit 256 px, Hairline, Titel in Mono und fett; die Hairlines ragen 16 px über den rechten Rand der Sidebar	zwei Spalten mit je 356 px, Hairline
<code>(width ≥ 90rem)</code>	eine Spalte mit 223 px, ohne Hairline	eine Spalte mit 736 px, ohne Hairline

Kein Wert in der Bedingung trennt die zwei Orte. Eine Media Query fragt nur den Viewport ab. Er ist 1280 px breit, die Bedingung trifft für beide Orte zu oder für keinen. Die Container Query fragt für jede Liste die Breite ihres Query Containers ab.



1.10 Design-System

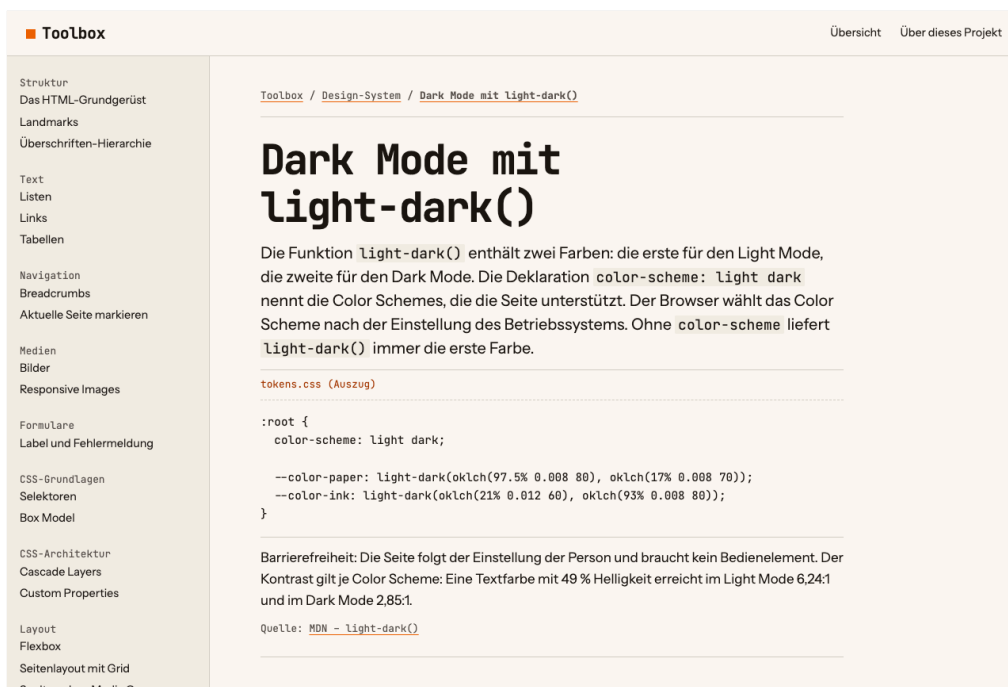
Voraussetzungen: [Responsive](#) (Kapitel 1.9)

Lernziele

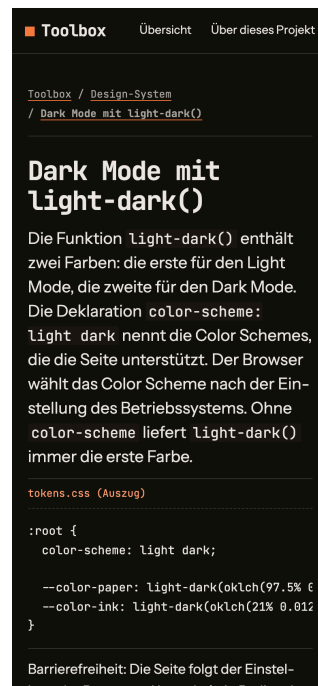
- Die Tokens der Toolbox nach ihrer Aufgabe lesen, einen festen Wert durch ein Token ersetzen und die Silbentrennung über `lang` und `­` steuern.
- Eine Farbe in `oklch()` aus Helligkeit, Chroma und Farbton lesen, eine Variante ableiten und ihren Kontrast zum Hintergrund in den DevTools gegen 4,5:1 nachweisen.
- Für ein Element vorhersagen, welchen Wert `light-dark()` im Light Mode und im Dark Mode liefert, und die Aufgaben von `color-scheme` und `prefers-color-scheme` unterscheiden.

1. Vorschau

Zielstand nach diesem Kapitel: Die Toolbox hat die Kategorie „Design-System“ mit drei Einträgen. Der Dateiname im Code-Block hat die Akzentfarbe in einer Variante für Text.



Dieselbe Seite bei Smartphone-Breite im Dark Mode. Jede Farbe hat dort ihren zweiten Wert. Die langen Zeilen im Code-Block sind abgeschnitten; der Code-Block scrollt waagrecht:



2. Warum

Steht eine Farbe als fester Wert in vielen Regeln, verlangt eine neue Akzentfarbe eine Änderung in jeder dieser Regeln. Ein Design-System legt die Werte der Gestaltung deshalb einmal fest: Farben, Schriften, Größen und Abstände. Jede Regel des Stylesheets verwendet diese Werte über ihren Namen. Ein neuer Wert entsteht aus einem vorhandenen, zum Beispiel eine Textfarbe aus der Akzentfarbe. Ob Text lesbar ist, entscheidet der Kontrast zum Hintergrund, im Light Mode und im Dark Mode.

Weiterführend: [MDN – oklch\(\)](#) · [MDN – light-dark\(\)](#) · [MDN – Color contrast](#)

3. Bauen

Ausgangsstand ist die Toolbox aus dem Kapitel Responsive (Kapitel 1.9): 23 Seiten und ein Stylesheet in sechs Dateien.

Die Messwerte dieses Kapitels gelten unter drei Bedingungen: `overrides.css` enthält in der Regel `:root` nur `--accent-hue: 45;`. Hast du einen anderen Wert eingetragen, behalte ihn; nur die Kontrastwerte der Akzentfarbe weichen dann ab. Die Schriftgröße in den Einstellungen des Browsers ist 16 px. Der Browser stellt den Light Mode dar. Steht dein Betriebssystem auf dunkel, führe im Command Menu den Befehl **Emulate CSS prefers-color-scheme: light** aus (siehe Browser & DevTools).

3.1 Tokens lesen

Ein Token ist eine Custom Property für einen Wert der Gestaltung. Die Datei `tokens.css` enthält alle Tokens der Toolbox in einer Regel mit dem Selektor `:root` (siehe CSS-Fundament (Kapitel 1.7)). Die Tokens bilden fünf Gruppen:

Gruppe	Tokens	Beispiel
Farbe	--accent-hue, sechs Tokens --color-...	--color-paper: Hintergrund der Seite
Schrift	--font-body, --font-mono	--font-mono: Überschriften, Captions, Code
Schriftgröße	sechs Tokens --text-...	--text-lg: Einleitung, Wordmark, <h3>
Abstand	acht Tokens --space-...	--space-md: Abstand zwischen zwei Absätzen
Maß	--radius-sm, --rule-width, --focus-width, --content-width, --sidebar-width	--content-width: größte Breite einer Textzeile

Der Name eines Tokens nennt die Aufgabe, nicht das Aussehen. `--color-paper` ist im Light Mode hell und im Dark Mode dunkel. Ein Name nach dem Aussehen, zum Beispiel `--color-white`, wäre im Dark Mode falsch.

Die übrigen CSS-Dateien enthalten keinen Farbwert. Schriftnamen stehen dort nur in den Regeln `@font-face` von `base.css`. Weise das nach:

1. Öffne in VS Code die View **Search** und trage unter **files to include** `*.css` ein.
2. Suche `oklch()`. Alle Treffer stehen in `tokens.css`, mit eigener Gestaltung auch in `overrides.css`.
3. Suche `Instrument Sans`. Die Treffer stehen in `tokens.css` und in den Regeln `@font-face` von `base.css`.

Suche danach `line-height`. Die Treffer stehen in drei Dateien und enthalten vier feste Werte. Ein fester Wert hat keinen Namen, der seine Aufgabe nennt. [Typografie](#) gibt zwei dieser Werte einen Namen.

3.2 Typografie

Die Toolbox verwendet zwei Schriften. Jedes Token nennt eine Liste: Der Browser verwendet die erste Schrift der Liste, die verfügbar ist. Die weiteren Einträge sind Ersatzschriften.

Token	Erste Schrift	Verwendung
--font-body	Instrument Sans	Fließtext
--font-mono	JetBrains Mono	Überschriften, Captions, Code

Die Schriftgrößen stehen in `rem`:

Token	Wert	Bei 16 px Schriftgröße im Browser
--text-xs	0.8rem	12,8 px
--text-sm	0.9rem	14,4 px
--text-base	1.0625rem	17 px
--text-lg	1.25rem	20 px
--text-xl	1.5625rem	25 px
--text-display	clamp(2rem, 1.4rem + 2.6vw, 3.25rem)	32 px bis 52 px

Zeilenhöhe. Die Eigenschaft `line-height` legt die Höhe einer Zeile fest. Die Datei `base.css` enthält dafür zwei feste Werte: 1.6 in der Regel `html` und 1.15 in der Regel `h1`, `h2`, `h3`. Ergänze in `tokens.css` nach der Zeile `--text-display` diese Zeilen:

```
/* Zeilenhöhe als Zahl ohne Einheit: ein Vielfaches der Schriftgröße des
Elements */
--leading-tight: 1.15;
--leading-body: 1.6;
```

Ersetze in `base.css` die Deklaration in der Regel `html`:

```
line-height: var(--leading-body);
```

Ersetze die Deklaration in der Regel `h1`, `h2`, `h3`:

```
line-height: var(--leading-tight);
```

Prüfe das Ergebnis mit `ueber.html` bei 1280 px Breite des Viewports. Wähle im Panel **Elements** nacheinander `<h1>`, den ersten Absatz nach `<h2>Stylesheet</h2>` und die `<figcaption>` unter dem Bild aus. Der Tab **Computed** nennt für `line-height` diese Werte:

Element	Schriftgröße	line-height
<code><h1></code>	52 px	59.8px
Absatz unter „Stylesheet“	17 px	27.2px
Caption unter dem Bild	12,8 px	20.48px

Die Werte 1.5 für die Einleitung in `layout.css` und 1.55 für den Code-Block in `components.css` gelten je für ein Bauteil und bleiben feste Werte. Ein Token erhalten die Werte für Text und Überschriften der ganzen Seite.

Silbentrennung. Die Deklaration `hyphens: auto` in der Regel `html` trennt Wörter am Zeilenende. Der Browser verwendet dafür die Regeln der Sprache aus dem Attribut `lang`. Das englische Wort „Stylesheet“ trennt er nach deutschen Regeln: Bei 414 px Breite endet eine Zeile im ersten Absatz von `ueber.html` mit „Style-“.

Die Zeichenreferenz `­` nennt eine Trennstelle in einem Wort. Der Browser stellt sie nur am Zeilenende dar, als Trennstrich. Der erste Absatz von `ueber.html` lautet danach:

```
<p>Die Toolbox dokumentiert Bauteile moderner Webseiten und wächst mit
jedem Kapitel. Das Aussehen legt das <a href="assets/css/
main.css">Style&shy;sheet</a> fest.</p>
```

1. Trage in der View **Search** unter **files to include** `*.html` ein, aktiviere **Match Case** (Symbol Aa) und suche `Stylesheet`. Mit **Match Case** trifft die Suche das Attribut `rel="stylesheet"` nicht. Die View zeigt mindestens **5 results in 2 files** (`index.html` und `ueber.html`), dazu Treffer in deinen eigenen Texten. Zeigt sie einen Treffer in jeder HTML-Datei, ist **Match Case** aus.
2. Trage im Feld **Replace** `Style­sheet` ein und klicke auf **Replace All**. Jedes Vorkommen erhält die Trennstelle, auch in `<meta>` und `<h2>`; der Browser zeigt sie nur am Zeilenende. Deaktiviere danach **Match Case**.
3. Lade `ueber.html` bei 414 px Breite neu. Die Zeile endet mit „Style-“.

3.3 Farbe mit oklch()

Jede Farbe der Toolbox steht in der Funktion `oklch()`. Die Textfarbe hat zwei Werte, den ersten für den Light Mode:

```
--color-ink: light-dark(oklch(21% 0.012 60), oklch(93% 0.008 80));
```

Angabe	Bedeutung	Bereich	In <code>oklch(21% 0.012 60)</code>
erste	Helligkeit	0 % (Schwarz) bis 100 % (Weiß)	21 %: dunkel
zweite	Chroma	0 (Grau) bis etwa 0,4 (kräftige Farbe)	0,012: fast Grau
dritte	Farbton	Winkel von 0 bis 360	60: Orange

Öffne `tokens.css`. Ordne die sechs Farben nach der Helligkeit ihres ersten Werts und notiere die Farbe mit dem höchsten Chroma.

→ [Lösung 1](#) am Ende des Kapitels

Die Helligkeit in `oklch()` folgt der Wahrnehmung: Zwei Farben mit gleicher Helligkeit und verschiedenem Farbton erscheinen ähnlich hell. Eine Variante einer Farbe entsteht deshalb durch eine andere Helligkeit bei gleichem Farbton.

3.4 Dark Mode

Die Regel `:root` in `tokens.css` enthält diese Zeilen:

```
/* light-dark(hell, dunkel): Der Browser folgt der Systemeinstellung. */
color-scheme: light dark;

--color-paper: light-dark(oklch(97.5% 0.008 80), oklch(17% 0.008 70));
```

Teil	Art	Aufgabe
<code>color-scheme</code>	Eigenschaft	nennt die Color Schemes , die ein Element unterstützt
<code>light-dark()</code>	Funktion im Wert einer Farbe	enthält zwei Farben und liefert eine davon

Mit `color-scheme: light dark` unterstützt die Seite beide Color Schemes. Der Browser wählt das Color Scheme nach der Einstellung der Person. `light-dark()` liefert im Light Mode die erste Farbe und im Dark Mode die zweite.

Ältere Stylesheets definieren jede Farbe ein zweites Mal in einem Block `@media (prefers-color-scheme: dark)`. Die Bedingung `prefers-color-scheme` fragt ab, welches Color Scheme die Person im Betriebssystem eingestellt hat. Mit `light-dark()` stehen beide Werte in einer Deklaration. `light-dark()` enthält nur Farben. Für jeden anderen Wert bleibt die Media Query.

Weise beide Werte in den DevTools nach:

1. Öffne `index.html` und wähle im Panel **Elements** das Element `<html>` aus.
2. Lies im Tab **Computed** den Wert von `background-color`. Er lautet `oklch(0.975 0.008 80)`: der erste Wert von `--color-paper`, die Helligkeit als Anteil statt in Prozent.
3. Führe im Command Menu den Befehl **Emulate CSS prefers-color-scheme: dark** aus (siehe Browser & DevTools).
4. Lies den Wert erneut. Er lautet `oklch(0.17 0.008 70)`, der zweite Wert von `--color-paper`.
5. Führe den Befehl **Emulate CSS prefers-color-scheme: light** aus.

Im Dark Mode ist der Hintergrund dunkel und der Text hell: `--color-paper` hat 17 % statt 97,5 % Helligkeit, `--color-ink` 93 % statt 21 %.

3.5 Kontrast

Der Kontrast zweier Farben ist ein Verhältnis zwischen 1:1 (gleiche Farbe) und 21:1 (Schwarz auf Weiß). Die [Web Content Accessibility Guidelines \(WCAG\)](#) verlangen für Text mindestens 4,5:1. Für großen Text genügen 3:1: ab 24 px oder ab 18,66 px in fetter Schrift.

Textfarbe	Hintergrund	Light Mode	Dark Mode
<code>--color-ink</code>	<code>--color-paper</code>	16,5:1	15,56:1
<code>--color-muted</code>	<code>--color-paper</code>	6,93:1	7,99:1
<code>--color-muted</code>	<code>--color-paper-2</code>	6,34:1	7,4:1
<code>--color-accent</code>	<code>--color-paper</code>	3,11:1	7,25:1

Die Akzentfarbe erreicht im Light Mode 3,11:1. Die Toolbox verwendet sie deshalb nur für Markierungen, Linien und den Fokusrahmen, nicht für Text.

Der Dateiname im Code-Block erhält die Akzentfarbe als Textfarbe. Dafür entsteht eine Variante mit demselben Farbton, einer anderen Helligkeit und einem geringeren Chroma. Ergänze in `tokens.css` nach der Deklaration `--color-accent` diese Zeilen:

```
/* Akzent als Textfarbe: Kontrast zu --color-paper mindestens 4.5:1 */
--color-accent-text: light-dark(
  oklch(49% 0.16 var(--accent-hue)),
  oklch(79% 0.13 var(--accent-hue))
);
```

Color Scheme	<code>--color-accent</code>	<code>--color-accent-text</code>
Light Mode	66 % Helligkeit, Chroma 0,19	49 %, Chroma 0,16: dunkler, weil der Hintergrund hell ist
Dark Mode	72 % Helligkeit, Chroma 0,17	79 %, Chroma 0,13: heller, weil der Hintergrund dunkel ist

Ergänze in `components.css` die Regel `.code > figcaption` um eine Deklaration. Die Regel lautet danach:

```
.code > figcaption {
  margin-block-start: 0;
  padding-block: var(--space-xs);
  border-block-end: var(--rule-width) dashed var(--color-rule);
  color: var(--color-accent-text);
}
```

Weise den Kontrast in den DevTools nach:

1. Lade `ueber.html` neu und schalte den **Inspect mode** ein (siehe Browser & DevTools, Elements).
2. Bewege den Mauszeiger über den Dateinamen „main.css“ im Code-Block. Der Tooltip nennt unter **Contrast** den Wert 6.24.

3. Emuliere den Dark Mode wie in [Dark Mode](#), schalte den **Inspect mode** erneut ein und bewege den Mauszeiger über den Dateinamen. Der Wert ist 9.49.
4. Führe den Befehl **Emulate CSS prefers-color-scheme: light** aus.

Den Tooltip beschreibt [Chrome DevTools – Make your website more readable](#).

3.6 Commit

1. Übertrage alle Änderungen mit `git add .` in die Staging Area.
2. Prüfe mit `git status`, dass alle geänderten Dateien unter **Changes to be committed** stehen.
3. Erstelle einen Commit mit der Nachricht `feat: Tokens für Zeilenhöhe und Akzent als Textfarbe` ergänzen und übertrage ihn mit `git push`.

4. Verstehen

Lege im Lernprotokoll die Überschrift **## Design-System (Datum)** an. Der Ablauf je Experiment ist: Vorhersage notieren, ausführen, Ergebnis und Erklärung notieren.

Nimm jede Änderung nach dem Experiment zurück: `Strg/Cmd + Z` oder `git restore` mit dem Pfad der Datei.

Predict 1: Zeilenhöhe mit Einheit

Ändere in `tokens.css` die Zeile `--leading-body: 1.6; in --leading-body: 160%;`.

- **Vorhersage:** Wie hoch ist danach eine Zeile der Caption unter dem Bild (Schriftgröße 12,8 px)? Wie hoch ist eine Zeile im Absatz unter „Stylesheet“ (17 px)?
- **Prüfen:** Lade `ueber.html` bei 1280 px Breite neu. Wähle beide Elemente aus und lies im Tab **Computed** den Wert von `line-height`.

→ [Auflösung 2](#) am Ende des Kapitels

Predict 2: Silbentrennung ohne lang

Lade `index.html` bei 320 px Breite und notiere die Wörter mit Trennstrich in den Beschreibungen der Einträge. Entferne danach das Attribut `lang="de"` am Element `<html>`.

- **Vorhersage:** Wie trennt der Browser die Wörter in den Beschreibungen der Einträge bei 320 px Breite? Wie trennt er sie mit `lang="en"`?
- **Prüfen:**
 1. Lade `index.html` neu und vergleiche die Zeilenenden mit deinen Notizen.
 2. Setze `lang="en"` und lade die Seite neu.

→ [Auflösung 3](#) am Ende des Kapitels

Predict 3: Gleiche Helligkeit, anderer Farbton

Der Dateiname im Code-Block hat die Farbe `oklch(49% 0.16 var(--accent-hue))`. Die Funktion `hsl()` beschreibt eine Farbe mit Farbton, Sättigung und Helligkeit; `hsl(var(--accent-hue) 100% 33%)` hat volle Sättigung und 33 % Helligkeit. Ändere in `overrides.css` in der Regel `:root` den Wert von `--accent-hue` nacheinander in 45, 110 und 250.

- **Vorhersage:** Bleibt der Kontrast des Dateinamens für jeden der drei Farbtöne über 4,5:1? Gilt dasselbe für `hsl(var(--accent-hue) 100% 33%)`?
- **Prüfen:**
 1. Lade `ueber.html` im Light Mode nach jeder Änderung neu und lies den Kontrast des Dateinamens „main.css“ im Tooltip des **Inspect mode**.
 2. Ersetze in `tokens.css` die Zeile `oklch(49% 0.16 var(--accent-hue))`, durch `hsl(var(--accent-hue) 100% 33%)`,
 3. Wiederhole Schritt 1 für die drei Farbtöne.

→ [Auflösung 4](#) am Ende des Kapitels

Break-it 1: Eine Textfarbe für beide Color Schemes

Ersetze in `tokens.css` in der Deklaration `--color-accent-text` die zweite Farbe durch die erste: `oklch(49% 0.16 var(--accent-hue))`.

- **Vorhersage:** Reicht der Kontrast des Dateinamens im Dark Mode? Ab welcher Helligkeit der zweiten Farbe erreicht er 4.5?
- **Prüfen:**
 1. Lade `ueber.html` neu und lies den Kontrast des Dateinamens im Light Mode und im Dark Mode.
 2. Wähle im Dark Mode im Panel **Elements** das Element `<html>` aus. Erhöhe im Tab **Styles** in der Deklaration `--color-accent-text` die Helligkeit der zweiten Farbe in Schritten von 5 %, ab 49%. Schalte nach jeder Änderung den **Inspect mode** ein und lies den Kontrast des Dateinamens. Notiere die erste Helligkeit mit mindestens 4.5.
 3. Setze die zweite Farbe wieder auf `oklch(79% 0.13 var(--accent-hue))` und führe den Befehl **Emulate CSS prefers-color-scheme: light** aus.

→ [Auflösung 5](#) am Ende des Kapitels

Break-it 2: color-scheme entfernen

Lösche in `tokens.css` die Zeile `color-scheme: light dark;`.

- **Vorhersage:** Welche Farben hat die Seite, wenn die DevTools den Dark Mode emulieren? Welche Farben hat sie mit `color-scheme: dark`, wenn die DevTools den Light Mode emulieren?
- **Prüfen:**
 1. Lade `index.html` neu und führe den Befehl **Emulate CSS prefers-color-scheme: dark** aus.
 2. Setze die Zeile wieder ein, mit dem Wert `dark` statt `light dark`.
 3. Führe den Befehl **Emulate CSS prefers-color-scheme: light** aus und lade die Seite neu.

→ [Auflösung 6](#) am Ende des Kapitels

Predict 4: color-scheme: dark an der Sidebar

Ergänze in `layout.css` in der Regel `body > nav` die Deklaration `color-scheme: dark;`. Die Regel enthält bereits `background-color: var(--color-paper-2);`. Die Textfarbe legt die Regel `html` in `base.css` fest: `color: var(--color-ink);`.

- **Vorhersage:** Welche Farbe haben im Light Mode der Hintergrund der Sidebar, die Namen der Kategorien und die Links? Welche Farbe haben die Links, wenn die Regel zusätzlich `color: var(--color-ink);` enthält?
- **Prüfen:**
 1. Lade `index.html` bei 1280 px Breite im Light Mode neu. Lies den Kontrast eines Namens einer Kategorie und eines Links im Tooltip des **Inspect mode**.
 2. Ergänze in derselben Regel die Deklaration `color: var(--color-ink);`, lade die Seite neu und lies den Kontrast erneut.

→ Auflösung 7 am Ende des Kapitels

5. Toolbox-Eintrag

Schreibe drei weitere Einträge. Jeder Eintrag erhält eine eigene Seite:

Datei in patterns	Eintrag	Kategorie	Anchor
<code>design-tokens.html</code>	Design-Tokens	Design-System	<code>#design-system</code>
<code>oklch.html</code>	Farben mit <code>oklch()</code>	Design-System	<code>#design-system</code>
<code>light-dark.html</code>	Dark Mode mit <code>light-dark()</code>	Design-System	<code>#design-system</code>

Lege die Seiten mit den Schritten 1 bis 3 aus Navigation & mehrere Seiten, Übrige Seiten und Übersicht (Kapitel 1.3) an. Die Schritte kopieren `landmarks.html`, ändern `<title>` und `description` und passen die Breadcrumbs an. Ersetze danach in jeder Seite das ganze `<article>` durch einen neuen Eintrag mit der Überschrift `<h1>`. Jeder Eintrag enthält:

1. **Beschreibung und Zweck** in zwei bis fünf Sätzen, in eigenen Worten.
2. **Code:** je Datei einen Code-Block mit einem Ausschnitt aus dem eigenen Stylesheet.
3. **Barrierefreiheit:** die Auswirkung auf die Lesbarkeit. Screenreader und Tastaturbedienung ändern die drei Einträge nicht.
4. **Quelle:** einen Link auf die zugehörige MDN-Seite.

Eintrag	Code-Block mit Caption	Barrierefreiheit	Quelle
Design-Tokens	zwei Code-Blöcke: Regel <code>:root</code> nur mit den Tokens der Zeilenhöhe <code>tokens.css</code> (Auszug); Regel <code>html::base.css</code> (Auszug)	Auflösung von <u>Predict 1</u>	<u>MDN – line-height</u>
Farben mit <code>oklch()</code>	Regel <code>:root</code> nur mit der Deklaration <code>--color-accent-text·tokens.css</code> (Auszug)	Werte aus <u>Kontrast</u>	<u>MDN – oklch()</u>
Dark Mode mit <code>light-dark()</code>	Regel <code>:root</code> nur mit <code>color-scheme, --color-paper</code> und <code>--color-ink·tokens.css</code> (Auszug)	Auflösung von <u>Break-it 1</u>	<u>MDN – light-dark()</u>

```

<article>
  <h1>Dark Mode mit light-dark()</h1>
  <p>Die Funktion <code>light-dark()</code> enthält zwei Farben: die erste
für den Light Mode, die zweite für den Dark Mode. ...</p>
  <figure class="code">
    <figcaption>tokens.css (Auszug)</figcaption>
    <pre tabindex="0"><code>...</code></pre>
  </figure>
  <p>Barrierefreiheit: ...</p>
  <p>Quelle: <a href="https://developer.mozilla.org/...">MDN - ...</a></p>
</article>

```

Ergänze in index.html nach dem </section> des Abschnitts „Responsive“, direkt vor </main>, den Abschnitt der neuen Kategorie:

```

<section id="design-system">
  <h2>Design-System</h2>
  <ul class="pattern-list">
    <li>
      <a href="patterns/design-tokens.html">Design-Tokens</a>
      <p>Jeder Wert der Gestaltung unter einem Namen nach seiner Aufgabe</p>
    </li>
    <li>
      <a href="patterns/oklch.html">Farben mit oklch()</a>
      <p>Helligkeit, Chroma und Farbton; Kontrast ab 4,5:1</p>
    </li>
    <li>
      <a href="patterns/light-dark.html">Dark Mode mit light-dark()</a>
      <p>Je Color Scheme eine Farbe, der Browser wählt</p>
    </li>
  </ul>
</section>

```

Ergänze in der Sidebar aller 26 Seiten nach dem der Kategorie „Responsive“ die neue Kategorie. In index.html und ueber.html endet die Sidebar danach so:

```

<p>Design-System</p>
<ul class="pattern-list">
  <li><a href="patterns/design-tokens.html">Design-Tokens</a></li>
  <li><a href="patterns/oklch.html">Farben mit oklch()</a></li>
  <li><a href="patterns/light-dark.html">Dark Mode mit light-dark()</a></li>
</ul>
</nav>

```

Ändere alle Seiten mit der View **Search** wie in Responsive, Toolbox-Eintrag (Kapitel 1.9): **Use Regular Expression**, Suche `^</nav>`, den Code-Block im Feld **Replace**.

1. Trage unter **files to include** index.html, ueber.html ein. Die View zeigt **2 results in 2 files**. Klicke auf **Replace All**.
2. Lösche im Feld **Replace** die drei Vorkommen von patterns/. Ersetze unter **files to include** den Eintrag durch patterns. Die View zeigt **24 results in 24 files**. Klicke auf **Replace All**.

3. Deaktiviere **Use Regular Expression**.
4. Die drei neuen Seiten sind Kopien von `landmarks.html`. Entferne dort `aria-current="page"` am Link „Landmarks“ der Sidebar und setze es an den eigenen Link.

Lade `index.html` neu. Unter „Design-System“ stehen die Links „Design-Tokens“, „Farben mit `oklch()`“ und „Dark Mode mit `light-dark()`“. Prüfe auf den drei neuen Seiten jeden Link der Breadcrumbs.

Die Referenz-Toolbox zeigt die vollständige Toolbox, wie sie am Ende aussehen kann.

Ersetze in `README.md` den Abschnitt `## Struktur` durch diesen Inhalt:

```
## Struktur

...

toolbox/
├── README.md           ← diese Datei
├── learning-notizen.md ← Lernprotokoll: Vorhersagen und Beobachtungen
├── .gitignore          ← Systemdateien, die nicht ins Repository gehören
├── index.html          ← Übersicht mit Links auf alle Einträge
├── ueber.html          ← Über dieses Projekt
├── patterns/           ← je Eintrag eine Seite
│   ├── html-grundgeruest.html
│   ├── landmarks.html
│   ├── ueberschriften.html
│   ├── listen.html
│   ├── links.html
│   ├── tabellen.html
│   ├── breadcrumbs.html
│   ├── aktuelle-seite.html
│   ├── bilder.html
│   ├── responsive-images.html
│   ├── label-fehlermeldung.html
│   ├── selektoren.html
│   ├── box-model.html
│   ├── cascade-layers.html
│   ├── custom-properties.html
│   ├── flexbox.html
│   ├── seitenlayout.html
│   ├── auto-fill-grid.html
│   ├── media-queries.html
│   ├── clamp.html
│   ├── container-queries.html
│   ├── design-tokens.html
│   ├── oklch.html
│   └── light-dark.html
├── assets/
│   └── css/
│       ├── main.css      ← einzige eingebundene Datei: Reihenfolge der Ebenen,
Importe
│       ├── tokens.css    ← Tokens: Farben, Schriften, Schriftgrößen,
Zeilenhöhen, Abstände
│       ├── base.css      ← Schriften, Reset, HTML-Elemente ohne Klassen
│       └── layout.css     ← Seitenlayout: Grid Areas, Sidebar ab 64rem, Flexbox
im Header
```

```

|   | components.css ← Komponenten mit einer Klasse: Code-Block, Pattern-
Liste |
|   | overrides.css ← eigene Gestaltung (freiwillig), gewinnt gegen alle
anderen Dateien
|   | fonts/          ← Schriften + Lizenzen (vorgegeben)
|   | img/            ← Bilder (vorgegeben, CC0 1.0)
...

```

Header, Sidebar und Footer stehen in jeder HTML-Datei. Eine Änderung an der Sidebar betrifft alle sechsundzwanzig Seiten.

Jede Regel des Stylesheets steht in einer der fünf Ebenen ``tokens``, ``base``, ``layout``, ``components`` und ``overrides``. Eigene Gestaltung steht nur in ``overrides.css``; dort legt ``--accent-hue`` die Akzentfarbe fest.

Das Stylesheet folgt Mobile-first: Die Regeln ohne Bedingung beschreiben die Seite mit einer Spalte. Ab dem Breakpoint 64rem stellt der Browser die Sidebar dar.

Farbwerte stehen nur in ``tokens.css`` und ``overrides.css``, Schriftnamen außerdem in den Regeln ``@font-face`` von ``base.css``. Jede Farbe ist ein Token mit zwei Werten in ``light-dark()``: Die Seite folgt dem Color Scheme des Betriebssystems. Text hat in beiden Color Schemes einen Kontrast von mindestens 4,5:1 zum Hintergrund.

1. Setze in `README.md` in der Tabelle `## Fortschritt „Design-System“` auf „erledigt“ und ergänze die Zeile „Nicht jeder Browser kann alles“ mit „offen“.
2. Übertrage alle Änderungen mit `git add .` in die Staging Area.
3. Prüfe mit `git status`, dass alle geänderten Dateien unter **Changes to be committed** stehen, auch `learning-notizen.md`.
4. Erstelle einen Commit mit der Nachricht `feat: Einträge zu Design-Tokens, oklch() und light-dark()` ergänzen und übertrage ihn mit `git push`.

6. Prüfen

- ☐ Der [W3C-Validator](#) zeigt für jede der sechsundzwanzig Seiten die Meldung „Document checking completed. No errors or warnings to show.“
- ☐ Das Stylesheet enthält keine Änderung aus einem Experiment: Auf GitHub ändert der Commit `feat: Einträge zu Design-Tokens, oklch() und light-dark()` ergänzen keine Datei im Ordner `assets/css`.
- ☐ Die Funktion `oklch()` kommt in den CSS-Dateien nur in `tokens.css` vor, mit eigener Gestaltung auch in `overrides.css`.
- ☐ `base.css` enthält `line-height` nur mit `var(--leading-body)` und `var(--leading-tight)`.
- ☐ Der Dateiname im Code-Block hat im Light Mode und im Dark Mode einen Kontrast von mindestens 4,5:1.
- ☐ `learning-notizen.md` enthält zu jedem Experiment Vorhersage, Ergebnis und Erklärung.
- ☐ Die Einträge „Design-Tokens“, „Farben mit `oklch()`“ und „Dark Mode mit `light-dark()`“ stehen unter „Design-System“ in der Übersicht.

- ☐ Die Commits feat: Tokens für Zeilenhöhe und Akzent als Textfarbe ergänzen und feat: Einträge zu Design-Tokens, `oklch()` und `light-dark()` ergänzen sind auf GitHub.

Quellen

- [MDN – oklch\(\)](#)
- [MDN – hsl\(\)](#)
- [MDN – light-dark\(\)](#)
- [MDN – color-scheme](#)
- [MDN – prefers-color-scheme](#)
- [MDN – Color contrast](#)
- [MDN – line-height](#)
- [MDN – hyphens](#)
- [MDN – Using CSS custom properties](#)
- [W3C WAI – Understanding SC 1.4.3: Contrast \(Minimum\)](#)
- [Design Tokens Community Group](#)
- [Chrome DevTools – Make your website more readable](#)
- [W3C Nu HTML Checker](#)

Auflösungen

Lösung 1

Zu [3.3 Farbe mit oklch\(\)](#)

Token	Helligkeit	Chroma	Farbton	Aufgabe
--color-paper	97,5 %	0,008	80	Hintergrund der Seite
--color-paper-2	94,5 %	0,011	80	Hintergrund der Sidebar und von <code>
--color-rule	85 %	0,012	75	Linien
--color-accent	66 %	0,19	--accent-hue	Markierungen, Linien unter Links, Fokusrahmen
--color-muted	45 %	0,012	70	gedämpfter Text
--color-ink	21 %	0,012	60	Text

Fünf Farben haben ein Chroma von höchstens 0,012 und einen Farbton zwischen 60 und 80: Grautöne mit einem Anteil Orange. Sie unterscheiden sich in der Helligkeit. Nur die Akzentfarbe hat ein hohes Chroma.

Auflösung 2

Zu [Predict 1: Zeilenhöhe mit Einheit](#)

Element	Schriftgröße	mit 1.6	mit 160%
Caption unter dem Bild	12,8 px	20.48px	25.6px
Absatz	17 px	27.2px	25.6px
Link in der Sidebar	14,4 px	23.04px	25.6px
<h1>	52 px	59.8px	59.8px

Die Deklaration `line-height` steht in der Regel `html`. Das Element `<html>` hat die Schriftgröße 16 px. Einen Wert mit Einheit berechnet der Browser an diesem Element: 160 % von 16 px sind 25,6 px. Die übrigen Elemente erben diese Länge.

Eine Zahl ohne Einheit erben die Elemente als Zahl. Jedes Element multipliziert sie mit seiner eigenen Schriftgröße.

Die `<h1>` bleibt unverändert: Die Regel `h1, h2, h3` legt für sie `--leading-tight` fest.

Auflösung 3

Zu [Predict 2: Silbentrennung ohne lang](#)

Attribut	Trennungen in den Beschreibungen
<code>lang="de"</code>	„Acces-sibility“, „be-nannte“, „Maxi-mum“, „Style-sheets“
ohne <code>lang</code>	„Style-sheets“
<code>lang="en"</code>	„benan-nte“, „Style-sheets“

Ohne `lang` trennt der Browser nur an der Trennstelle `aus­`. Ein anderes Wort, das nicht in die Zeile passt, steht vollständig in der nächsten Zeile. Nur „Header-Zellen“ bricht am Bindestrich um.

hyphens: auto braucht die Sprache des Texts. Der Browser wählt mit ihr die Regeln für die Trennung. Mit lang="en" wendet er englische Regeln auf deutsche Wörter an und trennt „benannte“ falsch.

Das Attribut lang wirkt damit auf die Darstellung, nicht nur auf Screenreader (siehe [Das Skelett](#) (Kapitel 1.1)).

Auflösung 4

Zu [Predict 3: Gleiche Helligkeit, anderer Farbton](#)

--accent-hue	oklch(49% 0.16 ...)	hsl(... 100% 33%)
45	6.24	3.44
110	5.67	2.92
250	5.82	12.07

Mit oklch() liegt der Kontrast für jeden Farbton zwischen 4,98:1 und 6,38:1. Mit hsl() liegt er zwischen 2,35:1 und 12,45:1.

Die dritte Angabe in hsl() heißt ebenfalls Helligkeit. Sie folgt nicht der Wahrnehmung: Gelb und Grün erscheinen bei gleichem Wert heller als Blau. In oklch() bedeutet derselbe Wert für jeden Farbton eine ähnliche Helligkeit.

Eine eigene Akzentfarbe ändert nur --accent-hue. Der Kontrast von --color-accent-text zu --color-paper bleibt für jeden Farbton über 4,5:1.

Der Winkel bezeichnet in den zwei Funktionen verschiedene Farbtöne: 110 ist in oklch() ein Gelbgrün und in hsl() ein Grün.

Auflösung 5

Zu [Break-it 1: Eine Textfarbe für beide Color Schemes](#)

Farbe des Dateinamens	Light Mode	Dark Mode
--color-accent-text (Ausgangsstand)	6.24	9.49
erste Farbe in beiden Color Schemes	6.24	2.85

Eine Farbe mit 49 % Helligkeit genügt auf dem hellen Hintergrund und genügt nicht auf dem dunklen. Eine Textfarbe braucht deshalb je Color Scheme einen eigenen Wert.

In Schritten von 5 % zeigt der Tooltip bei 59 % 4.36 und bei 64 % 5.34. Die Grenze liegt bei 60 % (4.54), für jeden Farbton bei 61 %. Der Lösungsstand verwendet 79 %: Der Kontrast bleibt für jede eigene Akzentfarbe deutlich über 4,5:1.

Der Browser meldet einen zu geringen Kontrast nicht im Panel **Console**. Der Nachweis ist die Messung in beiden Color Schemes.

Auflösung 6

Zu [Break-it 2: color-scheme entfernen](#)

Deklaration	Einstellung der Person	Darstellung
color-scheme: light dark (Ausgangsstand)	dunkel	dunkel

Deklaration	Einstellung der Person	Darstellung
keine	dunkel	hell
color-scheme: dark	hell	dunkel

Ohne `color-scheme` unterstützt die Seite nur das helle Color Scheme. `light-dark()` liefert dann immer die erste Farbe. Die Einstellung der Person ändert nichts. Der Browser meldet keinen Fehler.

Mit `color-scheme: dark` unterstützt die Seite nur das dunkle Color Scheme. `light-dark()` liefert immer die zweite Farbe.

`light-dark()` wertet die Einstellung der Person nicht selbst aus. Die Funktion folgt dem Color Scheme, das der Browser für das Element verwendet. Dieses Color Scheme wählt er aus den Werten von `color-scheme`. Eine Media Query mit `prefers-color-scheme: dark` trifft im dritten Fall nicht zu: Sie fragt nur die Einstellung ab.

Auflösung 7

Zu [Predict 4: color-scheme: dark an der Sidebar](#)

Teil der Sidebar	nur <code>color-scheme: dark</code>	zusätzlich <code>color: var(--color-ink)</code>
Hintergrund	dunkel	dunkel
Namen der Kategorien	hell, 7.4	hell, 7.4
Links	dunkel auf dunkel, 1	hell, 14.42

Der Browser wertet `light-dark()` an dem Element aus, dessen Deklaration die Farbe verwendet. Die Deklarationen `background-color` an der Sidebar und `color: var(--color-muted)` an den Namen der Kategorien gelten für Elemente mit dunklem Color Scheme. Sie erhalten die zweite Farbe.

Die Regel `a` in `base.css` setzt für Links `color: inherit`. Die Deklaration `color: var(--color-ink)` steht am Element `<html>` mit hellem Color Scheme. Der Browser berechnet dort die erste Farbe, und die Sidebar erbt das Ergebnis.

Mit einer eigenen Deklaration `color` an der Sidebar wertet der Browser `--color-ink` dort aus. Ein Bereich mit eigenem Color Scheme legt deshalb Hintergrund und Textfarbe fest.

In [CSS-Fundament, Predict 3](#) (Kapitel 1.7) setzt der Browser `var()` an dem Element ein, das die Custom Property definiert. Das gilt auch hier: Der berechnete Wert von `--color-ink` an `<html>` ist `light-dark(oklch(21% 0.012 60), oklch(93% 0.008 80))` und enthält beide Farben. `var()` ist darin eingesetzt. Die Wahl zwischen den zwei Farben trifft der Browser erst bei `color` oder `background-color`, an dem Element mit dieser Eigenschaft.



1.11 Nicht jeder Browser kann alles

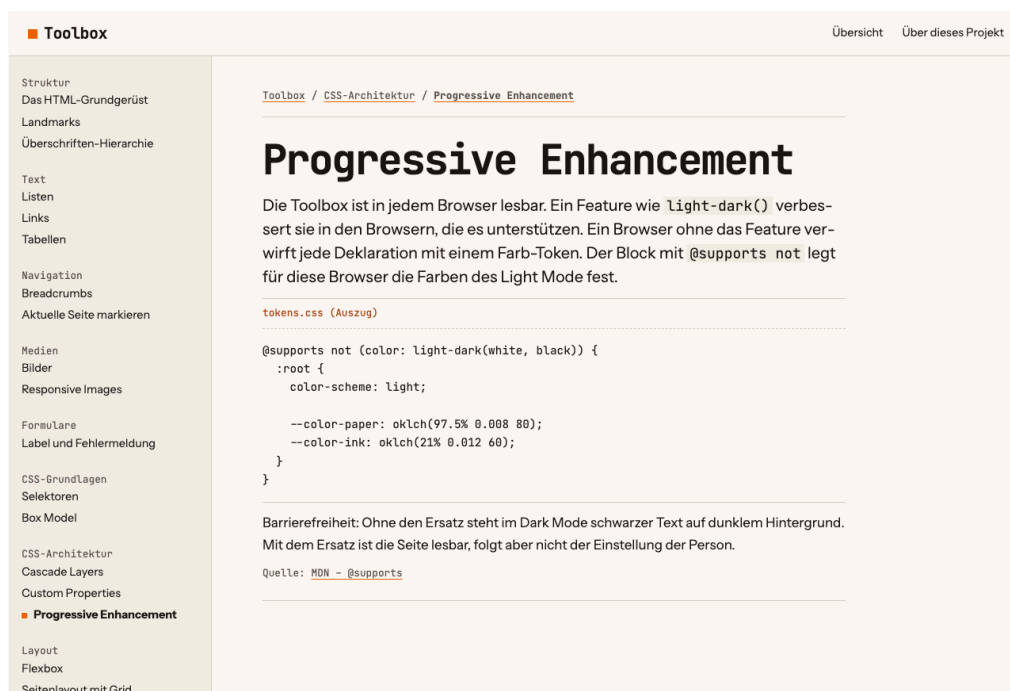
Voraussetzungen: Design-System (Kapitel 1.10)

Lernziele

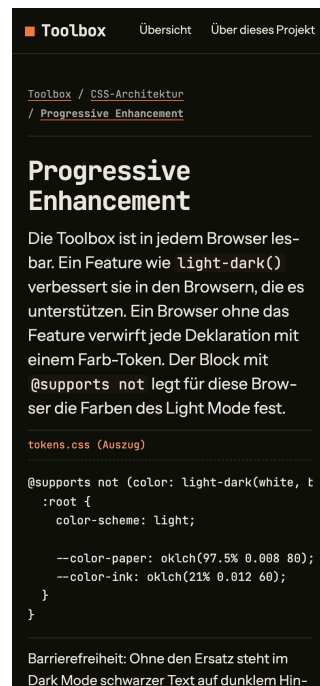
- Auf MDN und caniuse nachsehen, ob ein Feature in den verbreiteten Browsern funktioniert.
- Vorhersagen, was ein Browser mit einer Deklaration, einem Wert oder einem Selektor tut, den er nicht kennt, und das Ergebnis in den DevTools nachweisen.
- Einen Ersatz mit `@supports` schreiben, der die Toolbox in einem Browser ohne `light-dark()` lesbar hält, und ihn mit einer Simulation prüfen.

1. Vorschau

Zielstand nach diesem Kapitel: `tokens.css` enthält einen Ersatz für Browser ohne `light-dark()`. Die Kategorie „CSS-Architektur“ hat den Eintrag „Progressive Enhancement“.



Dieselbe Seite bei Smartphone-Breite im Dark Mode:



2. Warum

Manche Browser kennen ein neues Feature noch nicht. Nicht jede Person verwendet die neueste Version eines Browsers; auf iPhone und iPad hängt die Version von Safari an der Version von iOS. CSS, das ein Browser nicht kennt, verwirft er. Mit Progressive Enhancement bleibt eine Seite ohne ein neueres Feature lesbar.

Weiterführend: MDN – Baseline (compatibility) · MDN – Progressive Enhancement · MDN – @supports

3. Bauen

Ausgangsstand ist die Toolbox aus dem Kapitel Design-System (Kapitel 1.10): 26 Seiten und ein Stylesheet in sechs Dateien.

Die Messwerte dieses Kapitels gelten für Chrome und für eine Datei `overrides.css`, die keine Farb-Tokens festlegt.

3.1 Nachsehen, ob ein Feature funktioniert

MDN zeigt auf der Seite eines Features über dem ersten Abschnitt den Kasten Baseline. Er nennt eine von drei Stufen für die verbreiteten Browser Chrome, Edge, Firefox und Safari:

Stufe	Bedeutung
Widely available	Das Feature funktioniert seit mindestens 30 Monaten in allen verbreiteten Browsern.
Newly available	Das Feature funktioniert in der aktuellen Version aller verbreiteten Browser.
Limited availability	Mindestens ein verbreiteter Browser kennt das Feature nicht.

caniuse zeigt je Browser, ab welcher Version ein Feature funktioniert.

1. Öffne die [MDN-Seite von light-dark\(\)](#) und lies die Stufe im Kasten Baseline ab.
2. Öffne [caniuse – light-dark\(\)](#). Rote Felder stehen für Versionen ohne Unterstützung. Lies für Chrome, Firefox und Safari die letzte Version ohne Unterstützung ab.
3. Lege im Lernprotokoll die Überschrift ## Nicht jeder Browser kann alles (Datum) an und notiere darunter die Stufe und die drei Versionen.

Der Kasten zeigt „Baseline 2024“ und Newly available, ab dem 13.11.2026 Widely available. Chrome kennt `light-dark()` bis Version 122 nicht, Firefox bis 119, Safari bis 17.4.

3.2 Was der Browser verwirft

Ein Browser verwirft CSS, das er nicht kennt. Wie viel er verwirft, hängt von der Stelle ab:

Unbekannt ist	Der Browser verwirft
eine Eigenschaft	die Deklaration
ein Wert oder eine Funktion im Wert	die Deklaration
ein Selektor in einer Selektorliste	die ganze Regel
eine At-Rule: eine Anweisung mit @ wie @layer oder @media	die At-Rule mit ihrem Inhalt

Die übrigen Deklarationen und Regeln gelten weiter.

Für die Toolbox bedeutet das: `text-wrap: balance` verwirft ein Browser ohne das Feature. Die Überschriften brechen dann um wie jeder andere Text. `label:has(+ input)` verwirft ein Browser ohne `:has()`: Das Label steht dann neben dem Feld und ist nicht fett. Die Toolbox bleibt in beiden Fällen lesbar.

3.3 Ersatz mit @supports

Bei `light-dark()` bleibt die Toolbox im Dark Mode nicht lesbar. Alle Farben sind [Custom Properties](#) mit `light-dark()`. Für einen Browser ohne die Funktion ist jede Deklaration ungültig, die ein Farb-Token mit `var()` einsetzt.

Ein [Ersatzwert](#) davor in derselben Regel hilft hier nicht. Der Browser prüft den Wert einer Custom Property erst beim Einsetzen mit `var()`. Zu diesem Zeitpunkt hat die Kaskade die Deklaration mit `var()` bereits ausgewählt und den Ersatzwert verworfen. Ist der eingesetzte Wert ungültig, erhält eine [vererbte](#) Eigenschaft den Wert des umgebenden Elements, jede andere ihren [Anfangswert](#).

Die At-Rule [@supports](#) enthält Regeln, die nur gelten, wenn eine Bedingung zutrifft. Die Bedingung (`color: light-dark(white, black)`) trifft zu, wenn der Browser diese Deklaration kennt. Mit `not` trifft sie nur in Browsern ohne `light-dark()` zu.

Ergänze am Ende von `tokens.css` nach der Regel `:root` diesen Block:

```
/* Ersatz für Browser ohne light-dark(): nur Light Mode */
@supports not (color: light-dark(white, black)) {
  :root {
    color-scheme: light;

    --color-paper: oklch(97.5% 0.008 80);
    --color-paper-2: oklch(94.5% 0.011 80);
    --color-rule: oklch(85% 0.012 75);
```

```

--color-muted: oklch(45% 0.012 70);
--color-ink: oklch(21% 0.012 60);
--color-accent: oklch(66% 0.19 var(--accent-hue));
--color-accent-text: oklch(49% 0.16 var(--accent-hue));
}
}

```

Der Block definiert jedes Farb-Token mit dem ersten Wert aus `light-dark()`. `color-scheme: light` legt fest, dass der Browser seine eigenen Farben ebenfalls hell darstellt, zum Beispiel Scrollbalken und Felder. Ein Browser ohne `light-dark()` zeigt die Toolbox damit im Light Mode, unabhängig von der Einstellung der Person.

In Chrome gilt der Block nicht, weil Chrome `light-dark()` kennt. Die Wirkung zeigt eine Simulation: Chrome kennt die Funktion `lite-dark()` nicht und verhält sich damit wie ein Browser ohne `light-dark()`.

Achtung: Ohne `tokens.css` unter **files to include** ersetzt **Replace All** auch den Linktext „Dark Mode mit `light-dark()`“ in allen HTML-Dateien.

1. Öffne in VS Code die View **Search** und trage unter **files to include** `tokens.css` ein.
2. Suche `light-dark(` und trage im Feld **Replace** `lite-dark(` ein. Die View nennt 10 Treffer, auch in der Bedingung von `@supports`.
3. Führe **Replace All** aus. **Replace All** speichert die Datei.
4. Lade `index.html` neu und führe im Command Menu den Befehl **Emulate CSS prefers-color-scheme: dark** aus.
5. Wähle im Panel **Elements** das Element `<html>` aus und lies im Tab **Computed** den Wert von `background-color`.
6. Ersetze in der View **Search** `lite-dark(` wieder durch `light-dark(` und lade `index.html` neu.
7. Führe den Befehl **Emulate CSS prefers-color-scheme: light** aus.

In Schritt 5 ist die Seite hell, obwohl die DevTools den Dark Mode emulieren. `background-color` hat den Wert `oklch(0.975 0.008 80)`: den Wert von `--color-paper` aus dem Block. In Schritt 6 ist die Seite wieder dunkel.

3.4 Commit

1. Übertrage alle Änderungen mit `git add .` in die Staging Area.
2. Prüfe mit `git status`, dass alle geänderten Dateien unter **Changes to be committed** stehen.
3. Erstelle einen Commit mit der Nachricht `feat: Ersatz für Browser ohne light-dark()` ergänzen und übertrage ihn mit `git push`.

4. Verstehen

Trage die Experimente unter der Überschrift aus Nachsehen, ob ein Feature funktioniert ein. Der Ablauf je Experiment ist: Vorhersage notieren, ausführen, Ergebnis und Erklärung notieren.

Nimm jede Änderung nach dem Experiment zurück: `Strg/Cmd + Z` oder `git restore` mit dem Pfad der Datei.

Predict 1: Tippfehler im Wert

Öffne `patterns/seitenlayout.html` mit der Device Toolbar bei 320 px Breite und notiere die zwei Zeilen der Überschrift „Seitenlayout mit Grid“. Ändere danach in `base.css` in der Regel `h1`, `h2`, `h3` die Deklaration `text-wrap: balance;` in `text-wrap: balanced;`.

- **Vorhersage:** Meldet der Browser den Fehler im Panel **Console**? Wie bricht die Überschrift um? Wie zeigt der Tab **Styles** die Deklaration?
- **Prüfen:**
 1. Lade die Seite neu und lies das Panel **Console**.
 2. Vergleiche die Zeilen der Überschrift mit deiner Notiz.
 3. Wähle die `<h1>` aus und suche die Deklaration im Tab **Styles**.

→ [Auflösung 1](#) am Ende des Kapitels

Break-it 1: Selektorliste aus einem alten Tutorial

Ältere Tutorials ergänzen `:user-invalid` um die Pseudoklasse `:-moz-ui-invalid`, die Firefox früher verwendet hat. Ersetze in `base.css` die Zeile `input:user-invalid {` durch diese Zeilen:

```
input:user-invalid,  
input:-moz-ui-invalid {
```

`:is()` fasst Selektoren zusammen: `:is(A, B)` trifft jedes Element, das A oder B entspricht.

- **Vorhersage:** Erscheint der Balken am Feld „Suchbegriff“ nach einer ungültigen Eingabe? Erscheint er, wenn beide Selektoren in `:is()` stehen?
- **Prüfen:**
 1. Lade `ueber.html` neu.
 2. Gib im Feld „Suchbegriff“ ein Zeichen ein, lösche es und drücke Tab.
 3. Wähle das Feld im Panel **Elements** aus und lies im Tab **Computed** den Wert von `border-inline-start-width`.
 4. Ersetze die zwei Zeilen durch `:is(input:user-invalid, input:-moz-ui-invalid)` { und wiederhole Schritt 1 bis 3.

→ [Auflösung 2](#) am Ende des Kapitels

Break-it 2: Ohne Ersatz im Dark Mode

Lösche in `tokens.css` den Block `@supports not (...)` mit dem Kommentar davor. Führe danach die Simulation aus [Ersatz mit @supports](#), Schritt 1 bis 4, aus. Die View nennt in Schritt 2 jetzt 8 Treffer.

- **Vorhersage:** Wie sieht die Seite im Dark Mode aus? Wie im Light Mode?
- **Prüfen:**
 1. Lies im Tab **Computed** am Element `<html>` die Werte von `background-color` und `color`.
 2. Emuliere den Light Mode und vergleiche.
 3. Nimm die Änderungen in `tokens.css` mit `git restore` zurück. Die Datei hat danach wieder den Stand aus [Commit](#), mit `light-dark()` und dem Block.

→ [Auflösung 3](#) am Ende des Kapitels

5. Toolbox-Eintrag

Schreibe einen weiteren Eintrag:

Datei in patterns	Eintrag	Kategorie	Anchor
progressive-enhancement.html	Progressive Enhancement	CSS-Architektur	#css-architektur

Lege die Seite mit den Schritten 1 bis 3 aus [Navigation & mehrere Seiten](#), [Übrige Seiten](#) und [Übersicht](#) (Kapitel 1.3) an. Die Schritte kopieren `landmarks.html`, ändern `<title>` und `description` und passen die Breadcrumbs an. Ersetze danach das ganze `<article>` durch einen neuen Eintrag mit der Überschrift `<h1>`. Der Eintrag enthält:

1. **Beschreibung und Zweck** in zwei bis fünf Sätzen, in eigenen Worten.
2. **Code:** einen Code-Block mit einem Ausschnitt aus dem eigenen `tokens.css`.
3. **Barrierefreiheit:** die Auswirkung auf die Lesbarkeit in einem Browser ohne das Feature.
4. **Quelle:** einen Link auf die zugehörige MDN-Seite.

Eintrag	Code-Block mit Caption	Barrierefreiheit	Quelle
Progressive Enhancement	Block <code>@supports not (...)</code> nur mit <code>color-scheme</code> , -- <code>color-paper</code> und -- <code>color-ink</code> . <code>tokens.css</code> (Auszug)	Auflösung von Break-it 2	MDN – @supports

Der Eintrag nennt keine Stufe von Baseline und keine Browserversion. Die Stufe ändert sich mit der Zeit, der Eintrag beschreibt die Technik.

```
<article>
  <h1>Progressive Enhancement</h1>
  <p>Die Toolbox ist in jedem Browser lesbar. ...</p>
  <figure class="code">
    <figcaption>tokens.css (Auszug)</figcaption>
    <pre tabindex="0"><code>...</code></pre>
  </figure>
  <p>Barrierefreiheit: ...</p>
  <p>Quelle: <a href="https://developer.mozilla.org/...">MDN - ...</a></p>
</article>
```

Ergänze in `index.html` im Abschnitt „CSS-Architektur“ nach dem `</li>` des Eintrags „Custom Properties“:

```
<li>
  <a href="patterns/progressive-enhancement.html">Progressive
  Enhancement</a>
  <p>Lesbar in jedem Browser, Ersatz mit <code>@supports</code></p>
</li>
```

Ergänze in der Sidebar aller 27 Seiten den neuen Link am Ende der Kategorie „CSS-Architektur“, vor `<p>Layout</p>`:

```

    <li><a href="patterns/progressive-enhancement.html">Progressive
Enhancement</a></li>
  </ul>
<p>Layout</p>

```

Ändere alle Seiten mit der View **Search** wie in Responsive, Toolbox-Eintrag (Kapitel 1.9), mit einer Suche über zwei Zeilen:

1. Aktiviere **Use Regular Expression** und suche `^</ul>\n<p>Layout</p>\n` steht für den Zeilenumbruch; vor `</ul>` und vor `<p>` stehen je sechs Leerzeichen.
2. Füge den Code-Block in das Feld **Replace** ein. Trage unter **files to include** `index.html`, `ueber.html` ein. Die View zeigt **2 results in 2 files**. Klicke auf **Replace All**.
3. Lösche im Feld **Replace** `patterns/`. Ersetze unter **files to include** den Eintrag durch `patterns`. Die View zeigt **25 results in 25 files**. Klicke auf **Replace All**.
4. Deaktiviere **Use Regular Expression**.
5. Die neue Seite ist eine Kopie von `landmarks.html`. Entferne dort `aria-current="page"` am Link „Landmarks“ der Sidebar und setze es an den eigenen Link.

Lade `index.html` neu. Unter „CSS-Architektur“ steht der Link „Progressive Enhancement“ nach „Custom Properties“. Prüfe auf der neuen Seite jeden Link der Breadcrumbs.

Die Referenz-Toolbox zeigt die vollständige Toolbox, wie sie am Ende aussehen kann.

Ersetze in `README.md` den Abschnitt `## Struktur` durch diesen Inhalt:

Struktur

...

toolbox/

```

├── README.md           ← diese Datei
├── learning-notizen.md ← Lernprotokoll: Vorhersagen und Beobachtungen
├── .gitignore          ← Systemdateien, die nicht ins Repository gehören
├── index.html          ← Übersicht mit Links auf alle Einträge
├── ueber.html          ← Über dieses Projekt
├── patterns/           ← je Eintrag eine Seite
│   ├── html-grundgeruest.html
│   ├── landmarks.html
│   ├── ueberschriften.html
│   ├── listen.html
│   ├── links.html
│   ├── tabellen.html
│   ├── breadcrumbs.html
│   ├── aktuelle-seite.html
│   ├── bilder.html
│   ├── responsive-images.html
│   ├── label-fehlermeldung.html
│   ├── selektoren.html
│   ├── box-model.html
│   ├── cascade-layers.html
│   ├── custom-properties.html
│   ├── progressive-enhancement.html
│   ├── flexbox.html
│   ├── seitenlayout.html
│   └── auto-fill-grid.html

```



Header, Sidebar und Footer stehen in jeder HTML-Datei. Eine Änderung an der Sidebar betrifft alle siebenundzwanzig Seiten.

Jede Regel des Stylesheets steht in einer der fünf Ebenen ``tokens``, ``base``, ``layout``, ``components`` und ``overrides``. Eigene Gestaltung steht nur in ``overrides.css``; dort legt ``--accent-hue`` die Akzentfarbe fest.

Das Stylesheet folgt Mobile-first: Die Regeln ohne Bedingung beschreiben die Seite mit einer Spalte. Ab dem Breakpoint 64rem stellt der Browser die Sidebar dar.

Farbwerte stehen nur in ``tokens.css`` und ``overrides.css``, Schriftnamen außerdem in den Regeln ``@font-face`` von ``base.css``. Jede Farbe ist ein Token mit zwei Werten in ``light-dark()``: Die Seite folgt dem Color Scheme des Betriebssystems. Text hat in beiden Color Schemes einen Kontrast von mindestens 4,5:1 zum Hintergrund. Ein Browser ohne ``light-dark()`` erhält über ``@supports`` die Farben des Light Mode.

1. Setze in `README.md` in der Tabelle `## Fortschritt` „Nicht jeder Browser kann alles“ auf „erledigt“ und ergänze die Zeile „Interaktion ohne JavaScript“ mit „offen“.
2. Übertrage alle Änderungen mit `git add .` in die Staging Area.
3. Prüfe mit `git status`, dass alle geänderten Dateien unter **Changes to be committed** stehen, auch `learning-notizen.md`.
4. Erstelle einen Commit mit der Nachricht `feat: Eintrag zu Progressive Enhancement ergänzen` und übertrage ihn mit `git push`.

6. Prüfen

- ☐ Der [W3C-Validator](#) zeigt für jede der siebenundzwanzig Seiten die Meldung „Document checking completed. No errors or warnings to show.“
- ☐ Das Stylesheet enthält keine Änderung aus einem Experiment: Auf GitHub ändert der Commit feat: Eintrag zu Progressive Enhancement ergänzen keine Datei im Ordner assets/css.
- ☐ tokens.css enthält light-dark(an allen Stellen, nicht lite-dark(.
- ☐ Der Block @supports not (color: light-dark(white, black)) definiert jedes Token, das in :root light-dark() verwendet.
- ☐ Simulation: Die Seite ist im Dark Mode hell und lesbar.
- ☐ Ohne Simulation folgt die Seite im Light Mode und im Dark Mode der Einstellung.
- ☐ learning-notizen.md enthält die Stufe und die Versionen aus [Nachsehen, ob ein Feature funktioniert](#) und zu jedem Experiment Vorhersage, Ergebnis und Erklärung.
- ☐ Der Eintrag „Progressive Enhancement“ steht unter „CSS-Architektur“ in der Übersicht.
- ☐ Die Commits feat: Ersatz für Browser ohne light-dark() ergänzen und feat: Eintrag zu Progressive Enhancement ergänzen sind auf GitHub.

Quellen

- [MDN – Baseline \(compatibility\)](#)
- [MDN – Progressive Enhancement](#)
- [MDN – @supports](#)
- [MDN – :is\(\)](#)
- [MDN – CSS error handling](#)
- [MDN – Using CSS custom properties](#)
- [MDN – color-scheme](#)
- [caniuse – light-dark\(\)](#)
- [W3C Nu HTML Checker](#)

Auflösungen

Auflösung 1

Zu [Predict 1: Tippfehler im Wert](#)

Deklaration	Erste Zeile	Zweite Zeile
text-wrap: balance	„Seitenlayout“	„mit Grid“
text-wrap: balanced	„Seitenlayout mit“	„Grid“

Das Panel **Console** enthält keine Meldung. Der Browser kennt den Wert `balanced` nicht und verwirft die Deklaration. Die Überschrift bricht um wie ein Absatz: Die erste Zeile nimmt so viele Wörter auf, wie hineinpassen.

`balance` verteilt die Wörter auf gleich lange Zeilen. Ein Tippfehler und ein Feature, das der Browser nicht unterstützt, haben dieselbe Wirkung: Der Browser verwirft die Deklaration. Das zeigt nur der Tab **Styles**: Er streicht die Deklaration durch und zeigt ein Warnsymbol.

Auflösung 2

Zu [Break-it 1: Selektorliste aus einem alten Tutorial](#)

Selektor	Balken am Feld	<code>border-inline-start-width</code>
<code>input:user-invalid</code> (Ausgangsstand)	ja	8px
<code>input:user-invalid, input:-moz-ui-invalid</code>	nein	1px
<code>:is(input:user-invalid, input:-moz-ui-invalid)</code>	ja	8px

Chrome kennt `-moz-ui-invalid` nicht. Ein unbekannter Selektor in einer Selektorliste verwirft die ganze Regel, auch den Teil, den der Browser kennt. Das Feld ist ungültig und hat trotzdem keinen Balken. Das Panel **Console** enthält keine Meldung.

In `:is()` verwirft der Browser nur den unbekannten Selektor. Die übrigen Selektoren der Liste gelten weiter.

Auflösung 3

Zu [Break-it 2: Ohne Ersatz im Dark Mode](#)

Color Scheme	<code>background-color</code>	<code>color</code>	Darstellung
Dark Mode	<code>rgba(0, 0, 0, 0)</code>	<code>rgb(0, 0, 0)</code>	schwarzer Text auf dunkelgrauem Hintergrund
Light Mode	<code>rgba(0, 0, 0, 0)</code>	<code>rgb(0, 0, 0)</code>	schwarzer Text auf weißem Hintergrund

Jede Deklaration mit einem Farb-Token ist ungültig. Der Hintergrund ist transparent, `color` hat am Element `<html>` den Anfangswert Schwarz, und alle Elemente darin erben ihn. Linien und Rahmen fehlen: Sie stehen in Kurzschreibweisen wie `border-block-end: ... var(--color-rule)`, und mit dem ungültigen Token erhält jeder Teil seinen Anfangswert, auch der Stil `none`. Das Akzent-Quadrat und der Fokusrahmen fehlen ebenfalls.

`color-scheme: light dark` gilt weiter, weil die Deklaration kein Token verwendet. Im Dark Mode ist deshalb der Hintergrund des Browsers dunkel, der Text aber schwarz. Die Seite ist nicht lesbar. Der Browser meldet keinen Fehler.

Der Block aus [Ersatz mit @supports](#) legt `color-scheme: light` und alle Farben fest. Mit ihm ist die Seite im Dark Mode hell und lesbar.



1.12 Interaktion ohne JavaScript

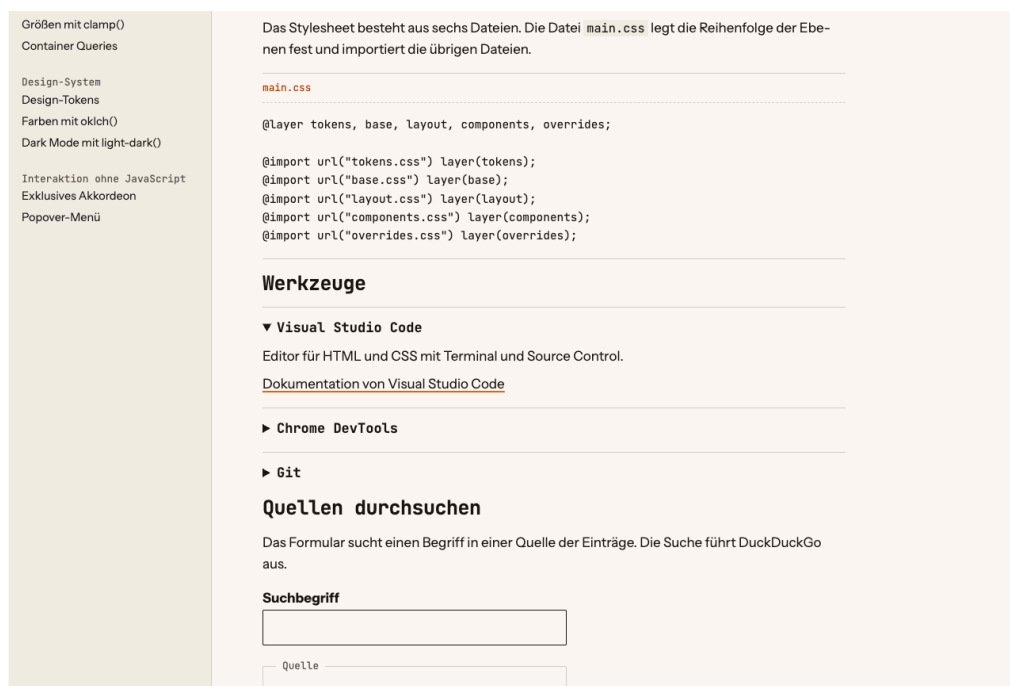
Voraussetzungen: Nicht jeder Browser kann alles (Kapitel 1.11)

Lernziele

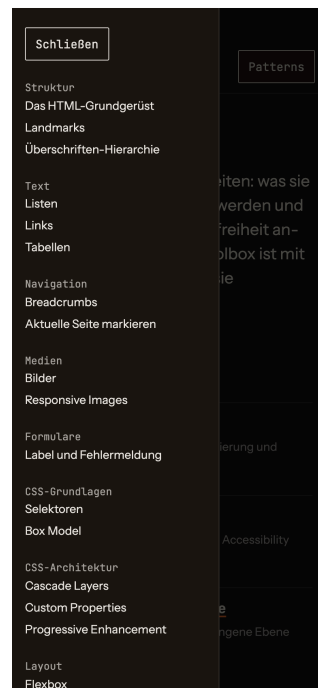
- Die Sidebar unter 64rem mit popover und popovertarget als Menü bauen, ab 64rem die Vorgaben des Browsers überschreiben und mit Tastatur und Accessibility Tree prüfen.
- Das Öffnen und Schließen des Menüs mit transition, @starting-style und allow-discrete animieren und die Bewegung bei prefers-reduced-motion: reduce abschalten.
- Mit :has() ein Element nach dem Zustand eines Nachfahren gestalten und ein exklusives Akkordeon mit <details name> bauen.

1. Vorschau

Zielstand nach diesem Kapitel: Unter 64rem öffnet der Button „Patterns“ im Header die Sidebar als Popover. Die Seite „Über dieses Projekt“ zeigt die Werkzeuge als Akkordeon. Die neue Kategorie „Interaktion ohne JavaScript“ hat zwei Einträge.



Die Übersicht bei Smartphone-Breite im Dark Mode, mit offenem Menü:



2. Warum

Unter 64rem fehlt der Toolbox bisher die Sidebar: Zu den Patterns führt nur die Übersicht. Menüs und Akkordeons brauchten lange JavaScript für Öffnen, Schließen und Fokus. HTML hat dafür heute Attribute und Elemente, deren Verhalten der Browser übernimmt: Tastatur, Fokus und Zustand im Accessibility Tree. CSS fragt diesen Zustand über Pseudoklassen wie `:popover-open` ab. Bewegung ergänzt die Gestaltung; Personen, die im System Bewegung reduziert haben, erhalten sie nicht.

Weiterführend: [MDN – Popover API](#) · [MDN – `:has\(\)`](#) · [MDN – `@starting-style`](#) · [MDN – `<details>`](#)

3. Bauen

Ausgangsstand ist die Toolbox aus dem Kapitel [Nicht jeder Browser kann alles](#) (Kapitel 1.11): 27 Seiten. Unter 64rem stellt der Browser die Sidebar wegen `display: none` nicht dar.

Die Messwerte dieses Kapitels gelten für Chrome und für die Toolbox ohne eigene Gestaltung in `overrides.css`. Sie setzen die Schriftgröße 16 px in den Einstellungen des Browsers voraus.

3.1 Popover im HTML

Ein **Popover** ist ein Element mit dem Attribut `popover`. Der Browser stellt es nur dar, solange es offen ist. Ein Button mit dem Attribut `popover-target` öffnet und schließt es.

Füge in `index.html` im Header nach dem `</nav>` der Main Navigation, direkt vor `</header>`, diesen Button ein:

```
<button type="button" popover-target="patterns">Patterns</button>
```

Ersetze in `index.html` die Zeile `<nav aria-label="Patterns">` durch diese zwei Zeilen:

```
<nav id="patterns" aria-label="Patterns" popover>
  <button type="button" popovertarget="patterns"
  popovertargetaction="hide">Schließen</button>
```

Attribut	Wirkung
popover	Die Sidebar ist ein Popover.
id="patterns"	Name des Elements, über den ein Button das Popover findet
popovertarget="patterns"	Der Button öffnet und schließt das Popover mit dieser id.
popovertargetaction="hide"	Der Button schließt das Popover nur.
type="button"	Der Button sendet kein Formular ab.

Übertrage beide Änderungen mit der View **Search** in die übrigen 26 Seiten. Leere **files to include** und trage bei beiden Suchen unter **files to exclude** `index.html` ein. Das Feld **Replace** nimmt mehrere Zeilen an (siehe VS Code, Suchen und Ersetzen).

Suche `</header>` und füge diese zwei Zeilen in das Feld **Replace** ein. Die erste Zeile beginnt mit zwei Leerzeichen, weil vor `</header>` bereits vier stehen.

```
<button type="button" popovertarget="patterns">Patterns</button>
</header>
```

Die View zeigt **26 results in 26 files**. Klicke auf **Replace All**.

Suche `<nav aria-label="Patterns">` und füge diese zwei Zeilen in das Feld **Replace** ein. Die erste Zeile beginnt ohne Leerzeichen, weil vor `<nav` bereits vier stehen.

```
<nav id="patterns" aria-label="Patterns" popover>
  <button type="button" popovertarget="patterns"
  popovertargetaction="hide">Schließen</button>
```

Die View zeigt **26 results in 26 files**. Klicke auf **Replace All**.

Lade `index.html` bei 390 px Breite neu und klicke auf „Patterns“. Es erscheint nichts. Das Popover ist offen, aber `display: none` aus `layout.css` gilt weiter. Wähle im Panel **Elements** den Button „Patterns“. Der Tab **Accessibility** zeigt trotzdem den Zustand `expanded: true`. Bei 1280 px Breite steht die Sidebar als 36 px schmaler Streifen mit schwarzem Rahmen am linken Rand. Popover im Stylesheet behebt beides.

3.2 Popover im Stylesheet

Der Browser gibt jedem Popover eigene Vorgaben. Wähle im Panel **Elements** das Element `<nav id="patterns">` (siehe Browser & DevTools, Elements). Der Tab **Styles** zeigt unter **user agent stylesheet** die Vorgaben des Browsers für ein Popover:

Eigenschaft	Vorgabe des Browsers
display	none, solange das Popover geschlossen ist
position	fixed: steht fest im Viewport, scrollt nicht mit und ist nicht Teil des Grids
inset	0
width, height	fit-content: so groß wie der Inhalt
border	solid, 3 px

Eigenschaft	Vorgabe des Browsers
padding	0.25em
overflow	auto
color, background-color	CanvasText, Canvas: die Farben des Browsers

Jede Deklaration aus dem eigenen Stylesheet gewinnt gegen eine Vorgabe des Browsers, unabhängig von Ebene und Spezifität. Deshalb gilt nach Popover im HTML `display: none` aus `layout.css` auch für das offene Popover.

Ersetze in `layout.css` die erste Regel `body > nav` (mit `display: none`) und den Kommentar „Sidebar: Navigation mit allen Patterns“ davor durch diese Regeln:

```
/* ---- Sidebar: Navigation mit allen Patterns ----
   Unter 64rem ein Popover am linken Rand, ab 64rem eine Spalte des Grids. */
body > nav {
  grid-area: nav;
  inset: 0 auto 0 0;
  inline-size: var(--sidebar-width);
  block-size: 100dvh;
  padding: var(--space-lg) var(--space-md);
  border: 0;
  border-inline-end: var(--rule-width) solid var(--color-rule);
  color: inherit;
  background-color: var(--color-paper-2);
}

/* Fläche zwischen dem offenen Popover und der Seite */
body > nav::backdrop {
  background-color: var(--color-backdrop);
}
```

Deklaration	Wirkung unter 64rem
ohne display	Den Wert für ein geschlossenes und ein offenes Popover setzt der Browser.
<code>inset: 0 auto 0 0</code>	Abstand oben, rechts, unten, links: am linken Rand über die volle Höhe
<code>inline-size: var(--sidebar-width)</code>	256 px wie die Sidebar ab 64rem. Die Pattern-Liste hat damit dieselbe Form (Container Query aus <u>Responsive</u> (Kapitel 1.9)).
<code>block-size: 100dvh</code>	Höhe des Viewports
<code>border: 0</code>	entfernt den Rahmen des Browsers; die Hairline rechts setzt die nächste Zeile
<code>color: inherit</code>	Textfarbe der Seite statt CanvasText
<code>::backdrop</code>	Fläche zwischen dem offenen Popover und der Seite

`position: fixed` und `overflow: auto` übernimmt die Regel vom Browser. Ist die Liste höher als der Viewport, scrollt sie im Popover.

Ergänze in `tokens.css` in der Regel `:root` nach dem Token `--color-accent-text` das Token für die Fläche:

```
/* Fläche hinter einem offenen Popover, halb durchsichtig */
--color-backdrop: light-dark(oklch(21% 0.012 60 / 0.45), oklch(8% 0.008 70 / 0.7));
```

Der Wert nach / ist die Deckkraft: 0.45 entspricht 45 %. Ergänze im Block `@supports not (...)` nach `--color-accent-text` den Wert des Light Mode. So enthält der Ersatz aus Nicht jeder Browser kann alles (Kapitel 1.11) weiter jedes Token, das `:root` mit `light-dark()` definiert.

```
--color-backdrop: oklch(21% 0.012 60 / 0.45);
```

Ergänze in `layout.css` nach der Regel `body > nav > ul + p` den Abstand unter dem Button „Schließen“:

```
/* Der Button „Schließen“ über der ersten Kategorie */
body > nav > button {
  margin-block-end: var(--space-lg);
}
```

Ab 64rem steht die Sidebar wieder in ihrer Spalte des Grids. Dafür überschreibt der Block `@media` die Vorgaben des Browsers und der Grundregel. Ergänze im Block `@media (width ≥ 64rem)` in der Regel `body > nav` nach `display: block;` diese Zeilen:

```
position: static;
block-size: auto;
```

Ergänze im selben Block nach der Regel `body > nav` diese Regel:

```
body > header > button,
body > nav > button {
  display: none;
}
```

- `position: static` stellt die Sidebar zurück in das Grid.
- `block-size: auto` lässt die Sidebar über die ganze Höhe des Grid Areas `nav` wachsen.
- Beide Buttons sind ab 64rem ohne Aufgabe.

Prüfe das Ergebnis mit `index.html`:

1. Stelle die Breite auf 390 px, lade die Seite neu und klicke auf „Patterns“. Die Sidebar steht 256 px breit am linken Rand über die volle Höhe. Die Seite daneben ist abgedunkelt.
2. Drücke Tab. Der Fokus steht auf „Schließen“, nach einem weiteren Tab auf „Das HTML-Grundgerüst“.
3. Drücke Escape. Das Popover ist geschlossen, der Fokus steht wieder auf „Patterns“.
4. Öffne das Popover erneut. Wähle im Panel **Elements** den Button „Patterns“ und öffne den Tab **Accessibility**. Der Button hat den Zustand `expanded: true`.
5. Klicke auf die abgedunkelte Fläche. Das Popover schließt sich.
6. Stelle die Breite auf 1280 px. Die Sidebar steht 256 px breit in der ersten Spalte, der Button „Patterns“ fehlt. Tab führt von „Über dieses Projekt“ zu „Das HTML-Grundgerüst“.

3.3 Scrollen sperren mit :has()

Bei offenem Popover scrollt die Seite hinter der abgedunkelten Fläche weiter. Die Sperre gehört an das Element `<html>`, solange das Popover offen ist.

A:has(B) trifft ein Element A, wenn ein Nachfahre von A dem Selektor B entspricht.

Ergänze in `layout.css` nach der Regel für den Button „Schließen“:

```
/* Bei offenem Popover scrollt die Seite dahinter nicht. */
html:has(body > nav:popover-open) {
  overflow: hidden;
}
```

:popover-open trifft ein offenes Popover. Die Regel trifft damit `<html>`, solange die Sidebar als Popover offen ist. `overflow: hidden` am Element `<html>` gilt für den Viewport: Die Seite scrollt nicht.

1. Lade `index.html` bei 390 px Breite neu und öffne das Popover.
2. Scrolle über der abgedunkelten Fläche. Die Seite bleibt stehen.
3. Wähle im Panel **Elements** das Element `<html>`. Der Tab **Computed** zeigt `overflow-y: hidden`.

3.4 Bewegung

Eine Transition ändert den Wert einer Eigenschaft über eine Dauer, statt ihn sofort zu setzen. Das Popover soll von links einfahren: `translate` wechselt von `-100% 0` (waagrecht eine Breite nach links, senkrecht keine Verschiebung) nach `0` (keine Verschiebung).

Ergänze in `tokens.css` nach `--sidebar-width`:

```
--duration-menu: 0.2s;
```

Ergänze in `layout.css` nach der Regel `html:has(...)`:

```
/* ---- Bewegung: Das Popover fährt von links ein ----
   Nur, wenn die Person im System Bewegung nicht reduziert hat. */
@media (prefers-reduced-motion: no-preference) {
  body > nav {
    translate: -100% 0;
    transition:
      translate var(--duration-menu),
      display var(--duration-menu) allow-discrete,
      overlay var(--duration-menu) allow-discrete;
  }

  body > nav:popover-open {
    translate: 0;
  }

  /* Startwert der Transition */
  @starting-style {
    body > nav:popover-open {
      translate: -100% 0;
    }
  }
}
```

```

    }
  }
}

```

Teil	Wirkung
@media (prefers-reduced-motion: no-preference)	Die Regeln gelten nur, wenn die Person im System Bewegung nicht reduziert hat.
translate: -100% 0	Das geschlossene Popover steht links außerhalb des Viewports.
translate: 0	Das offene Popover steht am linken Rand.
transition: translate ...	Der Wechsel von translate dauert 0,2 s.
display ... allow-discrete, overlay ... allow-discrete	display und overlay erhalten eine Transition.
@starting-style	Startwert für die Transition

display und overlay haben keine Zwischenwerte. allow-discrete erlaubt ihnen trotzdem eine Transition. overlay legt fest, ob das Popover über allen anderen Elementen liegt.

Ab 64rem steht die Sidebar in ihrer Spalte und nicht links außerhalb. Ergänze im Block @media (width ≥ 64rem) in der Regel body > nav nach block-size: auto;;

```
translate: none;
```

1. Lade index.html bei 390 px Breite neu und öffne das Popover. Es fährt in 0,2 s von links ein. Nach Escape fährt es nach links aus.
2. Führe im Command Menu den Befehl **Emulate CSS prefers-reduced-motion: reduce** aus und wiederhole Schritt 1. Das Popover erscheint und verschwindet ohne Bewegung.
3. Führe den Befehl **Do not emulate CSS prefers-reduced-motion** aus.

3.5 Exklusives Akkordeon

Ein Element <details> zeigt nur seine erste Zeile, das Element <summary>. Ein Klick auf die Zeile öffnet den übrigen Inhalt. Einträge mit demselben Wert im Attribut name bilden eine Gruppe, ein exklusives Akkordeon: Der Browser hält höchstens einen davon offen.

Ersetze in ueber.html unter der Überschrift „Werkzeuge“ die Liste durch diese drei Einträge:

```

<details name="werkzeuge" open>
  <summary>Visual Studio Code</summary>
  <p>Editor für HTML und CSS mit Terminal und Source Control.</p>
  <p><a href="https://code.visualstudio.com/docs">Dokumentation von Visual
Studio Code</a></p>
</details>
<details name="werkzeuge">
  <summary>Chrome DevTools</summary>
  <p>Werkzeuge im Browser für DOM, Styles, Accessibility Tree und
Netzwerk.</p>
  <p><a href="https://developer.chrome.com/docs/devtools">Dokumentation der
Chrome DevTools</a></p>
</details>

```



```
<details name="werkzeuge">
  <summary>Git</summary>
  <p>Versionsverwaltung: jeder Stand der Toolbox als Commit.</p>
  <p><a href="https://git-scm.com/doc">Dokumentation von Git</a></p>
</details>
```

- name="werkzeuge" verbindet die drei Einträge zu einer Gruppe.
- open öffnet den ersten Eintrag beim Laden der Seite.

Ergänze in base.css nach der Regel button: hover:

```
/* Akkordeon: Hairline über jedem Eintrag, summary in Mono */
details {
  padding-block-start: var(--space-sm);
  border-block-start: var(--rule-width) solid var(--color-rule);
}

summary {
  font-family: var(--font-mono);
  font-weight: 700;
  cursor: pointer;
}

details > * + * {
  margin-block-start: var(--space-xs);
}
```

Das Dreieck vor jeder Zeile setzt der Browser. Es zeigt nach rechts, solange der Eintrag geschlossen ist.

1. Lade ueber.html neu und klicke auf „Chrome DevTools“. Der Eintrag öffnet sich, „Visual Studio Code“ schließt sich.
2. Drücke Tab, bis der Fokus auf „Git“ steht, und drücke Enter. „Git“ öffnet sich, „Chrome DevTools“ schließt sich. Die Leertaste schließt „Git“ wieder.
3. Wähle im Panel **Elements** die Zeile <summary>Git</summary> und öffne den Tab **Accessibility**. Die Zeile hat den Zustand expanded: false. Klicke auf „Git“: Der Zustand wechselt auf expanded: true.

3.6 Commit

1. Übertrage alle Änderungen mit git add . in die Staging Area.
2. Prüfe mit git status, dass alle geänderten Dateien unter **Changes to be committed** stehen.
3. Erstelle einen Commit mit der Nachricht feat: Popover-Menü und exklusives Akkordeon ergänzen und übertrage ihn mit git push.

4. Verstehen

Lege im Lernprotokoll die Überschrift ## Interaktion ohne JavaScript (Datum) an. Der Ablauf je Experiment ist: Vorhersage notieren, ausführen, Ergebnis und Erklärung notieren.

Nimm jede Änderung nach dem Experiment zurück: Strg/Cmd + Z oder `git restore` mit dem Pfad der Datei.

Predict 1: Ohne `display: block` ab 64rem

Lösche in `layout.css` im Block `@media (width ≥ 64rem)` in der Regel `body > nav` die Zeile `display: block;`. Die Grundregel `body > nav` enthält seit Popover im Stylesheet kein `display: none` mehr.

- **Vorhersage:** Steht die Sidebar bei 1280 px Breite in ihrer Spalte? Welchen Link erreicht der Fokus mit Tab nach „Über dieses Projekt“?
- **Prüfen:**
 1. Lade `patterns/listen.html` bei 1280 px Breite neu.
 2. Drücke mehrmals Tab und beobachte, welchen Link der Fokus nach „Über dieses Projekt“ erreicht.

→ Auflösung 1 am Ende des Kapitels

Predict 2: Ohne `@starting-style`

Lösche in `layout.css` den Block `@starting-style` mit dem Kommentar davor.

- **Vorhersage:** Fährt das Popover beim Öffnen ein? Fährt es beim Schließen aus?
- **Prüfen:** Setze in `tokens.css` `--duration-menu` auf 2s, damit die Bewegung gut sichtbar ist. Öffne das Popover in `index.html` bei 390 px Breite und schließe es mit Escape.

→ Auflösung 2 am Ende des Kapitels

Break-it 1: Ohne `allow-discrete`

Lösche in `layout.css` in der Deklaration `transition` beide Vorkommen von `allow-discrete`.

- **Vorhersage:** Fährt das Popover beim Öffnen ein? Fährt es beim Schließen aus?
- **Prüfen:** Setze in `tokens.css` `--duration-menu` auf 2s, damit die Bewegung gut sichtbar ist. Öffne das Popover in `index.html` bei 390 px Breite und schließe es mit Escape.

→ Auflösung 3 am Ende des Kapitels

Break-it 2: `:has()` ohne `:popover-open`

Ersetze in `layout.css` den Selektor `html:has(body > nav:popover-open)` durch `html:has(body > nav)`.

- **Vorhersage:** Scrollt die Seite, solange das Popover geschlossen ist?
- **Prüfen:**
 1. Lade `index.html` bei 390 px Breite neu und scrolle, ohne das Popover zu öffnen.
 2. Stelle die Breite auf 1280 px und scrolle erneut.
 3. Wähle im Panel **Elements** das Element `<html>` und lies im Tab **Computed** den Wert von `overflow-y`.

→ Auflösung 4 am Ende des Kapitels

Predict 3: Zwei Einträge mit open

Ergänze in ueber.html am Eintrag „Chrome DevTools“ das Attribut open: <details name="werkzeuge" open>.

- **Vorhersage:** Wie viele Einträge sind nach dem Laden der Seite offen?
- **Prüfen:** Lade ueber.html neu und suche im Panel **Elements** die drei Elemente <details>.

→ Auflösung 5 am Ende des Kapitels

5. Toolbox-Eintrag

Schreibe zwei weitere Einträge in einer neuen Kategorie:

Datei in patterns	Eintrag	Kategorie	Anchor
akkordeon.html	Exklusives Akkordeon	Interaktion ohne JavaScript	#interaktion
popover-menue.html	Popover-Menü	Interaktion ohne JavaScript	#interaktion

Lege die Seiten mit den Schritten 1 bis 3 aus Navigation & mehrere Seiten, Übrige Seiten und Übersicht (Kapitel 1.3) an. Die Schritte kopieren landmarks.html, ändern <title> und description und passen die Breadcrumbs an. Ersetze danach in jeder Seite das ganze <article> durch einen neuen Eintrag mit der Überschrift <h1>. Jeder Eintrag enthält:

1. **Beschreibung und Zweck** in zwei bis fünf Sätzen, in eigenen Worten.
2. **Code:** einen Code-Block mit einem Ausschnitt aus der eigenen Toolbox.
3. **Barrierefreiheit:** das Verhalten mit der Tastatur und den Zustand im Accessibility Tree.
4. **Quelle:** einen Link auf die zugehörige MDN-Seite.

Eintrag	Code-Block mit Caption	Barrierefreiheit	Quelle
Exklusives Akkordeon	die ersten zwei Elemente <details>·ueber.html (Auszug)	Schritte 2 und 3 aus <u>Exklusives Akkordeon</u>	<u>MDN – <details></u>
Popover-Menü	Button „Patterns“ und die Zeile <nav id="patterns" ...>·index.html (Auszug)	Schritte 2 bis 4 aus <u>Popover im Stylesheet</u> , Verhalten bei reduzierter Bewegung	<u>MDN – Popover API</u>

```
<article>
  <h1>Popover-Menü</h1>
  <p>Unter 64rem öffnet der Button „Patterns“ im Header die Sidebar als
  Popover. ...</p>
  <figure class="code">
    <figcaption>index.html (Auszug)</figcaption>
    <pre tabindex="0"><code>...</code></pre>
  </figure>
  <p>Barrierefreiheit: ...</p>
  <p>Quelle: <a href="https://developer.mozilla.org/...">MDN - ...</a></p>
</article>
```

Ein Code-Block zeigt HTML als Text. Schreibe dafür < statt <, zum Beispiel <details name="werkzeuge">. Das Zeichen > darf unverändert stehen, > ist gleichwertig.

Ergänze in `index.html` nach dem `</section>` des Abschnitts „Design-System“, direkt vor `</main>`, den Abschnitt der neuen Kategorie:

```
<section id="interaktion">
  <h2>Interaktion ohne JavaScript</h2>
  <ul class="pattern-list">
    <li>
      <a href="patterns/akkordeon.html">Exklusives Akkordeon</a>
      <p>Höchstens ein Eintrag der Gruppe ist offen</p>
    </li>
    <li>
      <a href="patterns/popover-menue.html">Popover-Menü</a>
      <p>Die Sidebar unter 64rem, geöffnet mit einem Button</p>
    </li>
  </ul>
</section>
```

Ergänze in der Sidebar aller 29 Seiten nach dem `</ul>` der Kategorie „Design-System“ die neue Kategorie. In `index.html` und `ueber.html` endet die Sidebar danach so:

```
<p>Interaktion ohne JavaScript</p>
<ul class="pattern-list">
  <li><a href="patterns/akkordeon.html">Exklusives Akkordeon</a></li>
  <li><a href="patterns/popover-menue.html">Popover-Menü</a></li>
</ul>
</nav>
```

Ändere alle Seiten mit der View **Search** wie in Responsive, Toolbox-Eintrag (Kapitel 1.9): **Use Regular Expression**, Suche `^ </nav>`, den Code-Block im Feld **Replace**.

1. Leere **files to include** und **files to exclude** und trage unter **files to include** `index.html`, `ueber.html` ein. Die View zeigt **2 results in 2 files**. Klicke auf **Replace All**.
2. Lösche im Feld **Replace** die zwei Vorkommen von `patterns/`. Ersetze unter **files to include** den Eintrag durch `patterns`. Die View zeigt **27 results in 27 files**. Klicke auf **Replace All**.
3. Deaktiviere **Use Regular Expression**.
4. Die zwei neuen Seiten sind Kopien von `landmarks.html`. Entferne dort `aria-current="page"` am Link „Landmarks“ der Sidebar und setze es an den eigenen Link.

Lade `index.html` neu. Unter „Interaktion ohne JavaScript“ stehen die Links „Exklusives Akkordeon“ und „Popover-Menü“. Prüfe auf den zwei neuen Seiten jeden Link der Breadcrumbs.

Die Referenz-Toolbox zeigt die vollständige Toolbox, wie sie am Ende aussehen kann.

Ersetze in `README.md` den Abschnitt `## Struktur` durch diesen Inhalt:

```
## Struktur

...

toolbox/
├── README.md           ← diese Datei
├── learning-notizen.md ← Lernprotokoll: Vorhersagen und Beobachtungen
├── .gitignore          ← Systemdateien, die nicht ins Repository gehören
├── index.html          ← Übersicht mit Links auf alle Einträge
└── ueber.html          ← Über dieses Projekt
```



Header, Sidebar und Footer stehen in jeder HTML-Datei. Eine Änderung an der Sidebar betrifft alle neunundzwanzig Seiten.

Jede Regel des Stylesheets steht in einer der fünf Ebenen `tokens`, `base`, `layout`, `components` und `overrides`. Eigene Gestaltung steht nur in `overrides.css`; dort legt `--accent-hue` die Akzentfarbe fest.

Das Stylesheet folgt Mobile-first: Die Regeln ohne Bedingung beschreiben die Seite mit einer Spalte. Unter 64rem öffnet der Button „Patterns“ die Sidebar als

Popover. Ab dem Breakpoint 64rem steht die Sidebar in einer eigenen Spalte. Die Seite enthält kein JavaScript.

Farbwerte stehen nur in ``tokens.css`` und ``overrides.css``, Schriftnamen außerdem in den Regeln ``@font-face`` von ``base.css``. Jede Farbe ist ein Token mit zwei Werten in ``light-dark()``: Die Seite folgt dem Color Scheme des Betriebssystems. Text hat in beiden Color Schemes einen Kontrast von mindestens 4,5:1 zum Hintergrund. Ein Browser ohne ``light-dark()`` erhält über ``@supports`` die Farben des Light Mode.

1. Setze in README.md in der Tabelle ## Fortschritt „Interaktion ohne JavaScript“ auf „erledigt“ und ergänze die Zeile „Barrierefreiheit prüfen“ mit „offen“.
2. Übertrage alle Änderungen mit `git add` in die Staging Area.
3. Prüfe mit `git status`, dass alle geänderten Dateien unter **Changes to be committed** stehen, auch `learning-notizen.md`.
4. Erstelle einen Commit mit der Nachricht `feat: Einträge zu Popover-Menü und Akkordeon ergänzen` und übertrage ihn mit `git push`.

6. Prüfen

- ☐ Der W3C-Validator zeigt für jede der neunundzwanzig Seiten die Meldung „Document checking completed. No errors or warnings to show.“
- ☐ Das Stylesheet enthält keine Änderung aus einem Experiment: Auf GitHub ändert der Commit `feat: Einträge zu Popover-Menü und Akkordeon ergänzen` keine Datei im Ordner `assets/css`.
- ☐ Jede Seite hat im Header den Button „Patterns“ und in der Sidebar den Button „Schließen“.
- ☐ Unter 64rem öffnet „Patterns“ die Sidebar, Escape schließt sie, der Fokus steht danach auf „Patterns“.
- ☐ Ab 64rem steht die Sidebar in ihrer Spalte, beide Buttons fehlen.
- ☐ Mit **Emulate CSS prefers-reduced-motion: reduce** erscheint das Popover ohne Bewegung.
- ☐ Der Block `@supports not (...)` in `tokens.css` definiert auch `--color-backdrop`.
- ☐ In `ueber.html` schließt ein Klick auf „Git“ den offenen Eintrag „Visual Studio Code“.
- ☐ `learning-notizen.md` enthält zu jedem Experiment Vorhersage, Ergebnis und Erklärung.
- ☐ Die Einträge „Exklusives Akkordeon“ und „Popover-Menü“ stehen unter „Interaktion ohne JavaScript“ in der Übersicht.
- ☐ Die Commits `feat: Popover-Menü und exklusives Akkordeon ergänzen` und `feat: Einträge zu Popover-Menü und Akkordeon ergänzen` sind auf GitHub.

Quellen

- [MDN – Popover API](#)
- [MDN – popover](#)
- [MDN – :popover-open](#)
- [MDN – ::backdrop](#)
- [MDN – :has\(\)](#)
- [MDN – transition](#)
- [MDN – transition-behavior](#)

- [MDN – @starting-style](#)
- [MDN – overlay](#)
- [MDN – prefers-reduced-motion](#)
- [MDN – <details>](#)
- [MDN – <summary>](#)
- [W3C Nu HTML Checker](#)

Auflösungen

Auflösung 1

Zu Predict 1: Ohne display: block ab 64rem

Nein. Die erste Spalte ist leer, der Hauptinhalt beginnt weiterhin 256 px nach dem linken Rand. Der Button „Patterns“ ist ab 64rem ausgeblendet. Tab führt von „Über dieses Projekt“ direkt zum ersten Link der Breadcrumbs. Die Patterns sind nur noch über die Übersicht erreichbar.

Der Browser setzt für ein geschlossenes Popover `display: none`. Kein `display` in der Grundregel bedeutet deshalb nicht `display: block`. Erst die Deklaration im Block `@media` überschreibt die Vorgabe des Browsers.

Auflösung 2

Zu Predict 2: Ohne @starting-style

Vorgang	Mit <code>@starting-style</code>	Ohne <code>@starting-style</code>
Öffnen	fährt in 2 s ein	steht sofort am linken Rand
Schließen	fährt in 2 s aus	fährt in 2 s aus

Eine Transition braucht einen Wert vor der Änderung. Das geschlossene Popover hat `display: none`, und für ein Element ohne Darstellung startet der Browser keine Transition. Beim Öffnen erhält das Popover deshalb sofort `translate: 0`. `@starting-style` liefert diesen Startwert.

Beim Schließen ist das Popover dargestellt und hat den Wert 0. Die Transition nach -100% 0 braucht keinen Startwert aus `@starting-style`.

Auflösung 3

Zu Break-it 1: Ohne allow-discrete

Das Popover fährt beim Öffnen ein. Beim Schließen verschwindet es sofort.

`display` hat keine Zwischenwerte. Ohne `allow-discrete` setzt der Browser beim Schließen sofort `display: none`. Die Transition von `translate` läuft an einem Element ohne Darstellung und ist nicht zu sehen. Mit `allow-discrete` bleibt `display: block` bis zum Ende der Dauer.

Ohne `allow-discrete` bei `overlay` liegt das Popover beim Schließen sofort nicht mehr über der Seite: Die abgedunkelte Fläche verschwindet, und die Seite verdeckt das Popover.

Beim Öffnen wechselt `display` in jedem Fall sofort auf `block`, sonst wäre die Bewegung nicht zu sehen. `allow-discrete` wirkt deshalb nur beim Schließen.

Auflösung 4

Zu Break-it 2: `:has()` ohne `:popover-open`

Nein, die Seite scrollt bei keiner Breite. `<html>` hat `overflow-y: hidden`.

Jede Seite enthält die Sidebar `body > nav`, offen oder geschlossen. `html:has(body > nav)` trifft `<html>` deshalb immer. Erst `:popover-open` macht die Regel vom Zustand der Sidebar abhängig.

Auflösung 5

Zu Predict 3: Zwei Einträge mit open

Nur „Visual Studio Code“ ist offen. Im Panel **Elements** hat nur das erste Element `<details>` das Attribut `open`. Am Eintrag „Chrome DevTools“ fehlt es, obwohl es in der Datei steht.

Der Browser hält in einer Gruppe höchstens einen Eintrag offen, auch beim Laden. Er liest die Einträge in der Reihenfolge der Datei. Beim zweiten Eintrag mit `open` ist bereits einer offen, und der Browser entfernt das Attribut.



1.13 Barrierefreiheit prüfen

Voraussetzungen: Interaktion ohne JavaScript (Kapitel 1.12)

Lernziele

- Begründen, für wen Barrierefreiheit gilt (dauerhaft, vorübergehend, situativ) und welche Pflichten BFSG und BITV 2.0 mit Verweis auf die WCAG festlegen.
- Ein Erfolgskriterium der WCAG 2.2 über Prinzip, Richtlinie und Stufe finden und daraus eine prüfbare Anforderung formulieren: Kriterium, Erwartung, Prüfung.
- Eine Seite nach ausgewählten Erfolgskriterien prüfen, Befunde protokollieren und beheben; unterscheiden, was eine automatische Prüfung findet und was nur Tastatur, Kontrastmessung und Simulation zeigen.

1. Vorschau

Zielstand nach diesem Kapitel: Die Toolbox hat ein Audit-Protokoll `audit.md` und eine Liste prüfbarer Anforderungen `anforderung.md`. Der erste Tab-Stopp jeder Seite ist ein Skip-Link. Die neue Kategorie „Barrierefreiheit“ hat einen Eintrag.

The screenshot shows a web application titled 'Toolbox'. At the top, there is a navigation bar with a link 'Zum Inhalt springen' and a status bar with 'Übersicht' and 'Über dieses Projekt'. The left sidebar contains a list of navigation items: Struktur, Das HTML-Grundgerüst, Landmarks, Überschriften-Hierarchie, Text, Listen, Links, Tabellen, Navigation, Breadcrumbs, Aktuelle Seite markieren, Medien, Bilder, Responsive Images, Formulare, Label und Fehlermeldung, CSS-Grundlagen, Selektoren, Box Model, CSS-Architektur, Cascade Layers, Custom Properties, Progressive Enhancement, Layout, Flexbox, and Seitenlayout mit Grid. The main content area is titled 'Toolbox' and contains a description: 'Bauteile moderner Webseiten: was sie tun, warum sie so gebaut werden und worauf es bei der Barrierefreiheit ankommt. Jede Seite der Toolbox ist mit den Patterns gebaut, die sie beschreibt.' Below this, there are several sections: 'Struktur' with sub-sections 'Das HTML-Grundgerüst' (Dokumenttyp, Sprache, Zeichenkodierung und Viewport) and 'Landmarks' (Bereiche der Seite mit einer Rolle im Accessibility Tree); 'Text' with sub-sections 'Listen' (Drei Arten von Listen für drei Arten von Inhalt) and 'Links' (Relativer Pfad, absolute URL oder Anchor); and 'Tabelle' (Daten in Zeilen und Spalten mit Header-Zellen).

Der Eintrag „Skip-Link“ bei Smartphone-Breite im Dark Mode, ebenfalls nach einmal Tab:



2. Warum

Eine Seite ist barrierefrei, wenn Menschen sie unabhängig von ihren Fähigkeiten und ihrer Situation wahrnehmen und bedienen können. Für viele digitale Angebote schreibt das Gesetz Barrierefreiheit vor, und die WCAG legen fest, woran sie gemessen wird. Ein fehlendes Label kostet beim Bauen eine Zeile. Prüft der Auftraggeber das Ergebnis bei der Abnahme, kostet es einen Fehlerbericht und eine neue Version. Wer die Kriterien kennt, vermeidet Barrieren und formuliert die Kriterien als Anforderung für alle, die an einer Seite bauen.

Weiterführend: [W3C – Introduction to Web Accessibility](#) · [MDN – Accessibility](#)

3. Bauen

Ausgangsstand ist die Toolbox aus dem Kapitel [Interaktion ohne JavaScript](#) (Kapitel 1.12): 29 Seiten. Die Abschnitte 3.1 bis 3.3 erklären den Rahmen, 3.4 und 3.5 arbeiten an einer Übungsseite, 3.6 bis 3.9 an der Toolbox.

Die Messwerte dieses Kapitels gelten für Chrome im Light Mode und für die Toolbox ohne eigene Gestaltung in `overrides.css`. Steht dein Betriebssystem auf dunkel, führe im Command Menu den Befehl **Emulate CSS prefers-color-scheme: light** aus.

3.1 Wer eine Seite wie benutzt

Nach Schätzung der WHO leben weltweit 1,3 Milliarden Menschen mit einer erheblichen Behinderung, 16 % der Bevölkerung ([WHO – Disability](#)). Etwa einer von zwölf Männern hat eine Farbschwäche ([NEI – Color Blindness](#)).

Eine Einschränkung ist nicht immer dauerhaft. Eine Barriere trifft auch Personen mit einer vorübergehenden Einschränkung und Personen in einer Situation, die einschränkt:

Bereich	dauerhaft	vorübergehend	situativ	Was die Seite dafür braucht
Sehen	blind, Sehbehinderung	Augentropfen nach einer Untersuchung	Sonnenlicht auf dem Display	Struktur im Accessibility Tree, Alt-Text, Kontrast, Zoom ohne Verlust
Farbe sehen	Farbsehschwäche	–	Display mit Blaulichtfilter	Farbe nie als einziges Merkmal
Bewegen	Lähmung, Tremor	gebrochener Arm	ein Kind auf dem Arm	Bedienung mit der Tastatur, ausreichend große Ziele
Hören	gehörlos	Mittelohrentzündung	Bahnfahrt ohne Kopfhörer	Untertitel, Transkript
Verstehen	Lernbehinderung	Erschöpfung nach einer Operation	Ablenkung, fremde Sprache	klare Struktur, eindeutige Labels und Fehlermeldungen

Personen mit Behinderung verwenden Hilfsmittel und Einstellungen des Systems:

- Ein **Screenreader** liest den Accessibility Tree vor, den du seit [Das Skelett](#) (Kapitel 1.1) prüfst.
- **Zoom** und eine größere **Schriftgröße** vergrößern die Seite. Durch [Responsive](#) (Kapitel 1.9) bleibt der Inhalt der Toolbox bei Zoom bis 400 % im Viewport; der Browser schneidet nichts ab.
- Die **Tastatur** ersetzt die Maus, auch über Schalter oder Spracheingabe, die Tastendrücke senden.
- **Reduzierte Bewegung** im System schaltet die Bewegung aus [Interaktion ohne JavaScript](#) (Kapitel 1.12) ab.

Beschreibungen mit Personen und Situationen enthält [W3C – How People with Disabilities Use the Web](#).

3.2 Rechtlicher Rahmen

Zwei Regelwerke legen in Deutschland fest, welche digitalen Angebote barrierefrei sein müssen:

Regelwerk	Gilt für	Seit
Barrierefreiheitsstärkungsgesetz (BFSG)	Produkte und Dienstleistungen von Unternehmen für Verbraucher, unter anderem Online-Shops, Bankdienstleistungen, E-Books, Telekommunikation, Websites und Apps im Personenverkehr	28. Juni 2025
Barrierefreie-Informationstechnik-Verordnung (BITV 2.0)	Websites, Apps und elektronische Verwaltungsabläufe der öffentlichen Stellen des Bundes	2011, zuletzt geändert 2023

- Das BFSG setzt die EU-Richtlinie 2019/882 um, den [European Accessibility Act](#). Die Richtlinie gilt in der ganzen EU; jeder Mitgliedstaat setzt sie in eigenes Recht um.
- Ein Online-Shop zählt als Dienstleistung im elektronischen Geschäftsverkehr (§ 1 BFSG), auch wenn er Produkte verkauft.

- Das BFSG nimmt Kleinstunternehmen aus, die Dienstleistungen anbieten (§ 3 BFSG). Ein Kleinstunternehmen hat weniger als zehn Beschäftigte und höchstens 2 Millionen Euro Jahresumsatz oder Bilanzsumme (§ 2 BFSG).
- Beide Regelwerke gehen davon aus, dass ein Angebot die Anforderungen erfüllt, wenn es harmonisierten europäischen Normen entspricht (§ 4 BFSG, § 3 BITV 2.0). Für Websites ist das die Norm EN 301 549. Sie übernimmt für Websites die Erfolgskriterien der WCAG der Stufen A und AA. Die Fassung von 2026 verwendet die WCAG 2.2 ([W3C – WCAG 2 Overview](#)).

Die Kette lautet damit: Gesetz, Norm EN 301 549, WCAG 2.2 Stufe AA.

In Projekten für Online-Shops, Banken oder Behörden ist Barrierefreiheit damit eine Anforderung wie jede andere. Sie steht im Vertrag, und der Auftraggeber prüft sie bei der Abnahme. Ob ein Angebot unter ein Gesetz fällt, klärt eine rechtliche Prüfung im Einzelfall.

Entscheide nach der Tabelle und der Liste darunter, ob das BFSG, die BITV 2.0 oder keines der beiden Regelwerke gilt:

1. Der Online-Shop einer Buchhandlung mit 40 Beschäftigten.
2. Die Website einer Bundesbehörde.
3. Der Online-Shop einer Person, die allein Schmuck verkauft und 30.000 Euro im Jahr umsetzt.

→ [Lösung 1](#) am Ende des Kapitels

3.3 Die WCAG lesen

Die Web Content Accessibility Guidelines ([WCAG](#)) sind ein Standard des W3C. Die aktuelle Fassung ist die WCAG 2.2; sie ist auch als ISO/IEC 40500:2025 veröffentlicht. Eine Seite, die die WCAG 2.2 erfüllt, erfüllt auch die WCAG 2.1 und 2.0.

Die WCAG gliedern sich in drei Teile:

Teil	Anzahl	Beispiel
Prinzip	4: wahrnehmbar (Perceivable), bedienbar (Operable), verständlich (Understandable), robust (Robust)	1 Perceivable
Richtlinie (Guideline)	13	1.4 Distinguishable
Erfolgskriterium (Success Criterion)	mehr als 80, je eine prüfbare Aussage mit einer Stufe	1.4.3 Contrast (Minimum), Stufe AA

Die Nummer eines Erfolgskriteriums nennt Prinzip, Richtlinie und Kriterium: 1.4.3 ist das dritte Kriterium der Richtlinie 1.4 im Prinzip 1.

Jedes Erfolgskriterium hat eine von drei Stufen: A, AA oder AAA. Eine Seite erfüllt die Stufe AA, wenn sie jedes Kriterium der Stufen A und AA erfüllt. Gesetze und EN 301 549 verlangen die Stufe AA.

Zwei Seiten des W3C erschließen den Standard:

- [How to Meet WCAG 2.2 \(Quick Reference\)](#) listet alle Erfolgskriterien und lässt sich nach Stufe und Thema filtern.
- [Understanding WCAG 2.2](#) erklärt je Kriterium Zweck, Beispiele und Techniken.

Die WCAG 3 sind ein Entwurf. Das W3C erwartet die fertige Fassung erst in einigen Jahren; die WCAG 2 bleiben danach gültig ([W3C – WCAG 3 Introduction](#)).

Suche in der Quick Reference zu jedem Problem das Erfolgskriterium und seine Stufe. Die Quick Reference ist englisch. Suche auf der Seite mit Strg/Cmd + F nach einem englischen Wort zum Problem, zum Beispiel „contrast“ für zu blassen Text. Findet die Suche nichts, überfliege die Titel der Kriterien unter der Richtlinie, die zum Problem passt. Nenne außerdem je Problem eine Person, die es dauerhaft trifft, und eine, die es situativ trifft.

1. Ein Video hat keine Untertitel.
2. Eine Seite lässt sich auf dem Smartphone nur im Hochformat bedienen.
3. Ein Button zum Schließen ist 16 × 16 px groß und steht direkt neben einem anderen Button.

→ [Lösung 2](#) am Ende des Kapitels

3.4 Eine Seite prüfen

Ein [Audit](#) prüft eine Seite gegen eine Liste von Erfolgskriterien und hält jeden Befund fest. Du übst an einer Seite mit absichtlichen Barrieren.

1. Lade [barrieren.zip](#) herunter und entpacke die Datei außerhalb der Toolbox (siehe Dateisystem & Pfade).
2. Öffne [barrieren/index.html](#) in Chrome.
3. Lege im Toolbox-Ordner die Datei [audit.md](#) mit diesem Anfang an:

```
# Audit

Geprüft nach WCAG 2.2, Stufe A und AA, in Chrome.

## Übungsseite „Workshop: Formulare im Web“ (Datum)

| Kriterium | Stufe | Prüfung | Ergebnis | Maßnahme |
| --- | --- | --- | --- | --- |
```

Prüfe die Übungsseite nach diesen vier Erfolgskriterien:

Erfolgskriterium	Verlangt	Prüfung
1.1.1 Non-text Content (A)	Jedes Bild mit Inhalt hat eine Textalternative.	Name des Bildes im Tab Accessibility des Panels Elements
1.3.1 Info and Relationships (A)	Struktur, die man sieht, steht auch im Code: Ein Feld ist mit seinem Label verbunden.	Name jedes Felds im Tab Accessibility
1.4.3 Contrast (Minimum) (AA)	Text hat mindestens 4,5:1 Kontrast zum Hintergrund.	Kontrast im Tooltip des Inspect mode , wie in Design-System (Kapitel 1.10)
2.4.7 Focus Visible (AA)	Das Element mit dem Fokus ist sichtbar markiert.	Mit Tab durch die Seite

Ein Element, das der Fokus mit Tab erreicht, ist ein [Tab-Stopp](#). Hat ein Bild oder ein Feld im Tab **Accessibility** keinen Namen, ist 1.1.1 oder 1.3.1 nicht erfüllt.

Trage je Kriterium eine Zeile ein. Die Spalte **Ergebnis** nennt „erfüllt“ oder „nicht erfüllt“ und die Stelle. Die Spalte **Maßnahme** nennt die Änderung, die das Kriterium erfüllt. Diese Zeile aus dem Lösungsstand zeigt das Format:

```
| 2.4.7 Focus Visible | AA | Mit `Tab` durch die Seite | nicht erfüllt: Kein Tab-Stopp hat einen sichtbaren Fokus. | Regel `*:focus { outline: none; }` löschen |
```

→ [Lösung 3](#) am Ende des Kapitels

Die Experimente in [Verstehen](#) ergänzen zwei weitere Kriterien.

3.5 Anforderung formulieren

„Die Seite ist barrierefrei“ oder „Die Seite erfüllt die WCAG“ lässt sich nicht abnehmen. Die Aussage nennt weder Stufe noch Kriterium noch die Prüfung, mit der eine andere Person das Ergebnis nachvollzieht. Eine prüfbare Anforderung nennt drei Teile:

Teil	Beispiel
Kriterium mit Stufe	2.1.1 Keyboard (A)
Erwartung an diese Seite	Jede Funktion ist mit der Tastatur bedienbar.
Prüfung	Jede Seite ohne Maus bedienen

Eine Anforderung in dieser Form gibst du an jede Person oder jedes Werkzeug weiter, das an der Seite baut. Wer die Seite abnimmt, prüft das Ergebnis mit der genannten Prüfung.

Lege im Toolbox-Ordner die Datei anforderung.md mit diesem Anfang an:

```
# Anforderungen an die Barrierefreiheit
```

```
Gilt für jede Seite der Toolbox. Maßstab ist die WCAG 2.2, Stufe AA. Jede Anforderung nennt ein Erfolgskriterium und eine Prüfung.
```

```
| Kriterium | Anforderung | Prüfung |
| --- | --- | --- |
```

Ergänze je Kriterium aus [Eine Seite prüfen](#) eine Zeile. [Die Toolbox prüfen](#), Predict 1 und Predict 2 ergänzen weitere Kriterien. Diese Zeile zeigt das Format; die Zeile zu 2.1.1 trägst du in Predict 2 ein:

```
| 2.1.1 Keyboard (A) | Jede Funktion ist mit der Tastatur bedienbar. | Jede Seite ohne Maus bedienen |
```

→ [Lösung 4](#) am Ende des Kapitels

3.6 Die Toolbox prüfen

Die Toolbox erfüllt die Kriterien aus [Eine Seite prüfen](#). Zwei weitere Kriterien betreffen Seiten mit Navigation auf jeder Seite und eigener Gestaltung:

Erfolgskriterium	Verlangt
2.4.1 Bypass Blocks (A)	Blöcke, die auf jeder Seite wiederkehren, lassen sich überspringen.

Erfolgskriterium	Verlangt
<u>1.4.11 Non-text Contrast</u> (AA)	Rahmen von Bedienelementen und ihre Zustände, auch der Fokus, haben mindestens 3:1 Kontrast zur angrenzenden Fläche.

Ergänze in `audit.md` unter dem Abschnitt der Übungsseite:

```
## Toolbox (Datum)

| Kriterium | Stufe | Prüfung | Ergebnis | Maßnahme |
| --- | --- | --- | --- | --- |
```

Prüfe 2.4.1:

1. Lade `index.html` bei 1280 px Breite neu.
2. Drücke Tab, bis der Fokus auf dem ersten Link im Hauptinhalt steht. Zähle die Tab-Stopps davor.

Vor dem Hauptinhalt stehen 30 Tab-Stopps: 3 im Header und 27 in der Sidebar. Unter 64rem sind es 4, weil das Popover geschlossen ist.

Prüfe 1.4.11 für den Fokusrahmen. Er hat bisher die Farbe `--color-accent` (Regel `:focus-visible` in `base.css`). Der Color Picker zeigt den Kontrast nur für eine Textfarbe. Setze die Farbe des Rahmens deshalb probeweise als Textfarbe:

1. Wähle im Panel **Elements** in der Sidebar den Link „Landmarks“.
2. Lege im Tab **Styles** in **element.style** die Deklaration `color: var(--color-accent)` an (siehe Browser & DevTools, Elements).
3. Öffne mit dem Farbfeld vor dem Wert den Color Picker. Er zeigt unter **Contrast ratio** 2.84.
4. Wiederhole die Schritte mit dem Link „Landmarks“ im Hauptinhalt. Der Wert ist 3.11.

Die Deklaration in **element.style** ändert keine Datei; Neuladen entfernt sie.

Der Fokusrahmen in der Sidebar erreicht 3:1 nicht. Die Fläche der Sidebar ist dunkler als die Fläche des Hauptinhalts. Trage beide Befunde in `audit.md` ein. Ergänze in `anforderung.md` je eine Zeile zu 2.4.1 und 1.4.11. Skip-Link behebt 2.4.1, Fokusrahmen behebt 1.4.11.

→ Lösung 5 am Ende des Kapitels

3.7 Skip-Link

Ein Skip-Link ist der erste Tab-Stopp einer Seite und führt zum Hauptinhalt. Er erscheint erst, wenn er den Fokus hat.

Füge in `index.html` direkt nach `<body>` diese Zeile ein:

```
<a class="skip-link" href="#inhalt">Zum Inhalt springen</a>
```

Ändere `<main>` in:

```
<main id="inhalt">
```

Der Link verweist mit `#inhalt` auf das Element mit `id="inhalt"`.

Übertrage beide Änderungen mit der View **Search** in die übrigen 28 Seiten (siehe VS Code, Suchen und Ersetzen). Leere **files to include** und **files to exclude**. Trage bei beiden Suchen unter

files to include *.html und unter **files to exclude** index.html ein. Ohne *.html trifft die Suche auch learning-notizen.md, wenn dort <body> steht.

Suche <body> und füge diese zwei Zeilen in das Feld **Replace** ein:

```
<body>
  <a class="skip-link" href="#inhalt">Zum Inhalt springen</a>
```

Die View zeigt **28 results in 28 files**. Klicke auf **Replace All**.

Suche <main> und ersetze durch <main id="inhalt">. Die View zeigt **28 results in 28 files**. Klicke auf **Replace All**.

Ergänze am Ende von components.css:

```
/* -----
   Skip-Link: erster Tab-Stopp jeder Seite, sichtbar nur mit Fokus
   ----- */
.skip-link {
  position: absolute;
  inset-block-start: var(--space-xs);
  inset-inline-start: var(--space-xs);
  padding: var(--space-xs) var(--space-sm);
  color: var(--color-paper);
  background-color: var(--color-ink);
  translate: 0 -200%;
}

.skip-link:focus-visible {
  translate: none;
}
```

Deklaration	Wirkung
position: absolute	Der Link steht außerhalb des Grids und verschiebt keinen Bereich der Seite.
inset-block-start, inset-inline-start	Abstand zum oberen und linken Rand
color, background-color	Textfarbe und Hintergrund getauscht: Der Link hebt sich vom Header ab.
translate: 0 -200%	Der Link steht über dem oberen Rand des Viewports. Er bleibt im Accessibility Tree und ist ein Tab-Stopp.
.skip-link:focus-visible	Mit dem Fokus steht der Link an seiner Position.

Prüfe das Ergebnis mit index.html bei 1280 px Breite:

1. Lade die Seite neu und drücke Tab. Oben links erscheint „Zum Inhalt springen“.
2. Drücke Enter. Die Adresse endet auf #inhalt. Die Seite zeigt keinen Fokusrahmen: <main> kann keinen Fokus erhalten, der Fokus steht auf <body>.
3. Drücke Tab. Der Fokus steht auf „Das HTML-Grundgerüst“ im Hauptinhalt.

3.8 Fokusrahmen

--color-accent-text ist der Akzent als Textfarbe aus Design-System (Kapitel 1.10). Das Token hat mindestens 4,5:1 Kontrast zu --color-paper und mit dem Standard-Farbton 5,7:1 zur Sidebar.

Ändere in base.css die Regel :focus-visible mit dem Kommentar davor in:

```
/* Sichtbarer Fokus für alle, die mit der Tastatur navigieren.
   Akzent als Textfarbe: mindestens 3:1 zu jeder Fläche (WCAG 1.4.11). */
:focus-visible {
  outline: var(--focus-width) solid var(--color-accent-text);
  outline-offset: 2px;
}
```

1. Wiederhole die Messung aus Die Toolbox prüfen mit color: var(--color-accent-text). Der Color Picker zeigt in der Sidebar 5.68.
2. Drücke auf index.html mehrmals Tab. Der Fokusrahmen in der Sidebar ist dunkler als vorher.
3. Ergänze in audit.md bei 2.4.1 und 1.4.11 in der Spalte **Maßnahme** hinter der Maßnahme „: erledigt“, zum Beispiel Skip-Link: erledigt.

Mit einer eigenen Akzentfarbe in overrides.css ändern sich die Werte. Im Light Mode liegt der Fokusrahmen in der Sidebar mit --color-accent für jeden Farbton unter 3:1, mit --color-accent-text für jeden Farbton über 4,4:1.

3.9 Commit

1. Übertrage alle Änderungen mit git add . in die Staging Area.
2. Prüfe mit git status, dass alle geänderten Dateien unter **Changes to be committed** stehen.
3. Erstelle einen Commit mit der Nachricht feat: Audit, Skip-Link und Fokusrahmen ergänzen und übertrage ihn mit git push.

4. Verstehen

Lege im Lernprotokoll die Überschrift ## Barrierefreiheit prüfen (Datum) an. Der Ablauf je Experiment ist: Vorhersage notieren, ausführen, Ergebnis und Erklärung notieren.

Predict 1 und Predict 2 arbeiten an der Übungsseite und ergänzen je eine Zeile in audit.md und anforderung.md. Break-it 1 ändert components.css. Nimm diese Änderung nach dem Experiment zurück: Strg/Cmd + Z oder git restore assets/css/components.css.

Predict 1: Farbige Punkte am Termin

Die Übungsseite zeigt bei jedem Termin einen farbigen Punkt. Über den Terminen steht „Grün: Plätze frei · Rot: ausgebucht“.

- **Vorhersage:** Die Legende nennt beide Farben in Worten. Erkennt eine Person mit Deuteranopie, einer Form der Rot-Grün-Sehschwäche, welcher Termin ausgebucht ist?
- **Prüfen:**
 1. Öffne im Command Menu mit **Show Rendering** den Tab **Rendering** (Chrome DevTools – Rendering).

2. Wähle unter **Emulate vision deficiencies** die Option **Deuteranopia (no green)**.
3. Wähle danach **No emulation**.

→ Auflösung 6 am Ende des Kapitels

Predict 2: Der Text zur Anfahrt

Unter „Ort“ steht das gepunktet unterstrichene Wort „Anfahrt“. Mit der Maus darüber erscheint ein Text zur Anfahrt.

- **Vorhersage:** Erreicht eine Person mit der Tastatur den Text zur Anfahrt?
- **Prüfen:**
 1. Öffne `barrieren/style.css` in VS Code mit **File → Open File...** (macOS: **File → Open...**, siehe VS Code). Die Toolbox bleibt geöffnet.
 2. Lösche die Regel `*:focus { outline: none; }`, die Maßnahme zu 2.4.7.
 3. Lade die Übungsseite neu.
 4. Drücke Tab, bis der Fokus auf „Anmelden“ steht. Beobachte jeden Tab-Stopp.

→ Auflösung 7 am Ende des Kapitels

Predict 3: Was findet der Validator?

`audit.md` enthält jetzt sechs Befunde zur Übungsseite.

- **Vorhersage:** Wie viele der sechs Befunde meldet der W3C-Validator?
- **Prüfen:** Wähle im W3C-Validator unter **Check by** die Option **file upload** und danach die Datei `barrieren/index.html`. Klicke auf **Check**.

→ Auflösung 8 am Ende des Kapitels

Break-it 1: Skip-Link mit `display: none`

Ändere in `components.css` in der Regel `.skip-link` die Zeile `translate: 0 -200%;` in `display: none;`. Ändere in der Regel `.skip-link:focus-visible` die Zeile `translate: none;` in `display: block;`.

- **Vorhersage:** Erscheint der Skip-Link nach dem ersten Tab? Was zeigt der Tab **Accessibility** für den Skip-Link?
- **Prüfen:**
 1. Lade `index.html` neu und drücke einmal Tab.
 2. Wähle im Panel **Elements** den Skip-Link und öffne den Tab **Accessibility**.

→ Auflösung 9 am Ende des Kapitels

5. Toolbox-Eintrag

Schreibe einen weiteren Eintrag in einer neuen Kategorie:

Datei in patterns	Eintrag	Kategorie	Anchor
<code>skip-link.html</code>	Skip-Link	Barrierefreiheit	<code>#barrierefreiheit</code>

Lege die Seite mit den Schritten 1 bis 3 aus Navigation & mehrere Seiten, Übrige Seiten und Übersicht (Kapitel 1.3) an. Die Schritte kopieren `landmarks.html`, ändern `<title>` und

description und passen die Breadcrumbs an. Ersetze danach das ganze `<article>` durch einen neuen Eintrag mit der Überschrift `<h1>`. Der Eintrag enthält:

1. **Beschreibung und Zweck** in zwei bis fünf Sätzen, in eigenen Worten.
2. **Code:** einen Code-Block mit einem Ausschnitt aus `components.css`.
3. **Barrierefreiheit:** das Erfolgskriterium und das Verhalten mit der Tastatur.
4. **Quelle:** einen Link auf die Seite „Understanding“ des Erfolgskriteriums.

Eintrag	Code-Block mit Caption	Barrierefreiheit	Quelle
Skip-Link	die Regeln <code>.skip-link</code> und <code>.skip-link:focus-visible</code> <code>components.css</code> (Auszug)	2.4.1 und die Auflösung von <u>Break-it 1</u>	<u>W3C – Understanding SC 2.4.1: Bypass Blocks</u>

```
<article>
  <h1>Skip-Link</h1>
  <p>Der Skip-Link ist der erste Tab-Stopp jeder Seite und führt direkt zum
  Hauptinhalt. ...</p>
  <figure class="code">
    <figcaption>components.css (Auszug)</figcaption>
    <pre tabindex="0"><code>...</code></pre>
  </figure>
  <p>Barrierefreiheit: ...</p>
  <p>Quelle: <a href="https://www.w3.org/WAI/WCAG22/Understanding/bypass-
  blocks.html">W3C – Understanding SC 2.4.1: Bypass Blocks</a></p>
</article>
```

Ergänze in `index.html` nach dem `</section>` des Abschnitts „Interaktion ohne JavaScript“, direkt vor `</main>`, den Abschnitt der neuen Kategorie:

```
<section id="barrierefreiheit">
  <h2>Barrierefreiheit</h2>
  <ul class="pattern-list">
    <li>
      <a href="patterns/skip-link.html">Skip-Link</a>
      <p>Der erste Tab-Stopp führt direkt zum Hauptinhalt</p>
    </li>
  </ul>
</section>
```

Ergänze in der Sidebar aller 30 Seiten nach dem `</ul>` der Kategorie „Interaktion ohne JavaScript“ die neue Kategorie. In `index.html` und `ueber.html` endet die Sidebar danach so:

```
<p>Barrierefreiheit</p>
<ul class="pattern-list">
  <li><a href="patterns/skip-link.html">Skip-Link</a></li>
</ul>
</nav>
```

Ändere alle Seiten mit der View **Search** wie in Responsive, Toolbox-Eintrag (Kapitel 1.9): **Use Regular Expression**, Suche `</nav>`, den Code-Block im Feld **Replace**.

1. Leere **files to include** und **files to exclude** und trage unter **files to include** `index.html`, `ueber.html` ein. Die View zeigt **2 results in 2 files**. Klicke auf **Replace All**.
2. Lösche im Feld **Replace** das Vorkommen von `patterns/`. Ersetze unter **files to include** den Eintrag durch `patterns`. Die View zeigt **28 results in 28 files**. Klicke auf **Replace All**.
3. Deaktiviere **Use Regular Expression**.
4. Die neue Seite ist eine Kopie von `landmarks.html`. Entferne dort `aria-current="page"` am Link „Landmarks“ der Sidebar und setze es an den eigenen Link.

Lade `index.html` neu. Unter „Barrierefreiheit“ steht der Link „Skip-Link“. Prüfe auf der neuen Seite jeden Link der Breadcrumbs.

Die Referenz-Toolbox zeigt die vollständige Toolbox, wie sie am Ende aussehen kann.

Ersetze in `README.md` den Abschnitt `## Struktur` durch diesen Inhalt:

Struktur

...

toolbox/

- `README.md` ← diese Datei
- `learning-notizen.md` ← Lernprotokoll: Vorhersagen und Beobachtungen
- `audit.md` ← Audit-Protokoll: Kriterium, Prüfung, Ergebnis, Maßnahme
- `anforderung.md` ← Anforderungen an die Barrierefreiheit, je mit Prüfung
- `.gitignore` ← Systemdateien, die nicht ins Repository gehören
- `index.html` ← Übersicht mit Links auf alle Einträge
- `ueber.html` ← Über dieses Projekt
- `patterns/` ← je Eintrag eine Seite
 - `html-grundgeruest.html`
 - `landmarks.html`
 - `ueberschriften.html`
 - `listen.html`
 - `links.html`
 - `tabellen.html`
 - `breadcrumbs.html`
 - `aktuelle-seite.html`
 - `bilder.html`
 - `responsive-images.html`
 - `label-fehlermeldung.html`
 - `selektoren.html`
 - `box-model.html`
 - `cascade-layers.html`
 - `custom-properties.html`
 - `progressive-enhancement.html`
 - `flexbox.html`
 - `seitenlayout.html`
 - `auto-fill-grid.html`
 - `media-queries.html`
 - `clamp.html`
 - `container-queries.html`
 - `design-tokens.html`
 - `oklch.html`
 - `light-dark.html`
 - `akkordeon.html`
 - `popover-menue.html`
 - `skip-link.html`

```

└─ assets/
    └─ css/
        └─ main.css      ← einzige eingebundene Datei: Reihenfolge der Ebenen,
Importe
    └─ tokens.css        ← Tokens: Farben, Schriften, Schriftgrößen,
Zeilenhöhen, Abstände
    └─ base.css          ← Schriften, Reset, HTML-Elemente ohne Klassen
    └─ layout.css        ← Seitenlayout: Grid Areas, Sidebar als Popover und ab
64rem, Flexbox im Header
    └─ components.css    ← Komponenten mit einer Klasse: Code-Block, Pattern-
Liste, Skip-Link
    └─ overrides.css     ← eigene Gestaltung (freiwillig), gewinnt gegen alle
anderen Dateien
    └─ fonts/            ← Schriften + Lizenzen (vorgegeben)
    └─ img/              ← Bilder (vorgegeben, CC0 1.0)
...

```

Header, Sidebar und Footer stehen in jeder HTML-Datei. Eine Änderung an der Sidebar betrifft alle dreißig Seiten.

Jede Regel des Stylesheets steht in einer der fünf Ebenen ``tokens``, ``base``, ``layout``, ``components`` und ``overrides``. Eigene Gestaltung steht nur in ``overrides.css``; dort legt ``--accent-hue`` die Akzentfarbe fest.

Das Stylesheet folgt Mobile-first: Die Regeln ohne Bedingung beschreiben die Seite mit einer Spalte. Unter 64rem öffnet der Button „Patterns“ die Sidebar als Popover. Ab dem Breakpoint 64rem steht die Sidebar in einer eigenen Spalte. Die Seite enthält kein JavaScript.

Farbwerte stehen nur in ``tokens.css`` und ``overrides.css``, Schriftnamen außerdem in den Regeln ``@font-face`` von ``base.css``. Jede Farbe ist ein Token mit zwei Werten in ``light-dark()``: Die Seite folgt dem Color Scheme des Betriebssystems. Text hat in beiden Color Schemes einen Kontrast von mindestens 4,5:1 zum Hintergrund, der Fokusrahmen mindestens 3:1. Ein Browser ohne ``light-dark()`` erhält über ``@supports`` die Farben des Light Mode.

1. Setze in README.md in der Tabelle ## Fortschritt „Barrierefreiheit prüfen“ auf „erledigt“.
2. Übertrage alle Änderungen mit `git add .` in die Staging Area.
3. Prüfe mit `git status`, dass alle geänderten Dateien unter **Changes to be committed** stehen, auch `learning-notizen.md`.
4. Erstelle einen Commit mit der Nachricht `feat: Eintrag Skip-Link ergänzen` und übertrage ihn mit `git push`.

6. Prüfen

- ☐ `audit.md` enthält sechs Befunde zur Übungsseite und zwei zur Toolbox, je mit Prüfung, Ergebnis und Maßnahme.
- ☐ `anforderung.md` nennt je Kriterium aus `audit.md` eine Anforderung mit Prüfung.
- ☐ Der W3C-Validator zeigt für jede der 30 Seiten der Toolbox die Meldung „Document checking completed. No errors or warnings to show.“

- ☐ Der erste Tab-Stopp jeder Seite ist „Zum Inhalt springen“. Nach Enter und Tab steht der Fokus im Hauptinhalt.
- ☐ Der Fokusrahmen hat in der Sidebar mindestens 3:1 Kontrast.
- ☐ Auf GitHub ändert der Commit feat: Eintrag Skip-Link ergänzen keine Datei im Ordner assets/css.
- ☐ learning-notizen.md enthält zu jedem Experiment Vorhersage, Ergebnis und Erklärung.
- ☐ Der Eintrag „Skip-Link“ steht unter „Barrierefreiheit“ in der Übersicht.
- ☐ Die Commits feat: Audit, Skip-Link und Fokusrahmen ergänzen und feat: Eintrag Skip-Link ergänzen sind auf GitHub.

Quellen

- [W3C – Web Content Accessibility Guidelines \(WCAG\) 2.2](#)
- [W3C – How to Meet WCAG 2.2 \(Quick Reference\)](#)
- [W3C – Understanding WCAG 2.2](#)
- [W3C – WCAG 2 Overview](#)
- [W3C – WCAG 3 Introduction](#)
- [W3C – Introduction to Web Accessibility](#)
- [W3C – How People with Disabilities Use the Web](#)
- [WHO – Disability](#)
- [NEI – Color Blindness](#)
- [Barrierefreiheitsstärkungsgesetz \(BFSG\)](#)
- [Barrierefreie-Informationstechnik-Verordnung \(BITV 2.0\)](#)
- [EU-Richtlinie 2019/882 \(European Accessibility Act\)](#)
- [MDN – Accessibility](#)
- [MDN – :focus-visible](#)
- [Chrome DevTools – Emulate vision deficiencies](#)
- [W3C Nu HTML Checker](#)

Auflösungen

Lösung 1

Zu [3.2 Rechtlicher Rahmen](#)

Angebot	Regelwerk	Grund
Online-Shop der Buchhandlung	BFSG	Ein Online-Shop ist eine Dienstleistung für Verbraucher.
Website der Bundesbehörde	BITV 2.0	Die Behörde ist eine öffentliche Stelle des Bundes.
Online-Shop der Einzelperson	keines	Ein Kleinunternehmen, das eine Dienstleistung anbietet, ist ausgenommen.

Lösung 2

Zu [3.3 Die WCAG lesen](#)

Problem	Erfolgskriterium	Stufe	dauerhaft · situativ (Beispiele)
Video ohne Untertitel	1.2.2 Captions (Prerecorded)	A	gehörlos · Bahnfahrt ohne Kopfhörer
nur Hochformat	1.3.4 Orientation	AA	Tablet fest am Rollstuhl montiert · Tablet quer in einer Halterung
Button mit 16 × 16 px	2.5.8 Target Size (Minimum): mindestens 24 × 24 px oder ausreichend Abstand	AA	Tremor · ein Kind auf dem Arm

Lösung 3

Zu [3.4 Eine Seite prüfen](#)

Kriterium	Stufe	Prüfung	Ergebnis	Maßnahme
1.1.1 Non-text Content	A	Name des Bildes im Tab Accessibility	nicht erfüllt: Der Raumplan hat kein alt, sein Name ist leer.	alt="Raumplan des zweiten Obergeschosses: Raum 2.14 liegt rechts oben, der Aufzug links unten am Flur"
1.3.1 Info and Relationships	A	Feld „E-Mail“ im Tab Accessibility	nicht erfüllt: Das Feld hat keinen Namen. „E-Mail“ ist ein Absatz, kein Label.	<label for="email">E-Mail</label> und id="email" am Feld
1.4.3 Contrast (Minimum)	AA	Kontrast im Tooltip des Inspect mode	nicht erfüllt: Hinweis zur Anmeldung 2.84	Textfarbe #666 (5.74)
2.4.7 Focus Visible	AA	Mit Tab durch die Seite	nicht erfüllt: Kein Tab-Stopp hat einen sichtbaren Fokus.	Regel *:focus { outline: none; } löschen

Lösung 4

Zu 3.5 Anforderung formulieren

Kriterium	Anforderung	Prüfung
1.1.1 Non-text Content (A)	Jedes Bild mit Inhalt hat einen Alt-Text, der den Inhalt nennt. Ein Bild ohne Inhalt hat alt="".	Im Tab Accessibility hat jedes Bild mit Inhalt einen Namen.
1.3.1 Info and Relationships (A)	Jedes Feld hat ein Label. Überschriften, Listen und Tabellen sind mit den passenden Elementen ausgezeichnet.	Im Tab Accessibility hat jedes Feld einen Namen.
1.4.3 Contrast (Minimum) (AA)	Text hat einen Kontrast von mindestens 4,5:1 zum Hintergrund.	Kontrast im Tooltip des Inspect mode , im Light Mode und im Dark Mode
2.4.7 Focus Visible (AA)	Jedes Element mit Fokus hat einen sichtbaren Rahmen.	Mit Tab durch die Seite: Der Fokus ist auf jedem Tab-Stopp zu sehen.

Lösung 5

Zu 3.6 Die Toolbox prüfen

audit.md:

Kriterium	Stufe	Prüfung	Ergebnis	Maßnahme
2.4.1 Bypass Blocks	A	Tab-Stopps bis zum ersten Link im Hauptinhalt zählen, 1280 px	nicht erfüllt: 30 Tab-Stopps	Skip-Link
1.4.11 Non-text Contrast	AA	Akzent probeweise als Textfarbe in der Sidebar, Kontrast im Color Picker	nicht erfüllt: Fokusrahmen 2.84 zur Sidebar	Fokusrahmen in --color-accent-text

anforderung.md:

Kriterium	Anforderung	Prüfung
2.4.1 Bypass Blocks (A)	Der erste Tab-Stopp ist ein Link zum Hauptinhalt.	Seite neu laden, Tab, Enter, Tab: Der Fokus steht auf dem ersten Link im Hauptinhalt.
1.4.11 Non-text Contrast (AA)	Rahmen von Feldern und Buttons und der Fokusrahmen haben mindestens 3:1 zur angrenzenden Fläche.	Farbe probeweise als Textfarbe setzen und den Kontrast im Color Picker ablesen

Auflösung 6

Zu Predict 1: Farbige Punkte am Termin

Nein. Alle drei Punkte erscheinen in fast demselben Olivton. Die Legende hilft nur einer Person, die Rot und Grün unterscheidet.

Die Seite verletzt 1.4.1 Use of Color (A): Farbe ist das einzige Merkmal, das freie und ausgebuchte Termine unterscheidet. Eine Maßnahme ist der Text „ausgebucht“ am Termin. Trage die Zeile in `audit.md` und die Anforderung in `anforderung.md` ein.

Auflösung 7

Zu Predict 2: Der Text zur Anfahrt

Nein. Die Tab-Stopps sind die Gruppe „Termin“, die Felder „Name“ und „E-Mail“ und der Button „Anmelden“. „Anfahrt“ ist ein `<span>` und kein Tab-Stopp. Die Regel `.info:hover .info-text` reagiert nur auf den Mauszeiger.

Die Seite verletzt 2.1.1 Keyboard (A). Eine Maßnahme ist ein Absatz mit der Anfahrt auf der Seite. Trage die Zeile in `audit.md` und die Anforderung in `anforderung.md` ein.

Die drei Radio Buttons sind ein einziger Tab-Stopp. Die Pfeiltasten wählen innerhalb der Gruppe.

Auflösung 8

Zu Predict 3: Was findet der Validator?

Einen. Der Validator meldet nur das fehlende Attribut `alt` am Raumplan.

Der Validator prüft die Regeln der HTML-Spezifikation. Ein Absatz vor einem Feld, ein Hinweis in Grau, eine Regel mit `outline: none` und ein Text unter `:hover` sind gültiges HTML und gültiges CSS. Ob ein Alt-Text den Inhalt des Bildes nennt, prüft er ebenfalls nicht. Werkzeuge zur automatischen Prüfung finden Verstöße, die sich am Code ablesen lassen. Die Prüfung mit Tastatur, Kontrastmessung und Simulation bleibt Aufgabe einer Person.

Auflösung 9

Zu Break-it 1: Skip-Link mit `display: none`

Nein. Der erste Tab-Stopp ist die Wordmark „Toolbox“. Der Tab **Accessibility** zeigt für den Skip-Link „Accessibility node not exposed“ und „Element is not rendered“: Er ist nicht Teil des Accessibility Tree.

Ein Element mit `display: none` hat keine Darstellung. Der Browser nimmt es aus der Reihenfolge der Tab-Stopps und aus dem Accessibility Tree. Es kann deshalb nie den Fokus erhalten, und die Regel mit `:focus-visible` trifft nie zu. `translate` verschiebt nur die Darstellung, der Link bleibt Tab-Stopp.



Anhang



A Glossar

Begriffe aus dem Skript, alphabetisch sortiert. Englische Fachbegriffe bleiben englisch; die deutsche Übersetzung steht in der Definition. Die Kapitel verlinken auf diese Einträge.

Arbeitsweise

Lernprotokoll (`learning-notizen.md`)

Definition: Datei im Projekt der Toolbox. Sie enthält je Experiment die Vorhersage, das Ergebnis und die Erklärung. Sie ist der Nachweis des Verstehens und gehört nicht auf die Website.

Siehe auch: [Toolbox](#)

Lernziele (Kapitel)

Definition: Drei Stichpunkte am Anfang jedes Kapitels. Sie nennen, was du nach dem Kapitel kannst. Anhand der Lernziele entscheidest du, welche Teile du bearbeitest.

Pfad-Training

Definition: Übungsprojekt aus dem Material des Kapitels *Dateisystem & Pfade* mit drei Seiten und absichtlichen Pfadfehlern. Es verbindet die Kapitel *Dateisystem & Pfade*, *VS Code* und *Browser & DevTools*.

Siehe auch: [Pfad](#)

Toolbox

Definition: Das Lernprojekt der Kapitel zu HTML und CSS: eine Website, die Web-Patterns dokumentiert und mit genau diesen Patterns gebaut ist. Unterzeile „Bauteile für das Web“. Die Referenz-Toolbox zeigt die vollständige Toolbox.

Siehe auch: [Lernprotokoll](#)

Tooling

Definition: Das Handwerkszeug der Softwareentwicklung: Dateisystem, Editor, Browser-Dev-Tools und Git. Die Kapitel zu Tooling stehen vor den Kapiteln zu HTML und CSS.

Fachbegriffe

@starting-style

Definition: At-Rule mit Startwerten für eine Transition. Sie gilt, wenn ein Element zum ersten Mal dargestellt wird, zum Beispiel ein Popover beim Öffnen nach `display: none`. Ohne sie fehlt der Wert vor der Änderung, und die Transition findet nicht statt.

Siehe auch: [Transition](#), [Popover](#), [MDN Web Docs](#)

@supports

Definition: At-Rule mit einer Bedingung. Die Regeln im Block gelten nur, wenn der Browser die Deklaration in der Bedingung kennt; mit `not` nur, wenn er sie nicht kennt. Eine Bedingung mit einer Custom Property trifft immer zu, weil eine Custom Property jeden Wert annimmt.

Siehe auch: [Progressive Enhancement](#), [Ersatzwert](#), [MDN Web Docs](#)

:has()

Definition: Pseudoklasse, die ein Element trifft, wenn ein Nachfahre dem Selektor in den Klammern entspricht. Beginnt der Selektor mit `+`, gilt er für das folgende Geschwister. `html:has(body > nav:popover-open)` trifft `<html>`, solange die Sidebar als Popover offen ist.

Siehe auch: [Selektor](#), [Popover](#), [MDN Web Docs](#)

Accessibility Tree

Definition: Vom Browser aus dem DOM abgeleitete Struktur, die assistiven Technologien (z. B. Screenreadern) Rolle, Name und Zustand jedes Elements bereitstellt. In den DevTools einsehbar.

Siehe auch: [DOM](#), [Landmark](#), [MDN Web Docs](#)

Akkordeon (exklusiv)

Definition: Mehrere Elemente `<details>` mit demselben Wert im Attribut `name`. Der Browser hält höchstens einen Eintrag der Gruppe offen, auch beim Laden der Seite. Die Zeile `<summary>` ist ein Tab-Stopp mit dem Zustand `expanded`.

Siehe auch: [Accessibility Tree](#), [MDN Web Docs](#)

Alt-Text

Definition: Der Text im Attribut `alt` eines `<img>`. Er ersetzt das Bild: Screenreader lesen ihn vor, und der Browser zeigt ihn an, wenn die Datei nicht lädt. Er ist der Name des Bildes im Accessibility Tree. Ein dekoratives Bild erhält `alt=""` und fehlt dann im Accessibility Tree.

Siehe auch: [Accessibility Tree](#), [Caption](#), [W3C Web Accessibility Initiative](#)

aria-current

Definition: Attribut, das in einer Gruppe verwandter Elemente das aktuelle kennzeichnet. Der Wert `page` kennzeichnet in einer Navigation den Link, der auf die geöffnete Seite verweist. Das Stylesheet der Toolbox leitet die sichtbare Markierung aus dem Attribut ab.

Siehe auch: [Main Navigation](#), [Breadcrumbs](#), [MDN Web Docs](#)

Anchor

Definition: Stelle einer Seite, auf die ein Link verweist: ein Element mit dem Attribut `id`. Der Link nennt `#` und die `id` (`href="#struktur"`), auf einer anderen Seite mit Pfad davor (`../index.html#struktur`).

Siehe auch: [Linktext](#), [Skip-Link](#), [MDN Web Docs](#)

Anfangswert

Definition: Wert, den CSS für eine Eigenschaft festlegt. Er gilt, wenn keine Deklaration die Eigenschaft für das Element festlegt und die Eigenschaft nicht vererbt wird, zum Beispiel `transparent` für `background-color` oder `none` für `outline`.

Siehe auch: [Vererbung](#), [Custom Property](#), [MDN Web Docs](#)

Attribut

Definition: Angabe im öffnenden Tag eines Elements, geschrieben als Name und Wert: `lang="de"`, `href="index.html"`. Ein Attribut ergänzt oder steuert das Element.

Siehe auch: [Element](#), [Tag](#), [MDN Web Docs](#)

Audit

Definition: Prüfung einer Seite gegen eine Liste von Erfolgskriterien. Das Audit-Protokoll (`audit.md`) nennt je Kriterium Stufe, Prüfung, Ergebnis mit Stelle und Maßnahme.

Siehe auch: [Erfolgskriterium](#), [WCAG](#)

Baseline

Definition: Einstufung eines Features der Web-Plattform nach seiner Verfügbarkeit in den verbreiteten Browsern Chrome, Edge, Firefox und Safari. `Limited availability`: Mindestens ein verbreiteter Browser unterstützt es nicht. `Newly available`: Die aktuelle Version jedes verbreiteten Browsers unterstützt es. `Widely available`: seit 30 Monaten `Newly available`. Der Name des Programms und die Namen der Stufen werden nicht übersetzt.

Siehe auch: [Progressive Enhancement](#), [Web Platform Status](#), [Can I use](#)

Box Model

Definition: Modell, nach dem der Browser jedes Element als Box darstellt: Inhalt, padding (Innenabstand), border (Rahmen) und margin (Außenabstand). `box-sizing` legt fest, ob `width` nur den Inhalt (`content-box`) oder die Box bis einschließlich border (`border-box`) meint.

Siehe auch: [Margin Collapsing](#), [MDN Web Docs](#)

Branch

Definition: Benannte Entwicklungslinie in einem Git-Repository. Ein neues Repository hat einen Branch, in diesem Skript `main`; Commits entstehen auf dem aktuellen Branch.

Siehe auch: [Commit](#), [Repository](#)

Breadcrumbs

Definition: Navigation, die die Lage einer Seite innerhalb der Website nennt: eine geordnete Liste (`<ol>`) von Links von der Startseite bis zur aktuellen Seite. Der englische Begriff wird nicht übersetzt. Der letzte Link trägt `aria-current="page"`.

Siehe auch: [aria-current](#), [Pattern](#), [W3C Web Accessibility Initiative](#)

Breakpoint

Definition: Breite des Viewports, ab der die Bedingung einer Media Query zutrifft. Die Toolbox hat einen Breakpoint bei 64rem: Ab dieser Breite steht die Sidebar als Spalte neben dem Hauptinhalt.

Siehe auch: [Media Query](#), [Mobile-first](#)

Caption

Definition: Sichtbare Beschriftung eines Bildes im Element `<figcaption>` innerhalb von `<figure>` oder einer Tabelle im Element `<caption>`. Die Caption eines Bildes ergänzt es für alle; der Alt-Text ersetzt es. Die Caption einer Tabelle ist ihr Name im Accessibility Tree.

Siehe auch: [Alt-Text](#), [Header-Zelle](#), [MDN Web Docs](#)

Cascade Layers

Definition: Ebenen eines Stylesheets, festgelegt mit `@layer`. Eine Regel in einer späteren Ebene gewinnt gegen eine Regel in einer früheren Ebene, unabhängig von der Spezifität; eine Regel außerhalb jeder Ebene gewinnt gegen alle Ebenen. Im Fließtext heißt die einzelne Schicht „Ebene“. Die Toolbox hat die Ebenen `tokens`, `base`, `layout`, `components`, `overrides`.

Siehe auch: [Kaskade](#), [Spezifität](#), [MDN Web Docs](#)

clamp()

Definition: CSS-Funktion mit drei Angaben: Minimum, Wert und Maximum. Der Wert wächst zum Beispiel mit der Breite des Viewports (`vw`) und bleibt zwischen den zwei Grenzen. Eine Größe ändert sich so ohne Breakpoint.

Siehe auch: [Breakpoint](#), [Custom Property](#)

Clone

Definition: Kopie eines Repositorys von einem Remote auf den eigenen Rechner mit `git clone`, in VS Code **Git: Clone**. Der Klon enthält alle Commits und ist mit dem Remote als `origin` verbunden.

Siehe auch: [Remote](#), [Repository](#)

Code-Block

Definition: Block in Markdown zwischen zwei Zeilen mit mindestens drei Backticks; Zeilenumbrüche und Leerzeichen bleiben erhalten. Nach den öffnenden Backticks steht optional die Sprache. In HTML entspricht er `<pre><code>`.

Siehe auch: [Markdown](#)

Color Scheme

Definition: Helles oder dunkles Farbschema einer Seite: Light Mode oder Dark Mode. Die Eigenschaft `color-scheme` nennt die Color Schemes, die ein Element unterstützt; die Media Query `prefers-color-scheme` fragt ab, welches die Person im Betriebssystem eingestellt hat. Der englische Begriff wird nicht übersetzt.

Siehe auch: [light-dark\(\)](#), [Media Query](#), [MDN Web Docs](#)

Commit

Definition: Gespeicherter Stand eines Git-Repositorys mit Nachricht, Autor:in und eindeutiger Kennung (Hash). Entsteht in zwei Schritten: `git add` (in die Staging Area), `git commit`.

Siehe auch: [Repository](#), [Staging Area](#)

Container Query

Definition: Bedingung im Stylesheet (`@container`), unter der Regeln gelten, zum Beispiel ab einer Breite des Query Containers. Eine Komponente passt sich damit an den Platz an ihrem Ort an, nicht an den Viewport.

Siehe auch: [Query Container](#), [Media Query](#)

Container und Item

Definition: Ein Container ist ein Element mit `display: flex` (Flex-Container) oder `display: grid` (Grid-Container). Seine Kinder sind die Items. Eigenschaften des Containers legen die Anordnung aller Items fest; Enkel sind keine Items.

Siehe auch: [Flexbox](#), [Grid](#)

Conventional Commits

Definition: Konvention für Commit-Nachrichten der Form `<type>: <description>`, z. B. `fix: Pfad zum Logo korrigieren`. Die Spezifikation definiert `feat` und `fix` und lässt weitere Typen zu.

Siehe auch: [Commit](#), [Conventional Commits 1.0.0](#)

CSS

Definition: Cascading Style Sheets – regelbasierte Sprache für die Darstellung von HTML. Ein Stylesheet besteht aus Regeln; der Browser wendet jede Regel auf alle Elemente an, auf die ihr Selektor zutrifft.

Siehe auch: [Regel](#), [Stylesheet](#), [Kaskade](#), [MDN Web Docs](#)

Custom Property

Definition: Eigenschaft mit frei gewähltem Namen, der mit `--` beginnt (`--accent-hue: 45;`). Die Funktion `var()` setzt ihren Wert in eine Deklaration ein. Custom Properties werden vererbt; der Browser prüft den Wert erst beim Einsetzen.

Siehe auch: [Deklaration](#), [MDN Web Docs](#)

Dateiendung

Definition: Teil des Dateinamens nach dem letzten Punkt (.html, .css, .md). Bestimmt, wie Programme die Datei behandeln. Windows und macOS blenden sie standardmäßig aus – für Webprojekte immer einblenden.

Deklaration

Definition: Paar aus Eigenschaft und Wert in einer Regel eines Stylesheets, z. B. `font-size: var(--text-xs);`. Eine Regel besteht aus einem Selektor und einem Block von Deklarationen.

Siehe auch: [Selektor](#), [Kaskade](#)

DevTools

Definition: In den Browser eingebaute Entwicklerwerkzeuge (Elements, Console, Network, Accessibility ...), die zeigen, was der Browser geladen und aus dem Code gemacht hat. Änderungen dort wirken nur im DOM, nicht in der Datei.

Siehe auch: [DOM](#)

DOM

Definition: Document Object Model – die Baumstruktur, die der Browser aus dem HTML-Quelltext erzeugt. Kann vom Quelltext abweichen, weil der Browser Fehler stillschweigend korrigiert.

Siehe auch: [Accessibility Tree](#)

Element

Definition: Baustein eines HTML-Dokuments aus öffnendem Tag, Inhalt und schließendem Tag, zum Beispiel `<title>Toolbox</title>`. Leere Elemente wie `<meta>` und `<link>` haben keinen Inhalt und keinen schließenden Tag.

Siehe auch: [Tag](#), [Attribut](#), [DOM](#), [MDN Web Docs](#)

Erfolgskriterium

Definition: Prüfbare Aussage der WCAG mit Nummer und Stufe (A, AA, AAA), zum Beispiel 1.4.3 Contrast (Minimum), Stufe AA. Die Nummer nennt Prinzip, Richtlinie und Kriterium. Englisch: Success Criterion.

Siehe auch: [WCAG](#), [Audit](#), [W3C Web Accessibility Initiative](#)

Ersatzwert

Definition: Wert für den Fall, dass ein anderer Wert nicht gilt. Als zweite Angabe in `var()` gilt er, wenn die Custom Property nicht definiert ist. Als Deklaration vor einer Deklaration mit einem neueren Feature gilt er in Browsern ohne das Feature. Vor einer Deklaration mit `var()` wirkt er nicht: Der Browser prüft den eingesetzten Wert erst nach der Kaskade.

Siehe auch: [Custom Property](#), [@supports](#), [Kaskade](#)

Fetch

Definition: Abruf neuer Commits von einem Remote mit `git fetch`. Fetch aktualisiert `origin/main` und ändert keinen lokalen Branch.

Siehe auch: [Pull](#), [Remote](#)

Flexbox

Definition: Layout-Verfahren für Items in einer Richtung (Hauptachse): in einer Zeile oder in einer Spalte, mit Umbruch (`flex-wrap`) und Verteilung des freien Platzes je Zeile (`justify-content`). Der Inhalt der Items bestimmt die Aufteilung. Die Querachse verläuft senkrecht zur Hauptachse.

Siehe auch: [Container und Item](#), [Grid](#), [Gap](#), [MDN Web Docs](#)

Fokus

Definition: Das Element, an das Eingaben über die Tastatur gehen. Ein Klick oder die Taste Tab setzt ihn; der Browser zeigt ihn meist als Rahmen um das Element.

Siehe auch: [Tab-Stopp](#)

Gap

Definition: Abstand zwischen den Items eines Flex- oder Grid-Containers, festgelegt mit `gap`. Das Gap steht nur zwischen Items, nicht am Rand des Containers.

Siehe auch: [Flexbox](#), [Grid](#)

Git

Definition: Werkzeug zur Versionskontrolle: speichert Stände (Commits) eines Projekts lokal im Ordner `.git`. GitHub, GitLab oder Codeberg sind Plattformen, auf denen Repositories online liegen.

Siehe auch: [Commit](#), [Remote](#)

Grid

Definition: Layout-Verfahren mit Spalten und Zeilen, die der Container für alle Items festlegt (`grid-template-columns`, `grid-template-rows`). Die Einheit `fr` ist ein Anteil am freien Platz; `minmax()` nennt die kleinste und die größte Größe einer Spalte oder Zeile.

Siehe auch: [Grid Area](#), [Flexbox](#), [Container und Item](#), [MDN Web Docs](#)

Grid Area

Definition: Bereich eines Grids mit einem Namen. `grid-template-areas` legt die Namen im Container fest, `grid-area` ordnet ein Item einem Grid Area zu. Das Seitenlayout der Toolbox hat die Grid Areas `header`, `main` und `footer`.

Siehe auch: [Grid](#), [Landmark](#)

Header-Zelle

Definition: Zelle einer Tabelle, die eine Spalte oder eine Zeile benennt: das Element `<th>`. Das Attribut `scope` nennt die Richtung (`col`, `row`). Im Accessibility Tree hat sie die Rolle `columnheader` oder `rowheader`; eine Datenzelle (`<td>`) hat die Rolle `cell`.

Siehe auch: [Caption](#), [Accessibility Tree](#)

Kaskade

Definition: Verfahren, mit dem der Browser aus mehreren Deklarationen derselben Eigenschaft für ein Element eine auswählt. In den Materialien in drei Schritten: Ebene, Spezifität, Reihenfolge im Quelltext.

Siehe auch: [Cascade Layers](#), [Spezifität](#), [MDN Web Docs](#)

Klasse

Definition: Selbst gewählter Name im Attribut `class` eines Elements (`class="code"`). Der Selektor `.code` wählt jedes Element mit dieser Klasse aus. Der Name nennt die Aufgabe, nicht das Aussehen.

Siehe auch: [Selektor](#), [Komponente](#), [Spezifität](#), [MDN Web Docs](#)

Komponente

Definition: Wiederverwendbarer Baustein einer Seite mit eigenem HTML-Aufbau und eigenen Regeln im Stylesheet, ausgewählt über eine Klasse (`.code`). Der Name der Klasse nennt die Aufgabe, nicht das Aussehen.

Siehe auch: [Pattern](#), [Klasse](#)

Kontrast

Definition: Verhältnis der Helligkeit zweier Farben zwischen 1:1 und 21:1, berechnet nach WCAG. Text braucht zum Hintergrund mindestens 4,5:1, großer Text (ab 24 px oder ab 18,66 px fett) 3:1. Die DevTools nennen den Wert im Tooltip des **Inspect mode**.

Siehe auch: [oklch\(\)](#), [W3C Web Accessibility Initiative](#)

Label

Definition: Das Element `<label>`: der sichtbare Text zu einem Feld eines Formulars. Über `for` und `id` ist er der Name des Felds im Accessibility Tree; ein Klick auf das Label setzt den Fokus in das Feld. Das Attribut `name` ist dagegen der Schlüssel des Werts in den gesendeten Daten.

Siehe auch: [Placeholder](#), [Accessibility Tree](#)

Landmark

Definition: Seitenbereich mit fester Rolle im Accessibility Tree, zu dem Screenreader direkt springen können. HTML-Elemente → Rollen: `<header>` → `banner`, `<nav>` → `navigation`, `<main>` → `main`, `<footer>` → `contentinfo` (header/footer nur als direkte Kinder von `<body>`).

Siehe auch: [Accessibility Tree](#)

Layout Shift

Definition: Verschiebung von sichtbarem Inhalt, nachdem die Seite dargestellt ist, zum Beispiel weil ein Bild ohne reservierte Fläche lädt. Die Attribute `width` und `height` am `<img>` reservieren die Fläche vor dem Laden.

Siehe auch: [Lazy Loading](#), [MDN Web Docs](#)

Lazy Loading

Definition: Aufgeschobenes Laden: Mit `loading="lazy"` fordert der Browser eine Bilddatei erst an, wenn das Bild in die Nähe des Viewports kommt. Wirkt nur auf Seiten, die mehrere Viewports lang sind.

Siehe auch: [Layout Shift](#), [Viewport](#), [web.dev](#)

light-dark()

Definition: CSS-Funktion mit zwei Farben: Sie liefert im Light Mode die erste und im Dark Mode die zweite. Sie wirkt nur mit `color-scheme` und wird an dem Element ausgewertet, dessen Deklaration die Farbe verwendet.

Siehe auch: [Color Scheme](#), [Token](#), [MDN Web Docs](#)

Linktext

Definition: Der Text zwischen `<a>` und `</a>`. Er ist der Name des Links im Accessibility Tree und nennt das Ziel so, dass er ohne den umgebenden Satz verständlich ist.

Siehe auch: [Accessibility Tree](#)

Main Navigation

Definition: Die Navigation im Header jeder Seite mit den Zielen der Website. Enthält eine Seite mehrere Elemente `<nav>`, erhält jedes über `aria-label` einen Namen; in der Toolbox der Studierenden trägt sie den Namen „Main“.

Siehe auch: [Landmark](#), [Breadcrumbs](#)

Margin Collapsing

Definition: Zusammenfallen der Margins zweier Block-Boxen untereinander: Der Abstand ist der größere der beiden Werte, nicht ihre Summe. Padding und Rahmen fallen nie zusammen.

Siehe auch: [Box Model](#), [MDN Web Docs](#)

Markdown

Definition: Textformat, das die Struktur eines Textes (Überschriften, Listen, Links, Code, Tabellen) mit Zeichen im Text kennzeichnet. Das Skript folgt CommonMark, Tabellen stammen aus GitHub Flavored Markdown (GFM).

Siehe auch: [README](#), [Code-Block](#), [CommonMark Spec 0.31.2](#)

Media Query

Definition: Bedingung im Stylesheet (@media), unter der Regeln gelten, zum Beispiel ab einer Breite des Viewports. Ein Grid mit `repeat(auto-fill, minmax())` passt die Spaltenzahl ohne Media Query an den Platz an.

Siehe auch: [Breakpoint](#), [Mobile-first](#), [Container Query](#), [Grid](#), [Viewport](#)

Merge

Definition: Zusammenführen der Commits eines Branch in einen anderen Branch. Ein Merge-Commit hat zwei Vorgänger: den letzten Commit beider Branches.

Siehe auch: [Branch](#), [Merge-Konflikt](#), [Pull Request](#)

Merge-Konflikt

Definition: Zustand, in dem Git zwei Änderungen an derselben Stelle einer Datei nicht automatisch zusammenführen kann. Git schreibt beide Fassungen mit Konfliktmarkern in die Datei; eine Person entscheidet über den Inhalt.

Siehe auch: [Merge](#)

Mobile-first

Definition: Reihenfolge im Stylesheet: Die Regeln ohne Bedingung beschreiben die schmale Form der Seite. Jede Media Query ergänzt Regeln ab einem Breakpoint und steht nach ihnen.

Siehe auch: [Media Query](#), [Breakpoint](#)

oklch()

Definition: CSS-Funktion für eine Farbe aus Helligkeit (0 % bis 100 %), Chroma (0 für Grau bis etwa 0,4) und Farbton (Winkel von 0 bis 360). Die Helligkeit folgt der Wahrnehmung: Farben mit gleicher Helligkeit haben zu einem Hintergrund einen ähnlichen Kontrast.

Siehe auch: [Kontrast](#), [MDN Web Docs](#)

Override-Datei (overrides.css)

Definition: Datei `assets/css/overrides.css` der Toolbox für eigene Gestaltung. Sie gehört zur Ebene `overrides`, die in `main.css` zuletzt steht; eine Regel darin gewinnt gegen die Regeln aller anderen Dateien. Eigene Gestaltung ist freiwillig. Die Kapitel ändern diese Datei nicht.

Siehe auch: [Cascade Layers](#), [Custom Property](#)

Pattern

Definition: Bewährte, wiederverwendbare Lösung für ein wiederkehrendes Problem – hier: ein Baustein einer Webseite (z. B. Breadcrumbs, Landmarks) mit festem HTML-Aufbau und bekannten Regeln zur Barrierefreiheit.

Siehe auch: [Toolbox](#)

Pfad

Definition: Weg zu einer Datei. **Relativ** (.. /assets/img/logo.svg) von der Datei aus, in der er steht – funktioniert, wo auch immer das Projekt liegt. **Absolut** (C:\Users\..., /Users/...) vom obersten Ordner aus (Windows: Laufwerk, macOS: /) – funktioniert nur auf einem Rechner. In HTML und CSS für Dateien des eigenen Projekts immer relativ und immer mit /. Ein Link auf eine andere Website enthält eine absolute URL mit Schema (https://git-scm.com/); sie ist kein Pfad auf dem eigenen Rechner.

Pflichtfeld

Definition: Feld eines Formulars mit dem Attribut `required`. Der Browser sendet das Formular nur ab, wenn das Feld einen Wert enthält, und zeigt sonst eine Fehlermeldung.

Siehe auch: [Validierung](#)

Placeholder

Definition: Das Attribut `placeholder`: ein Beispieltext in einem leeren Feld. Er verschwindet mit der ersten Eingabe und ersetzt kein Label.

Siehe auch: [Label](#)

Popover

Definition: Element mit dem Attribut `popover`. Der Browser stellt es nur dar, solange es offen ist; offen liegt es über allen anderen Elementen der Seite. Ein Button mit `popover-target` öffnet und schließt es, Escape und ein Klick daneben schließen es. Die Pseudoklasse `:popover-open` trifft ein offenes Popover.

Siehe auch: [:has\(\)](#), [@starting-style](#), [MDN Web Docs](#)

Progressive Enhancement

Definition: Vorgehen, bei dem eine Seite mit den Features funktioniert, die jeder Browser unterstützt, und neuere Features sie in Browsern mit Unterstützung verbessern. Ein Feature ohne Unterstützung fällt ohne Schaden weg oder erhält einen Ersatz mit `@supports`. Der englische Begriff wird nicht übersetzt.

Siehe auch: [Baseline](#), [@supports](#), [MDN Web Docs](#)

Pull

Definition: Abruf neuer Commits von einem Remote und Übernahme in den aktuellen Branch mit `git pull`, in VS Code **Git: Pull**. Pull besteht aus Fetch und Merge.

Siehe auch: [Fetch](#), [Push](#)

Pull Request

Definition: Anfrage auf einer Plattform wie GitHub, die Commits eines Branch in einen anderen Branch zu übernehmen, meist nach `main`. Zeigt alle Änderungen und sammelt Kommentare, Reviews und Freigaben.

Siehe auch: [Review](#), [Merge](#), [Branch](#)

Push

Definition: Übertragen der lokalen Commits zu einem Remote mit `git push`, in VS Code **Sync Changes**. Danach zeigt etwa GitHub denselben Stand.

Siehe auch: [Remote](#), [Commit](#)

Query Container

Definition: Element mit der Deklaration `container-type`, dessen Breite eine Container Query abfragt. Der Browser berechnet seine Breite ohne den Inhalt. Der Begriff unterscheidet das Element vom Flex-Container und vom Grid-Container.

Siehe auch: [Container Query](#), [Container](#) und [Item](#)

README

Definition: Datei `README.md` im obersten Ordner eines Projekts. Sie beschreibt Zweck, Aufbau und Start des Projekts für Personen, die es nicht angelegt haben; GitHub zeigt sie auf der Startseite des Repositorys.

Siehe auch: [Markdown](#), [Repository](#)

Regel

Definition: Baustein eines Stylesheets aus einem Selektor und einem Block von Deklarationen in geschweiften Klammern: `figcaption { color: var(--color-muted); }`.

Siehe auch: [Selektor](#), [Deklaration](#), [CSS](#)

Remote

Definition: Verbundene Kopie eines Repositorys auf einem anderen Rechner, meist einer Plattform wie GitHub. Standardname `origin`; `git push` überträgt Commits dorthin.

Siehe auch: [Repository](#)

Repository

Definition: Ein von Git verwaltetes Projekt: Arbeitsordner plus die gesamte Versionsgeschichte im Ordner `.git`. In das Repository gelangt, was ein Commit enthält; die Commits liegen im Ordner `.git`.

Siehe auch: [Commit](#), [Git](#)

Responsive Images

Definition: Mehrere Dateien für ein Bild, aus denen der Browser eine wählt. In der Toolbox: `<picture>` mit `<source media>` liefert je nach Breite des Viewports eine andere Datei; dargestellt wird immer das `<img>`.

Siehe auch: [Viewport](#), [Alt-Text](#), [MDN Web Docs](#)

Review

Definition: Prüfung eines Pull Request durch eine andere Person vor dem Merge. Ergebnis ist ein Kommentar, eine Freigabe (**Approve**) oder die Bitte um Änderungen (**Request changes**).

Siehe auch: [Pull Request](#)

Selektor

Definition: Teil einer Regel, der die Elemente auswählt, für die die Regel gilt: ein Element (a), eine Klasse (.code), ein Attribut ([aria-current="page"]), eine Pseudoklasse (:focus-visible) oder eine Kombination (nav a:jedes <a> in <nav>; main > p:jedes <p> direkt in <main>; * + *:jedes Element direkt nach einem anderen). Eine Liste mit Komma (main > * + *, article > * + *) gilt für jeden ihrer Selektoren.

Siehe auch: [Spezifität](#), [Deklaration](#)

Sidebar

Definition: Spalte neben dem Hauptinhalt. In der Toolbox enthält sie die Navigation mit allen Patterns (<nav aria-label="Patterns">). Ab 64rem steht sie als Spalte neben dem Hauptinhalt; darunter öffnet der Button „Patterns“ sie als [Popover](#).

Siehe auch: [Main Navigation](#), [Breakpoint](#)

Skip-Link

Definition: Link am Anfang jeder Seite, der zum Hauptinhalt führt (href="#inhalt"). Er ist der erste [Tab-Stopp](#) und erscheint erst mit dem Fokus.

Siehe auch: [Landmark](#), [W3C Web Accessibility Initiative](#)

Spezifität

Definition: Gewicht eines Selektors aus drei Zahlen: IDs, dann Klassen, Attribute und Pseudoklassen, dann Elemente und Pseudoelemente (nav ol a = 0,0,3; .code = 0,1,0). Sie entscheidet zwischen zwei Regeln derselben Ebene.

Siehe auch: [Kaskade](#), [Selektor](#), [MDN Web Docs](#)

Staging Area

Definition: Zwischenbereich in Git für Änderungen, die in den nächsten Commit sollen (git add). Erlaubt, zusammengehörige Änderungen gezielt zu einem Commit zu bündeln.

Siehe auch: [Commit](#)

Stylesheet

Definition: Datei mit CSS-Regeln, meist mit der Endung .css. Eine HTML-Seite lädt sie mit <link rel="stylesheet" href="...">; findet der Browser die Datei nicht, erscheint die Seite ohne Gestaltung. Regeln für nur eine Seite stehen im Element <style>, Deklarationen für nur ein Element im Attribut style.

Siehe auch: [Pfad](#), [Selektor](#), [CSS](#)

Tab-Stopp

Definition: Element, das der Fokus mit Tab erreicht: Links, Buttons, Felder und Elemente mit `tabindex="0"`. Eine Gruppe von Radio Buttons ist ein Tab-Stopp; die Pfeiltasten wählen darin.

Siehe auch: [Skip-Link](#), [W3C Web Accessibility Initiative](#)

Tag

Definition: Auszeichnung in spitzen Klammern, die ein Element beginnt (`<title>`) oder beendet (`</title>`). Element und Tag sind nicht dasselbe: Das Element umfasst beide Tags und den Inhalt dazwischen.

Siehe auch: [Element](#)

Token

Definition: Custom Property für einen Wert der Gestaltung: Farbe, Schrift, Schriftgröße, Zeilenhöhe, Abstand oder Maß. Der Name nennt die Aufgabe, nicht das Aussehen (`--color-paper`, nicht `--color-white`). Die Toolbox definiert alle Tokens in `tokens.css`; das Pattern heißt „Design-Tokens“.

Siehe auch: [Custom Property](#), [Design Tokens Community Group](#)

Transition

Definition: Übergang einer Eigenschaft von einem Wert zu einem anderen über eine Dauer, festgelegt mit `transition`. Eigenschaften ohne Zwischenwerte wie `display` wechseln nur mit `allow-discrete` erst am Ende der Dauer. Personen, die im System Bewegung reduziert haben, erhalten in der Toolbox keine Transition (`prefers-reduced-motion`).

Siehe auch: [@starting-style](#), [MDN Web Docs](#)

Überschriften-Hierarchie

Definition: Gliederung einer Seite durch `<h1>` bis `<h6>`: genau ein `<h1>`, keine übersprungene Ebene, Ebene nach Struktur statt nach Schriftgröße.

Siehe auch: [Accessibility Tree](#)

Validierung

Definition: Prüfung der Eingaben eines Formulars durch den Browser vor dem Absenden, gesteuert über Attribute wie `required`, `type` und `pattern`. Der Selektor `:invalid` trifft auf ein Feld mit ungültigem Wert zu, `:user-invalid` erst nach einer Eingabe der Person oder nach dem Absenden.

Siehe auch: [Pflichtfeld](#)

Vererbung

Definition: Ein Element übernimmt den Wert einer Eigenschaft vom umgebenden Element, solange keine Regel die Eigenschaft für das Element selbst festlegt. Custom Properties, Textfarbe und Schrift werden vererbt, Abstände und Hintergrund nicht. Eine Custom Property wird mit ihrem berechneten Wert vererbt: `var()` ist darin eingesetzt, eine Funktion wie `light-dark()`

bleibt stehen.

Siehe auch: [Custom Property](#), [Kaskade](#), [MDN Web Docs](#)

Viewport

Definition: Der sichtbare Bereich der Seite im Browserfenster. Ohne `<meta name="viewport">` rechnen Mobilbrowser mit einer virtuellen Breite von ca. 980 px und verkleinern die Seite.

WCAG

Definition: Web Content Accessibility Guidelines des W3C, aktuell in der Fassung 2.2: vier Prinzipien, 13 Richtlinien und Erfolgskriterien mit den Stufen A, AA und AAA. BFG und BITV 2.0 verlangen über EN 301 549 die Stufe AA.

Siehe auch: [Erfolgskriterium](#), [Kontrast](#), [W3C Web Accessibility Initiative](#)

Wordmark

Definition: Name einer Website als gestalteter Schriftzug, meist oben links im Header und als Link auf die Startseite. In der Toolbox ist es der Link „Toolbox“ mit dem farbigen Quadrat.

Siehe auch: [Landmark](#)

Workspace

Definition: Ein in VS Code geöffneter Ordner. Explorer, Pfad-Vervollständigung und Terminal beziehen sich auf diesen Ordner.

Zeichenkodierung

Definition: Regel, nach der ein Programm die gespeicherten Bytes einer Datei in Zeichen übersetzt. UTF-8 enthält alle Zeichen, auch Umlaute wie „ä“ und „ß“. Eine HTML-Seite nennt sie mit `<meta charset="utf-8">`.

Siehe auch: [MDN Web Docs](#)